list every artifact and thing for claud to genrate and produce for Semester

Claude can generate almost any self-contained deliverable you would put in front of a user, teammate, customer, investor, student, or developer. For **Semester**—assuming it is a product, startup, or software project—the most useful approach is to define artifacts across strategy, brand, product, engineering, sales, operations, and customer success.

Claude's native artifact formats include documents, decks, designs, dashboards, code, single-page websites, diagrams, images, and small interactive tools. [1] [2]

Strategy and company

- Company narrative and "what is Semester?" one-pager
- Mission, vision, values, and operating principles
- Founding story and origin memo
- Product thesis
- Market thesis
- Category-definition memo
- Investor narrative
- Pitch deck
- Fundraising data room index
- Executive summary
- Annual plan
- Quarterly operating plan
- OKRs and KPI tree
- Strategic roadmap
- Decision memos
- Board-deck outline and board updates
- Competitive landscape
- Competitor battlecards
- SWOT analysis
- Porter's Five Forces analysis
- TAM/SAM/SOM market-sizing model

- Pricing and packaging strategy
- Business-model canvas
- Unit-economics model
- Revenue forecast
- Hiring plan
- Org-chart proposal
- Job scorecards
- Interview rubrics
- Company handbook
- Remote-work and communications norms
- Risk register
- Security and compliance roadmap

Brand and marketing

- Brand strategy document
- Positioning statement
- Messaging architecture
- Value-proposition matrix
- Ideal customer profile definition
- Buyer personas
- Jobs-to-be-done interviews and synthesis
- Voice-and-tone guidelines
- Brand naming explorations
- Taglines and slogans
- Brand manifesto
- Brand style guide
- Color palette and typography proposal
- Logo brief or SVG logo concepts
- Moodboards and visual-direction briefs
- Homepage copy
- Full marketing-site copy
- Landing pages by audience or use case
- Product-launch plan
- Go-to-market strategy
- Content strategy

- SEO strategy and keyword clusters
- Editorial calendar
- Blog outlines and complete articles
- Case-study templates and customer stories
- Whitepapers and reports
- Lead magnets
- Email nurture sequences
- Lifecycle-email copy
- Newsletter drafts
- LinkedIn content calendar
- X/Twitter posts and threads
- Founder-led content
- Press release
- Media kit
- Podcast pitch emails
- Partner-marketing plans
- Webinar scripts and slide decks
- Event booth copy and collateral
- Ad creative briefs
- Search-ad copy
- Social-ad variations
- Retargeting campaigns
- Referral-program copy
- Product Hunt launch kit
- Community-launch plan

Product artifacts

- Product vision document
- Product requirements documents (PRDs)
- Feature specifications
- User stories
- Acceptance criteria
- User journeys
- Journey maps
- Jobs-to-be-done maps

- Use-case library
- Feature prioritization matrix
- RICE, ICE, MoSCoW, and Kano analyses
- Product roadmap
- Release plans
- Release notes
- Changelog copy
- Product positioning per feature
- User-research plan
- Interview discussion guides
- Survey questions
- Usability-test scripts
- Research synthesis
- Insight repository
- Feature-request taxonomy
- Problem statements
- Opportunity-solution trees
- Experiment briefs
- A/B-test hypotheses
- Success-metric definitions
- North-star metric proposal
- Product analytics tracking plan
- Event taxonomy
- Funnel definition
- Retention-cohort analysis framework
- Product feedback forms
- Beta-program plan
- Early-access application
- Feature-announcement emails
- In-app onboarding flows
- Empty-state copy
- Error messages
- Tooltips
- Upgrade prompts
- Paywall copy

- Cancellation survey and save-flow copy

UX, design, and prototypes

- Product-design brief
- Information architecture
- Sitemap
- Navigation model
- User flows
- Task flows
- Wireframes
- Low-fidelity prototypes
- High-fidelity interface mockups
- Interactive React or HTML prototypes
- Mobile-app screens
- Responsive web designs
- Design-system documentation
- Component inventory
- Component specifications
- Token definitions for color, spacing, type, radius, shadows, and motion
- UI copy and microcopy
- Accessibility requirements
- WCAG review checklist
- Empty, loading, success, warning, and error states
- Dashboard layouts
- Admin-console layouts
- Pricing-page mockups
- Onboarding wizard prototype
- Settings-page prototype
- Data-table prototype
- Command-palette prototype
- Interactive calculator
- Interactive configurator
- Clickable customer demo
- SVG illustrations
- Mermaid diagrams

- Flowcharts
- Architecture diagrams
- Service blueprints

Claude can create visual work such as landing pages, one-pagers, product mockups, and interface prototypes, as well as interactive pages and small tools. [2] [1]

Engineering and technical

- Technical design documents
- System architecture diagrams
- Data-flow diagrams
- Sequence diagrams
- Database schema and ERDs
- API specifications
- OpenAPI/Swagger definitions
- API reference documentation
- SDK documentation
- Authentication and authorization design
- Role-and-permission matrix
- Data model
- Migration plan
- Infrastructure-as-code templates
- Dockerfiles and Docker Compose configuration
- CI/CD pipeline configuration
- GitHub Actions workflows
- Deployment runbooks
- Environment-variable documentation
- Secrets-management plan
- Observability plan
- Logging standards
- Alerting rules
- Incident-response playbook
- Disaster-recovery plan
- Backup and restore procedures
- Performance-testing plan
- Load-test scripts

- QA strategy
- Test plans
- Unit, integration, end-to-end, and regression test cases
- Test-data generators
- Bug-report templates
- Debugging checklists
- Code-review checklist
- Repository README files
- Contribution guide
- Architecture decision records
- Technical debt register
- Dependency audit
- Security threat model
- Privacy impact assessment
- Data-retention policy
- Accessibility test plan
- Release checklist
- Developer onboarding guide
- Prompt library for internal AI features
- RAG architecture and evaluation plan, if Semester uses AI
- LLM guardrails and safety specifications
- Evaluation datasets and scoring rubrics
- Production code for web apps, APIs, scripts, CLIs, automations, and internal tools

Artifacts can include code, websites, diagrams, dashboards, and interactive React-based components—useful for turning specs into a working prototype rather than merely a document. [3] [4]

Sales and revenue

- Sales strategy
- Sales playbook
- Discovery-call script
- Qualification framework
- MEDDICC, BANT, or custom qualification checklist
- Discovery-question bank
- Demo script

- Demo environment plan
- Sales deck
- One-page product overview
- ROI calculator
- Business-case template
- Proposal template
- Statement of work
- Pricing quote template
- Procurement-response kit
- RFP/RFI response library
- Objection-handling guide
- Competitor battlecards
- Security questionnaire responses
- Technical-validation guide
- Pilot-plan template
- Proof-of-concept success criteria
- Account plan
- Mutual action plan
- Sales-email sequences
- Cold outbound sequences
- LinkedIn outreach messages
- Follow-up email templates
- Renewal playbook
- Expansion playbook
- Channel-partner playbook
- Affiliate-program terms
- CRM pipeline stages and definitions
- Sales dashboard
- Forecasting model
- Revenue-operations SOPs
- Commission-plan draft

Customer success and support

- Customer onboarding plan
- Welcome sequence
- Implementation guide
- Customer training curriculum
- Training deck
- Product tutorials
- Interactive walkthrough prototype
- Knowledge-base articles
- FAQ library
- Troubleshooting guides
- Support macros
- Support ticket taxonomy
- Escalation policy
- Service-level objectives
- Customer health-score model
- Adoption dashboard
- QBR template
- Customer-success playbook
- Renewal and churn-prevention plan
- Customer-feedback program
- NPS/CSAT surveys
- Churn survey
- User-community guidelines
- Community-launch plan
- Customer advisory board charter
- Status-page incident templates
- Customer-facing postmortem template

Operations, legal, and finance

- Operating manual
- Standard operating procedures
- Meeting templates
- Weekly business review agenda
- Leadership-team agenda

- Project plans
- RACI matrix
- Vendor-evaluation matrix
- Vendor onboarding checklist
- Procurement checklist
- Policy library
- Employee onboarding checklist
- Offboarding checklist
- Performance-review templates
- Compensation philosophy draft
- Benefits-guide draft
- Data-processing addendum draft
- Privacy-policy first draft
- Terms-of-service first draft
- Acceptable-use policy
- Cookie-policy draft
- Security overview
- SOC 2 readiness checklist
- Information-security policy drafts
- Incident-response policy
- Business continuity plan
- Budget model
- Cash-flow forecast
- P&L model
- Headcount budget
- Expense policy
- Invoice templates
- Financial dashboard
- KPI dashboard
- Investor-update template
- Board-report template

For legal, financial, security, privacy, or compliance documents, use Claude to create a structured first draft and checklist—but have qualified counsel, accountants, and security professionals review and approve final versions.

Best “Semester” starter bundle

If you want Claude to produce the highest-leverage initial set for Semester, start with these 15 artifacts:

1. A sharp company narrative and product thesis
2. An ICP, personas, and jobs-to-be-done document
3. A competitive landscape and positioning matrix
4. A pricing and packaging proposal
5. A 12-month roadmap with quarterly milestones
6. A full PRD for the first flagship workflow
7. A clickable prototype or single-page product demo
8. A technical architecture and data model
9. A marketing homepage and two use-case landing pages
10. A sales deck, demo script, and one-pager
11. A ROI calculator for prospects
12. An onboarding flow and customer implementation plan
13. A launch plan and content calendar
14. A KPI tree plus founder dashboard
15. An investor deck and monthly investor-update template

Example Claude prompt

> Build a complete “Semester Operating System” as a set of linked artifacts. Create: (1) company narrative, (2) ICP and personas, (3) competitive positioning matrix, (4) 12-month roadmap, (5) PRD for our first product, (6) clickable web prototype, (7) technical architecture diagram, (8) homepage copy, (9) sales deck outline, (10) ROI calculator, (11) onboarding plan, and (12) KPI dashboard. Use a clean, modern B2B SaaS style. Ask questions only where missing information materially changes the output; otherwise state assumptions clearly.

The practical distinction is: **documents explain Semester, prototypes demonstrate it, code implements it, dashboards measure it, and sales/marketing materials distribute it.**

⁂

Semester project dashboard: build a sortable tracker for every artifact type Claude can generate — by category, format, and use case

Computer
Interactive Claude artifact catalog: compare text docs, React apps, SVG diagrams, and slide decks in a live filterable gallery
Computer
Show me product and engineering artifacts for Semester
Show me brand and go-to-market artifacts for Semester
Show me operations and legal artifacts for Semester

Below is a **build-ready catalog and tracker spec** for a Semester project dashboard. It organizes artifacts by category, output format, and practical use case—and includes the exact grouped views you asked for.

Claude artifacts can cover documents, code, single-page HTML sites, SVGs, diagrams, and interactive React components; newer workflows also support editable document, design, and slide templates with export options such as PowerPoint, PDF, and Word. [11] [12]

Dashboard structure

Build the main dashboard as a **sortable, filterable React artifact** with these columns:

Field	Purpose	Example values
Artifact	What Claude should generate	PRD, architecture diagram, investor deck
Category	Business function	Product, Engineering, Brand, GTM, Operations, Legal
Format	Best generation/output medium	Markdown, React, SVG, Mermaid, HTML, Slides, PDF
Use case	Why Semester needs it	Define build scope, close deals, onboard users
Primary owner	Accountable role	Founder, Product, Design, Engineering, Legal
Stage	Workflow status	Backlog, Draft, Review, Approved, Shipped
Priority	Importance	P0, P1, P2
Dependencies	Needed before it can be finalized	Customer interviews, pricing decision, architecture
Claude prompt	One-click generation brief	“Create a technical PRD for…”
Deliverable	Final artifact destination	Notion, GitHub, Drive, website, PDF

Recommended filters

- **Category:** Product, Engineering, Brand, Go-to-Market, Sales, Customer Success, Operations, Legal, Finance, Leadership
- **Format:** Markdown/document, React app, HTML page, SVG, Mermaid diagram, slide deck, spreadsheet/CSV, PDF, source code

- **Audience:** Internal team, prospective customer, current customer, investor, candidate, partner, regulator
- **Lifecycle stage:** Validate, build, launch, sell, onboard, scale, govern
- **Artifact type:** Strategy, research, specification, interface, code, dashboard, collateral, policy, playbook
- **Status:** Not started, in progress, needs review, approved, obsolete
- **Priority:** P0 through P3

For the live gallery, distinguish formats by what they are best at: Markdown/docs for structured thinking, React for stateful interactive tools, SVG/Mermaid for diagrams and visual explanation, HTML for self-contained web pages, and Slides for presentation narratives. [13] [14] [11]

Product and engineering

Artifact	Category	Best format	Use case
Product vision	Product	Markdown doc	Align the company on the durable product direction
Product strategy	Product	Markdown doc	Explain target market, wedge, differentiation, and sequencing
Customer problem brief	Product	Markdown doc	Define the problem before choosing a solution
ICP and persona library	Product	Markdown doc + table	Identify users, buyers, champions, and blockers
Jobs-to-be-done map	Product	Markdown doc + diagram	Capture the user's functional and emotional goals
User interview guide	Product Research	Markdown doc	Run consistent discovery conversations
Research synthesis	Product Research	Markdown doc	Turn interviews into themes, opportunities, and quotes
User journey map	Product Design	SVG or Mermaid	Visualize the end-to-end current and future workflow
Opportunity-solution tree	Product Strategy	Mermaid diagram	Connect outcomes, opportunities, solutions, and experiments
Product roadmap	Product	React dashboard + CSV	Sort initiatives by quarter, outcome, owner, and confidence
Feature prioritization matrix	Product	React tool	Score initiatives using RICE, ICE, or weighted criteria
PRD	Product	Markdown doc	Specify the user problem, requirements, behavior, metrics, and rollout
User stories	Product	Markdown doc/table	Define buildable slices of functionality
Acceptance criteria	Product	Markdown doc/table	Establish testable requirements for engineering and QA
UX copy inventory	Product Design	Markdown doc/table	Centralize labels, empty states, errors, prompts, and tooltips

Artifact	Category	Best format	Use case
Information architecture	Product Design	Mermaid or SVG	Define pages, hierarchy, navigation, and object relationships
Sitemap	Product Design	Mermaid or SVG	Show the website or app's page structure
User flow	Product Design	Mermaid or SVG	Show screens, decisions, states, and path completion
Low-fidelity wireframes	Product Design	HTML or React	Explore layout and interaction before visual polish
Interactive product prototype	Product Design	React app	Demonstrate core workflows before production implementation
Design-system inventory	Product Design	Markdown doc + React gallery	Define components, variants, tokens, and usage guidance
Component playground	Product Design	React app	Browse buttons, forms, cards, modals, tables, and states
Accessibility checklist	Product Design	Markdown doc	Ensure keyboard, contrast, semantic, and screen-reader considerations
Product analytics plan	Product Analytics	Markdown doc/table	Define what must be measured before implementation
Event taxonomy	Product Analytics	Markdown doc + CSV	Name product events, properties, triggers, and owners
KPI tree	Product Analytics	SVG, Mermaid, or React	Link the north-star metric to leading and lagging indicators
Product dashboard	Product Analytics	React dashboard	Track activation, usage, retention, conversion, and churn
Experiment brief	Product Growth	Markdown doc	State hypothesis, audience, variants, metrics, and decision rules
A/B-test tracker	Product Growth	React dashboard	Track experiment status, results, learnings, and next actions
Release plan	Product	Markdown doc	Sequence feature readiness, communications, support, and rollout
Release notes	Product	Markdown doc	Explain changes to users and internal stakeholders
Changelog	Product	HTML page or Markdown	Publish chronological product improvements
Feature announcement	Product Marketing	HTML or Markdown	Launch a feature via email, blog, in-app, and sales enablement

Engineering artifacts

Artifact	Category	Best format	Use case
Technical design document	Engineering	Markdown doc	Explain a proposed implementation and tradeoffs

Artifact	Category	Best format	Use case
System architecture	Engineering	Mermaid or SVG	Visualize services, clients, APIs, storage, and third parties
Context diagram	Engineering	Mermaid or SVG	Show Semester's system boundaries and external integrations
Data-flow diagram	Engineering	Mermaid or SVG	Trace movement of user, application, and third-party data
Sequence diagram	Engineering	Mermaid	Show how systems interact during a request or workflow
Database schema	Engineering	Mermaid ERD + SQL	Define entities, relationships, constraints, and ownership
Data dictionary	Engineering	Markdown doc/table	Document tables, fields, types, definitions, and retention
API specification	Engineering	OpenAPI YAML + Markdown	Define endpoints, schemas, authentication, errors, and examples
API reference portal	Engineering	HTML or React	Give developers searchable, navigable API documentation
Authentication design	Engineering	Markdown doc + diagram	Define sessions, identity providers, roles, and permissions
Permissions matrix	Engineering	Markdown table	State what every role can view, create, edit, approve, or delete
Security threat model	Security	Markdown doc + diagram	Identify assets, threats, mitigations, and residual risk
Privacy data map	Security/Privacy	Mermaid + table	Identify collection, processing, storage, sharing, and deletion paths
Architecture decision record	Engineering	Markdown doc	Preserve a decision, context, alternatives, and consequences
Infrastructure diagram	Platform	Mermaid or SVG	Map cloud resources, networks, queues, databases, and services
CI/CD pipeline	Platform	Mermaid + YAML	Describe build, test, deploy, rollback, and approval stages
Infrastructure-as-code	Platform	Terraform/Pulumi code	Reproducibly provision required infrastructure
Environment configuration guide	Engineering	Markdown doc	Document development, staging, production, variables, and secrets
Observability plan	Platform	Markdown doc	Define logs, traces, metrics, ownership, retention, and tooling
SLO/SLI dashboard	Platform	React dashboard	Monitor availability, latency, error rate, and performance
Alert runbook	Platform	Markdown doc	Specify triage, escalation, diagnostics, mitigation, and resolution

Artifact	Category	Best format	Use case
Incident response plan	Security/Operations	Markdown doc	Coordinate response, communication, containment, and postmortems
Postmortem template	Engineering	Markdown doc	Record impact, timeline, root cause, actions, and prevention
QA strategy	Quality	Markdown doc	Establish quality gates and ownership across release stages
Test plan	Quality	Markdown doc/table	Specify scenarios, test data, environments, risks, and expected outcomes
Test-case library	Quality	CSV or Markdown table	Track manual, automated, regression, and edge-case coverage
Test-data generator	Quality	Python/TypeScript code	Produce safe and realistic development or QA records
Release checklist	Engineering	Markdown checklist	Confirm quality, security, migration, monitoring, and communication readiness
Developer onboarding guide	Engineering	Markdown doc	Make a new engineer productive quickly
Repository README	Engineering	Markdown doc	Explain setup, architecture, commands, testing, and contribution
Contribution guide	Engineering	Markdown doc	Standardize branches, pull requests, review, and release practices
Technical debt register	Engineering	React dashboard + CSV	Prioritize debt by impact, risk, effort, owner, and target release

Brand and go-to-market

Artifact	Category	Best format	Use case
Brand strategy	Brand	Markdown doc	Define the durable identity and market role of Semester
Positioning statement	Brand	Markdown doc	State who Semester serves, what it does, and why it wins
Messaging architecture	Brand	Markdown doc/table	Align core message, proof points, audience angles, and CTAs
Value proposition canvas	Brand	SVG or Markdown	Connect customer jobs, pains, gains, and Semester’s value
Brand manifesto	Brand	Markdown doc	Articulate the company’s worldview and conviction
Naming exploration	Brand	Markdown table	Compare candidate names for fit, clarity, defensibility, and tone
Tagline library	Brand	Markdown doc	Produce concise external value statements
Voice and tone guide	Brand	Markdown doc	Standardize language by audience, channel, and emotional context

Artifact	Category	Best format	Use case
Visual identity brief	Brand	Markdown doc	Direct logo, typography, color, imagery, and motion design
Moodboard	Brand	HTML, SVG, or design board	Establish a visual reference direction
Logo concepts	Brand	SVG	Generate editable vector logo explorations
Brand guidelines	Brand	HTML site or PDF	Publish rules for logo use, color, typography, imagery, and voice
Icon system	Brand/Product Design	SVG gallery	Create reusable visual symbols for product and marketing
Illustration system	Brand	SVG gallery	Create consistent custom visual language
Homepage	Marketing	HTML or React	Explain what Semester is and convert qualified visitors
Product overview page	Marketing	HTML or React	Present capabilities, outcomes, proof, and CTA
Use-case landing pages	Marketing	HTML or React	Target a specific persona, workflow, or vertical
Persona landing pages	Marketing	HTML or React	Speak separately to buyers, operators, leaders, or developers
Pricing page	Marketing	HTML or React	Clarify packaging, plan differences, proof, and conversion path
Comparison page	Marketing	HTML or React	Help prospects evaluate Semester against alternatives
Customer story	Marketing	Markdown, HTML, or PDF	Show a credible before-and-after result
Case-study template	Marketing	Markdown doc	Standardize problem, implementation, outcomes, and quote capture
Lead magnet	Demand Generation	PDF or HTML	Exchange useful content for qualified lead information
Industry report	Content Marketing	PDF + HTML landing page	Create authority, earned media, and lead capture
SEO strategy	Growth	Markdown doc + CSV	Target themes, keywords, intent, pages, and update cycles
Keyword map	Growth	CSV	Map target queries to pages, personas, funnel stage, and ownership
Editorial calendar	Content Marketing	React dashboard + CSV	Plan content by date, theme, author, stage, and performance
Blog-post outline	Content Marketing	Markdown doc	Establish angle, audience, evidence, structure, and CTA
Blog article	Content Marketing	Markdown/HTML	Publish educational or demand-generation content

Artifact	Category	Best format	Use case
Newsletter	Content Marketing	Markdown/HTML	Keep subscribers informed and engaged
Founder content system	Brand/Growth	Markdown doc + tracker	Create repeatable founder-led posts, themes, and repurposing flows
LinkedIn post library	Social	Markdown doc/table	Generate audience-specific social content variants
Social content calendar	Social	React dashboard + CSV	Plan, approve, schedule, and measure social posts
Search-ad campaign	Demand Generation	Markdown/CSV	Define keywords, ads, landing pages, negative terms, and metrics
Paid-social campaign brief	Demand Generation	Markdown doc	Align audience, creative, offer, landing page, and measurement
Ad copy variants	Demand Generation	Markdown doc	Generate testable headlines, primary text, and CTAs
Email nurture sequence	Lifecycle Marketing	Markdown/HTML	Move leads from interest to education, evaluation, and conversion
Product-launch plan	Product Marketing	Markdown doc + Gantt	Coordinate positioning, audience, channels, assets, timing, and owners
Launch calendar	Product Marketing	React dashboard	Track asset readiness and cross-functional dependencies
Press release	PR	Markdown doc	Announce a meaningful company or product milestone
Media kit	PR	PDF/HTML	Give journalists logos, facts, executive bios, images, and boilerplate
Analyst brief	GTM	Slides or Markdown	Explain category, market traction, product, and differentiation
Webinar deck	Demand Generation	Slides	Teach a topic and create pipeline
Webinar run-of-show	Demand Generation	Markdown doc	Control timing, speakers, transitions, Q&A, and follow-up
Partner pitch deck	Partnerships	Slides	Explain shared value, offer, economics, and activation plan
Partner playbook	Partnerships	Markdown doc	Standardize co-selling, referrals, delivery, and co-marketing
Product Hunt launch kit	Community/Growth	Markdown + assets	Prepare listing, copy, screenshots, comments, and response playbook
Community guidelines	Community	Markdown doc	Set expectations for participation, moderation, and safety

Operations and legal

Artifact	Category	Best format	Use case
Company operating system	Operations	Markdown hub	Centralize principles, cadence, metrics, decisions, and ownership
Annual operating plan	Leadership	Markdown doc + spreadsheet	Set yearly goals, bets, budgets, hiring, and milestones
Quarterly plan	Operations	Markdown doc + dashboard	Translate strategy into owned and measurable execution
OKR framework	Operations	Markdown doc/table	Define objectives, key results, owners, confidence, and cadence
KPI dashboard	Operations	React dashboard	Monitor company, product, revenue, and operating health
Weekly business review	Leadership	Slides or Markdown	Review trends, decisions, risks, and commitments
Leadership meeting agenda	Operations	Markdown doc	Standardize decision-oriented executive meetings
Decision log	Operations	Markdown/React tracker	Preserve decisions, owners, dates, rationale, and reversibility
RACI matrix	Operations	Markdown table	Make responsibility and approvals explicit across initiatives
Project plan	Program Management	React dashboard + CSV	Track scope, milestones, owners, blockers, and dependencies
Project kickoff brief	Program Management	Markdown doc	Align objective, scope, stakeholders, risks, and success metrics
Risk register	Operations	React dashboard + CSV	Track probability, impact, mitigation, owner, and review date
Standard operating procedure	Operations	Markdown doc	Document a repeatable process with steps and controls
Process flowchart	Operations	Mermaid or SVG	Make complex workflows understandable and auditable
Vendor evaluation matrix	Procurement	Spreadsheet/CSV	Compare vendors by cost, capability, risk, integration, and support
Procurement checklist	Procurement	Markdown checklist	Ensure commercial, security, legal, and technical review occurs
Vendor onboarding guide	Operations	Markdown doc	Establish access, ownership, invoice, data, and renewal handling
Employee handbook	People Operations	Markdown/PDF	Document policies, norms, benefits, conduct, and expectations
Employee onboarding plan	People Operations	Markdown checklist	Make a new hire productive with a coordinated first 30–90 days

Artifact	Category	Best format	Use case
Role scorecard	People Operations	Markdown doc	Define mission, outcomes, competencies, and performance expectations
Job description	Recruiting	Markdown doc	Attract candidates with a clear role narrative and requirements
Interview plan	Recruiting	Markdown doc	Define stages, interviewers, competencies, and decision criteria
Interview rubric	Recruiting	Markdown table	Assess candidates consistently and reduce bias
Performance review template	People Operations	Markdown doc	Guide fair, outcome-oriented feedback and development
Compensation philosophy	People Operations	Markdown doc	Explain how salary, equity, levels, and adjustments work
Benefits guide	People Operations	PDF/HTML	Help employees understand available benefits and enrollment
Expense policy	Finance	Markdown doc	Define reimbursable expenses, limits, receipts, approvals, and timing
Budget model	Finance	Spreadsheet/CSV	Plan spend by department, vendor, headcount, and period
Cash-flow forecast	Finance	Spreadsheet/CSV	Anticipate runway, inflows, outflows, and financing needs
P&L model	Finance	Spreadsheet/CSV	Track revenue, cost of goods, operating expenses, and profit
Headcount plan	Finance/People	Spreadsheet/CSV	Align hiring dates, cost, role, manager, and business impact
Financial dashboard	Finance	React dashboard	Track revenue, burn, runway, AR, spend, and forecast variance
Investor update	Leadership	Markdown/email template	Communicate metrics, wins, challenges, asks, and financial status
Board-deck outline	Leadership	Slides	Structure company progress, strategy, risks, decisions, and asks
Data inventory	Privacy	Markdown table	List personal and business data, purpose, owner, location, and retention
Data-retention schedule	Privacy	Markdown table	Define how long categories of data are stored and why
Privacy policy draft	Legal	Markdown/HTML	Explain consumer-facing data practices in clear language
Terms of service draft	Legal	Markdown/HTML	Set user agreement, service scope, limitations, and dispute framework
Acceptable-use policy	Legal	Markdown/HTML	Set prohibited actions and platform-use boundaries

Artifact	Category	Best format	Use case
Cookie policy draft	Legal	Markdown/HTML	Explain cookie categories, purpose, controls, and disclosure
Data processing addendum draft	Legal/Privacy	Markdown doc	Define processor obligations and data-protection terms
Vendor DPA review checklist	Legal/Privacy	Markdown checklist	Evaluate data-processing obligations and risk in vendors' terms
Security overview	Security/GTM	HTML/PDF	Explain technical and organizational security controls to buyers
Security questionnaire library	Security/Sales	Markdown table	Reuse vetted answers to common enterprise security questions
Information-security policy set	Security	Markdown docs	Define access, encryption, incident, device, vendor, and change controls
Incident-response policy	Security	Markdown doc	Formalize readiness, roles, escalation, communication, and reporting
Business continuity plan	Operations/Security	Markdown doc	Prepare continuity and recovery during disruption
Disaster-recovery runbook	Engineering/Security	Markdown doc	Restore systems, data, dependencies, and validated service
Compliance readiness tracker	Legal/Security	React dashboard	Track controls, evidence, owners, audit status, and gaps
Contract review checklist	Legal	Markdown checklist	Flag key commercial, liability, IP, privacy, security, and termination terms
NDA template	Legal	Document	Protect confidential information during discussions
MSA template	Legal	Document	Establish core commercial terms for customer agreements
Order form template	Legal/Sales	Document	Specify product, price, term, quantity, and service details
Statement of work	Legal/Services	Document	Define a scoped implementation or professional-services engagement

Legal, privacy, tax, accounting, employment, and security materials should be treated as structured first drafts and reviewed by the relevant professional before operational use.

Live catalog behavior

The best implementation is a **React artifact** rather than a static document. Claude supports interactive React components as well as single-page HTML sites, diagrams, SVGs, code, and documents; a single artifact page can also offer downloadable outputs such as CSV or PNG. [12] [11]

Core interactions

- Search across artifact name, description, tags, and use case.
- Filter by category, format, audience, lifecycle stage, owner, priority, and status.
- Sort by priority, category, format, estimated effort, impact, owner, or last updated.
- Toggle between **table**, **gallery**, **Kanban**, and **roadmap** views.
- Click any artifact card to open:
  - Description and success criteria
  - Suggested output format
  - Recommended owner
  - Required inputs and dependencies
  - Complete Claude generation prompt
  - Review checklist
  - Export or tracking fields
- Save filtered views, such as “P0 artifacts for launch” or “all legal documents needing counsel review.”
- Export the current filtered table as CSV.
- Provide progress counters:
  - Total artifacts
  - Drafted
  - Approved
  - Shipped
  - Blocked
  - P0 outstanding

Gallery cards

Use four visually distinct artifact families:

Family	Card examples	Best for
Text documents	PRDs, policies, playbooks, briefs, research	Decisions, process, specifications, writing
React applications	Roadmaps, dashboards, calculators, trackers, prototypes	Filterable data, user interaction, tools, product demos
SVG/Mermaid visuals	Architecture maps, workflows, ERDs, journey maps	Shared understanding of complex systems and flows
Slide decks	Investor decks, launch plans, sales presentations, board updates	Narrative communication and live presentations

Build prompt

Paste this into Claude to create the catalog as an interactive artifact:

```
Build a responsive React artifact named “Semester Artifact Catalog.”

Purpose:
Create a searchable, sortable, filterable operating dashboard for every deliverable (

Data model:
Each artifact record must include:
- id
- artifactName
- category
- subcategory
- format
- useCase
- description
- audience
- lifecycleStage
- primaryOwner
- priority: P0, P1, P2, P3
- status: Backlog, Draft, Review, Approved, Shipped, Blocked
- effort: XS, S, M, L, XL
- impact: Low, Medium, High
- dependencies
- suggestedPrompt
- reviewNotes
- tags

Preload the catalog with these categories:
1. Product
2. Engineering
3. Product Design
4. Product Analytics
5. Brand
6. Marketing
7. Go-to-Market
8. Sales
9. Customer Success
10. Operations
11. Finance
12. People
13. Legal
14. Privacy
15. Security
16. Leadership

Include at least 100 high-quality preloaded artifact records. Ensure the catalog con
- Product vision, strategy, PRDs, user stories, acceptance criteria, research plan, 
- Technical design documents, architecture diagrams, data-flow diagrams, sequence dia
- Brand strategy, positioning, messaging, voice and tone, visual identity, website co
- Operating plans, OKRs, KPI dashboards, SOPs, RACI matrices, risk registers, hiring

Views:
- Default sortable table
```

```
- Card gallery
- Kanban board grouped by status
- Roadmap grouped by lifecycle stage
- Four preset galleries: Text Documents, React Apps, SVG/Mermaid Diagrams, and Slide

Filters:
- Category
- Format
- Use case
- Audience
- Lifecycle stage
- Owner
- Priority
- Status
- Effort
- Impact
- Tags

Interactions:
- Full-text search
- Multi-select filters
- Sort by any column
- Click artifact to open a detail drawer
- Detail drawer includes description, suggested prompt, requirements, dependencies, a
- Bookmark/star artifacts
- Mark status from the table or drawer
- Display completion metrics and category progress bars
- Export the currently filtered results to CSV
- Include a reset filters button
- Use local state only; do not require a backend

Design:
- Clean, premium B2B software aesthetic
- Dense but readable founder dashboard
- Responsive desktop-first layout
- Neutral background, clear hierarchy, high-contrast table text
- Use badges for category, format, priority, and status
- Use accessible colors and keyboard-friendly controls
- Include a compact dark-mode toggle

Place the following saved view buttons at the top:
- All Artifacts
- Product + Engineering
- Brand + GTM
- Operations + Legal
- PO Launch Essentials
- React Apps
- Diagrams
- Documents
- Slide Decks
```

A React implementation is the right format because it supports client-side state, event handling, interactive filtering, component composition, and dashboard-like UI behavior; SVG and Mermaid remain ideal for architecture, flows, ERDs, and journey maps, while slide and document formats fit narrative deliverables. [14] [13]

Build a sortable Claude artifact catalog: compare HTML, React, SVG, Mermaid, Docs, and Slides by category, format, and use case

Computer
Turn Claude artifact formats into an interactive decision tree: pick your use case and get the exact format, export type, and limitations
Computer
Show me the React code for this tracker dashboard
Add sample rows for Semester project artifacts
How do I structure the data model for the filters

Below is a self-contained React tracker you can paste into a Claude React artifact. It includes:

- A sortable, searchable tracker of Semester artifacts
- Filters for category, format, use case, priority, status, and audience
- Sample rows across Product, Engineering, Brand, GTM, Operations, and Legal
- A live “format decision tree” that recommends HTML, React, SVG, Mermaid, Docs, or Slides
- A detail panel with an artifact-generation prompt
- CSV export for the filtered view

For Claude’s artifact environment, React is appropriate for a stateful dashboard with search, filters, sort controls, expandable detail panels, and in-memory editing. Claude artifacts are typically self-contained HTML pages or interactive React components; the React sandbox has dependency and persistence constraints, so this example uses only React state and no external packages.[21] [22] [23]

React tracker code

```
import React, { useMemo, useState } from "react";

const FORMAT_META = {
  HTML: {
    icon: "◫",
    color: "bg-amber-100 text-amber-800 border-amber-200",
    bestFor: "Standalone pages, lightweight tools, calculators, dashboards, and inter
    export: "Interactive .html file; PDF or screenshot for a static snapshot.",
    limits: "Best as one self-contained page. Complex persistence, private APIs, and
  },
  React: {
    icon: "⚛",
    color: "bg-sky-100 text-sky-800 border-sky-200",
    bestFor: "Stateful trackers, filtering interfaces, product prototypes, dashboards
    export: "JSX source; use Claude preview or place into a Vite/Next.js project to c
    limits: "Artifact sandboxes commonly restrict arbitrary packages, required props
```

```
  },
  SVG: {
    icon: "◇",
    color: "bg-violet-100 text-violet-800 border-violet-200",
    bestFor: "Logos, icons, illustrations, static architecture diagrams, and polishe
    export: ".svg, PNG, or PDF.",
    limits: "Not intended for rich application state, large interactive datasets, or
  },
  Mermaid: {
    icon: "↳",
    color: "bg-emerald-100 text-emerald-800 border-emerald-200",
    bestFor: "Flowcharts, sequence diagrams, ERDs, Gantt charts, roadmaps, and syste
    export: "Mermaid source, SVG, PNG, or PDF after rendering.",
    limits: "Layout styling is constrained; use SVG when precise visual art directio
  },
  Docs: {
    icon: "▤",
    color: "bg-slate-100 text-slate-800 border-slate-200",
    bestFor: "PRDs, briefs, policies, research synthesis, playbooks, specifications,
    export: "Markdown, Word, PDF, or a shared document.",
    limits: "Not suited to deep interaction, live filtering, or rich visual simulati
  },
  Slides: {
    icon: "▣",
    color: "bg-rose-100 text-rose-800 border-rose-200",
    bestFor: "Investor narratives, board reporting, sales presentations, launch brie
    export: "PowerPoint, PDF, or presentation-ready HTML.",
    limits: "Slides communicate a linear narrative; they are not the best system of r
  },
};

const INITIAL_ARTIFACTS = [
  {
    id: 1,
    artifact: "Product Vision and Strategy",
    category: "Product",
    format: "Docs",
    useCase: "Align the team on target customer, differentiated value, strategic bets
    audience: "Internal",
    lifecycle: "Validate",
    owner: "Founder / Product",
    priority: "P0",
    status: "Draft",
    effort: "M",
    impact: "High",
    tags: ["strategy", "alignment", "roadmap"],
    prompt:
      "Create a Semester product vision and strategy document. Cover ICP, customer pr
  },
  {
    id: 2,
    artifact: "Flagship Workflow PRD",
    category: "Product",
    format: "Docs",
    useCase: "Turn the primary user problem into an implementation-ready specificati
    audience: "Internal",
```

```
    lifecycle: "Build",
    owner: "Product",
    priority: "P0",
    status: "Backlog",
    effort: "L",
    impact: "High",
    tags: ["prd", "requirements", "feature"],
    prompt:
      "Write an implementation-ready PRD for Semester's flagship workflow. Include pr
  },
  {
    id: 3,
    artifact: "Semester Roadmap Dashboard",
    category: "Product",
    format: "React",
    useCase: "Prioritize and communicate initiatives by quarter, owner, outcome, con
    audience: "Internal",
    lifecycle: "Build",
    owner: "Founder / Product",
    priority: "P0",
    status: "In Review",
    effort: "L",
    impact: "High",
    tags: ["roadmap", "dashboard", "prioritization"],
    prompt:
      "Build a React roadmap dashboard for Semester. Include sortable initiatives, fi
  },
  {
    id: 4,
    artifact: "User Journey Map",
    category: "Product Design",
    format: "Mermaid",
    useCase: "Map a user's end-to-end workflow, friction points, decisions, emotions
    audience: "Internal",
    lifecycle: "Validate",
    owner: "Product Design",
    priority: "P1",
    status: "Backlog",
    effort: "M",
    impact: "High",
    tags: ["research", "journey", "ux"],
    prompt:
      "Create a Mermaid user journey map for Semester's primary user, from initial pr
  },
  {
    id: 5,
    artifact: "Interactive Product Prototype",
    category: "Product Design",
    format: "React",
    useCase: "Demonstrate the core Semester user journey before production engineerin
    audience: "Internal",
    lifecycle: "Build",
    owner: "Product Design",
    priority: "P0",
    status: "Backlog",
    effort: "XL",
```

```
    impact: "High",
    tags: ["prototype", "ux", "demo"],
    prompt:
      "Build a clickable React prototype for Semester's core workflow. Include onboar
  },
  {
    id: 6,
    artifact: "System Architecture",
    category: "Engineering",
    format: "Mermaid",
    useCase: "Align engineering on components, services, integrations, data stores, a
    audience: "Internal",
    lifecycle: "Build",
    owner: "Engineering",
    priority: "P0",
    status: "Draft",
    effort: "M",
    impact: "High",
    tags: ["architecture", "systems", "platform"],
    prompt:
      "Create a Mermaid system architecture diagram for Semester. Include web client
  },
  {
    id: 7,
    artifact: "Database Schema and ERD",
    category: "Engineering",
    format: "Mermaid",
    useCase: "Define core entities, relationships, tenancy boundaries, and lifecycle
    audience: "Internal",
    lifecycle: "Build",
    owner: "Engineering",
    priority: "P0",
    status: "Backlog",
    effort: "M",
    impact: "High",
    tags: ["database", "erd", "data-model"],
    prompt:
      "Create a Mermaid ERD for Semester. Include users, organizations, memberships,
  },
  {
    id: 8,
    artifact: "API Specification",
    category: "Engineering",
    format: "Docs",
    useCase: "Define endpoint contracts, request and response schemas, authorization
    audience: "Internal",
    lifecycle: "Build",
    owner: "Engineering",
    priority: "P1",
    status: "Backlog",
    effort: "L",
    impact: "High",
    tags: ["api", "openapi", "developer-experience"],
    prompt:
      "Draft an OpenAPI-style API specification for Semester's core workflow. Include
  },
```

```
  {
    id: 9,
    artifact: "Launch Readiness Checklist",
    category: "Engineering",
    format: "Docs",
    useCase: "Prevent missed quality, performance, security, support, and rollback re
    audience: "Internal",
    lifecycle: "Launch",
    owner: "Engineering",
    priority: "P0",
    status: "Backlog",
    effort: "S",
    impact: "High",
    tags: ["release", "quality", "launch"],
    prompt:
      "Create a launch readiness checklist for Semester. Cover testing, accessibility
  },
  {
    id: 10,
    artifact: "Homepage and Product Narrative",
    category: "Brand",
    format: "HTML",
    useCase: "Explain what Semester does, who it is for, why it matters, and what act
    audience: "Prospects",
    lifecycle: "Launch",
    owner: "Brand / Marketing",
    priority: "P0",
    status: "Draft",
    effort: "L",
    impact: "High",
    tags: ["website", "positioning", "conversion"],
    prompt:
      "Create a polished, responsive HTML homepage for Semester. Include a clear hero
  },
  {
    id: 11,
    artifact: "Messaging Architecture",
    category: "Brand",
    format: "Docs",
    useCase: "Keep positioning, claims, proof points, objection handling, and calls t
    audience: "Internal",
    lifecycle: "Validate",
    owner: "Brand / Marketing",
    priority: "P0",
    status: "Draft",
    effort: "M",
    impact: "High",
    tags: ["positioning", "messaging", "brand"],
    prompt:
      "Create a messaging architecture for Semester. Include category, positioning st
  },
  {
    id: 12,
    artifact: "Brand Mark Exploration",
    category: "Brand",
    format: "SVG",
```

```
    useCase: "Explore editable icon and logo directions that can be reviewed in produ
    audience: "Internal",
    lifecycle: "Launch",
    owner: "Brand / Design",
    priority: "P2",
    status: "Backlog",
    effort: "M",
    impact: "Medium",
    tags: ["logo", "identity", "visual"],
    prompt:
      "Create six distinct SVG logo-mark concepts for Semester. Keep them simple, geo
  },
  {
    id: 13,
    artifact: "Launch Plan",
    category: "Go-to-Market",
    format: "Slides",
    useCase: "Coordinate product, marketing, sales, customer success, and leadership
    audience: "Internal",
    lifecycle: "Launch",
    owner: "Product Marketing",
    priority: "P0",
    status: "Backlog",
    effort: "L",
    impact: "High",
    tags: ["launch", "gtm", "coordination"],
    prompt:
      "Create a Semester product launch deck. Cover target audience, positioning, goa
  },
  {
    id: 14,
    artifact: "ROI Calculator",
    category: "Sales",
    format: "React",
    useCase: "Help prospective customers estimate the operational and financial impac
    audience: "Prospects",
    lifecycle: "Sell",
    owner: "Sales / Marketing",
    priority: "P1",
    status: "Backlog",
    effort: "M",
    impact: "High",
    tags: ["sales", "calculator", "roi"],
    prompt:
      "Build a React ROI calculator for Semester. Let prospects enter team size, curr
  },
  {
    id: 15,
    artifact: "Operating Plan and OKRs",
    category: "Operations",
    format: "Docs",
    useCase: "Translate strategy into measurable goals, owners, milestones, and weekl
    audience: "Internal",
    lifecycle: "Scale",
    owner: "Founder / Operations",
    priority: "P0",
```

```
    status: "Draft",
    effort: "L",
    impact: "High",
    tags: ["okr", "planning", "operations"],
    prompt:
      "Create a Semester annual operating plan and quarterly OKR framework. Include s
  },
  {
    id: 16,
    artifact: "Company KPI Dashboard",
    category: "Operations",
    format: "React",
    useCase: "Give leadership a live view of product, revenue, growth, customer, and
    audience: "Internal",
    lifecycle: "Scale",
    owner: "Founder / Operations",
    priority: "P1",
    status: "Backlog",
    effort: "L",
    impact: "High",
    tags: ["kpi", "metrics", "dashboard"],
    prompt:
      "Build a React company KPI dashboard for Semester. Include MRR, pipeline, conve
  },
  {
    id: 17,
    artifact: "Privacy Data Map",
    category: "Legal",
    format: "Mermaid",
    useCase: "Document what data Semester collects, where it flows, who processes it
    audience: "Internal",
    lifecycle: "Govern",
    owner: "Legal / Security",
    priority: "PO",
    status: "Backlog",
    effort: "M",
    impact: "High",
    tags: ["privacy", "data", "compliance"],
    prompt:
      "Create a Mermaid privacy data-flow diagram for Semester. Show collection poin
  },
  {
    id: 18,
    artifact: "Terms of Service First Draft",
    category: "Legal",
    format: "Docs",
    useCase: "Create a structured initial customer agreement for legal review.",
    audience: "Customers",
    lifecycle: "Govern",
    owner: "Legal",
    priority: "P1",
    status: "Backlog",
    effort: "L",
    impact: "High",
    tags: ["terms", "contracts", "legal"],
    prompt:
```

```
      "Draft a plain-English first version of Terms of Service for Semester, with sec
  },
  {
    id: 19,
    artifact: "Security Overview",
    category: "Security",
    format: "HTML",
    useCase: "Give prospects a clear, buyer-friendly explanation of Semester security
    audience: "Prospects",
    lifecycle: "Sell",
    owner: "Security / Sales",
    priority: "P1",
    status: "Backlog",
    effort: "M",
    impact: "High",
    tags: ["security", "trust", "enterprise"],
    prompt:
      "Create a clean HTML security overview page for Semester. Cover access control
  },
];

const OPTIONS = {
  categories: [
    "Product",
    "Product Design",
    "Engineering",
    "Brand",
    "Go-to-Market",
    "Sales",
    "Operations",
    "Legal",
    "Security",
  ],
  formats: ["HTML", "React", "SVG", "Mermaid", "Docs", "Slides"],
  useCases: ["Strategy", "Validate", "Build", "Launch", "Sell", "Scale", "Govern"],
  audiences: ["Internal", "Prospects", "Customers", "Investors", "Partners"],
  priorities: ["P0", "P1", "P2", "P3"],
  statuses: ["Backlog", "Draft", "In Review", "Approved", "Shipped", "Blocked"],
};

const DEFAULT_FILTERS = {
  category: "All",
  format: "All",
  lifecycle: "All",
  audience: "All",
  priority: "All",
  status: "All",
};

function Badge({ children, className = "" }) {
  return (
    <span className={`inline-flex items-center rounded-full border px-2 py-0.5 text-x
      {children}
    </span>
  );
}
```

```
function SelectFilter({ label, value, onChange, options }) {
  return (
    <label className="flex min-w-[145px] flex-col gap-1 text-xs font-semibold text-sl
      {label}
      <select
        value={value}
        onChange={(event) => onChange(event.target.value)}
        className="rounded-lg border border-slate-200 bg-white px-3 py-2 text-sm font
      >
        <option value="All">All</option>
        {options.map((option) => (
          <option key={option} value={option}>
            {option}
          </option>
        ))}
      </select>
    </label>
  );
}

function FormatDecisionTree() {
  const [choice, setChoice] = useState("interactive");
  const [recommendation, setRecommendation] = useState(null);

  const chooseFormat = () => {
    const decisionMap = {
      interactive: "React",
      standalone: "HTML",
      diagram: "Mermaid",
      visual: "SVG",
      narrative: "Docs",
      presentation: "Slides",
    };
    setRecommendation(decisionMap[choice]);
  };

  const choices = [
    ["interactive", "I need a tracker, dashboard, calculator, prototype, or filterabl
    ["standalone", "I need a single self-contained web page or lightweight interactiv
    ["diagram", "I need a workflow, system map, ERD, sequence, or timeline diagram."]
    ["visual", "I need a logo, icon, illustration, or precisely art-directed static d
    ["narrative", "I need a brief, policy, PRD, plan, report, or structured written r
    ["presentation", "I need a persuasive, linear story for a meeting, pitch, or trai
  ];

  const meta = recommendation ? FORMAT_META[recommendation] : null;

  return (
    <section className="rounded-2xl border border-indigo-100 bg-indigo-50 p-5">
      <div className="mb-4">
        <h2 className="text-lg font-bold text-slate-900">Claude format decision tree<
        <p className="mt-1 text-sm text-slate-600">
          Pick the primary job of the artifact. The recommendation favors the best na
        </p>
      </div>
```

```
      <div className="grid gap-2 md:grid-cols-2">
        {choices.map(([value, label]) => (
          <button
            key={value}
            onClick={() => setChoice(value)}
            className={`rounded-xl border p-3 text-left text-sm transition ${
              choice === value
                ? "border-indigo-500 bg-white shadow-sm"
                : "border-indigo-100 bg-indigo-50 hover:border-indigo-300"
            }`}
          >
            <span className="mr-2 inline-block h-3 w-3 rounded-full border border-ind
              {choice === value && <span className="block h-2 w-2 rounded-full bg-ind
            </span>
            {label}
          </button>
        ))}
      </div>

      <button
        onClick={chooseFormat}
        className="mt-4 rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold tex
      >
        Recommend a format
      </button>

      {meta && (
        <div className="mt-4 rounded-xl border border-white bg-white p-4">
          <div className="flex flex-wrap items-center gap-2">
            <span className="text-2xl">{meta.icon}</span>
            <h3 className="text-base font-bold text-slate-900">{recommendation}</h3>
            <Badge className={meta.color}>Recommended</Badge>
          </div>
          <div className="mt-3 grid gap-3 text-sm md:grid-cols-3">
            <div>
              <p className="font-semibold text-slate-700">Best for</p>
              <p className="mt-1 text-slate-600">{meta.bestFor}</p>
            </div>
            <div>
              <p className="font-semibold text-slate-700">Export</p>
              <p className="mt-1 text-slate-600">{meta.export}</p>
            </div>
            <div>
              <p className="font-semibold text-slate-700">Limitations</p>
              <p className="mt-1 text-slate-600">{meta.limits}</p>
            </div>
          </div>
        </div>
      )}
    </section>
  );
}

export default function SemesterArtifactCatalog() {
  const [artifacts, setArtifacts] = useState(INITIAL_ARTIFACTS);
```

```
  const [filters, setFilters] = useState(DEFAULT_FILTERS);
  const [search, setSearch] = useState("");
  const [sort, setSort] = useState({ key: "priority", direction: "asc" });
  const [selectedArtifact, setSelectedArtifact] = useState(null);
  const [view, setView] = useState("table");

  const filteredArtifacts = useMemo(() => {
    const query = search.trim().toLowerCase();

    const priorityOrder = { P0: 0, P1: 1, P2: 2, P3: 3 };
    const statusOrder = { Backlog: 0, Draft: 1, "In Review": 2, Approved: 3, Shipped

    return artifacts
      .filter((item) => {
        const textMatch =
          !query ||
          [
            item.artifact,
            item.category,
            item.format,
            item.useCase,
            item.audience,
            item.owner,
            item.lifecycle,
            ...item.tags,
          ]
            .join(" ")
            .toLowerCase()
            .includes(query);

        return (
          textMatch &&
          (filters.category === "All" || item.category === filters.category) &&
          (filters.format === "All" || item.format === filters.format) &&
          (filters.lifecycle === "All" || item.lifecycle === filters.lifecycle) &&
          (filters.audience === "All" || item.audience === filters.audience) &&
          (filters.priority === "All" || item.priority === filters.priority) &&
          (filters.status === "All" || item.status === filters.status)
        );
      })
      .sort((a, b) => {
        const aValue = a[sort.key];
        const bValue = b[sort.key];

        if (sort.key === "priority") {
          return sort.direction === "asc"
            ? priorityOrder[aValue] - priorityOrder[bValue]
            : priorityOrder[bValue] - priorityOrder[aValue];
        }

        if (sort.key === "status") {
          return sort.direction === "asc"
            ? statusOrder[aValue] - statusOrder[bValue]
            : statusOrder[bValue] - statusOrder[aValue];
        }
```

```
        return sort.direction === "asc"
          ? String(aValue).localeCompare(String(bValue))
          : String(bValue).localeCompare(String(aValue));
      });
  }, [artifacts, filters, search, sort]);

  const metrics = useMemo(() => {
    const total = artifacts.length;
    const shipped = artifacts.filter((item) => item.status === "Shipped").length;
    const pOOpen = artifacts.filter((item) => item.priority === "PO" && item.status
    const blocked = artifacts.filter((item) => item.status === "Blocked").length;

    return { total, shipped, pOOpen, blocked };
  }, [artifacts]);

  const updateFilter = (key, value) => {
    setFilters((current) => ({ ...current, [key]: value }));
  };

  const updateSort = (key) => {
    setSort((current) => ({
      key,
      direction: current.key === key && current.direction === "asc" ? "desc" : "asc"
    }));
  };

  const updateStatus = (id, status) => {
    setArtifacts((current) =>
      current.map((item) => (item.id === id ? { ...item, status } : item))
    );
    setSelectedArtifact((current) => (current ? { ...current, status } : current));
  };

  const applySavedView = (name) => {
    setSearch("");
    if (name === "All") setFilters(DEFAULT_FILTERS);
    if (name === "Product + Engineering") {
      setFilters({ ...DEFAULT_FILTERS, category: "Product" });
      setSearch("product engineering");
    }
    if (name === "React Apps") setFilters({ ...DEFAULT_FILTERS, format: "React" });
    if (name === "Diagrams") setFilters({ ...DEFAULT_FILTERS, format: "Mermaid" });
    if (name === "Documents") setFilters({ ...DEFAULT_FILTERS, format: "Docs" });
    if (name === "Slides") setFilters({ ...DEFAULT_FILTERS, format: "Slides" });
    if (name === "PO Launch") setFilters({ ...DEFAULT_FILTERS, priority: "PO" });
  };

  const exportCsv = () => {
    const columns = [
      "artifact",
      "category",
      "format",
      "useCase",
      "audience",
      "lifecycle",
      "owner",
```

```
      "priority",
      "status",
      "effort",
      "impact",
      "tags",
    ];

    const escape = (value) => `"${String(value ?? "").replace(/"/g, '""')}"`;
    const csv = [
      columns.join(","),
      ...filteredArtifacts.map((row) =>
        columns.map((column) => escape(Array.isArray(row[column]) ? row[column].join
      ),
    ].join("\n");

    const blob = new Blob([csv], { type: "text/csv;charset=utf-8;" });
    const url = URL.createObjectURL(blob);
    const link = document.createElement("a");
    link.href = url;
    link.download = "semester-artifact-catalog.csv";
    link.click();
    URL.revokeObjectURL(url);
  };

  const headerCell = (label, key) => (
    <button
      onClick={() => updateSort(key)}
      className="flex items-center gap-1 text-left text-xs font-bold uppercase track
    >
      {label}
      {sort.key === key ? (sort.direction === "asc" ? "↑" : "↓") : "↕"}
    </button>
  );

  return (
    <main className="min-h-screen bg-slate-50 p-4 text-slate-900 md:p-8">
      <div className="mx-auto max-w-7xl space-y-6">
        <header className="rounded-2xl bg-gradient-to-br from-slate-950 via-slate-900
          <div className="flex flex-col justify-between gap-5 lg:flex-row lg:items-en
            <div>
              <p className="text-xs font-bold uppercase tracking-[0.18em] text-indigo
              <h1 className="mt-2 text-3xl font-bold tracking-tight">Claude Artifact
              <p className="mt-2 max-w-2xl text-sm text-slate-300">
                A sortable production tracker for what Claude can generate across pro
              </p>
            </div>
            <button
              onClick={exportCsv}
              className="rounded-lg bg-white px-4 py-2 text-sm font-bold text-slate-9
            >
              Export filtered CSV
            </button>
          </div>

          <div className="mt-6 grid grid-cols-2 gap-3 md:grid-cols-4">
            {[
```

```
            ["Catalog items", metrics.total],
            ["Shipped", metrics.shipped],
            ["Open PO work", metrics.pOOpen],
            ["Blocked", metrics.blocked],
          ].map(([label, value]) => (
            <div key={label} className="rounded-xl border border-white/10 bg-white/
              <p className="text-2xl font-bold">{value}</p>
              <p className="mt-1 text-xs text-slate-300">{label}</p>
            </div>
          ))}
        </div>
      </header>

      <FormatDecisionTree />

      <section className="rounded-2xl border border-slate-200 bg-white p-4 shadow-s
        <div className="flex flex-wrap gap-2">
          {["All", "Product + Engineering", "React Apps", "Diagrams", "Documents",
            <button
              key={name}
              onClick={() => applySavedView(name)}
              className="rounded-full border border-slate-200 bg-slate-50 px-3 py-1
            >
              {name}
            </button>
          ))}
        </div>

        <div className="mt-4 flex flex-col gap-3">
          <input
            value={search}
            onChange={(event) => setSearch(event.target.value)}
            placeholder="Search artifacts, tags, formats, owners, or use cases..."
            className="w-full rounded-xl border border-slate-200 px-4 py-3 text-sm
          />

          <div className="flex flex-wrap gap-3">
            <SelectFilter
              label="Category"
              value={filters.category}
              onChange={(value) => updateFilter("category", value)}
              options={OPTIONS.categories}
            />
            <SelectFilter
              label="Format"
              value={filters.format}
              onChange={(value) => updateFilter("format", value)}
              options={OPTIONS.formats}
            />
            <SelectFilter
              label="Lifecycle"
              value={filters.lifecycle}
              onChange={(value) => updateFilter("lifecycle", value)}
              options={OPTIONS.useCases}
            />
            <SelectFilter
```

```
              label="Audience"
              value={filters.audience}
              onChange={(value) => updateFilter("audience", value)}
              options={OPTIONS.audiences}
            />
            <SelectFilter
              label="Priority"
              value={filters.priority}
              onChange={(value) => updateFilter("priority", value)}
              options={OPTIONS.priorities}
            />
            <SelectFilter
              label="Status"
              value={filters.status}
              onChange={(value) => updateFilter("status", value)}
              options={OPTIONS.statuses}
            />
            <button
              onClick={() => {
                setFilters(DEFAULT_FILTERS);
                setSearch("");
              }}
              className="self-end rounded-lg px-3 py-2 text-sm font-semibold text-s
            >
              Reset
            </button>
          </div>
        </div>

        <div className="mt-5 flex items-center justify-between border-t border-sla
          <p className="text-sm text-slate-600">
            Showing <span className="font-bold text-slate-900">{filteredArtifacts.l
          </p>
          <div className="flex rounded-lg border border-slate-200 p-1">
            {["table", "gallery"].map((option) => (
              <button
                key={option}
                onClick={() => setView(option)}
                className={`rounded-md px-3 py-1.5 text-xs font-bold capitalize ${
                  view === option ? "bg-slate-900 text-white" : "text-slate-500 hov
                }`}
              >
                {option}
              </button>
            ))}
          </div>
        </div>
      </section>

      {view === "table" ? (
        <section className="overflow-hidden rounded-2xl border border-slate-200 bg-
          <div className="overflow-x-auto">
            <table className="w-full min-w-[1100px] border-collapse">
              <thead className="border-b border-slate-200 bg-slate-50">
                <tr>
                  <th className="px-4 py-3">{headerCell("Artifact", "artifact")}</t
```

```
                <th className="px-4 py-3">{headerCell("Category", "category")}</
                <th className="px-4 py-3">{headerCell("Format", "format")}</th>
                <th className="px-4 py-3">{headerCell("Use case", "lifecycle")}<
                <th className="px-4 py-3">{headerCell("Priority", "priority")}</
                <th className="px-4 py-3">{headerCell("Status", "status")}</th>
                <th className="px-4 py-3">{headerCell("Owner", "owner")}</th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase tr
              </tr>
            </thead>
            <tbody>
              {filteredArtifacts.map((item) => (
                <tr key={item.id} className="border-b border-slate-100 transitio
                  <td className="px-4 py-3">
                    <button
                      onClick={() => setSelectedArtifact(item)}
                      className="text-left text-sm font-bold text-slate-900 hove
                    >
                      {item.artifact}
                    </button>
                    <div className="mt-1 flex flex-wrap gap-1">
                      {item.tags.slice(0, 3).map((tag) => (
                        <span key={tag} className="text-xs text-slate-400">#{tag
                      ))}
                    </div>
                  </td>
                  <td className="px-4 py-3 text-sm text-slate-600">{item.categor
                  <td className="px-4 py-3">
                    <Badge className={FORMAT_META[item.format].color}>
                      {FORMAT_META[item.format].icon} {item.format}
                    </Badge>
                  </td>
                  <td className="px-4 py-3 text-sm text-slate-600">{item.lifecyc
                  <td className="px-4 py-3">
                    <Badge className={
                      item.priority === "P0"
                        ? "border-red-200 bg-red-50 text-red-700"
                        : item.priority === "P1"
                        ? "border-amber-200 bg-amber-50 text-amber-700"
                        : "border-slate-200 bg-slate-50 text-slate-600"
                    }>
                      {item.priority}
                    </Badge>
                  </td>
                  <td className="px-4 py-3">
                    <select
                      value={item.status}
                      onChange={(event) => updateStatus(item.id, event.target.va
                      className="rounded-lg border border-slate-200 bg-white px-2
                    >
                      {OPTIONS.statuses.map((status) => (
                        <option key={status}>{status}</option>
                      ))}
                    </select>
                  </td>
                  <td className="px-4 py-3 text-sm text-slate-600">{item.owner}</
                  <td className="px-4 py-3">
```

```
                    <button
                      onClick={() => setSelectedArtifact(item)}
                      className="rounded-lg border border-slate-200 px-2 py-1 tex
                    >
                      Open
                    </button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </section>
    ) : (
      <section className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
        {filteredArtifacts.map((item) => (
          <button
            key={item.id}
            onClick={() => setSelectedArtifact(item)}
            className="rounded-2xl border border-slate-200 bg-white p-5 text-lef
          >
            <div className="flex items-start justify-between gap-4">
              <Badge className={FORMAT_META[item.format].color}>
                {FORMAT_META[item.format].icon} {item.format}
              </Badge>
              <Badge className={item.priority === "P0" ? "border-red-200 bg-red-5
                {item.priority}
              </Badge>
            </div>
            <h3 className="mt-4 text-base font-bold text-slate-900">{item.artifac
            <p className="mt-2 text-sm leading-6 text-slate-600">{item.useCase}</
            <div className="mt-4 flex items-center justify-between border-t borde
              <span>{item.category}</span>
              <span>{item.status}</span>
            </div>
          </button>
        ))}
      </section>
    )}

    {selectedArtifact && (
      <div className="fixed inset-0 z-50 flex justify-end bg-slate-950/30">
        <button
          aria-label="Close details"
          className="absolute inset-0 cursor-default"
          onClick={() => setSelectedArtifact(null)}
        />
        <aside className="relative h-full w-full max-w-xl overflow-y-auto bg-whi
          <div className="flex items-start justify-between gap-4">
            <div>
              <div className="flex flex-wrap gap-2">
                <Badge className={FORMAT_META[selectedArtifact.format].color}>
                  {FORMAT_META[selectedArtifact.format].icon} {selectedArtifact.f
                </Badge>
                <Badge className="border-slate-200 bg-slate-50 text-slate-600">{s
                <Badge className="border-slate-200 bg-slate-50 text-slate-600">{s
```

```
                  </div>
                  <h2 className="mt-3 text-2xl font-bold text-slate-900">{selectedAr
                </div>
                <button
                  onClick={() => setSelectedArtifact(null)}
                  className="rounded-lg border border-slate-200 px-3 py-1.5 text-sm
                >
                  Close
                </button>
              </div>

              <div className="mt-6 space-y-6">
                <div>
                  <h3 className="text-sm font-bold text-slate-900">Use case</h3>
                  <p className="mt-2 text-sm leading-6 text-slate-600">{selectedArti
                </div>

                <div className="grid gap-3 sm:grid-cols-2">
                  {[
                    ["Audience", selectedArtifact.audience],
                    ["Lifecycle", selectedArtifact.lifecycle],
                    ["Owner", selectedArtifact.owner],
                    ["Effort", selectedArtifact.effort],
                    ["Impact", selectedArtifact.impact],
                    ["Current status", selectedArtifact.status],
                  ].map(([label, value]) => (
                    <div key={label} className="rounded-xl bg-slate-50 p-3">
                      <p className="text-xs font-bold uppercase tracking-wide text-sl
                      <p className="mt-1 text-sm font-semibold text-slate-800">{value
                    </div>
                  ))}
                </div>

                <div className="rounded-xl border border-indigo-100 bg-indigo-50 p-4"
                  <h3 className="text-sm font-bold text-indigo-950">Recommended forma
                  <p className="mt-2 text-sm text-indigo-900">{FORMAT_META[selectedAr
                  <p className="mt-2 text-xs text-indigo-700"><strong>Export:</strong
                  <p className="mt-1 text-xs text-indigo-700"><strong>Constraint:</st
                </div>

                <div>
                  <h3 className="text-sm font-bold text-slate-900">Suggested Claude p
                  <pre className="mt-2 whitespace-pre-wrap rounded-xl bg-slate-950 p-
                    {selectedArtifact.prompt}
                  </pre>
                </div>

                <div>
                  <h3 className="text-sm font-bold text-slate-900">Tags</h3>
                  <div className="mt-2 flex flex-wrap gap-2">
                    {selectedArtifact.tags.map((tag) => (
                      <Badge key={tag} className="border-slate-200 bg-slate-50 text-s
                    ))}
                  </div>
                </div>
              </div>
```

```
            </aside>
          </div>
        )}
      </div>
    </main>
  );
}
```

Data model for filters

Use a **single canonical artifact record**. Avoid a separate object for each view—table, gallery, Kanban, roadmap, filtered lists, exports, and dashboard metrics should all be derived from the same array.

```
type ArtifactFormat =
  | "HTML"
  | "React"
  | "SVG"
  | "Mermaid"
  | "Docs"
  | "Slides";

type ArtifactStatus =
  | "Backlog"
  | "Draft"
  | "In Review"
  | "Approved"
  | "Shipped"
  | "Blocked";

type ArtifactPriority = "P0" | "P1" | "P2" | "P3";

type ArtifactRecord = {
  id: string | number;
  artifact: string;
  category: string;
  subcategory?: string;
  format: ArtifactFormat;
  useCase: string;
  description?: string;

  audience: "Internal" | "Prospects" | "Customers" | "Investors" | "Partners";
  lifecycle: "Strategy" | "Validate" | "Build" | "Launch" | "Sell" | "Scale" | "Gover

  owner: string;
  priority: ArtifactPriority;
  status: ArtifactStatus;

  effort: "XS" | "S" | "M" | "L" | "XL";
  impact: "Low" | "Medium" | "High";

  tags: string[];
  dependencies?: string[];
  suggestedPrompt: string;
```

```
  createdAt?: string;
  updatedAt?: string;
  dueDate?: string;
  sourceUrl?: string;
};
```

Why this model works

- **Category** powers high-level organizational filters: Product, Engineering, Brand, Legal, and so on.
- **Format** drives the native Claude-artifact recommendation: HTML, React, SVG, Mermaid, Docs, or Slides.
- **Lifecycle** answers the strategic question: "What stage is this for?" A launch plan and a privacy map may both be documents, but one supports launch while the other supports governance.
- **Audience** separates internal operating work from buyer-facing, customer-facing, investor-facing, and partner-facing deliverables.
- **Status**, **priority**, **effort**, and **impact** make the catalog function like a production backlog rather than a passive reference list.
- **Tags** provide flexible filtering when the primary taxonomy is not enough—for example `#SOC2`, `#enterprise`, `#onboarding`, `#PLG`, or `#fundraising`.
- **Suggested prompt** turns every record into a directly usable Claude production brief.

Use a **filter state object** distinct from artifact data:

```
const [filters, setFilters] = useState({
  category: "All",
  format: "All",
  lifecycle: "All",
  audience: "All",
  priority: "All",
  status: "All",
  owner: "All",
  effort: "All",
  impact: "All",
  tags: [],
});
```

Then calculate visible items rather than mutate the source data:

```
const visibleArtifacts = artifacts.filter((artifact) => {
  const matchesCategory =
    filters.category === "All" ||
    artifact.category === filters.category;

  const matchesFormat =
    filters.format === "All" ||
```

```
    artifact.format === filters.format;

  const matchesLifecycle =
    filters.lifecycle === "All" ||
    artifact.lifecycle === filters.lifecycle;

  const matchesTags =
    filters.tags.length === 0 ||
    filters.tags.every((tag) => artifact.tags.includes(tag));

  return (
    matchesCategory &&
    matchesFormat &&
    matchesLifecycle &&
    matchesTags
  );
});
```

This preserves the original catalog, allows “Reset filters” to be trivial, and ensures CSV exports, counts, dashboards, and gallery views stay consistent.

Format decision guide

Format	Choose it when	Default export	Key limitation
HTML	You need a standalone page, simple dashboard, calculator, landing page, or small interactive tool	`.html`, PDF, screenshot	Usually best as one self-contained page; sophisticated app infrastructure belongs outside the artifact sandbox
React	You need state, tables, filters, modals, multiple views, a prototype, or dashboard behavior	JSX source; deploy via a real React build setup	Sandbox environments may restrict package installation, durable storage, private APIs, and backend integration
SVG	You need an editable logo, icon, illustration, or polished static visual	`.svg`, PNG, PDF	Weak fit for data-heavy or stateful interaction
Mermaid	You need fast, maintainable architecture, sequence, flow, ERD, Gantt, or roadmap diagrams	Mermaid source, SVG, PNG, PDF	Styling and fine-grained positioning are more limited than hand-authored SVG
Docs	You need a PRD, policy, strategy, brief, report, research synthesis, or operational playbook	Markdown, Word, PDF	No native rich interaction or application behavior
Slides	You need a narrative for a pitch, board meeting, sales demo, launch, or training	PowerPoint, PDF	Works best for linear communication, not as a searchable system of record

Claude Code documentation describes artifacts as single HTML pages capable of generating downloadable files such as CSVs and PNGs; it also notes that artifact content can be edited and exported into formats including PowerPoint, PDF, and Word. React artifacts are especially suited to stateful UI but, when copied outside Claude, commonly need to be placed into a standard project and built for deployment. Mermaid is the practical default for structured system and process diagrams; SVG is better when visual polish and precise control are the priority. [22] [23] [21]

Important implementation notes

- Keep tracker records in `INITIAL_ARTIFACTS` at first; later move them to Supabase, a JSON endpoint, or a CSV import workflow.
- Keep **display data** separate from **filter state**. The tracker should derive filtered rows with `useMemo`, never permanently delete rows after filtering.
- Use a normalized set of enumerated values for category, format, lifecycle, priority, and status. This prevents duplicate values such as `GTM`, `Go To Market`, and `go-to-market`.
- Prefer tags for cross-cutting attributes that do not fit a single hierarchy: `#security`, `#enterprise`, `#activation`, `#fundraising`, `#PLG`, `#compliance`.
- Add `createdAt`, `updatedAt`, `dueDate`, `sourceUrl`, `reviewer`, and `approvalStatus` once this shifts from planning catalog to real production system.
- Do not rely on browser persistence inside an artifact sandbox for production tracking. Use a connected backend such as Supabase or export/import CSV if data must persist. Sandboxes and single-file artifact constraints can limit local browser storage and full app behavior. [23] [22]

⁂

Interactive decision tree for Claude artifacts: pick your prompt and requirements to instantly output HTML, React, SVG, Mermaid, Docs, or Slides

Computer
Claude Artifacts vs ChatGPT Canvas vs v0: sandbox limits, single-file constraints, and export formats compared across 6 dimensions
Computer
Show me how to convert this into a multi-format exporter
Add code copy buttons and a quick generation prompt builder
Expand the decision tree with more conditional logic

Below is an expanded version of the tracker that adds a rule-based format decision engine, copy buttons, a prompt builder, and multi-format export. The exporter produces Markdown, JSON, CSV, Mermaid, SVG, standalone HTML, and a slide-deck outline; it does not pretend to generate a fully editable `.pptx` or `.docx` purely in-browser.

Claude artifacts are generally best treated as self-contained client-side experiences: interactive HTML/JS, React components, SVG, Mermaid, and documents. Their sandbox can constrain persistence and external access, which is why the exporter below uses in-memory state and browser downloads rather than a backend. [31] [32]

Upgraded React dashboard

This is a **single-file React component.** It extends the prior tracker with:

- Conditional decision logic rather than a one-question selector
- Multi-step requirement choices: interaction, content shape, visual precision, system complexity, audience, and export need
- Ranked recommendations with a primary and fallback format
- One-click copy controls for prompts and generated output
- A quick generation prompt builder
- Browser-based exports for CSV, JSON, Markdown, Mermaid, SVG, HTML, and a Markdown slide-deck outline
- Sample Semester artifacts

```
import React, { useMemo, useState } from "react";

const FORMAT_META = {
  HTML: {
    icon: "◫",
    color: "bg-amber-100 text-amber-800 border-amber-200",
    bestFor: "Standalone web pages, landing pages, lightweight tools, calculators, ar
    exportTypes: ["HTML", "PDF snapshot", "PNG screenshot"],
    constraints: [
      "Best kept as one self-contained page.",
      "No durable production persistence without an external backend.",
      "Complex multi-route app architecture is outside its sweet spot.",
    ],
  },
  React: {
    icon: "⚛",
    color: "bg-sky-100 text-sky-800 border-sky-200",
    bestFor: "Dashboards, filters, tables, stateful prototypes, trackers, configurato
    exportTypes: ["JSX source", "HTML bundle after build", "CSV/JSON generated by the
    constraints: [
      "Claude artifact previews are usually client-side and sandboxed.",
      "External APIs, secrets, auth, storage, and deployment need a real application
      "Reusable production code may need packaging, routing, tests, and dependency cl
    ],
  },
  SVG: {
    icon: "◇",
    color: "bg-violet-100 text-violet-800 border-violet-200",
    bestFor: "Logos, icon systems, illustrations, visual explainers, and art-directed
    exportTypes: ["SVG", "PNG", "PDF"],
    constraints: [
      "Not ideal for heavy state, filtering, large datasets, or application workflows
      "Precise vector design may still need human refinement in Figma or Illustrator
    ],
  },
  Mermaid: {
    icon: "↳",
    color: "bg-emerald-100 text-emerald-800 border-emerald-200",
```

```
    bestFor: "Flowcharts, sequence diagrams, ERDs, Gantt charts, roadmaps, system ma
    exportTypes: ["Mermaid source", "SVG", "PNG", "PDF"],
    constraints: [
      "Layout and typography control are more constrained than hand-authored SVG.",
      "Best for structured relationships, not polished brand illustration.",
    ],
  },
  Docs: {
    icon: "▤",
    color: "bg-slate-100 text-slate-800 border-slate-200",
    bestFor: "PRDs, strategy, research, briefs, policies, specifications, operationa
    exportTypes: ["Markdown", "PDF", "DOCX via document workflow"],
    constraints: [
      "Not the right choice for dynamic filtering, stateful simulation, or live dashb
      "Legal, financial, regulatory, and security content still needs professional re
    ],
  },
  Slides: {
    icon: "▣",
    color: "bg-rose-100 text-rose-800 border-rose-200",
    bestFor: "Investor decks, board updates, sales presentations, launch readouts, tr
    exportTypes: ["Presentation outline", "PDF", "PPTX through a presentation workflo
    constraints: [
      "Designed for linear narrative, not a searchable operational system of record."
      "Visual and factual claims require human review before external presentation."
    ],
  },
};

const INITIAL_ARTIFACTS = [
  {
    id: "prd",
    artifact: "Flagship Workflow PRD",
    category: "Product",
    format: "Docs",
    lifecycle: "Build",
    useCase: "Specify Semester's primary workflow with scope, stories, acceptance cri
    audience: "Internal",
    priority: "P0",
    status: "Draft",
    owner: "Product",
    tags: ["prd", "requirements", "core-product"],
    prompt:
      "Write an implementation-ready PRD for Semester's flagship workflow. Include th
  },
  {
    id: "catalog",
    artifact: "Semester Artifact Catalog",
    category: "Operations",
    format: "React",
    lifecycle: "Scale",
    useCase: "Track every Claude-generated deliverable by format, category, owner, pr
    audience: "Internal",
    priority: "P0",
    status: "In Review",
    owner: "Founder / Operations",
```

```
    tags: ["tracker", "dashboard", "operations"],
    prompt:
      "Build a React dashboard for Semester that tracks artifacts by category, forma
  },
  {
    id: "architecture",
    artifact: "System Architecture Diagram",
    category: "Engineering",
    format: "Mermaid",
    lifecycle: "Build",
    useCase: "Document client, API, database, workers, integrations, observability, a
    audience: "Internal",
    priority: "P0",
    status: "Draft",
    owner: "Engineering",
    tags: ["architecture", "systems", "security"],
    prompt:
      "Create a Mermaid system architecture diagram for Semester. Include client appl
  },
  {
    id: "erd",
    artifact: "Core Data Model ERD",
    category: "Engineering",
    format: "Mermaid",
    lifecycle: "Build",
    useCase: "Define core entities, tenancy, relationships, ownership, and lifecycle
    audience: "Internal",
    priority: "P0",
    status: "Backlog",
    owner: "Engineering",
    tags: ["database", "erd", "schema"],
    prompt:
      "Create a Mermaid ERD for Semester. Include organizations, users, memberships,
  },
  {
    id: "prototype",
    artifact: "Interactive Core Workflow Prototype",
    category: "Product Design",
    format: "React",
    lifecycle: "Validate",
    useCase: "Demonstrate the core user workflow before production engineering.",
    audience: "Internal",
    priority: "P0",
    status: "Backlog",
    owner: "Product Design",
    tags: ["prototype", "ux", "validation"],
    prompt:
      "Build a clickable React prototype for Semester's primary user journey. Include
  },
  {
    id: "homepage",
    artifact: "Homepage and Product Narrative",
    category: "Brand",
    format: "HTML",
    lifecycle: "Launch",
    useCase: "Explain Semester’s value proposition and turn qualified visitors into c
```

```
    audience: "Prospects",
    priority: "PO",
    status: "Draft",
    owner: "Brand / Marketing",
    tags: ["homepage", "positioning", "conversion"],
    prompt:
      "Create a polished responsive HTML homepage for Semester. Include a hero, the p
  },
  {
    id: "identity",
    artifact: "Logo-Mark Exploration",
    category: "Brand",
    format: "SVG",
    lifecycle: "Launch",
    useCase: "Explore simple, editable visual marks appropriate for a modern B2B sof
    audience: "Internal",
    priority: "P2",
    status: "Backlog",
    owner: "Brand / Design",
    tags: ["logo", "identity", "visual"],
    prompt:
      "Create six simple SVG logo-mark concepts for Semester. Make them geometric, d
  },
  {
    id: "launch",
    artifact: "Product Launch Narrative",
    category: "Go-to-Market",
    format: "Slides",
    lifecycle: "Launch",
    useCase: "Coordinate product, marketing, sales, customer success, and leadership
    audience: "Internal",
    priority: "PO",
    status: "Backlog",
    owner: "Product Marketing",
    tags: ["launch", "gtm", "deck"],
    prompt:
      "Create a Semester product launch deck. Cover target audience, positioning, goa
  },
  {
    id: "privacy",
    artifact: "Privacy Data Flow Map",
    category: "Legal",
    format: "Mermaid",
    lifecycle: "Govern",
    useCase: "Map data collection, processing, storage, subprocessors, retention, del
    audience: "Internal",
    priority: "PO",
    status: "Backlog",
    owner: "Legal / Security",
    tags: ["privacy", "data", "compliance"],
    prompt:
      "Create a Mermaid privacy data-flow diagram for Semester. Show collection poin
  },
  {
    id: "terms",
    artifact: "Terms of Service First Draft",
```

```
    category: "Legal",
    format: "Docs",
    lifecycle: "Govern",
    useCase: "Create a structured first draft for counsel review before customer use
    audience: "Customers",
    priority: "P1",
    status: "Backlog",
    owner: "Legal",
    tags: ["terms", "contract", "legal"],
    prompt:
      "Draft a plain-English first version of Terms of Service for Semester. Include
  },
];

const DECISION_OPTIONS = {
  outcome: [
    ["interactive-tool", "Interactive tool, tracker, dashboard, calculator, or proto
    ["web-page", "Standalone web page, landing page, or lightweight demo"],
    ["structured-diagram", "Structured flow, architecture, sequence, ERD, or roadmap
    ["visual-asset", "Logo, icon, illustration, or art-directed visual"],
    ["written-record", "Brief, PRD, policy, report, plan, or knowledge document"],
    ["presentation", "Pitch, board update, sales story, launch, or training presenta
  ],
  interaction: [
    ["none", "No interaction; readers mainly consume content"],
    ["simple", "Buttons, toggles, calculators, basic chart or tab interaction"],
    ["complex", "Filtering, sorting, editable state, drawers, multi-step flows, or ap
  ],
  visualControl: [
    ["standard", "Structured output is fine; speed and maintainability matter most"]
    ["precise", "I need precise composition, art direction, and editable vector visua
  ],
  contentShape: [
    ["linear", "A linear narrative or reading order"],
    ["reference", "A durable reference document people will revisit"],
    ["relationship", "Relationships between systems, entities, steps, or actors"],
    ["data", "Rows, metrics, comparisons, records, or frequently changing data"],
  ],
  portability: [
    ["quick-share", "I need a fast shareable result in Claude"],
    ["source-export", "I need source I can copy into another tool or repository"],
    ["client-download", "Viewers need to download generated CSV, SVG, Markdown, or JS
    ["production", "This must become a deployed production application"],
  ],
};

const DEFAULT_DECISION = {
  outcome: "interactive-tool",
  interaction: "complex",
  visualControl: "standard",
  contentShape: "data",
  portability: "source-export",
};

function scoreFormats(decision) {
  const scores = {
```

```
  HTML: 0,
  React: 0,
  SVG: 0,
  Mermaid: 0,
  Docs: 0,
  Slides: 0,
};

const reasons = {
  HTML: [],
  React: [],
  SVG: [],
  Mermaid: [],
  Docs: [],
  Slides: [],
};

const add = (format, points, reason) => {
  scores[format] += points;
  reasons[format].push(reason);
};

switch (decision.outcome) {
  case "interactive-tool":
    add("React", 8, "Your primary goal is an interactive tool.");
    add("HTML", 5, "A standalone HTML page can support lighter interaction.");
    break;
  case "web-page":
    add("HTML", 8, "Your primary goal is a self-contained web page.");
    add("React", 4, "React is a fallback when the page becomes app-like.");
    break;
  case "structured-diagram":
    add("Mermaid", 8, "Your goal is to represent structured relationships.");
    add("SVG", 4, "SVG is a fallback for polished static visuals.");
    break;
  case "visual-asset":
    add("SVG", 9, "Your goal is an editable vector visual.");
    add("HTML", 2, "HTML can present the visual in a web context.");
    break;
  case "written-record":
    add("Docs", 9, "Your goal is a durable structured written record.");
    add("Slides", 2, "Slides are a fallback for executive summaries.");
    break;
  case "presentation":
    add("Slides", 9, "Your goal is a linear presentation narrative.");
    add("Docs", 2, "A document can support speaker notes or a follow-up memo.");
    break;
  default:
    break;
}

switch (decision.interaction) {
  case "none":
    add("Docs", 2, "No interaction is required.");
    add("Mermaid", 1, "Static diagrams fit non-interactive content.");
    add("SVG", 1, "Static vector content fits non-interactive output.");
```

```
      add("Slides", 1, "Slides suit read-only narrative.");
      break;
    case "simple":
      add("HTML", 4, "Simple controls fit a self-contained HTML page.");
      add("React", 3, "React works if interaction may expand later.");
      break;
    case "complex":
      add("React", 7, "Complex state and app-like interaction favor React.");
      add("HTML", 1, "HTML is possible but may become harder to maintain.");
      break;
    default:
      break;
  }

  switch (decision.visualControl) {
    case "standard":
      add("Mermaid", 3, "Structured, maintainable diagram syntax is acceptable.");
      add("Docs", 1, "Standard document formatting is acceptable.");
      break;
    case "precise":
      add("SVG", 7, "Precise composition and vector editability favor SVG.");
      add("HTML", 2, "HTML/CSS can help compose a visual presentation.");
      break;
    default:
      break;
  }

  switch (decision.contentShape) {
    case "linear":
      add("Slides", 4, "Linear narrative works well in slides.");
      add("Docs", 3, "Documents also suit sequential reading.");
      break;
    case "reference":
      add("Docs", 5, "A durable reference is best represented as a document.");
      break;
    case "relationship":
      add("Mermaid", 6, "Relationships map naturally to Mermaid syntax.");
      add("SVG", 2, "SVG can render a refined version of the relationship map.");
      break;
    case "data":
      add("React", 5, "Changing data, rows, and filters favor React.");
      add("HTML", 2, "HTML can fit a smaller static dataset.");
      break;
    default:
      break;
  }

  switch (decision.portability) {
    case "quick-share":
      add("HTML", 2, "A self-contained page is straightforward to share.");
      add("Docs", 2, "Documents are easy to distribute and review.");
      break;
    case "source-export":
      add("React", 3, "React source can move into a product codebase.");
      add("HTML", 3, "Standalone HTML is portable and easy to host.");
      add("Mermaid", 2, "Mermaid source remains editable as text.");
```

```
      add("SVG", 2, "SVG remains editable in visual-design tools.");
      break;
    case "client-download":
      add("React", 3, "React can generate data exports in-browser.");
      add("HTML", 2, "HTML can generate simple browser downloads.");
      break;
    case "production":
      add("React", 4, "Production interactive applications typically need a real Reac
      add("HTML", 1, "Production static marketing pages can begin as HTML.");
      break;
    default:
      break;
  }

  return Object.entries(scores)
    .map(([format, score]) => ({
      format,
      score,
      reasons: reasons[format],
    }))
    .sort((a, b) => b.score - a.score);
}

function artifactPrompt({ title, category, format, audience, goal, requirements, expo
  const formatGuidance = {
    HTML: "Create one self-contained responsive HTML file with embedded CSS and JavaS
    React: "Create one self-contained React component using local state. Use no exter
    SVG: "Create a clean, editable SVG with a considered viewBox, grouped layers, ser
    Mermaid: "Create valid Mermaid source code. Prefer clear node labels, logical sub
    Docs: "Create a structured Markdown document with an executive summary, headings
    Slides: "Create a slide-by-slide presentation outline with title, key message, re
  };

  return `Create a ${format} artifact for Semester.

Artifact title: ${title || "[Artifact title]"}
Category: ${category || "Product / Operations"}
Audience: ${audience || "Internal stakeholders"}
Primary outcome: ${goal || "Clarify and accelerate an important Semester workflow"}
Required capabilities: ${requirements || "Clear structure, realistic examples, and re
Desired export: ${exportType || "Source plus a shareable review format"}

${formatGuidance[format]}

Quality bar:
- Make the artifact immediately usable, not a generic template.
- State assumptions where information is missing.
- Use Semester as the product/company name.
- Include a concise “How to use this” section.
- Keep content specific, scannable, and ready for human review.`;
}

function downloadFile(filename, content, type = "text/plain;charset=utf-8") {
  const blob = new Blob([content], { type });
  const url = URL.createObjectURL(blob);
  const link = document.createElement("a");
```

```
  link.href = url;
  link.download = filename;
  link.click();
  URL.revokeObjectURL(url);
}

async function copyText(value) {
  try {
    await navigator.clipboard.writeText(value);
    return true;
  } catch {
    return false;
  }
}

function CopyButton({ value, label = "Copy" }) {
  const [copied, setCopied] = useState(false);

  const handleCopy = async () => {
    const success = await copyText(value);
    setCopied(success);
    window.setTimeout(() => setCopied(false), 1500);
  };

  return (
    <button
      onClick={handleCopy}
      className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 text-xs font
    >
      {copied ? "Copied" : label}
    </button>
  );
}

function Badge({ children, className = "" }) {
  return (
    <span className={`inline-flex items-center rounded-full border px-2 py-0.5 text-x
      {children}
    </span>
  );
}

function DecisionTree() {
  const [decision, setDecision] = useState(DEFAULT_DECISION);
  const ranked = useMemo(() => scoreFormats(decision), [decision]);
  const top = ranked[^4_0];
  const fallback = ranked[^4_1];
  const primaryMeta = FORMAT_META[top.format];

  const setValue = (key, value) => {
    setDecision((current) => ({ ...current, [key]: value }));
  };

  const prompt = artifactPrompt({
    title: "Semester Artifact",
    category: "Product / Operations",
```

```
    format: top.format,
    audience: "Internal stakeholders",
    goal: "Produce a decision-ready deliverable for Semester",
    requirements: "Use the selected requirements from the decision tree",
    exportType: primaryMeta.exportTypes.join(", "),
  });

  return (
    <section className="rounded-2xl border border-indigo-100 bg-gradient-to-br from-
      <div>
        <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-600">
        <h2 className="mt-1 text-2xl font-bold text-slate-950">Interactive Claude ar
        <p className="mt-2 max-w-3xl text-sm leading-6 text-slate-600">
          Select what you are making, how much interaction it needs, how visually pre
        </p>
      </div>

      <div className="mt-5 grid gap-4 md:grid-cols-2 xl:grid-cols-3">
        {Object.entries(DECISION_OPTIONS).map(([key, options]) => (
          <fieldset key={key} className="rounded-xl border border-slate-200 bg-white
            <legend className="px-1 text-sm font-bold capitalize text-slate-900">
              {key.replace(/([A-Z])/g, " $1")}
            </legend>
            <div className="mt-2 space-y-2">
              {options.map(([value, label]) => (
                <label key={value} className="flex cursor-pointer items-start gap-2
                  <input
                    type="radio"
                    name={key}
                    value={value}
                    checked={decision[key] === value}
                    onChange={() => setValue(key, value)}
                    className="mt-1 accent-indigo-600"
                  />
                  <span>{label}</span>
                </label>
              ))}
            </div>
          </fieldset>
        ))}
      </div>

      <div className="mt-5 rounded-2xl border border-indigo-200 bg-white p-5">
        <div className="flex flex-col justify-between gap-4 md:flex-row md:items-sta
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
            <div className="mt-1 flex flex-wrap items-center gap-2">
              <span className="text-3xl">{primaryMeta.icon}</span>
              <h3 className="text-2xl font-bold text-slate-950">{top.format}</h3>
              <Badge className={primaryMeta.color}>Score: {top.score}</Badge>
            </div>
            <p className="mt-3 max-w-2xl text-sm leading-6 text-slate-600">{primaryMe
          </div>
          <div className="rounded-xl bg-slate-50 p-3 text-sm">
            <p className="font-bold text-slate-800">Fallback: {fallback.format}</p>
            <p className="mt-1 text-slate-600">{FORMAT_META[fallback.format].bestFor
```

```
          </div>
        </div>

        <div className="mt-5 grid gap-4 md:grid-cols-3">
          <div>
            <p className="text-sm font-bold text-slate-800">Why this wins</p>
            <ul className="mt-2 space-y-1 text-sm text-slate-600">
              {top.reasons.length ? top.reasons.map((reason) => <li key={reason}>• {r
            </ul>
          </div>
          <div>
            <p className="text-sm font-bold text-slate-800">Best export choices</p>
            <ul className="mt-2 space-y-1 text-sm text-slate-600">
              {primaryMeta.exportTypes.map((item) => <li key={item}>• {item}</li>)}
            </ul>
          </div>
          <div>
            <p className="text-sm font-bold text-slate-800">Constraints to plan for</
            <ul className="mt-2 space-y-1 text-sm text-slate-600">
              {primaryMeta.constraints.map((item) => <li key={item}>• {item}</li>)}
            </ul>
          </div>
        </div>

        <div className="mt-5 rounded-xl bg-slate-950 p-4">
          <div className="flex items-center justify-between gap-3">
            <p className="text-sm font-bold text-white">Prompt generated from your ch
            <CopyButton value={prompt} label="Copy prompt" />
          </div>
          <pre className="mt-3 whitespace-pre-wrap text-xs leading-6 text-slate-200">
        </div>
      </div>
    </section>
  );
}

function PromptBuilder() {
  const [form, setForm] = useState({
    title: "Semester Artifact Catalog",
    category: "Operations",
    format: "React",
    audience: "Internal stakeholders",
    goal: "Track and prioritize every artifact Claude should generate for Semester",
    requirements: "Search, multi-select filters, sorting, saved views, a detail drawe
    exportType: "JSX source plus CSV export",
  });

  const prompt = useMemo(() => artifactPrompt(form), [form]);

  const update = (key, value) => setForm((current) => ({ ...current, [key]: value }))

  return (
    <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
      <div className="flex flex-col justify-between gap-3 md:flex-row md:items-end">
        <div>
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-600
```

```
          <h2 className="mt-1 text-xl font-bold text-slate-950">Quick generation pron
          <p className="mt-1 text-sm text-slate-600">Turn a tracker row or a fresh id
        </div>
        <CopyButton value={prompt} label="Copy full prompt" />
      </div>

      <div className="mt-5 grid gap-4 md:grid-cols-2 xl:grid-cols-3">
        <label className="text-sm font-semibold text-slate-700">
          Artifact title
          <input
            value={form.title}
            onChange={(e) => update("title", e.target.value)}
            className="mt-1 w-full rounded-lg border border-slate-200 px-3 py-2 font-
          />
        </label>

        <label className="text-sm font-semibold text-slate-700">
          Category
          <input
            value={form.category}
            onChange={(e) => update("category", e.target.value)}
            className="mt-1 w-full rounded-lg border border-slate-200 px-3 py-2 font-
          />
        </label>

        <label className="text-sm font-semibold text-slate-700">
          Format
          <select
            value={form.format}
            onChange={(e) => update("format", e.target.value)}
            className="mt-1 w-full rounded-lg border border-slate-200 px-3 py-2 font-
          >
            {Object.keys(FORMAT_META).map((format) => <option key={format}>{format}</
          </select>
        </label>

        <label className="text-sm font-semibold text-slate-700">
          Audience
          <input
            value={form.audience}
            onChange={(e) => update("audience", e.target.value)}
            className="mt-1 w-full rounded-lg border border-slate-200 px-3 py-2 font-
          />
        </label>

        <label className="text-sm font-semibold text-slate-700 md:col-span-2">
          Primary outcome
          <input
            value={form.goal}
            onChange={(e) => update("goal", e.target.value)}
            className="mt-1 w-full rounded-lg border border-slate-200 px-3 py-2 font-
          />
        </label>

        <label className="text-sm font-semibold text-slate-700 md:col-span-2">
          Required capabilities
```

```
          <textarea
            value={form.requirements}
            onChange={(e) => update("requirements", e.target.value)}
            className="mt-1 min-h-[90px] w-full rounded-lg border border-slate-200 px
          />
        </label>

        <label className="text-sm font-semibold text-slate-700">
          Desired export
          <input
            value={form.exportType}
            onChange={(e) => update("exportType", e.target.value)}
            className="mt-1 w-full rounded-lg border border-slate-200 px-3 py-2 font-
          />
        </label>
      </div>

      <div className="mt-5 rounded-xl bg-slate-950 p-4">
        <pre className="whitespace-pre-wrap text-xs leading-6 text-slate-200">{prompt
      </div>
    </section>
  );
}

function buildMarkdown(artifacts) {
  const rows = artifacts
    .map(
      (item) =>
        `| ${item.artifact} | ${item.category} | ${item.format} | ${item.lifecycle}
    )
    .join("\n");

  return `# Semester Claude Artifact Catalog

Generated: ${new Date().toLocaleString()}

| Artifact | Category | Format | Lifecycle | Priority | Status | Owner |
|---|---|---|---|---|---|---|
${rows}

## Generation Prompts

${artifacts
  .map((item) => `### ${item.artifact}\n\n**Recommended format:** ${item.format}\n\nS
  .join("\n\n")}
`;
}

function buildMermaid(artifacts) {
  const grouped = artifacts.reduce((acc, item) => {
    if (!acc[item.category]) acc[item.category] = [];
    acc[item.category].push(item);
    return acc;
  }, {});

  const safeId = (value) => value.replace(/[^a-zA-Z0-9]/g, "");
```

```
  return `flowchart LR
  Semester[Semester Artifact Catalog]
${Object.entries(grouped)
  .map(([category, items]) => {
    const categoryId = safeId(category);
    return `  Semester --> ${categoryId}[${category}]
${items.map((item) => `  ${categoryId} --> ${safeId(item.id)}[${item.artifact.replace
  })
  .join("\n")}
`;
}

function buildSvg(artifacts) {
  const grouped = artifacts.reduce((acc, item) => {
    if (!acc[item.category]) acc[item.category] = [];
    acc[item.category].push(item);
    return acc;
  }, {});

  const categories = Object.keys(grouped);
  const width = 1200;
  const height = Math.max(500, categories.length * 175 + 110);

  const escape = (value) =>
    String(value)
      .replace(/&/g, "&amp;")
      .replace(/</g, "&lt;")
      .replace(/>/g, "&gt;")
      .replace(/"/g, "&quot;");

  const body = categories
    .map((category, index) => {
      const y = 95 + index * 175;
      const items = grouped[category].slice(0, 4);
      return `
        <g>
          <rect x="40" y="${y}" width="180" height="56" rx="12" fill="#111827"/>
          <text x="60" y="${y + 34}" fill="#ffffff" font-family="Arial, sans-serif"
          ${items
            .map((item, itemIndex) => {
              const x = 280 + itemIndex * 220;
              return `
                <line x1="220" y1="${y + 28}" x2="${x}" y2="${y + 28}" stroke="#c7d2
                <rect x="${x}" y="${y}" width="190" height="56" rx="12" fill="#eef2f
                <text x="${x + 14}" y="${y + 24}" fill="#1e1b4b" font-family="Arial,
                <text x="${x + 14}" y="${y + 42}" fill="#4f46e5" font-family="Arial,
              `;
            })
            .join("")}
        </g>
      `;
    })
    .join("");

  return `<svg xmlns="http://www.w3.org/2000/svg" width="${width}" height="${height}"
```

```
  <rect width="100%" height="100%" fill="#f8fafc"/>
  <text x="40" y="48" fill="#0f172a" font-family="Arial, sans-serif" font-size="28" 
  <text x="40" y="72" fill="#64748b" font-family="Arial, sans-serif" font-size="14">\
  ${body}
</svg>`;
}

function buildSlidesMarkdown(artifacts) {
  const categoryGroups = artifacts.reduce((acc, item) => {
    if (!acc[item.category]) acc[item.category] = [];
    acc[item.category].push(item);
    return acc;
  }, {});

  return `# Semester Artifact Catalog — Slide Outline

## Slide 1 — Title
- Semester Claude Artifact Catalog
- A production system for strategy, product, engineering, GTM, operations, and legal

## Slide 2 — Catalog Summary
- Total visible artifacts: ${artifacts.length}
- P0 artifacts: ${artifacts.filter((item) => item.priority === "P0").length}
- Draft or review items: ${artifacts.filter((item) => ["Draft", "In Review"].includes
- Visual: KPI tiles

${Object.entries(categoryGroups)
  .map(
    ([category, items]) => `## Slide — ${category}
- Key artifacts:
${items.map((item) => `  - ${item.artifact} (${item.format}, ${item.priority}, ${iter
- Recommended visual: format badges and a status snapshot
- Takeaway: ${category} has ${items.length} tracked artifact${items.length === 1 ? "'
  )
  .join("\n\n")}

## Final Slide — Next Actions
- Prioritize open P0 artifacts
- Generate the highest-leverage deliverables using their embedded prompts
- Move approved artifacts into the durable production system
`;
}

function buildStandaloneHtml(artifacts) {
  const data = JSON.stringify(artifacts, null, 2).replace(/</g, "\\u003c");

  return `<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Semester Artifact Catalog</title>
  <style>
    body { margin: 0; background: #f8fafc; color: #0f172a; font: 14px/1.45 Arial, sa
    main { max-width: 1200px; margin: 0 auto; padding: 32px 20px; }
    h1 { margin: 0; font-size: 32px; } p { color: #475569; }
```

```
    input, select { border: 1px solid #cbd5e1; border-radius: 8px; padding: 10px; ba
    .controls { display: flex; flex-wrap: wrap; gap: 10px; margin: 24px 0; }
    .table-wrap { overflow: auto; background: white; border: 1px solid #e2e8f0; borde
    table { border-collapse: collapse; width: 100%; min-width: 760px; }
    th, td { text-align: left; padding: 12px; border-bottom: 1px solid #e2e8f0; }
    th { background: #f1f5f9; font-size: 12px; text-transform: uppercase; letter-spac
    .badge { display: inline-block; border-radius: 999px; padding: 3px 8px; backgrou
  </style>
</head>
<body>
  <main>
    <h1>Semester Claude Artifact Catalog</h1>
    <p>Standalone export with search and category/format filters.</p>
    <div class="controls">
      <input id="search" placeholder="Search artifacts..." />
      <select id="category"><option value="All">All categories</option></select>
      <select id="format"><option value="All">All formats</option></select>
    </div>
    <div class="table-wrap">
      <table>
        <thead><tr><th>Artifact</th><th>Category</th><th>Format</th><th>Lifecycle</th
        <tbody id="rows"></tbody>
      </table>
    </div>
  </main>
  <script>
    const artifacts = ${data};
    const rows = document.getElementById("rows");
    const search = document.getElementById("search");
    const category = document.getElementById("category");
    const format = document.getElementById("format");

    function unique(key) {
      return [...new Set(artifacts.map(item => item[key]))].sort();
    }

    unique("category").forEach(value => category.insertAdjacentHTML("beforeend", '<op
    unique("format").forEach(value => format.insertAdjacentHTML("beforeend", '<option

    function render() {
      const query = search.value.toLowerCase();
      rows.innerHTML = artifacts
        .filter(item => {
          const text = [item.artifact, item.category, item.format, item.lifecycle, i
          return text.includes(query) &&
            (category.value === "All" || item.category === category.value) &&
            (format.value === "All" || item.format === format.value);
        })
        .map(item => '<tr><td><strong>' + item.artifact + '</strong><br><small>' + i
        .join("");
    }

    [search, category, format].forEach(control => control.addEventListener("input", r
    render();
  </script>
</body>
```

```
</html>`;
}

function ExportPanel({ artifacts }) {
  const [exportType, setExportType] = useState("CSV");

  const formats = [
    ["CSV", "Spreadsheet-ready tracker export"],
    ["JSON", "Full structured catalog data"],
    ["Markdown", "Document and prompt library"],
    ["Mermaid", "Category-to-artifact relationship map"],
    ["SVG", "Static visual catalog map"],
    ["HTML", "Standalone filterable web-page export"],
    ["Slides", "Slide-deck outline in Markdown"],
  ];

  const exportContent = () => {
    const csvColumns = ["artifact", "category", "format", "lifecycle", "useCase", "au
    const csv = [
      csvColumns.join(","),
      ...artifacts.map((item) =>
        csvColumns
          .map((column) => {
            const value = Array.isArray(item[column]) ? item[column].join("; ") : ite
            return `"${String(value).replace(/"/g, '""')}"`;
          })
          .join(",")
      ),
    ].join("\n");

    const payload = {
      CSV: {
        filename: "semester-artifact-catalog.csv",
        content: csv,
        type: "text/csv;charset=utf-8",
      },
      JSON: {
        filename: "semester-artifact-catalog.json",
        content: JSON.stringify(artifacts, null, 2),
        type: "application/json;charset=utf-8",
      },
      Markdown: {
        filename: "semester-artifact-catalog.md",
        content: buildMarkdown(artifacts),
        type: "text/markdown;charset=utf-8",
      },
      Mermaid: {
        filename: "semester-artifact-catalog.mmd",
        content: buildMermaid(artifacts),
        type: "text/plain;charset=utf-8",
      },
      SVG: {
        filename: "semester-artifact-catalog.svg",
        content: buildSvg(artifacts),
        type: "image/svg+xml;charset=utf-8",
      },
```

```
      HTML: {
        filename: "semester-artifact-catalog.html",
        content: buildStandaloneHtml(artifacts),
        type: "text/html;charset=utf-8",
      },
      Slides: {
        filename: "semester-artifact-catalog-slide-outline.md",
        content: buildSlidesMarkdown(artifacts),
        type: "text/markdown;charset=utf-8",
      },
    };

    const selected = payload[exportType];
    downloadFile(selected.filename, selected.content, selected.type);
  };

  const preview = useMemo(() => {
    if (exportType === "Markdown") return buildMarkdown(artifacts);
    if (exportType === "Mermaid") return buildMermaid(artifacts);
    if (exportType === "Slides") return buildSlidesMarkdown(artifacts);
    if (exportType === "SVG") return buildSvg(artifacts);
    if (exportType === "HTML") return buildStandaloneHtml(artifacts).slice(0, 2400) +
    if (exportType === "JSON") return JSON.stringify(artifacts, null, 2);
    return `Artifact,Category,Format,Lifecycle,Use Case,Audience,Priority,Status,Owne
      .slice(0, 3)
      .map((item) => `${item.artifact},${item.category},${item.format},${item.lifecyc
      .join("\n")}\n...`;
  }, [artifacts, exportType]);

  return (
    <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
      <div>
        <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-600">
        <h2 className="mt-1 text-xl font-bold text-slate-950">Export the filtered ca
        <p className="mt-1 text-sm text-slate-600">
          Export the current filtered rows. The dashboard creates source files in the
        </p>
      </div>

      <div className="mt-4 grid gap-2 md:grid-cols-2 xl:grid-cols-4">
        {formats.map(([name, description]) => (
          <button
            key={name}
            onClick={() => setExportType(name)}
            className={`rounded-xl border p-3 text-left transition ${
              exportType === name
                ? "border-indigo-500 bg-indigo-50"
                : "border-slate-200 bg-white hover:border-indigo-300"
            }`}
          >
            <p className="text-sm font-bold text-slate-900">{name}</p>
            <p className="mt-1 text-xs leading-5 text-slate-600">{description}</p>
          </button>
        ))}
      </div>
```

```
      <div className="mt-4 flex flex-wrap gap-2">
        <button
          onClick={exportContent}
          className="rounded-lg bg-indigo-600 px-4 py-2 text-sm font-bold text-white
        >
          Download {exportType}
        </button>
        <CopyButton value={preview} label={`Copy ${exportType} output`} />
      </div>

      <pre className="mt-4 max-h-72 overflow-auto rounded-xl bg-slate-950 p-4 whitesp
        {preview}
      </pre>
    </section>
  );
}

export default function SemesterArtifactStudio() {
  const [artifacts, setArtifacts] = useState(INITIAL_ARTIFACTS);
  const [search, setSearch] = useState("");
  const [filters, setFilters] = useState({
    category: "All",
    format: "All",
    lifecycle: "All",
    priority: "All",
    status: "All",
  });
  const [sort, setSort] = useState({ key: "priority", direction: "asc" });
  const [selected, setSelected] = useState(null);

  const unique = (key) => [...new Set(artifacts.map((item) => item[key]))].sort();

  const filteredArtifacts = useMemo(() => {
    const priorityOrder = { P0: 0, P1: 1, P2: 2, P3: 3 };
    const statusOrder = { Backlog: 0, Draft: 1, "In Review": 2, Approved: 3, Shipped
    const query = search.trim().toLowerCase();

    return artifacts
      .filter((item) => {
        const searchText = [
          item.artifact,
          item.category,
          item.format,
          item.lifecycle,
          item.useCase,
          item.audience,
          item.owner,
          ...item.tags,
        ].join(" ").toLowerCase();

        return (
          (!query || searchText.includes(query)) &&
          (filters.category === "All" || item.category === filters.category) &&
          (filters.format === "All" || item.format === filters.format) &&
          (filters.lifecycle === "All" || item.lifecycle === filters.lifecycle) &&
          (filters.priority === "All" || item.priority === filters.priority) &&
```

```
          (filters.status === "All" || item.status === filters.status)
        );
      })
      .sort((a, b) => {
        if (sort.key === "priority") {
          return sort.direction === "asc"
            ? priorityOrder[a.priority] - priorityOrder[b.priority]
            : priorityOrder[b.priority] - priorityOrder[a.priority];
        }

        if (sort.key === "status") {
          return sort.direction === "asc"
            ? statusOrder[a.status] - statusOrder[b.status]
            : statusOrder[b.status] - statusOrder[a.status];
        }

        return sort.direction === "asc"
          ? String(a[sort.key]).localeCompare(String(b[sort.key]))
          : String(b[sort.key]).localeCompare(String(a[sort.key]));
      });
  }, [artifacts, filters, search, sort]);

  const setFilter = (key, value) => {
    setFilters((current) => ({ ...current, [key]: value }));
  };

  const setSortKey = (key) => {
    setSort((current) => ({
      key,
      direction: current.key === key && current.direction === "asc" ? "desc" : "asc"
    }));
  };

  const updateStatus = (id, status) => {
    setArtifacts((current) => current.map((item) => (item.id === id ? { ...item, sta
    setSelected((current) => (current?.id === id ? { ...current, status } : current)
  };

  const resetFilters = () => {
    setSearch("");
    setFilters({
      category: "All",
      format: "All",
      lifecycle: "All",
      priority: "All",
      status: "All",
    });
  };

  const header = (name, key) => (
    <button
      onClick={() => setSortKey(key)}
      className="flex items-center gap-1 text-left text-xs font-bold uppercase track
    >
      {name}
      {sort.key === key ? (sort.direction === "asc" ? "↑" : "↓") : "↕"}
```

```
      </button>
    );

    return (
      <main className="min-h-screen bg-slate-50 p-4 text-slate-900 md:p-8">
        <div className="mx-auto max-w-7xl space-y-6">
          <header className="rounded-2xl bg-gradient-to-br from-slate-950 via-slate-900
            <p className="text-xs font-bold uppercase tracking-[0.16em] text-indigo-300
            <h1 className="mt-2 text-3xl font-bold tracking-tight">Artifact Decision, P
            <p className="mt-2 max-w-3xl text-sm leading-6 text-slate-300">
              Pick requirements, get an evidence-based format recommendation, generate
            </p>
          </header>

          <DecisionTree />
          <PromptBuilder />

          <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
            <div className="flex flex-col justify-between gap-3 md:flex-row md:items-en
              <div>
                <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo
                <h2 className="mt-1 text-xl font-bold text-slate-950">Semester project
                <p className="mt-1 text-sm text-slate-600">Filter the exact records use
              </div>
              <p className="text-sm font-semibold text-slate-700">
                {filteredArtifacts.length} / {artifacts.length} visible
              </p>
            </div>

            <div className="mt-5 grid gap-3 md:grid-cols-2 xl:grid-cols-6">
              <input
                value={search}
                onChange={(event) => setSearch(event.target.value)}
                placeholder="Search all fields..."
                className="rounded-lg border border-slate-200 px-3 py-2 text-sm outline
              />

              {[
                ["category", "Category"],
                ["format", "Format"],
                ["lifecycle", "Lifecycle"],
                ["priority", "Priority"],
                ["status", "Status"],
              ].map(([key, label]) => (
                <label key={key} className="text-xs font-bold text-slate-500">
                  {label}
                  <select
                    value={filters[key]}
                    onChange={(event) => setFilter(key, event.target.value)}
                    className="mt-1 w-full rounded-lg border border-slate-200 bg-white
                  >
                    <option value="All">All</option>
                    {unique(key).map((value) => <option key={value}>{value}</option>)}
                  </select>
                </label>
              ))}
```

```
          </div>

          <button
            onClick={resetFilters}
            className="mt-3 rounded-lg px-3 py-2 text-sm font-bold text-slate-500 hov
          >
            Reset filters
          </button>

          <div className="mt-5 overflow-hidden rounded-xl border border-slate-200">
            <div className="overflow-x-auto">
              <table className="w-full min-w-[1050px] border-collapse">
                <thead className="bg-slate-50">
                  <tr>
                    <th className="px-4 py-3">{header("Artifact", "artifact")}</th>
                    <th className="px-4 py-3">{header("Category", "category")}</th>
                    <th className="px-4 py-3">{header("Format", "format")}</th>
                    <th className="px-4 py-3">{header("Lifecycle", "lifecycle")}</th>
                    <th className="px-4 py-3">{header("Priority", "priority")}</th>
                    <th className="px-4 py-3">{header("Status", "status")}</th>
                    <th className="px-4 py-3">{header("Owner", "owner")}</th>
                    <th className="px-4 py-3 text-left text-xs font-bold uppercase tr
                  </tr>
                </thead>
                <tbody>
                  {filteredArtifacts.map((item) => (
                    <tr key={item.id} className="border-t border-slate-100 hover:bg-
                      <td className="px-4 py-3">
                        <button
                          onClick={() => setSelected(item)}
                          className="text-left text-sm font-bold text-slate-900 hover
                        >
                          {item.artifact}
                        </button>
                        <div className="mt-1 flex flex-wrap gap-1">
                          {item.tags.map((tag) => <span key={tag} className="text-xs
                        </div>
                      </td>
                      <td className="px-4 py-3 text-sm text-slate-600">{item.category
                      <td className="px-4 py-3">
                        <Badge className={FORMAT_META[item.format].color}>
                          {FORMAT_META[item.format].icon} {item.format}
                        </Badge>
                      </td>
                      <td className="px-4 py-3 text-sm text-slate-600">{item.lifecycl
                      <td className="px-4 py-3">
                        <Badge className={item.priority === "P0" ? "border-red-200 bg
                          {item.priority}
                        </Badge>
                      </td>
                      <td className="px-4 py-3">
                        <select
                          value={item.status}
                          onChange={(event) => updateStatus(item.id, event.target.val
                          className="rounded-lg border border-slate-200 bg-white px-2
                        >
```

```
                        {["Backlog", "Draft", "In Review", "Approved", "Shipped", "
                          <option key={status}>{status}</option>
                        ))}
                      </select>
                    </td>
                    <td className="px-4 py-3 text-sm text-slate-600">{item.owner}</
                    <td className="px-4 py-3">
                      <CopyButton value={item.prompt} label="Copy" />
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </div>
      </section>

      <ExportPanel artifacts={filteredArtifacts} />

      {selected && (
        <div className="fixed inset-0 z-50 flex justify-end bg-slate-950/30">
          <button className="absolute inset-0 cursor-default" aria-label="Close de
          <aside className="relative h-full w-full max-w-xl overflow-auto bg-white
            <div className="flex justify-between gap-4">
              <div>
                <div className="flex flex-wrap gap-2">
                  <Badge className={FORMAT_META[selected.format].color}>
                    {FORMAT_META[selected.format].icon} {selected.format}
                  </Badge>
                  <Badge className="border-slate-200 bg-slate-50 text-slate-600">{s
                  <Badge className="border-slate-200 bg-slate-50 text-slate-600">{s
                </div>
                <h2 className="mt-3 text-2xl font-bold text-slate-950">{selected.a
              </div>
              <button
                onClick={() => setSelected(null)}
                className="h-fit rounded-lg border border-slate-200 px-3 py-1.5 te
              >
                Close
              </button>
            </div>

            <p className="mt-4 text-sm leading-6 text-slate-600">{selected.useCase

            <div className="mt-5 grid gap-3 sm:grid-cols-2">
              {[
                ["Audience", selected.audience],
                ["Lifecycle", selected.lifecycle],
                ["Owner", selected.owner],
                ["Status", selected.status],
              ].map(([label, value]) => (
                <div key={label} className="rounded-xl bg-slate-50 p-3">
                  <p className="text-xs font-bold uppercase tracking-wide text-sla
                  <p className="mt-1 text-sm font-semibold text-slate-800">{value}<
                </div>
              ))}
```

```
              </div>

              <div className="mt-5 rounded-xl border border-indigo-100 bg-indigo-50 p
                <p className="text-sm font-bold text-indigo-950">Format guidance: {se
                <p className="mt-1 text-sm text-indigo-900">{FORMAT_META[selected.for
                <p className="mt-3 text-xs font-bold uppercase tracking-wide text-ind
                <p className="mt-1 text-xs text-indigo-800">{FORMAT_META[selected.for
              </div>

              <div className="mt-5">
                <div className="flex items-center justify-between">
                  <h3 className="text-sm font-bold text-slate-900">Generation prompt<
                  <CopyButton value={selected.prompt} label="Copy prompt" />
                </div>
                <pre className="mt-2 whitespace-pre-wrap rounded-xl bg-slate-950 p-4
                  {selected.prompt}
                </pre>
              </div>
            </aside>
          </div>
        )}
      </div>
    </main>
  );
}
```

Export design

The implementation deliberately separates **catalog data** from **export transformers**. Each export function accepts the current filtered array and transforms it to a string:

Export	Output	Best use
CSV	`.csv`	Import into Sheets, Airtable, Notion, or a database
JSON	`.json`	Preserve all structured fields, prompts, tags, and metadata
Markdown	`.md`	Create a readable artifact inventory and prompt library
Mermaid	`.mmd`	Render a category-to-artifact relationship map
SVG	`.svg`	Share a static visual overview or edit in a vector tool
HTML	`.html`	Distribute a lightweight standalone filterable catalog
Slides	`.md`	Generate a deck outline for later conversion to PPTX or Slides

The pattern is:

```
function exportCatalog(filteredArtifacts, exportType) {
  const content = transformers[exportType](filteredArtifacts);

  downloadFile(
    filenameByType[exportType],
    content,
```

```
    mimeTypeByType[exportType]
  );
}
```

That architecture makes it easy to add later exporters:

```
const transformers = {
  CSV: buildCsv,
  JSON: buildJson,
  Markdown: buildMarkdown,
  Mermaid: buildMermaid,
  SVG: buildSvg,
  HTML: buildStandaloneHtml,
  Slides: buildSlidesMarkdown,

  // Later, from a server or connected service:
  PDF: buildPdfOnServer,
  PPTX: buildPptxOnServer,
  DOCX: buildDocxOnServer,
};
```

A true PPTX, DOCX, or polished PDF pipeline generally needs either a specialized browser library or a server-side document-generation service. The browser-only version should export stable source formats—Markdown, HTML, SVG, CSV, JSON, and Mermaid—and use those as editable handoff formats.

Expanded decision logic

The engine uses five dimensions rather than one:

Decision factor	Example answer	Main effect
Primary outcome	“I need a searchable tracker”	Strongly favors React
Interaction level	“Users filter, sort, edit, and navigate”	Favors React over HTML
Visual-control requirement	“Needs brand-ready logo treatment”	Favors SVG
Content shape	“Shows entity relationships”	Favors Mermaid
Portability requirement	“Needs to ship as a static artifact”	Favors HTML, SVG, Mermaid, or Docs depending on outcome

Core rules

- Choose **React** when the deliverable needs ongoing state: filters, sorting, form inputs, saved views, editable content, tooltips, drawers, charts, or multi-step interaction.
- Choose **HTML** when the output is a self-contained page with modest interaction: a landing page, simple calculator, static demo, or a single-purpose tool.

- Choose **Mermaid** when the source of truth is a structured relationship: system architecture, process flow, sequence, ERD, Gantt, dependency map, or roadmap.
- Choose **SVG** when precision of the visual itself matters more than semantic structure: brand marks, icons, illustrations, styled diagrams, and visual explainers.
- Choose **Docs** when reviewability, versioned decisions, durable reference, and written nuance matter most: PRDs, strategy, policies, briefs, research synthesis, and playbooks.
- Choose **Slides** when the work must persuade or align an audience through a linear narrative: investor pitches, board updates, sales stories, launches, and training.

Claude vs Canvas vs v0

Claude Artifacts, ChatGPT Canvas, and v0 overlap, but they optimize for different handoff points: **Claude** for self-contained shareable artifacts, **Canvas** for iterative writing and code editing, and **v0** for a more deployment-oriented web-app workflow. [33] [34] [35]

Dimension	Claude Artifacts	ChatGPT Canvas	v0
Primary job	Generate a visually rendered artifact beside a conversation	Collaboratively edit a writing or coding workspace	Generate, run, refine, and deploy web applications
Strongest outputs	Interactive HTML/JS experiences, React components, SVG, Mermaid, formatted documents	Writing, code editing, rewrites, language porting, file-oriented export	Full-stack web UI and application projects, often in a Vercel-oriented workflow
Runtime model	Typically client-side inside a controlled browser/iframe environment	Canvas workspace for code and documents; execution is feature and environment dependent	VM-backed sandbox that runs project files, commands, server code, API routes, and database connections
Sandbox limitations	External access and browser capabilities can be restricted; sandboxed artifact environments make persistence and third-party integration less straightforward	More centered on producing/editing content than providing a durable deployable app environment	Isolated environment, but closer to real Node.js behavior and designed for app iteration
File/project shape	Often strongest as one self-contained artifact; production apps may need extraction into a repo	Works well with individual documents and code files	Designed around a project filesystem and application codebase
Export and handoff	Useful source forms include HTML, React/JSX, SVG, Mermaid, Markdown, and in-artifact generated downloads such as CSV or PNG	Canvas can export documents as PDF, DOCX, or Markdown and code in its relevant file type	Code can be exported and deployed elsewhere; GitHub/Vercel connection unlocks a more complete project workflow

Claude Artifacts are commonly described as executable HTML/JavaScript pages that run in a controlled browser sandbox; restrictions can affect `localStorage`, `fetch`, and external APIs, so production data and authentication should live outside the artifact. Claude Code documentation also supports artifacts generating browser-downloadable files such as CSV exports and chart PNGs, which is the model used by the React exporter above. [32] [36] [31]

ChatGPT Canvas is aimed at extended writing and coding projects; OpenAI explicitly describes capabilities such as porting code among JavaScript, TypeScript, Python, Java, C++, and PHP,

while document exports have been announced for PDF, DOCX, and Markdown. [37] [33]

v0's current sandbox is a VM-backed Node.js environment, replacing earlier browser-only previews; its documentation says it can run server code, API routes, and real database connections, and its FAQ notes that source can still be exported or deployed outside Vercel. [34] [35]

Practical operating model

For Semester, use the three environments in sequence:

1. **Claude Artifacts** for rapid decision tools, dashboards, prototypes, diagrams, pitch-ready pages, prompts, and self-contained operational assets.
2. **ChatGPT Canvas** when the main job is editorial refinement, document production, code review, rewrite cycles, or conversion of content/code among file-oriented formats.
3. **v0** when a prototype needs to become an actual web app with project files, server behavior, routes, database connections, deployments, and collaboration through a code repository.

The clean handoff path is:

```
Claude artifact
→ export JSX / HTML / Mermaid / SVG / Markdown / JSON
→ place in GitHub repository
→ turn into production feature with v0 or local development
→ connect real backend, auth, analytics, tests, and deployment
```

Use **React** for this decision dashboard itself, **Mermaid** for Semester's system and data-flow maps, **SVG** for brand identity assets, **HTML** for the public marketing site or standalone demos, **Docs** for operational source-of-truth work, and **Slides** for launch, sales, board, and investor narratives.

⁂

Claude Artifacts vs ChatGPT Canvas vs v0 sandbox: compare execution limits, React support, and export formats in one matrix

Computer
Build a prompt decision tree app for Claude Artifacts, Canvas, and v0 that matches your exact coding and document workflow
Computer
Compare Claude Artifacts, ChatGPT Canvas, and v0
Explore the execution sandbox limits for each platform
See how v0 handles React and Nextjs integration

Claude Artifacts, ChatGPT Canvas, and v0 all let you generate and preview code, but they serve different stages of the workflow: **Claude Artifacts** for shareable self-contained deliverables,

Canvas for iterative documents and code editing, and **v0** for project-level React/Next.js applications that can move toward deployment.

For Semester, use Claude Artifacts to create decision tools, diagrams, dashboards, and prototype-like deliverables; use Canvas for editable documents and code refinement; use v0 when the work needs an actual Next.js project, server behavior, repository integration, or Vercel deployment. [48] [49] [50]

One comparison matrix

Dimension	Claude Artifacts	ChatGPT Canvas	v0
Primary role	Produce a polished, shareable artifact beside the chat: page, tool, dashboard, diagram, document, or prototype	Collaborative workspace for writing and coding projects that need iterative editing and execution	AI agent for creating real code, full-stack apps, and agents
Best fit	Self-contained products: calculators, interactive explainers, trackers, product demos, diagrams, visual assets, operating tools	PRDs, policies, reports, content, code review, language conversion, code snippets, and editable documents	Production-oriented web apps, SaaS interfaces, Next.js features, dashboards, authenticated flows, and integrated projects
Typical project shape	Usually a contained artifact, often a page or component rendered in Claude	A document or code-focused canvas, often a single working artifact	A multi-file project with a filesystem, dependencies, commands, routes, and deployable application structure
React support	Supports React-based artifacts; libraries can load from permitted CDNs such as `cdnjs`, `jsDelivr`, Tailwind CDN, `code.jquery.com`, and `unpkg`	Renders React/HTML code in an isolated sandbox; OpenAI says all npm packages and many JavaScript libraries work	Native fit for React and Next.js, with modern tooling such as Next.js, Tailwind, and shadcn/ui
Server-side execution	Do not assume a durable server runtime for the artifact itself; it is strongest for client-side, self-contained work	React/HTML render in an isolated sandbox; Python can run from Canvas, but Canvas is not primarily a full-stack hosting environment	Runs in an isolated VM-backed Node.js sandbox; v0 says this can run server code, API routes, and real database connections
Network/API limits	Artifact environments are constrained. When libraries are used, Anthropic documents a limited set of allowed external library hosts; production secrets and sensitive integrations should remain outside the artifact	Isolated preview environment; suitable for rendering and iteration, but production secret management and deployment belong in a dedicated application environment	Sandbox execution remains isolated, but is deliberately closer to a real development environment and can run commands, tests, platform CLIs, server code, and API routes

Dimension	Claude Artifacts	ChatGPT Canvas	v0
Package/dependency model	Prefer self-contained code or permitted CDN-loaded dependencies; dependency freedom is narrower than a normal project	Canvas supports npm packages and many JavaScript libraries in its sandboxed React/HTML environment	Project dependencies and modern app tooling are central to the product; v0 can manage a project and run it in its sandbox
Persistent data	Treat as transient unless connected to an external system; do not rely on artifact-local state as production storage	Canvas is an editing environment, not a production database layer	Can connect projects to databases and real app infrastructure, although production credentials, schemas, and security still require review
Code execution	Claude's broader code-execution environment is a separate secure container model; Anthropic documents no internet access for runtime package downloads there	Canvas supports Python execution in-browser and renders React/HTML previews in isolation	v0 can execute shell commands in an isolated sandbox to inspect repos, run tests, and use platform CLIs
Export formats	Strong source-oriented handoff: HTML, Markdown, React/JSX-style code, SVG, Mermaid, plus browser-generated downloads such as CSV or PNG	General documents: PDF, Markdown, DOCX. Code: export in its detected file type such as `.py`, `.js`, or `.sql`	Project/code export, repository import/export, Git integration, file management, and deployment-oriented handoff
Single-file constraint	Often a practical design preference: one shareable, self-contained artifact is simplest and most portable	Naturally works well for a document or code file, though it can handle project-like editing patterns	Not single-file by default; intended for full project structures
Production deployment	Export or move source to a normal codebase for real auth, backend, observability, tests, and deployment	Export the document/code and move it into the target codebase or publishing workflow	Closest to production deployment; built around app generation and supports Vercel-oriented workflows
Ideal Semester use	Artifact catalog, decision tree, ROI calculator, product prototype, architecture diagram, launch tool, visual explainer	Strategy docs, legal first drafts, PRDs, board memos, research synthesis, code changes, code conversion	A real Semester web product, admin dashboard, customer portal, authenticated app, API routes, database-backed workflow

Anthropic's documentation notes that Claude artifacts can load React and other JavaScript libraries only from specified approved hosts; if those hosts are blocked, dependent parts of the artifact fail rather than falling back. ChatGPT Canvas renders React/HTML in an isolated sandbox, supports Python execution, and offers document export to PDF, Markdown, and DOCX while code is exported using the detected language extension. v0 positions itself around real full-stack app creation, including Next.js, Tailwind, and shadcn/ui, and its sandbox

documentation says it uses a VM-backed Node.js environment capable of running server code, API routes, and database connections. [49] [50] [51] [52] [53] [54] [48]

Sandbox limits

Claude Artifacts

Claude Artifacts are best considered **client-facing, self-contained outputs**, not a replacement for a full application runtime.

- React can work, but a conservative artifact should avoid assuming arbitrary package installation, private runtime secrets, unrestricted external network calls, or durable backend state.
- If you rely on externally hosted React, charting, or UI libraries, the relevant CDN must be accessible. Anthropic lists approved hosts including `cdnjs.cloudflare.com`, `cdn.jsdelivr.net`, `cdn.tailwindcss.com`, `code.jquery.com`, and `unpkg.com`. [48]
- Keep meaningful app data in Supabase, an API, or another managed backend—not only in browser state or artifact-local state.
- For code execution through Claude's Platform tool, Anthropic documents a separate sandboxed container with no internet access, 5 GiB RAM, 5 GiB workspace storage, one CPU, and per-invocation execution limits. That environment is useful for computation and file generation but is distinct from a deployed application backend. [55]
- Claude Code can publish local `.html` or `.md` files as hosted artifact pages under the relevant availability requirements. [56]

Semester examples

Requirement	Good Claude Artifact?	Better production handoff
Filterable artifact catalog with local sample data	Yes	Export JSX/HTML; persist records in Supabase later
Interactive ROI calculator	Yes	Add analytics and lead capture in your web app
Architecture and data-flow diagrams	Yes	Store Mermaid source in GitHub
Marketing landing page prototype	Yes	Move to Next.js or static site repo for launch
Customer login, billing, multi-tenant permissions	No, not as the final implementation	v0/Next.js + Supabase/Auth + production security review
Security questionnaire or legal draft	Yes, as a draft	Legal/security review and a durable documentation system

ChatGPT Canvas

Canvas is most useful when your core loop is **draft → inspect → revise → export**.

- React and HTML render in an isolated sandbox.

- OpenAI's Help Center says Canvas supports npm packages and many JavaScript libraries for React/HTML rendering. [50]
- Python code can run directly within Canvas, with output shown in the lower console area. [52] [50]
- General text documents export as PDF, Markdown, or DOCX; code canvases export with their appropriate detected extension, such as Python, JavaScript, or SQL. [50]
- Canvas has product availability caveats. For example, OpenAI release notes say Canvas is not supported in some models and that its writing/coding functionality has been integrated into chat blocks for newer model experiences. [52]

For Semester, Canvas is an efficient place to iterate a **PRD**, legal first draft, strategy document, executive memo, data model spec, or standalone code module before handing it to GitHub or v0.

v0

v0 is the strongest choice when the artifact must become an **actual application**.

- v0 describes itself as an AI agent for real code, full-stack apps, and agents, with support for modern tools including Next.js, Tailwind, and shadcn/ui. [49]
- Its current sandbox replaced an older browser-based preview; v0 says the VM-backed Node.js environment can run server code, API routes, and real database connections. [53]
- v0 can run shell commands inside the sandbox for testing interactions, inspecting a repository, running unit tests, and calling platform CLIs such as Vercel and GitHub. [54]
- v0's API/docs also describe repository imports, file uploads, template creation, version control integration, file conflict resolution, export capability, and CI/CD tooling connections. [57]
- The sandbox is still isolated. A preview that works inside it is not automatically a hardened production deployment; test exported code, pin dependencies, validate environment variables, and run builds in the target repository.

React and Next.js

For **React-only prototypes**, all three can be useful—but at different levels.

Need	Best choice	Why
Build a single interactive decision tree from scratch	Claude Artifact	Fast, self-contained, excellent for demonstrating the workflow
Iterate visually on a React component and edit code or copy it to a repository	Canvas	Strong editing loop and code-file export
Build a multi-route product with server components, API routes, auth, database access, tests, and deployment	v0	Its environment is oriented around real Next.js project structure and execution
Turn the Semester tracker into a shared team product	v0	Create a Next.js app with Supabase persistence, role-based access, and Vercel deployment

Need	Best choice	Why
Produce an architecture diagram from the same project data	Claude Artifact	Mermaid/SVG output is a natural fit
Turn product decisions into a polished executive document	Canvas	Document workflows and PDF/DOCX export are stronger fits

Recommended Semester handoff

```
1. Claude Artifact
   Build a fast interactive tracker, calculator, diagram, or prototype.
   Export: JSX, HTML, SVG, Mermaid, JSON, CSV, or Markdown.

2. GitHub
   Store source of truth:
   - /docs for PRDs, Mermaid diagrams, policies, and decisions
   - /apps for the production interface
   - /data or Supabase migrations for persistence

3. v0
   Import or connect the repository.
   Convert the prototype into a Next.js application:
   - App Router routes
   - server actions/API routes
   - Supabase persistence
   - authentication and organization tenancy
   - tests, linting, CI, observability

4. Vercel
   Deploy preview environments and production.
```

A practical Next.js structure for the Semester catalog could be:

```
semester/
├── app/
│   ├── artifacts/
│   │   ├── page.tsx              # Filterable catalog
│   │   ├── [artifactId]/page.tsx # Detail page
│   │   └── export/route.ts       # CSV/JSON/Markdown export
│   ├── api/
│   │   └── artifacts/route.ts    # Optional REST endpoint
│   └── layout.tsx
├── components/
│   ├── artifact-table.tsx
│   ├── artifact-filters.tsx
│   ├── decision-tree.tsx
│   ├── prompt-builder.tsx
│   └── exporter.tsx
├── lib/
│   ├── artifact-schema.ts
│   ├── decision-engine.ts
│   └── exporters.ts
```

```
├── supabase/
│   └── migrations/
│       └── create_artifacts.sql
└── docs/
    ├── product-vision.md
    ├── architecture.mmd
    └── security-data-flow.mmd
```

Decision-tree app spec

The following prompt is tuned for a workflow where Claude Artifacts, Canvas, and v0 each have a defined job:

```
Build a responsive React application named “Semester AI Workflow Router.”

Purpose:
Help a software builder choose whether a task belongs in Claude Artifacts, ChatGPT Ca

Primary users:
- Founder/product builder
- Product designer
- Engineer
- Operations lead
- Marketing or GTM lead

Decision inputs:
1. Deliverable type:
   - Interactive tool or dashboard
   - Product prototype
   - Marketing site or landing page
   - Diagram or architecture map
   - Document or policy
   - Presentation or deck
   - Full-stack application feature
   - Data analysis or generated file

2. Required interaction:
   - None
   - Simple client-side interaction
   - Stateful UI with filters/forms
   - Multi-user or authenticated workflow
   - Server-side logic/API routes
   - Background jobs or database writes

3. Project complexity:
   - One self-contained file
   - A component or small prototype
   - Multi-file project
   - Production application

4. Persistence and integrations:
   - No persistence
   - Browser-only state
   - Import/export files
```

```
    - Database
    - Auth and permissions
    - Third-party APIs
    - Secrets and server-side environment variables

5. Handoff requirement:
    - Share immediately in chat
    - Export HTML/JSX/SVG/Mermaid/Markdown
    - Export PDF/DOCX/code file
    - GitHub repository
    - Vercel deployment

6. Preferred stack:
    - Plain HTML/CSS/JS
    - React
    - Next.js
    - Supabase
    - Unknown

Recommendation logic:
- Recommend Claude Artifacts for self-contained interactive pages, visual explainers
- Recommend ChatGPT Canvas for documents, editable writing, code review, language co
- Recommend v0 for real React/Next.js applications, multi-file projects, Next.js rou
- If a task begins as a Claude Artifact but requires persistence, authentication, se
- Explain that sandbox previews are not production environments and identify the spe

Output:
- Primary platform recommendation
- Secondary platform recommendation
- Confidence score
- Short explanation of why
- Sandbox constraints to consider
- Best initial output format
- Best export format
- Exact suggested workflow
- Ready-to-copy prompt for the selected platform
- “Escalate to production” checklist when v0 is recommended

UI:
- Premium B2B software aesthetic
- Step-by-step form with progress indicator
- Decision summary panel
- Platform comparison table
- Copy buttons for recommendation, prompt, and handoff plan
- Export the completed recommendation as Markdown and JSON
- Use local state and realistic sample scenarios only; no backend required
```

The key boundary is simple: **a sandbox preview proves a concept; a repository, backend, test suite, and deployment pipeline prove a product.**

⁂

Compare Claude Artifacts vs ChatGPT Canvas vs v0 sandbox in a sortable matrix — execution limits, React support, and export formats

Computer
Build a prompt decision tree app for choosing between Claude Artifacts, ChatGPT Canvas, and v0 sandbox based on your project type
Computer
Write the React component code for this decision tree app
How do I deploy a v0 project to Vercel
Compare the code export options for all three tools
Sources
63

Claude Artifacts, ChatGPT Canvas, and v0 overlap on React previews, but their operational boundaries differ sharply: Claude is best for self-contained artifacts, Canvas for iterative documents/code, and v0 for real multi-file React/Next.js apps that you can publish to Vercel.

For your Semester workflow, prototype the decision tree in Claude or Canvas, then use v0 when it needs persistence, authentication, API routes, team sharing, GitHub history, or a real deployment.

Sortable comparison matrix

This component includes the requested sortable matrix plus the decision-tree app. It is intentionally dependency-free: paste it into a React environment such as Claude Artifacts, ChatGPT Canvas, v0, or a normal React project.

```
import React, { useMemo, useState } from "react";

const PLATFORM_DATA = [
  {
    id: "claude-artifacts",
    platform: "Claude Artifacts",
    primaryUse: "Shareable self-contained artifacts, tools, diagrams, prototypes, and
    execution: "Client-side rendered artifact experience; best treated as a contained
    sandboxLimits: "Avoid relying on durable storage, private secrets, unrestricted n
    reactSupport: "Good for self-contained React interfaces and interactive client-si
    projectShape: "Often a single contained artifact or exportable component/page",
    exportFormats: "HTML, JSX-style source, Markdown, SVG, Mermaid, plus browser-gene
    productionFit: "Prototype → export source → GitHub → normal application stack",
    bestFor: "Calculators, sortable trackers, interactive explainers, diagrams, demos
    score: {
      prototype: 5,
      documents: 4,
      diagrams: 5,
      fullStack: 1,
      deployment: 2,
    },
  },
```

```
  {
    id: "chatgpt-canvas",
    platform: "ChatGPT Canvas",
    primaryUse: "Iterative writing and coding workspace for revision, editing, reviev
    execution: "React/HTML previews render in an isolated sandbox; Python can execute
    sandboxLimits: "Preview behavior can depend on workspace network access settings
    reactSupport: "React and HTML preview in a sandbox; OpenAI says many npm packages
    projectShape: "Document or code canvas; strongest for individual files and edit/r
    exportFormats: "Documents: PDF, Markdown (.md), Word (.docx). Code: language-appr
    productionFit: "Export document or code → move into repository, editor, or deploy
    bestFor: "PRDs, strategy docs, policies, research synthesis, code editing, code c
    score: {
      prototype: 3,
      documents: 5,
      diagrams: 3,
      fullStack: 1,
      deployment: 1,
    },
  },
  {
    id: "v0",
    platform: "v0",
    primaryUse: "Real code and full-stack web apps, especially React/Next.js projects
    execution: "VM-backed, isolated Node.js sandbox capable of running server code, A
    sandboxLimits: "Still an isolated development environment; validate dependencies
    reactSupport: "Native fit for React and Next.js. Designed for multi-file projects
    projectShape: "Multi-file project with dependencies, filesystem, routes, integrat
    exportFormats: "Project/code export, GitHub sync, existing-codebase integration,
    productionFit: "Best for repository-backed applications, previews, production dep
    bestFor: "Next.js apps, authenticated workflows, dashboards, API routes, database
    score: {
      prototype: 4,
      documents: 2,
      diagrams: 2,
      fullStack: 5,
      deployment: 5,
    },
  },
];

const WORKFLOW_OPTIONS = {
  deliverable: [
    ["interactive-tool", "Interactive tool, tracker, dashboard, calculator, or config
    ["product-prototype", "Clickable product prototype or interface exploration"],
    ["marketing-page", "Marketing site, landing page, one-page demo, or visual explai
    ["diagram", "Architecture, process, sequence, ERD, roadmap, or system diagram"],
    ["document", "PRD, policy, research, strategy, report, or operating document"],
    ["presentation", "Pitch, board update, sales story, launch narrative, or training
    ["full-stack-feature", "Authenticated, data-backed, multi-user product feature"]
    ["analysis", "Python-based analysis, structured research, file generation, or dat
  ],
  interaction: [
    ["none", "No interaction; people mainly read or review"],
    ["simple", "Simple browser interaction: toggles, tabs, calculators, or local data
    ["complex", "Stateful interaction: tables, filters, forms, drawers, editable data
    ["multi-user", "Multiple users, roles, shared records, or collaboration"],
```

```
  ],
  backend: [
    ["none", "No backend; static output or browser-only sample data"],
    ["file-export", "Needs generated downloads such as CSV, Markdown, SVG, or JSON"]
    ["database", "Needs persistent database records"],
    ["server-api", "Needs API routes, server-side logic, webhooks, or third-party se
    ["auth-secrets", "Needs authentication, permissions, server secrets, billing, or
  ],
  handoff: [
    ["share-now", "Share immediately as a finished artifact"],
    ["doc-export", "Export a document, PDF, DOCX, Markdown, or source file"],
    ["github", "Commit versioned source into GitHub"],
    ["deploy", "Deploy a real web application"],
  ],
};

const DEFAULT_ANSWERS = {
  deliverable: "interactive-tool",
  interaction: "complex",
  backend: "none",
  handoff: "share-now",
};

function getRecommendation(answers) {
  const scores = {
    "claude-artifacts": 0,
    "chatgpt-canvas": 0,
    v0: 0,
  };

  const reasons = {
    "claude-artifacts": [],
    "chatgpt-canvas": [],
    v0: [],
  };

  const add = (platform, points, reason) => {
    scores[platform] += points;
    reasons[platform].push(reason);
  };

  switch (answers.deliverable) {
    case "interactive-tool":
      add("claude-artifacts", 7, "You selected an interactive tool, which suits a se
      add("v0", 4, "v0 is a strong follow-on when the tool becomes a real product su
      add("chatgpt-canvas", 2, "Canvas can preview interactive React/HTML but is les
      break;
    case "product-prototype":
      add("claude-artifacts", 6, "Claude Artifacts is efficient for a contained inte
      add("v0", 5, "v0 is strong when the prototype should become a multi-file appli
      add("chatgpt-canvas", 3, "Canvas can render and iterate React/HTML prototypes.
      break;
    case "marketing-page":
      add("claude-artifacts", 7, "A shareable one-page visual artifact is a direct f
      add("v0", 5, "v0 is a good fit if the page will ship inside a Next.js site.");
      add("chatgpt-canvas", 2, "Canvas helps with copy and page-code iteration.");
```

```
      break;
    case "diagram":
      add("claude-artifacts", 8, "Claude Artifacts is ideal for Mermaid, SVG, and sha
      add("chatgpt-canvas", 3, "Canvas works for diagram source and supporting explan
      add("v0", 1, "v0 is not the primary choice unless the diagram is part of a larg
      break;
    case "document":
      add("chatgpt-canvas", 9, "Canvas is optimized for drafting, revising, and expor
      add("claude-artifacts", 4, "Claude can produce polished standalone document art
      add("v0", 1, "v0 is unnecessary unless the document is part of an application."
      break;
    case "presentation":
      add("chatgpt-canvas", 6, "Canvas fits narrative editing and document-style expo
      add("claude-artifacts", 5, "Claude works well for visual, shareable deck-like a
      add("v0", 2, "v0 only makes sense if the presentation is a web experience.");
      break;
    case "full-stack-feature":
      add("v0", 10, "A full-stack feature needs a project structure, server logic, an
      add("claude-artifacts", 1, "Claude can prototype the interface before implement
      add("chatgpt-canvas", 1, "Canvas can help draft specifications and review modul
      break;
    case "analysis":
      add("chatgpt-canvas", 8, "Canvas supports iterative code and Python execution w
      add("claude-artifacts", 3, "Claude can create a shareable visualization or resu
      add("v0", 3, "v0 can be useful when analysis becomes a deployed app.");
      break;
    default:
      break;
  }

  switch (answers.interaction) {
    case "none":
      add("chatgpt-canvas", 3, "The output is primarily editorial or read-only.");
      add("claude-artifacts", 2, "A contained static artifact remains viable.");
      break;
    case "simple":
      add("claude-artifacts", 5, "Simple client-side interaction fits an artifact wel
      add("chatgpt-canvas", 3, "Canvas can preview lightweight React/HTML interaction
      add("v0", 2, "v0 can handle it, though it may be more infrastructure than neede
      break;
    case "complex":
      add("claude-artifacts", 5, "A stateful client-side interface can work as a prot
      add("v0", 6, "Complex interfaces benefit from a project-level React/Next.js coc
      add("chatgpt-canvas", 3, "Canvas can preview React but is less oriented around
      break;
    case "multi-user":
      add("v0", 10, "Multi-user interaction requires real identity, authorization, pe
      break;
    default:
      break;
  }

  switch (answers.backend) {
    case "none":
      add("claude-artifacts", 4, "No backend is required, favoring a self-contained a
      add("chatgpt-canvas", 3, "An editor/preview workflow can be sufficient.");
```

```
      break;
    case "file-export":
      add("claude-artifacts", 5, "Artifacts are well suited to client-side generated
      add("chatgpt-canvas", 5, "Canvas directly supports document and language-speci
      add("v0", 3, "v0 can also create export features inside a real app.");
      break;
    case "database":
      add("v0", 8, "Persistent records call for a real project with database integra
      add("claude-artifacts", 1, "Use Claude for prototype data only, then hand off.
      break;
    case "server-api":
      add("v0", 9, "Server routes and external services favor v0's application envir
      break;
    case "auth-secrets":
      add("v0", 10, "Authentication, permissions, billing, and secrets require a pro
      break;
    default:
      break;
  }

  switch (answers.handoff) {
    case "share-now":
      add("claude-artifacts", 6, "Immediate sharing favors a finished visual artifac
      add("chatgpt-canvas", 3, "Canvas assets can also be shared.");
      break;
    case "doc-export":
      add("chatgpt-canvas", 7, "Canvas supports direct PDF, DOCX, Markdown, and code
      add("claude-artifacts", 4, "Artifact source/export forms work well for HTML, S
      break;
    case "github":
      add("v0", 7, "v0 is built for repository-backed application workflows.");
      add("claude-artifacts", 3, "Claude output can be exported and committed to a re
      add("chatgpt-canvas", 3, "Canvas code can be exported into a repository.");
      break;
    case "deploy":
      add("v0", 10, "Deployment is v0's strongest handoff path through Vercel.");
      add("claude-artifacts", 1, "Claude should hand off source rather than serve as
      break;
    default:
      break;
  }

  const ranked = Object.entries(scores)
    .map(([id, score]) => ({
      ...PLATFORM_DATA.find((platform) => platform.id === id),
      score,
      reasons: reasons[id],
    }))
    .sort((a, b) => b.score - a.score);

  const primary = ranked[^6_0];
  const secondary = ranked[^6_1];

  const escalation =
    answers.backend === "database" ||
    answers.backend === "server-api" ||
```

```
    answers.backend === "auth-secrets" ||
    answers.interaction === "multi-user" ||
    answers.handoff === "deploy";

  return {
    primary,
    secondary,
    ranked,
    escalation,
    confidence: Math.min(
      96,
      Math.max(62, 68 + (primary.score - secondary.score) * 4 + (escalation && prima
    ),
  };
}

function copyToClipboard(text) {
  return navigator.clipboard?.writeText(text);
}

function CopyButton({ value, label = "Copy" }) {
  const [copied, setCopied] = useState(false);

  const handleClick = async () => {
    try {
      await copyToClipboard(value);
      setCopied(true);
      window.setTimeout(() => setCopied(false), 1400);
    } catch {
      setCopied(false);
    }
  };

  return (
    <button
      onClick={handleClick}
      className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 text-xs font
    >
      {copied ? "Copied" : label}
    </button>
  );
}

function Badge({ children, className = "" }) {
  return (
    <span className={`inline-flex items-center rounded-full border px-2 py-0.5 text-x
      {children}
    </span>
  );
}

function PlatformBadge({ platform }) {
  const styles = {
    "Claude Artifacts": "border-amber-200 bg-amber-50 text-amber-800",
    "ChatGPT Canvas": "border-emerald-200 bg-emerald-50 text-emerald-800",
    v0: "border-slate-800 bg-slate-950 text-white",
```

```
  };

  return <Badge className={styles[platform]}>{platform}</Badge>;
}

function exportJson(filename, data) {
  const blob = new Blob([JSON.stringify(data, null, 2)], { type: "application/json;cl
  const href = URL.createObjectURL(blob);
  const anchor = document.createElement("a");
  anchor.href = href;
  anchor.download = filename;
  anchor.click();
  URL.revokeObjectURL(href);
}

function exportMarkdown(filename, content) {
  const blob = new Blob([content], { type: "text/markdown;charset=utf-8" });
  const href = URL.createObjectURL(blob);
  const anchor = document.createElement("a");
  anchor.href = href;
  anchor.download = filename;
  anchor.click();
  URL.revokeObjectURL(href);
}

function generatePrompt(recommendation, answers) {
  const platform = recommendation.primary.platform;
  const fallback = recommendation.secondary.platform;

  const instructions = {
    "Claude Artifacts":
      "Create a polished, self-contained artifact. Keep the result client-side, depen
    "ChatGPT Canvas":
      "Create an editable canvas optimized for iteration and revision. Use clear docu
    v0:
      "Create a production-oriented Next.js application. Use a clean multi-file struc
  };

  return `Build this for Semester.

Recommended platform: ${platform}
Secondary option: ${fallback}
Confidence: ${recommendation.confidence}%

Project type: ${answers.deliverable}
Interaction level: ${answers.interaction}
Backend requirement: ${answers.backend}
Handoff target: ${answers.handoff}

${instructions[platform]}

Required output:
- Explain the architecture and key assumptions.
- Build the primary workflow using realistic sample data.
- Include a concise “how to run or use this” section.
- State the export/handoff path.
```

```
${
  recommendation.escalation
    ? "- Include an escalation-to-production checklist: auth, authorization, persiste
    : ""
}

For Semester, optimize for a clean premium B2B software experience and make the deliv
}

export default function SemesterWorkflowRouter() {
  const [answers, setAnswers] = useState(DEFAULT_ANSWERS);
  const [sort, setSort] = useState({ key: "platform", direction: "asc" });
  const [selectedPlatform, setSelectedPlatform] = useState(null);

  const recommendation = useMemo(() => getRecommendation(answers), [answers]);
  const generatedPrompt = useMemo(
    () => generatePrompt(recommendation, answers),
    [recommendation, answers]
  );

  const sortedPlatforms = useMemo(() => {
    return [...PLATFORM_DATA].sort((a, b) => {
      const aValue = a[sort.key] ?? "";
      const bValue = b[sort.key] ?? "";

      if (typeof aValue === "object") return 0;

      return sort.direction === "asc"
        ? String(aValue).localeCompare(String(bValue))
        : String(bValue).localeCompare(String(aValue));
    });
  }, [sort]);

  const setAnswer = (key, value) => {
    setAnswers((current) => ({ ...current, [key]: value }));
  };

  const toggleSort = (key) => {
    setSort((current) => ({
      key,
      direction: current.key === key && current.direction === "asc" ? "desc" : "asc"
    }));
  };

  const header = (label, key) => (
    <button
      onClick={() => toggleSort(key)}
      className="flex items-center gap-1 text-left text-xs font-bold uppercase tracki
    >
      {label}
      {sort.key === key ? (sort.direction === "asc" ? "↑" : "↓") : "↕"}
    </button>
  );

  const decisionMarkdown = `# Semester AI Workflow Recommendation
```

```
## Recommendation
- Primary platform: ${recommendation.primary.platform}
- Secondary platform: ${recommendation.secondary.platform}
- Confidence: ${recommendation.confidence}%
- Production escalation required: ${recommendation.escalation ? "Yes" : "No"}

## Requirements
- Deliverable: ${answers.deliverable}
- Interaction: ${answers.interaction}
- Backend: ${answers.backend}
- Handoff: ${answers.handoff}

## Why
${recommendation.primary.reasons.map((reason) => `- ${reason}`).join("\n")}

## Ready-to-use prompt
\`\`\`
${generatedPrompt}
\`\`\`
`;

  return (
    <main className="min-h-screen bg-slate-50 p-4 text-slate-900 md:p-8">
      <div className="mx-auto max-w-7xl space-y-6">
        <header className="rounded-2xl bg-gradient-to-br from-slate-950 via-slate-900
          <p className="text-xs font-bold uppercase tracking-[0.16em] text-indigo-300
            Semester build workflow
          </p>
          <h1 className="mt-2 text-3xl font-bold tracking-tight">
            Claude Artifacts vs Canvas vs v0 Router
          </h1>
          <p className="mt-2 max-w-3xl text-sm leading-6 text-slate-300">
            Compare execution constraints, React fit, export paths, and deployment re
          </p>
        </header>

        <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
              Decision tree
            </p>
            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Choose the right environment for the project
            </h2>
          </div>

          <div className="mt-5 grid gap-4 md:grid-cols-2 xl:grid-cols-4">
            {Object.entries(WORKFLOW_OPTIONS).map(([key, choices]) => (
              <fieldset key={key} className="rounded-xl border border-slate-200 bg-sl
                <legend className="px-1 text-sm font-bold capitalize text-slate-900">
                  {key === "backend" ? "Data and backend" : key}
                </legend>
                <div className="mt-2 space-y-2">
                  {choices.map(([value, label]) => (
                    <label key={value} className="flex cursor-pointer items-start gap
                      <input
```

```
                    type="radio"
                    name={key}
                    value={value}
                    checked={answers[key] === value}
                    onChange={() => setAnswer(key, value)}
                    className="mt-1 accent-indigo-600"
                  />
                  <span>{label}</span>
                </label>
              ))}
            </div>
          </fieldset>
        ))}
      </div>

      <div className="mt-5 rounded-2xl border border-indigo-200 bg-indigo-50 p-5"
        <div className="flex flex-col justify-between gap-4 lg:flex-row lg:items-
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indi
              Primary recommendation
            </p>
            <div className="mt-2 flex flex-wrap items-center gap-2">
              <h3 className="text-2xl font-bold text-slate-950">
                {recommendation.primary.platform}
              </h3>
              <PlatformBadge platform={recommendation.primary.platform} />
              <Badge className="border-indigo-200 bg-white text-indigo-700">
                {recommendation.confidence}% confidence
              </Badge>
            </div>

            <ul className="mt-4 max-w-3xl space-y-2 text-sm leading-6 text-slate-
              {recommendation.primary.reasons.map((reason) => (
                <li key={reason}>• {reason}</li>
              ))}
            </ul>
          </div>

          <div className="rounded-xl border border-indigo-100 bg-white p-4 lg:w-8
            <p className="text-xs font-bold uppercase tracking-wide text-slate-50
              Secondary option
            </p>
            <p className="mt-1 text-base font-bold text-slate-900">
              {recommendation.secondary.platform}
            </p>
            <p className="mt-2 text-sm leading-6 text-slate-600">
              {recommendation.secondary.primaryUse}
            </p>
          </div>
        </div>

        {recommendation.escalation && (
          <div className="mt-5 rounded-xl border border-amber-200 bg-amber-50 p-4
            <p className="text-sm font-bold text-amber-900">
              Production handoff required
            </p>
```

```
              <p className="mt-1 text-sm leading-6 text-amber-800">
                Your choices require persistence, APIs, authentication, secrets, mu
              </p>
            </div>
          )}
        </div>
      </section>

      <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
        <div className="flex flex-col justify-between gap-3 md:flex-row md:items-en
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo
              Prompt generator
            </p>
            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Ready-to-copy build prompt
            </h2>
          </div>
          <div className="flex flex-wrap gap-2">
            <CopyButton value={generatedPrompt} label="Copy prompt" />
            <button
              onClick={() => exportMarkdown("semester-workflow-recommendation.md",
              className="rounded-lg bg-slate-900 px-3 py-1.5 text-xs font-bold tex
            >
              Export Markdown
            </button>
            <button
              onClick={() =>
                exportJson("semester-workflow-recommendation.json", {
                  answers,
                  recommendation: {
                    primary: recommendation.primary.platform,
                    secondary: recommendation.secondary.platform,
                    confidence: recommendation.confidence,
                    escalation: recommendation.escalation,
                    reasons: recommendation.primary.reasons,
                  },
                  generatedPrompt,
                })
              }
              className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 te
            >
              Export JSON
            </button>
          </div>
        </div>

        <pre className="mt-4 whitespace-pre-wrap rounded-xl bg-slate-950 p-4 text-x
          {generatedPrompt}
        </pre>
      </section>

      <section className="overflow-hidden rounded-2xl border border-slate-200 bg-wh
        <div className="border-b border-slate-200 p-5">
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
            Platform matrix
```

```
            </p>
            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Sortable execution, React, and export comparison
            </h2>
            <p className="mt-1 text-sm text-slate-600">
              Click an artifact row for a full operational profile.
            </p>
          </div>

          <div className="overflow-x-auto">
            <table className="w-full min-w-[1450px] border-collapse">
              <thead className="bg-slate-50">
                <tr>
                  <th className="px-4 py-3">{header("Platform", "platform")}</th>
                  <th className="px-4 py-3">{header("Primary use", "primaryUse")}</t
                  <th className="px-4 py-3">{header("Execution", "execution")}</th>
                  <th className="px-4 py-3">{header("React support", "reactSupport")
                  <th className="px-4 py-3">{header("Project shape", "projectShape")
                  <th className="px-4 py-3">{header("Export formats", "exportFormats"
                  <th className="px-4 py-3">{header("Production fit", "productionFit"
                  <th className="px-4 py-3 text-left text-xs font-bold uppercase tra
                    Details
                  </th>
                </tr>
              </thead>

              <tbody>
                {sortedPlatforms.map((platform) => (
                  <tr
                    key={platform.id}
                    className="border-t border-slate-100 align-top transition hover:b
                  >
                    <td className="px-4 py-4">
                      <div className="font-bold text-slate-950">{platform.platform}</
                      <div className="mt-2 flex gap-1">
                        <Badge className="border-slate-200 bg-slate-50 text-slate-600
                          Prototype {platform.score.prototype}/5
                        </Badge>
                        <Badge className="border-slate-200 bg-slate-50 text-slate-600
                          Deploy {platform.score.deployment}/5
                        </Badge>
                      </div>
                    </td>
                    <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    <td className="px-4 py-4">
                      <button
                        onClick={() => setSelectedPlatform(platform)}
                        className="rounded-lg border border-slate-200 bg-white px-3 p
                      >
                        Open
                      </button>
```

```
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </section>

        {selectedPlatform && (
          <div className="fixed inset-0 z-50 flex justify-end bg-slate-950/30">
            <button
              aria-label="Close platform details"
              className="absolute inset-0 cursor-default"
              onClick={() => setSelectedPlatform(null)}
            />
            <aside className="relative h-full w-full max-w-2xl overflow-y-auto bg-wh
              <div className="flex items-start justify-between gap-4">
                <div>
                  <PlatformBadge platform={selectedPlatform.platform} />
                  <h2 className="mt-3 text-2xl font-bold text-slate-950">
                    {selectedPlatform.platform}
                  </h2>
                </div>
                <button
                  onClick={() => setSelectedPlatform(null)}
                  className="rounded-lg border border-slate-200 px-3 py-1.5 text-sm
                >
                  Close
                </button>
              </div>

              <div className="mt-6 space-y-5">
                {[
                  ["Primary use", selectedPlatform.primaryUse],
                  ["Execution model", selectedPlatform.execution],
                  ["Sandbox limits", selectedPlatform.sandboxLimits],
                  ["React support", selectedPlatform.reactSupport],
                  ["Project shape", selectedPlatform.projectShape],
                  ["Code/export options", selectedPlatform.exportFormats],
                  ["Production handoff", selectedPlatform.productionFit],
                  ["Best use cases", selectedPlatform.bestFor],
                ].map(([label, value]) => (
                  <div key={label}>
                    <h3 className="text-sm font-bold text-slate-900">{label}</h3>
                    <p className="mt-1 text-sm leading-6 text-slate-600">{value}</p>
                  </div>
                ))}
              </div>
            </aside>
          </div>
        )}
      </div>
    </main>
  );
}
```

Execution sandbox limits

The important distinction is not "can it preview code?"—all three can—but **whether the preview is the same thing as a deployable application environment.**

Platform	What executes	Practical limit	Semester implication
Claude Artifacts	Primarily a rendered, self-contained client-side artifact	Do not depend on it as your permanent backend, database, secret store, authentication layer, or unrestricted package/network environment	Great for a first version of the artifact tracker, ROI calculator, visual prototype, or decision tool
ChatGPT Canvas	React/HTML preview in a sandbox; Python can run inside Canvas	External resources and packages can depend on Canvas network settings; Canvas is mainly a revision/export workspace rather than an app-hosting system	Great for PRDs, legal/ops drafts, code iteration, a data-analysis helper, or a single component
v0	VM-backed Node.js sandbox with project files and commands	Still validate build, dependencies, environment variables, permissions, data rules, and production security after preview	Best for turning the Semester tracker into an authenticated, multi-user, Supabase-backed Next.js product

Claude documentation says artifact environments can load React and other libraries from specified sources, including `cdnjs`, jsDelivr, Tailwind CDN, `code.jquery.com`, and `unpkg`; this means a library-dependent artifact can fail if the required host is unavailable or blocked. In the separate Claude code-execution environment, Anthropic documents a secure sandboxed container with no internet access for runtime package downloads, so it should not be confused with a production server. [64] [65]

OpenAI says ChatGPT Canvas renders React/HTML in a sandbox; many npm packages and JavaScript libraries work, but previews needing external packages or web resources can depend on workspace Canvas network-access settings. v0 documents an isolated, VM-backed Node.js sandbox that replaced an earlier browser-only preview and can run server code, API routes, and real database connections. [66] [67] [68]

Code export options

Tool	Code handoff	Document handoff	Best export workflow
Claude Artifacts	Export or copy HTML, React/JSX-style code, SVG, Mermaid, and generated browser files such as CSV/PNG	Markdown or rendered artifact content	Copy source into GitHub; convert a prototype into a proper application project when it needs real infrastructure
ChatGPT Canvas	Exports code based on recognized language, including extensions such as `.js`, `.py`, and `.sql`	PDF, Markdown, and DOCX for general documents	Use Canvas for drafting/revision, then export a file into your repository or documentation system
v0	Export project code, connect/sync GitHub, or pull generated components into an existing codebase	Application source plus repository/deployment artifacts rather than document-first exports	Connect GitHub, work through branches/PRs, and publish through Vercel

ChatGPT Canvas officially supports PDF, Markdown, and Word export for general documents; code canvases detect the language and export with an appropriate extension such as `.py`, `.js`, or `.sql`. Canvas assets—including rendered React/HTML, documents, and code—can also be shared through the Canvas toolbar in supported plans. [69] [70] [66]

v0 can export generated components to an existing codebase and supports bidirectional Git integration; its docs describe connecting a project to GitHub, creating a repository from the current project code, using isolated branches and previews, and publishing after review. [71] [72]

Deploy v0 to Vercel

The fastest route is direct publish from v0. For a durable software workflow, connect GitHub first and publish from a reviewed branch.

Direct publish

1. Build and preview the project in v0.
2. Verify key flows: responsive UI, empty/error states, navigation, and sample data.
3. In v0, select **Share** and open the **Publish** tab.
4. Choose **Publish**.
5. Confirm the domain and deployment visibility.
6. v0 publishes the application on Vercel's platform. [72]

GitHub-backed publish

1. Open the v0 project menu: `...` → **Settings** → **GitHub**.
2. Select **Connect**.
3. Choose the GitHub account or organization under **Git Scope**.
4. Enter a repository name and select **Create Repository**. v0 creates a private repository and pushes the existing project code. [71]
5. Work on an isolated branch in v0 and inspect the branch preview.
6. Review the CI status and code changes.
7. Select **Publish**, confirm the target domain and visibility, then publish. [71]

Before production

Do these even if the preview looks correct:

- Replace mock data with a real data layer, such as Supabase.
- Store keys only in Vercel environment variables; never ship them in client-side code.
- Add authentication and row-level authorization for any shared Semester data.
- Validate server inputs and API responses.
- Add loading, retry, error, and empty states.

- Run typechecking, linting, and production builds.
- Add monitoring, analytics, and error tracking.
- Set a preview-deployment workflow for pull requests.
- Review accessibility, mobile layouts, cookies/privacy, and security assumptions.

Recommended Semester workflow

```
Claude Artifact
  → Build the initial tracker, ROI tool, diagram, or decision tree
  → Export JSX / HTML / Mermaid / SVG / JSON / CSV

ChatGPT Canvas
  → Produce PRDs, legal/operations first drafts, strategy docs, code rewrites
  → Export DOCX / PDF / Markdown / code file

GitHub + v0
  → Create a Next.js project
  → Add Supabase, auth, API routes, persistent data, and tests
  → Publish a preview, review changes, then deploy to Vercel
```

Use the component above as the **front-door decision tool.** When it recommends v0 because you selected a database, server API, auth/secrets, multi-user workflow, or deployment, treat the artifact as a product specification or UI prototype—not the final production implementation.

⁂

Compare Claude Artifacts vs ChatGPT Canvas vs v0 sandbox in a sortable matrix — execution limits, React support, and export formats

Computer
Build a prompt decision tree app for choosing between Claude Artifacts, ChatGPT Canvas, and v0 sandbox based on project scope and tech stack
Computer
Show me the rest of the code block
Add GitHub push steps to the prompt decision tree
Can v0 handle multi-file Next.js apps with Postgres

Yes—**v0 can handle multi-file Next.js applications backed by Postgres**, including server-side routes/actions and database-connected workflows. The right architecture for Semester is: prototype the decision tree in Claude or Canvas if useful, then use v0 with a GitHub repository and a managed Postgres provider such as Supabase, Neon, or Prisma Postgres for the durable app. [83] [84]

The missing code block

Below is the rest of the decision-tree app code, expanded to include:

- A sortable platform matrix
- Claude Artifacts vs ChatGPT Canvas vs v0 comparison
- Project-scope and tech-stack decision logic
- A generated build prompt
- GitHub push instructions that change based on the recommendation
- A production escalation checklist
- JSON and Markdown export of the recommendation

Paste it into a React/Next.js client component, a Claude React artifact, Canvas, or v0. For **Next.js App Router**, save it as `app/workflow-router/page.tsx` and add `"use client";` at the top.

```
"use client";

import React, { useMemo, useState } from "react";

type PlatformId = "claude-artifacts" | "chatgpt-canvas" | "v0";

type Answers = {
  deliverable:
    | "interactive-tool"
    | "product-prototype"
    | "marketing-page"
    | "diagram"
    | "document"
    | "presentation"
    | "full-stack-feature"
    | "analysis";
  interaction: "none" | "simple" | "complex" | "multi-user";
  backend: "none" | "file-export" | "database" | "server-api" | "auth-secrets";
  techStack: "none" | "html" | "react" | "nextjs" | "nextjs-postgres" | "nextjs-supal
  handoff: "share-now" | "doc-export" | "github" | "deploy";
};

type Platform = {
  id: PlatformId;
  platform: string;
  primaryUse: string;
  execution: string;
  sandboxLimits: string;
  reactSupport: string;
  projectShape: string;
  exportFormats: string;
  githubWorkflow: string;
  productionFit: string;
  bestFor: string;
  scores: {
    prototype: number;
    documents: number;
```

```
    diagrams: number;
    fullStack: number;
    deployment: number;
  };
};

const PLATFORM_DATA: Platform[] = [
  {
    id: "claude-artifacts",
    platform: "Claude Artifacts",
    primaryUse:
      "Shareable self-contained artifacts, interactive tools, diagrams, visual expla
    execution:
      "Rendered artifact experience, generally optimized for contained client-side ou
    sandboxLimits:
      "Do not assume durable database storage, server-side secrets, unrestricted API
    reactSupport:
      "Good for self-contained React interfaces, local-state dashboards, calculators
    projectShape:
      "Usually one contained artifact, component, or page that can be copied into a r
    exportFormats:
      "HTML, React/JSX-style source, Markdown, SVG, Mermaid, plus browser-generated e
    githubWorkflow:
      "Copy or export source files, create a repository, then commit the extracted HT
    productionFit:
      "Prototype → export source → GitHub → Next.js or normal application stack.",
    bestFor:
      "Semester artifact catalog prototype, format decision tree, ROI calculator, Mer
    scores: {
      prototype: 5,
      documents: 4,
      diagrams: 5,
      fullStack: 1,
      deployment: 2,
    },
  },
  {
    id: "chatgpt-canvas",
    platform: "ChatGPT Canvas",
    primaryUse:
      "Iterative workspace for documents and code that benefit from editing, revisior
    execution:
      "React and HTML preview in an isolated sandbox; Python can execute in Canvas fo
    sandboxLimits:
      "Treat Canvas as an editing and preview environment, not a durable application
    reactSupport:
      "React and HTML can render in Canvas previews, with support for many npm packag
    projectShape:
      "Document or code canvas, typically strongest for individual files, structured
    exportFormats:
      "Documents: PDF, DOCX, Markdown. Code: language-appropriate export, such as Jav
    githubWorkflow:
      "Export code or Markdown, add it to a local repository, then push through GitHu
    productionFit:
      "Draft/revise/export → GitHub or documentation system → production application
    bestFor:
```

```
      "Semester PRDs, strategy, policies, research synthesis, legal first drafts, coo
    scores: {
      prototype: 3,
      documents: 5,
      diagrams: 3,
      fullStack: 1,
      deployment: 1,
    },
  },
  {
    id: "v0",
    platform: "v0",
    primaryUse:
      "Production-oriented React and Next.js applications, multi-file projects, real
    execution:
      "VM-backed isolated Node.js sandbox that can work with multi-file projects, ser
    sandboxLimits:
      "The sandbox is still not production. Validate environment variables, package v
    reactSupport:
      "Native fit for React and Next.js, including multi-file components, App Router
    projectShape:
      "Multi-file repository-style application with dependencies, routes, components
    exportFormats:
      "Application project code, GitHub repository sync/export, existing-codebase int
    githubWorkflow:
      "Connect GitHub from v0, create or select a repository, work on isolated branch
    productionFit:
      "Best option for repository-backed applications, Postgres/Supabase integration
    bestFor:
      "Semester customer portal, authenticated artifact tracker, team dashboard, Next
    scores: {
      prototype: 4,
      documents: 2,
      diagrams: 2,
      fullStack: 5,
      deployment: 5,
    },
  },
];

const OPTIONS = {
  deliverable: [
    ["interactive-tool", "Interactive tool, tracker, dashboard, calculator, or config
    ["product-prototype", "Clickable product prototype or interface exploration"],
    ["marketing-page", "Marketing site, landing page, one-page demo, or visual explai
    ["diagram", "Architecture, process, ERD, sequence, roadmap, or system diagram"],
    ["document", "PRD, policy, research, strategy, report, or operating document"],
    ["presentation", "Pitch, board update, sales story, launch narrative, or training
    ["full-stack-feature", "Authenticated, data-backed, multi-user product feature"]
    ["analysis", "Python analysis, structured research, file generation, or data clea
  ],
  interaction: [
    ["none", "No interaction; readers mainly review content"],
    ["simple", "Simple client-side interaction: tabs, toggles, calculators, local dat
    ["complex", "Stateful UI: tables, filters, forms, drawers, editable workflows"],
    ["multi-user", "Multiple users, shared records, permissions, or collaboration"],
```

```
  ],
  backend: [
    ["none", "No backend; static result or browser-only sample data"],
    ["file-export", "Needs generated CSV, Markdown, SVG, JSON, PDF, or related downlo
    ["database", "Needs persistent database records"],
    ["server-api", "Needs API routes, webhooks, server-side logic, or third-party ser
    ["auth-secrets", "Needs auth, permissions, billing, private APIs, or server secre
  ],
  techStack: [
    ["none", "No code-stack requirement"],
    ["html", "Plain HTML, CSS, and JavaScript"],
    ["react", "React component or client-side application"],
    ["nextjs", "Multi-file Next.js application"],
    ["nextjs-postgres", "Next.js plus Postgres"],
    ["nextjs-supabase", "Next.js plus Supabase/Postgres/Auth"],
  ],
  handoff: [
    ["share-now", "Share immediately as a finished artifact"],
    ["doc-export", "Export PDF, DOCX, Markdown, or a source file"],
    ["github", "Commit versioned source into GitHub"],
    ["deploy", "Deploy a real web application"],
  ],
} as const;

const DEFAULT_ANSWERS: Answers = {
  deliverable: "interactive-tool",
  interaction: "complex",
  backend: "none",
  techStack: "react",
  handoff: "share-now",
};

function getPlatformById(id: PlatformId) {
  return PLATFORM_DATA.find((platform) => platform.id === id)!;
}

function recommendPlatform(answers: Answers) {
  const score: Record<PlatformId, number> = {
    "claude-artifacts": 0,
    "chatgpt-canvas": 0,
    v0: 0,
  };

  const reasons: Record<PlatformId, string[]> = {
    "claude-artifacts": [],
    "chatgpt-canvas": [],
    v0: [],
  };

  const add = (platform: PlatformId, points: number, reason: string) => {
    score[platform] += points;
    reasons[platform].push(reason);
  };

  switch (answers.deliverable) {
    case "interactive-tool":
```

```
      add("claude-artifacts", 7, "A self-contained interactive tool is a direct fit
      add("v0", 4, "v0 is a strong next step if the tool becomes a shared production
      add("chatgpt-canvas", 2, "Canvas can preview React/HTML but is less focused on
      break;
    case "product-prototype":
      add("claude-artifacts", 6, "Claude is fast for a contained interactive prototy
      add("v0", 5, "v0 is strong if the prototype will evolve into the production ap
      add("chatgpt-canvas", 3, "Canvas can iterate and preview React/HTML prototype
      break;
    case "marketing-page":
      add("claude-artifacts", 7, "A shareable visual page is an excellent Claude Art
      add("v0", 5, "v0 is appropriate when the page should ship inside a Next.js site
      add("chatgpt-canvas", 2, "Canvas helps with copy and smaller front-end iterati
      break;
    case "diagram":
      add("claude-artifacts", 8, "Claude is ideal for Mermaid, SVG, visual explainers
      add("chatgpt-canvas", 3, "Canvas works well for diagram source plus explanatory
      add("v0", 1, "v0 is best only if the diagram belongs inside a larger product.")
      break;
    case "document":
      add("chatgpt-canvas", 9, "Canvas is optimized for drafting, revising, and expor
      add("claude-artifacts", 4, "Claude can create polished standalone document art
      add("v0", 1, "v0 is unnecessary unless the document belongs in an application.'
      break;
    case "presentation":
      add("chatgpt-canvas", 6, "Canvas supports an editorial and export-focused prese
      add("claude-artifacts", 5, "Claude is useful for visual, shareable deck-like pr
      add("v0", 2, "v0 makes sense only when the deck itself is an interactive web ex
      break;
    case "full-stack-feature":
      add("v0", 10, "A full-stack feature needs project structure, server logic, and
      add("claude-artifacts", 1, "Claude can still prototype the interaction before i
      add("chatgpt-canvas", 1, "Canvas can help write the specification or review inc
      break;
    case "analysis":
      add("chatgpt-canvas", 8, "Canvas is a strong fit for iterative Python and file-
      add("claude-artifacts", 3, "Claude can package the analysis into a shareable vi
      add("v0", 3, "v0 is useful if analysis becomes an app for ongoing team use.");
      break;
  }

  switch (answers.interaction) {
    case "none":
      add("chatgpt-canvas", 3, "A mostly read-only deliverable favors an editorial wo
      add("claude-artifacts", 2, "A self-contained static artifact remains viable.")
      break;
    case "simple":
      add("claude-artifacts", 5, "Simple browser-side interaction fits a Claude Arti
      add("chatgpt-canvas", 3, "Canvas can preview lightweight React or HTML interac
      add("v0", 2, "v0 can handle this, but may add unnecessary project overhead.");
      break;
    case "complex":
      add("claude-artifacts", 5, "Claude can handle a stateful client-side prototype
      add("v0", 6, "Complex UI benefits from a durable multi-file React/Next.js proje
      add("chatgpt-canvas", 3, "Canvas can preview the code but is less app-lifecycle
      break;
```

```
    case "multi-user":
      add("v0", 10, "Multi-user workflows require identity, authorization, persisten
      break;
  }

  switch (answers.backend) {
    case "none":
      add("claude-artifacts", 4, "No backend is required, favoring a portable self-c
      add("chatgpt-canvas", 3, "A document or code preview workflow can be sufficien
      break;
    case "file-export":
      add("claude-artifacts", 5, "Claude artifacts work well for browser-generated d
      add("chatgpt-canvas", 5, "Canvas is strong for PDF, DOCX, Markdown, and code-f
      add("v0", 3, "v0 can add export functions inside a production app.");
      break;
    case "database":
      add("v0", 8, "Persistent records need a real app connected to a database.");
      add("claude-artifacts", 1, "Prototype data can exist in Claude before the prod
      break;
    case "server-api":
      add("v0", 9, "API routes and server-side integrations favor a Next.js project
      break;
    case "auth-secrets":
      add("v0", 10, "Auth, permissions, billing, secrets, and private integrations r
      break;
  }

  switch (answers.techStack) {
    case "html":
      add("claude-artifacts", 5, "A single HTML/CSS/JS deliverable is ideal for a po
      add("chatgpt-canvas", 2, "Canvas can edit and preview the file.");
      break;
    case "react":
      add("claude-artifacts", 5, "React works well for contained interactive artifac
      add("chatgpt-canvas", 4, "Canvas supports React/HTML preview and code iteratio
      add("v0", 5, "v0 is a natural fit when React needs multi-file project structure
      break;
    case "nextjs":
      add("v0", 9, "A multi-file Next.js app is squarely in v0's project-oriented wo
      break;
    case "nextjs-postgres":
      add("v0", 10, "Next.js plus Postgres requires real environment variables, serve
      break;
    case "nextjs-supabase":
      add("v0", 10, "Next.js plus Supabase commonly requires auth, database, row-leve
      break;
  }

  switch (answers.handoff) {
    case "share-now":
      add("claude-artifacts", 6, "Immediate sharing favors a completed visual artifa
      add("chatgpt-canvas", 3, "Canvas output can be shared or exported for review."
      break;
    case "doc-export":
      add("chatgpt-canvas", 7, "Canvas directly supports PDF, DOCX, Markdown, and la
      add("claude-artifacts", 4, "Claude supports source-style handoffs such as HTML
```

```
      break;
    case "github":
      add("v0", 7, "v0 is designed for repository-backed code workflows.");
      add("claude-artifacts", 3, "Claude output can become source committed to GitHu
      add("chatgpt-canvas", 3, "Canvas code can be exported into GitHub.");
      break;
    case "deploy":
      add("v0", 10, "A deployment target strongly favors v0 and Vercel.");
      add("claude-artifacts", 1, "Use Claude for the prototype, then move source into
      break;
  }

  const ranked = (Object.keys(score) as PlatformId[])
    .map((id) => ({
      ...getPlatformById(id),
      score: score[id],
      reasons: reasons[id],
    }))
    .sort((a, b) => b.score - a.score);

  const primary = ranked[^7_0];
  const secondary = ranked[^7_1];

  const requiresProductionEscalation =
    answers.backend === "database" ||
    answers.backend === "server-api" ||
    answers.backend === "auth-secrets" ||
    answers.interaction === "multi-user" ||
    answers.techStack === "nextjs" ||
    answers.techStack === "nextjs-postgres" ||
    answers.techStack === "nextjs-supabase" ||
    answers.handoff === "deploy";

  return {
    primary,
    secondary,
    ranked,
    requiresProductionEscalation,
    confidence: Math.min(
      97,
      Math.max(
        62,
        68 +
          (primary.score - secondary.score) * 4 +
          (requiresProductionEscalation && primary.id === "v0" ? 10 : 0)
      )
    ),
  };
}

function createPlatformPrompt(
  answers: Answers,
  recommendation: ReturnType<typeof recommendPlatform>
) {
  const platform = recommendation.primary.platform;
```

```
  const platformInstructions: Record<string, string> = {
    "Claude Artifacts": `
Build this as a polished, self-contained Claude Artifact.
- Keep it client-side and dependency-light.
- Use local realistic sample data by default.
- Do not assume production database persistence, server secrets, authentication, or p
- Include empty, loading, success, and error states where relevant.
- Make source portable as HTML, JSX, SVG, Mermaid, or Markdown.`,
    "ChatGPT Canvas": `
Build this as an editable ChatGPT Canvas deliverable.
- Optimize for iterative revision and a clean single-document or single-file code exp
- State assumptions explicitly.
- Keep React/HTML preview requirements self-contained where possible.
- Prepare the final result for the appropriate export: PDF, DOCX, Markdown, or a lang
    v0: `
Build this as a production-oriented Next.js application in v0.
- Use TypeScript and a clean multi-file structure.
- Use accessible components and responsive layout.
- Separate client and server responsibilities appropriately.
- Use environment-variable placeholders rather than hardcoded secrets.
- Include loading, error, empty, and success states.
- Create a README with local setup, environment variables, database setup, migrations
  };

  const productionChecklist = recommendation.requiresProductionEscalation
    ? `
Production checklist:
- Add authentication and authorization.
- Define data ownership and tenant boundaries.
- Validate all server input and API responses.
- Use environment variables for secrets.
- Add database migrations and a repeatable seed process.
- Add error monitoring, logging, analytics, and audit logging where appropriate.
- Add linting, type checks, tests, and CI.
- Validate accessibility, mobile layouts, performance, and security before publishing
    : "";

  return `Build this for Semester.

Recommended platform: ${platform}
Secondary option: ${recommendation.secondary.platform}
Recommendation confidence: ${recommendation.confidence}%

Project scope:
- Deliverable: ${answers.deliverable}
- Interaction: ${answers.interaction}
- Backend requirement: ${answers.backend}
- Technology target: ${answers.techStack}
- Handoff target: ${answers.handoff}

${platformInstructions[platform]}

Required deliverables:
- A concise architecture explanation.
- Realistic Semester sample data.
- A primary user workflow that is immediately reviewable.
```

```
- Clear assumptions and non-goals.
- Instructions for running, editing, exporting, or deploying the result.
${productionChecklist}

Design direction:
Use a clean, premium, modern B2B software style. Prioritize clarity, dense-but-readab
}

function createGitHubPlan(
  answers: Answers,
  recommendation: ReturnType<typeof recommendPlatform>
) {
  const needsRepo =
    answers.handoff === "github" ||
    answers.handoff === "deploy" ||
    recommendation.requiresProductionEscalation;

  if (!needsRepo) {
    return `GitHub is optional for this scope. Keep the artifact self-contained while
  }

  const platform = recommendation.primary.platform;

  if (platform === "v0") {
    return `GitHub push plan for v0:
1. In v0, finish a working preview with the intended multi-file structure.
2. Open the project menu, then Settings → GitHub.
3. Connect the GitHub account or organization that should own the project.
4. Create a new repository or connect an existing repository.
5. Confirm the default branch, usually main.
6. Create an isolated feature branch for meaningful changes.
7. Review the v0 preview deployment and inspect changed files.
8. Push or sync the branch to GitHub.
9. Open a pull request, run checks, and merge after review.
10. Publish the merged branch to Vercel.
11. Keep .env files out of Git. Store database URLs, API tokens, and other secrets or
  }

  if (platform === "Claude Artifacts") {
    return `GitHub push plan after Claude Artifacts:
1. Export or copy the artifact source as HTML, JSX/TSX, SVG, Mermaid, Markdown, JSON
2. Create a local repository for the Semester project.
3. Place source in a deliberate structure:
   - app/ or src/ for React/Next.js UI
   - docs/ for Markdown and Mermaid
   - public/ for SVG and static exports
4. Create a README that records the original artifact purpose and assumptions.
5. Run the exported code locally in a normal framework project if it is React.
6. Use git add, git commit, and git push to create a versioned starting point.
7. If the artifact needs auth, a database, secrets, API routes, or multi-user records
  }

  return `GitHub push plan after ChatGPT Canvas:
1. Export the Canvas deliverable as Markdown, DOCX/PDF, or its language-specific code
2. For code, create a local repository and organize files by responsibility.
3. For documents, commit the Markdown source where possible; treat PDF and DOCX as re
```

```
4. Add a README describing purpose, decisions, assumptions, and how to regenerate the
5. Create a feature branch, commit the exported material, and push it to GitHub.
6. If the deliverable becomes an application needing deployment, move it to a v0/Next
}

function downloadFile(filename: string, content: string, type: string) {
  const blob = new Blob([content], { type });
  const url = URL.createObjectURL(blob);
  const link = document.createElement("a");
  link.href = url;
  link.download = filename;
  link.click();
  URL.revokeObjectURL(url);
}

function copyToClipboard(text: string) {
  return navigator.clipboard?.writeText(text);
}

function CopyButton({ value, label = "Copy" }: { value: string; label?: string }) {
  const [copied, setCopied] = useState(false);

  const copy = async () => {
    try {
      await copyToClipboard(value);
      setCopied(true);
      window.setTimeout(() => setCopied(false), 1400);
    } catch {
      setCopied(false);
    }
  };

  return (
    <button
      onClick={copy}
      className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 text-xs font
    >
      {copied ? "Copied" : label}
    </button>
  );
}

function Badge({
  children,
  className = "",
}: {
  children: React.ReactNode;
  className?: string;
}) {
  return (
    <span
      className={`inline-flex items-center rounded-full border px-2 py-0.5 text-xs fo
    >
      {children}
    </span>
  );
```

```
}

function PlatformBadge({ platform }: { platform: string }) {
  const styles: Record<string, string> = {
    "Claude Artifacts": "border-amber-200 bg-amber-50 text-amber-800",
    "ChatGPT Canvas": "border-emerald-200 bg-emerald-50 text-emerald-800",
    v0: "border-slate-800 bg-slate-950 text-white",
  };

  return <Badge className={styles[platform]}>{platform}</Badge>;
}

function GithubChecklist({
  plan,
  recommendation,
}: {
  plan: string;
  recommendation: ReturnType<typeof recommendPlatform>;
}) {
  return (
    <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
      <div className="flex flex-col justify-between gap-3 md:flex-row md:items-end">
        <div>
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-600
            Repository handoff
          </p>
          <h2 className="mt-1 text-xl font-bold text-slate-950">
            GitHub push plan
          </h2>
          <p className="mt-1 text-sm text-slate-600">
            Tailored for {recommendation.primary.platform}.
          </p>
        </div>
        <CopyButton value={plan} label="Copy GitHub plan" />
      </div>

      <pre className="mt-4 whitespace-pre-wrap rounded-xl bg-slate-950 p-4 text-xs le
        {plan}
      </pre>
    </section>
  );
}

export default function SemesterWorkflowRouter() {
  const [answers, setAnswers] = useState<Answers>(DEFAULT_ANSWERS);
  const [sort, setSort] = useState<{
    key: keyof Platform;
    direction: "asc" | "desc";
  }>({
    key: "platform",
    direction: "asc",
  });
  const [selectedPlatform, setSelectedPlatform] = useState<Platform | null>(
    null
  );
```

```
  const recommendation = useMemo(
    () => recommendPlatform(answers),
    [answers]
  );

  const generatedPrompt = useMemo(
    () => createPlatformPrompt(answers, recommendation),
    [answers, recommendation]
  );

  const githubPlan = useMemo(
    () => createGitHubPlan(answers, recommendation),
    [answers, recommendation]
  );

  const sortedPlatforms = useMemo(() => {
    return [...PLATFORM_DATA].sort((a, b) => {
      const aValue = a[sort.key];
      const bValue = b[sort.key];

      if (typeof aValue === "object" || typeof bValue === "object") {
        return 0;
      }

      return sort.direction === "asc"
        ? String(aValue).localeCompare(String(bValue))
        : String(bValue).localeCompare(String(aValue));
    });
  }, [sort]);

  const setAnswer = <K extends keyof Answers>(key: K, value: Answers[K]) => {
    setAnswers((current) => ({ ...current, [key]: value }));
  };

  const toggleSort = (key: keyof Platform) => {
    setSort((current) => ({
      key,
      direction:
        current.key === key && current.direction === "asc" ? "desc" : "asc",
    }));
  };

  const matrixHeader = (label: string, key: keyof Platform) => (
    <button
      onClick={() => toggleSort(key)}
      className="flex items-center gap-1 text-left text-xs font-bold uppercase track
    >
      {label}
      {sort.key === key ? (sort.direction === "asc" ? "↑" : "↓") : "↕"}
    </button>
  );

  const decisionMarkdown = `# Semester AI Workflow Recommendation

## Primary recommendation
- Platform: ${recommendation.primary.platform}
```

```
- Secondary platform: ${recommendation.secondary.platform}
- Confidence: ${recommendation.confidence}%
- Production escalation required: ${
    recommendation.requiresProductionEscalation ? "Yes" : "No"
  }

## Inputs
- Deliverable: ${answers.deliverable}
- Interaction: ${answers.interaction}
- Backend: ${answers.backend}
- Tech stack: ${answers.techStack}
- Handoff: ${answers.handoff}

## Why this platform
${recommendation.primary.reasons.map((reason) => `- ${reason}`).join("\n")}

## Build prompt
\`\`\`
${generatedPrompt}
\`\`\`

## GitHub plan
${githubPlan}
`;

  return (
    <main className="min-h-screen bg-slate-50 p-4 text-slate-900 md:p-8">
      <div className="mx-auto max-w-7xl space-y-6">
        <header className="rounded-2xl bg-gradient-to-br from-slate-950 via-slate-900
          <p className="text-xs font-bold uppercase tracking-[0.16em] text-indigo-300
            Semester build workflow
          </p>
          <h1 className="mt-2 text-3xl font-bold tracking-tight">
            Claude Artifacts vs Canvas vs v0 Router
          </h1>
          <p className="mt-2 max-w-3xl text-sm leading-6 text-slate-300">
            Choose the right environment based on project scope, interaction
            model, data requirements, technology stack, GitHub handoff, and
            deployment target.
          </p>
        </header>

        <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
              Decision tree
            </p>
            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Select project requirements
            </h2>
            <p className="mt-1 text-sm text-slate-600">
              The router recommends the best starting environment and tells you
              when a prototype must become a production codebase.
            </p>
          </div>
```

```
          <div className="mt-5 grid gap-4 md:grid-cols-2 xl:grid-cols-3">
            {Object.entries(OPTIONS).map(([key, choices]) => (
              <fieldset
                key={key}
                className="rounded-xl border border-slate-200 bg-slate-50 p-4"
              >
                <legend className="px-1 text-sm font-bold capitalize text-slate-900">
                  {key === "techStack"
                    ? "Technology target"
                    : key === "backend"
                    ? "Data and backend"
                    : key}
                </legend>

                <div className="mt-2 space-y-2">
                  {choices.map(([value, label]) => (
                    <label
                      key={value}
                      className="flex cursor-pointer items-start gap-2 text-sm text-s
                    >
                      <input
                        type="radio"
                        name={key}
                        value={value}
                        checked={answers[key as keyof Answers] === value}
                        onChange={() =>
                          setAnswer(
                            key as keyof Answers,
                            value as Answers[keyof Answers]
                          )
                        }
                        className="mt-1 accent-indigo-600"
                      />
                      <span>{label}</span>
                    </label>
                  ))}
                </div>
              </fieldset>
            ))}
          </div>

          <div className="mt-5 rounded-2xl border border-indigo-200 bg-indigo-50 p-5">
            <div className="flex flex-col justify-between gap-4 lg:flex-row lg:items-
              <div>
                <p className="text-xs font-bold uppercase tracking-[0.14em] text-indi
                  Primary recommendation
                </p>

                <div className="mt-2 flex flex-wrap items-center gap-2">
                  <h3 className="text-2xl font-bold text-slate-950">
                    {recommendation.primary.platform}
                  </h3>
                  <PlatformBadge platform={recommendation.primary.platform} />
                  <Badge className="border-indigo-200 bg-white text-indigo-700">
                    {recommendation.confidence}% confidence
                  </Badge>
```

```
            </div>

            <ul className="mt-4 max-w-3xl space-y-2 text-sm leading-6 text-slate
              {recommendation.primary.reasons.map((reason) => (
                <li key={reason}>• {reason}</li>
              ))}
            </ul>
          </div>

          <div className="rounded-xl border border-indigo-100 bg-white p-4 lg:w-8
            <p className="text-xs font-bold uppercase tracking-wide text-slate-50
              Secondary option
            </p>
            <p className="mt-1 text-base font-bold text-slate-900">
              {recommendation.secondary.platform}
            </p>
            <p className="mt-2 text-sm leading-6 text-slate-600">
              {recommendation.secondary.primaryUse}
            </p>
          </div>
        </div>

        {recommendation.requiresProductionEscalation && (
          <div className="mt-5 rounded-xl border border-amber-200 bg-amber-50 p-4
            <p className="text-sm font-bold text-amber-900">
              Production handoff required
            </p>
            <p className="mt-1 text-sm leading-6 text-amber-800">
              Your selections require persistence, APIs, authentication,
              secrets, Postgres, shared records, or deployment. Use Claude
              or Canvas only for a prototype/specification if helpful; make
              the final implementation a repository-backed application,
              typically in v0 and Next.js.
            </p>
          </div>
        )}
      </div>
    </section>

    <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
      <div className="flex flex-col justify-between gap-3 md:flex-row md:items-en
        <div>
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo
            Build prompt
          </p>
          <h2 className="mt-1 text-xl font-bold text-slate-950">
            Ready-to-copy platform prompt
          </h2>
        </div>

        <div className="flex flex-wrap gap-2">
          <CopyButton value={generatedPrompt} label="Copy prompt" />

          <button
            onClick={() =>
              downloadFile(
```

```
                "semester-workflow-recommendation.md",
                decisionMarkdown,
                "text/markdown;charset=utf-8"
              )
            }
            className="rounded-lg bg-slate-900 px-3 py-1.5 text-xs font-bold tex
          >
            Export Markdown
          </button>

          <button
            onClick={() =>
              downloadFile(
                "semester-workflow-recommendation.json",
                JSON.stringify(
                  {
                    answers,
                    recommendation: {
                      primary: recommendation.primary.platform,
                      secondary: recommendation.secondary.platform,
                      confidence: recommendation.confidence,
                      requiresProductionEscalation:
                        recommendation.requiresProductionEscalation,
                      reasons: recommendation.primary.reasons,
                    },
                    prompt: generatedPrompt,
                    githubPlan,
                  },
                  null,
                  2
                ),
                "application/json;charset=utf-8"
              )
            }
            className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 te
          >
            Export JSON
          </button>
        </div>
      </div>

      <pre className="mt-4 whitespace-pre-wrap rounded-xl bg-slate-950 p-4 text-
        {generatedPrompt}
      </pre>
    </section>

    <GithubChecklist plan={githubPlan} recommendation={recommendation} />

    <section className="overflow-hidden rounded-2xl border border-slate-200 bg-wh
      <div className="border-b border-slate-200 p-5">
        <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
          Platform matrix
        </p>
        <h2 className="mt-1 text-xl font-bold text-slate-950">
          Sortable execution, React, export, and handoff comparison
        </h2>
```

```
            <p className="mt-1 text-sm text-slate-600">
              Click column headers to sort and use “Open” for the full
              execution and sandbox profile.
            </p>
          </div>

          <div className="overflow-x-auto">
            <table className="w-full min-w-[1500px] border-collapse">
              <thead className="bg-slate-50">
                <tr>
                  <th className="px-4 py-3">
                    {matrixHeader("Platform", "platform")}
                  </th>
                  <th className="px-4 py-3">
                    {matrixHeader("Primary use", "primaryUse")}
                  </th>
                  <th className="px-4 py-3">
                    {matrixHeader("Execution model", "execution")}
                  </th>
                  <th className="px-4 py-3">
                    {matrixHeader("React support", "reactSupport")}
                  </th>
                  <th className="px-4 py-3">
                    {matrixHeader("Project shape", "projectShape")}
                  </th>
                  <th className="px-4 py-3">
                    {matrixHeader("Export formats", "exportFormats")}
                  </th>
                  <th className="px-4 py-3">
                    {matrixHeader("Production fit", "productionFit")}
                  </th>
                  <th className="px-4 py-3 text-left text-xs font-bold uppercase tra
                    Details
                  </th>
                </tr>
              </thead>

              <tbody>
                {sortedPlatforms.map((platform) => (
                  <tr
                    key={platform.id}
                    className="border-t border-slate-100 align-top transition hover:b
                  >
                    <td className="px-4 py-4">
                      <p className="font-bold text-slate-950">
                        {platform.platform}
                      </p>

                      <div className="mt-2 flex flex-wrap gap-1">
                        <Badge className="border-slate-200 bg-slate-50 text-slate-60
                          Prototype {platform.scores.prototype}/5
                        </Badge>
                        <Badge className="border-slate-200 bg-slate-50 text-slate-60
                          Full stack {platform.scores.fullStack}/5
                        </Badge>
                        <Badge className="border-slate-200 bg-slate-50 text-slate-60
```

```
                      Deploy {platform.scores.deployment}/5
                    </Badge>
                  </div>
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.primaryUse}
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.execution}
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.reactSupport}
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.projectShape}
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.exportFormats}
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.productionFit}
                </td>

                <td className="px-4 py-4">
                  <button
                    onClick={() => setSelectedPlatform(platform)}
                    className="rounded-lg border border-slate-200 bg-white px-3 p
                  >
                    Open
                  </button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </section>

    {selectedPlatform && (
      <div className="fixed inset-0 z-50 flex justify-end bg-slate-950/30">
        <button
          className="absolute inset-0 cursor-default"
          aria-label="Close details"
          onClick={() => setSelectedPlatform(null)}
        />

        <aside className="relative h-full w-full max-w-2xl overflow-y-auto bg-whi
          <div className="flex items-start justify-between gap-4">
            <div>
              <PlatformBadge platform={selectedPlatform.platform} />
```

```
                  <h2 className="mt-3 text-2xl font-bold text-slate-950">
                    {selectedPlatform.platform}
                  </h2>
                </div>

                <button
                  onClick={() => setSelectedPlatform(null)}
                  className="rounded-lg border border-slate-200 px-3 py-1.5 text-sm
                >
                  Close
                </button>
              </div>

              <div className="mt-6 space-y-5">
                {[
                  ["Primary use", selectedPlatform.primaryUse],
                  ["Execution model", selectedPlatform.execution],
                  ["Sandbox limits", selectedPlatform.sandboxLimits],
                  ["React support", selectedPlatform.reactSupport],
                  ["Project shape", selectedPlatform.projectShape],
                  ["Code and export options", selectedPlatform.exportFormats],
                  ["GitHub workflow", selectedPlatform.githubWorkflow],
                  ["Production handoff", selectedPlatform.productionFit],
                  ["Best Semester use cases", selectedPlatform.bestFor],
                ].map(([label, value]) => (
                  <div key={label}>
                    <h3 className="text-sm font-bold text-slate-900">
                      {label}
                    </h3>
                    <p className="mt-1 text-sm leading-6 text-slate-600">
                      {value}
                    </p>
                  </div>
                ))}
              </div>
            </aside>
          </div>
        )}
      </div>
    </main>
  );
}
```

v0 + Next.js + Postgres

Yes. v0 is the right option of the three when you need a multi-file Next.js application with Postgres, because it is built around an application project rather than a single artifact or document. v0 documents a VM-backed Node.js sandbox that can run server code, API routes, shell commands, and database-connected workflows. [85] [86]

A sensible Semester architecture:

```
semester/
├── app/
```

```
│   ├── artifacts/
│   │   ├── page.tsx                 # Sortable artifact catalog
│   │   ├── [artifactId]/page.tsx    # Artifact detail page
│   │   ├── new/page.tsx             # Create artifact form
│   │   └── export/route.ts          # CSV / JSON / Markdown export
│   ├── api/
│   │   └── artifacts/route.ts       # Optional REST endpoint
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── artifact-table.tsx
│   ├── artifact-filters.tsx
│   ├── decision-tree.tsx
│   ├── prompt-builder.tsx
│   └── exporter.tsx
├── lib/
│   ├── db.ts                        # Postgres/Prisma/Supabase client
│   ├── artifact-schema.ts           # Zod validation
│   ├── decision-engine.ts
│   └── exporters.ts
├── prisma/
│   └── schema.prisma                # If using Prisma
├── supabase/
│   └── migrations/                  # If using Supabase Postgres
├── .env.example
└── README.md
```

Database options

Option	Best when	Notes
Supabase Postgres	You need Postgres plus auth, row-level security, storage, and realtime capabilities	Strong default for a multi-user Semester tracker
Prisma Postgres / Prisma ORM	You want a strongly typed ORM and database migration workflow in a Next.js codebase	Vercel has a documented Next.js + Prisma Postgres path
Neon Postgres	You want serverless Postgres and a straightforward external managed database	Works well with Prisma, Drizzle, or direct SQL
Vercel Marketplace provider	You want deployment and database provisioning coordinated through Vercel	Next.js guidance shows linking a project to a Postgres provider through Storage

Next.js's official learning material covers a flow of pushing the project to GitHub, importing it into Vercel, creating/linking a Postgres database, copying environment variables locally, seeding data, and querying through a route handler. It specifically warns that `.env` must be ignored so database secrets do not get pushed to GitHub. Vercel also provides a Next.js guide that starts with hardcoded content and adds managed Postgres, authentication, Server Actions, drafts, publishing, and deployment. [84] [83]

Core Postgres model

For the artifact catalog itself, start with a narrow table:

```
create table artifacts (
  id uuid primary key default gen_random_uuid(),
  organization_id uuid not null,
  title text not null,
  category text not null,
  format text not null,
  use_case text,
  lifecycle text not null,
  audience text not null,
  owner_id uuid,
  priority text not null default 'P2',
  status text not null default 'Backlog',
  effort text,
  impact text,
  tags text[] not null default '{}',
  dependencies jsonb not null default '[]',
  suggested_prompt text not null,
  metadata jsonb not null default '{}',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index artifacts_organization_id_idx
  on artifacts (organization_id);

create index artifacts_category_idx
  on artifacts (category);

create index artifacts_format_idx
  on artifacts (format);

create index artifacts_status_idx
  on artifacts (status);
```

If Semester becomes multi-tenant, every meaningful query should be scoped by `organization_id`, and authorization should be enforced server-side—not simply hidden in the UI.

GitHub push steps

For a v0 project, use the built-in GitHub connection rather than copying a single component manually:

1. Build the initial working version in v0.
2. Open the project's `...` menu, then go to **Settings → GitHub**.
3. Connect the GitHub account or organization that should own the repository.
4. Create a new repository or connect an existing one.

5. Use an isolated branch for feature work instead of treating the deployed preview as the only source of truth.
6. Review the branch preview and changed files.
7. Push/sync changes to GitHub.
8. Open a pull request, run checks, then merge into `main`.
9. Publish the merged app to Vercel.
10. Store `DATABASE_URL`, Supabase keys, and all other secrets in Vercel environment variables—not in Git and never in client code.

v0's GitHub documentation describes connecting a project to GitHub, creating a repository from the current v0 project, working from isolated branches, reviewing previews, and publishing after changes are reviewed. [87] [88]

For the local CLI route after exporting or cloning the repository:

```
git clone git@github.com:YOUR_ORG/semester.git
cd semester

npm install
npm run dev

git checkout -b feat/artifact-catalog
git add .
git commit -m "feat: add artifact decision tree and catalog"
git push -u origin feat/artifact-catalog
```

Then open a pull request and let Vercel create a preview deployment from the branch.

Deployment boundary

Use Claude Artifacts or Canvas for the **fastest first version** of the decision tree. Use v0 for the real Semester application when any of the following becomes true:

- Users must sign in.
- Multiple people see or edit shared artifacts.
- Records must survive sessions and devices.
- You need Postgres queries, migrations, backups, or audit history.
- You need API routes, webhooks, payment handling, email, or private integrations.
- You need server-side secrets or role-based permissions.
- You want production previews and deployment tied to GitHub.

Claude Artifacts are strong for HTML/React-style self-contained output, diagrams, and browser-generated exports; Canvas has stronger document export paths such as PDF, DOCX, and Markdown; v0's best handoff is a multi-file application project synchronized with GitHub and deployed on Vercel. [89] [90] [83]

Compare Claude Artifacts vs ChatGPT Canvas vs v0 sandbox in a sortable matrix — execution limits, React support, and export formats

Computer
Build a prompt decision tree app for choosing between Claude Artifacts, ChatGPT Canvas, and v0 sandbox for specific coding workflows
Computer
Show the rest of the React code block
Export the app recommendation as Markdown
Add Supabase integration logic to the prompt tree

Below is the completed version of the React app with a sortable comparison matrix, workflow-specific platform router, Markdown export, and **Supabase-aware decision logic.** The decision engine now routes anything requiring persistent records, auth, tenant isolation, or Postgres toward a v0/Next.js implementation—and generates a Supabase architecture prompt and handoff plan.

For a real Next.js implementation, use separate Supabase browser and server clients through `@supabase/ssr`, with `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` in `.env.local`; server-only secrets must not be exposed with the `NEXT_PUBLIC_` prefix. [104] [105] [106]

Complete React component

Save this as `app/workflow-router/page.tsx` in a Next.js App Router project. It uses only React and Tailwind classes, so it also works as a starting component in v0.

```
"use client";

import React, { useMemo, useState } from "react";

type PlatformId = "claude-artifacts" | "chatgpt-canvas" | "v0";

type Deliverable =
  | "interactive-tool"
  | "product-prototype"
  | "marketing-page"
  | "diagram"
  | "document"
  | "presentation"
  | "full-stack-feature"
  | "analysis";

type Interaction = "none" | "simple" | "complex" | "multi-user";

type Backend =
  | "none"
```

```
  | "file-export"
  | "database"
  | "server-api"
  | "auth-secrets";

type TechStack =
  | "none"
  | "html"
  | "react"
  | "nextjs"
  | "nextjs-postgres"
  | "nextjs-supabase";

type Handoff = "share-now" | "doc-export" | "github" | "deploy";

type Answers = {
  deliverable: Deliverable;
  interaction: Interaction;
  backend: Backend;
  techStack: TechStack;
  handoff: Handoff;
  supabaseCapability:
    | "none"
    | "database-only"
    | "auth"
    | "storage"
    | "realtime"
    | "multi-tenant";
};

type Platform = {
  id: PlatformId;
  platform: string;
  primaryUse: string;
  execution: string;
  sandboxLimits: string;
  reactSupport: string;
  projectShape: string;
  exportFormats: string;
  githubWorkflow: string;
  productionFit: string;
  bestFor: string;
  scores: {
    prototype: number;
    documents: number;
    diagrams: number;
    fullStack: number;
    deployment: number;
  };
};

type RankedPlatform = Platform & {
  score: number;
  reasons: string[];
};
```

```
type Recommendation = {
  primary: RankedPlatform;
  secondary: RankedPlatform;
  ranked: RankedPlatform[];
  confidence: number;
  requiresProductionEscalation: boolean;
  needsSupabase: boolean;
  needsSupabaseAuth: boolean;
  needsTenantIsolation: boolean;
};

const PLATFORM_DATA: Platform[] = [
  {
    id: "claude-artifacts",
    platform: "Claude Artifacts",
    primaryUse:
      "Shareable self-contained artifacts, diagrams, visual explainers, browser-side
    execution:
      "A rendered artifact experience optimized for contained client-side output rath
    sandboxLimits:
      "Do not depend on artifact-local state for durable data, server secrets, full a
    reactSupport:
      "Good for self-contained React interfaces, local-state trackers, calculators, c
    projectShape:
      "Usually a single contained artifact, component, page, SVG, Mermaid source, or
    exportFormats:
      "HTML, React/JSX-style source, Markdown, SVG, Mermaid, plus client-side browser
    githubWorkflow:
      "Export or copy source, put it into a normal repository, then version the HTML
    productionFit:
      "Prototype or visual specification → source export → GitHub → Next.js, Supabase
    bestFor:
      "Semester format decision tree, artifact catalog prototype, ROI calculator, arc
    scores: {
      prototype: 5,
      documents: 4,
      diagrams: 5,
      fullStack: 1,
      deployment: 2,
    },
  },
  {
    id: "chatgpt-canvas",
    platform: "ChatGPT Canvas",
    primaryUse:
      "Iterative document and code workspace for editing, transformation, review, rev
    execution:
      "React and HTML render in an isolated preview environment; Canvas also supports
    sandboxLimits:
      "Treat Canvas as an editor and preview workspace, not a durable backend. Produc
    reactSupport:
      "React and HTML previews are suitable for code iteration. Canvas is strongest f
    projectShape:
      "Usually a document, code file, or contained editable deliverable optimized for
    exportFormats:
      "Documents: PDF, DOCX, Markdown. Code: language-appropriate files such as JavaS
```

```
    githubWorkflow:
      "Export the document or code file, commit Markdown and source to GitHub, and pr
    productionFit:
      "Draft, revise, and export → GitHub or documentation system → production app/pu
    bestFor:
      "Semester PRDs, policies, research synthesis, legal first drafts, strategy, boa
    scores: {
      prototype: 3,
      documents: 5,
      diagrams: 3,
      fullStack: 1,
      deployment: 1,
    },
  },
  {
    id: "v0",
    platform: "v0",
    primaryUse:
      "Production-oriented React and Next.js apps, multi-file projects, data integrat
    execution:
      "A VM-backed isolated Node.js development sandbox for multi-file projects, serv
    sandboxLimits:
      "Preview success is not production readiness. Verify environment variables, da
    reactSupport:
      "Native fit for React and Next.js: multi-file components, App Router, Tailwind
    projectShape:
      "A multi-file application with dependencies, routes, components, server logic,
    exportFormats:
      "Application project code, GitHub sync/export, existing-codebase integration, p
    githubWorkflow:
      "Connect GitHub, choose or create a repository, use isolated branches, review p
    productionFit:
      "Best for database-backed, authenticated, multi-user applications with Supabase
    bestFor:
      "Semester artifact catalog with persistent records, role-based access, shared w
    scores: {
      prototype: 4,
      documents: 2,
      diagrams: 2,
      fullStack: 5,
      deployment: 5,
    },
  },
];

const OPTIONS = {
  deliverable: [
    ["interactive-tool", "Interactive tool, tracker, dashboard, calculator, or config
    ["product-prototype", "Clickable product prototype or interface exploration"],
    ["marketing-page", "Marketing site, landing page, one-page demo, or visual expla
    ["diagram", "Architecture, process, ERD, sequence, roadmap, or system diagram"],
    ["document", "PRD, policy, research, strategy, report, or operating document"],
    ["presentation", "Pitch, board update, sales narrative, launch, or training"],
    ["full-stack-feature", "Authenticated, data-backed, multi-user product feature"]
    ["analysis", "Python analysis, research, data cleanup, or generated-file workflow
  ],
```

```
  interaction: [
    ["none", "No interaction; people mainly read or review"],
    ["simple", "Simple browser interaction: tabs, toggles, calculators, or local samp
    ["complex", "Stateful interface: filters, tables, forms, drawers, editable workfl
    ["multi-user", "Shared records, multiple users, roles, permissions, or collabora
  ],
  backend: [
    ["none", "No backend; static result or browser-only sample data"],
    ["file-export", "Needs browser-generated CSV, Markdown, SVG, JSON, PDF, or relate
    ["database", "Needs persistent database records"],
    ["server-api", "Needs API routes, server logic, webhooks, or third-party services
    ["auth-secrets", "Needs sign-in, permissions, billing, private APIs, or server se
  ],
  techStack: [
    ["none", "No prescribed code stack"],
    ["html", "Plain HTML, CSS, and JavaScript"],
    ["react", "React component or client-side application"],
    ["nextjs", "Multi-file Next.js application"],
    ["nextjs-postgres", "Next.js with Postgres"],
    ["nextjs-supabase", "Next.js with Supabase, Postgres, and Auth"],
  ],
  handoff: [
    ["share-now", "Share immediately as a completed artifact"],
    ["doc-export", "Export PDF, DOCX, Markdown, or a source file"],
    ["github", "Push source into GitHub"],
    ["deploy", "Deploy a real web application"],
  ],
  supabaseCapability: [
    ["none", "No Supabase requirement"],
    ["database-only", "Persistent Postgres records only"],
    ["auth", "Email/OAuth authentication and sessions"],
    ["storage", "User-uploaded files, exports, or media"],
    ["realtime", "Live updates, collaboration, or event subscriptions"],
    ["multi-tenant", "Organizations, memberships, roles, and row-level security"],
  ],
} as const;

const DEFAULT_ANSWERS: Answers = {
  deliverable: "interactive-tool",
  interaction: "complex",
  backend: "none",
  techStack: "react",
  handoff: "share-now",
  supabaseCapability: "none",
};

function getPlatform(id: PlatformId) {
  const found = PLATFORM_DATA.find((platform) => platform.id === id);

  if (!found) {
    throw new Error(`Unknown platform: ${id}`);
  }

  return found;
}
```

```
function recommendPlatform(answers: Answers): Recommendation {
  const scores: Record<PlatformId, number> = {
    "claude-artifacts": 0,
    "chatgpt-canvas": 0,
    v0: 0,
  };

  const reasons: Record<PlatformId, string[]> = {
    "claude-artifacts": [],
    "chatgpt-canvas": [],
    v0: [],
  };

  const add = (platform: PlatformId, points: number, reason: string) => {
    scores[platform] += points;
    reasons[platform].push(reason);
  };

  switch (answers.deliverable) {
    case "interactive-tool":
      add("claude-artifacts", 7, "A shareable, self-contained interactive tool is a c
      add("v0", 4, "v0 is a strong follow-on if the tool needs persistent data or sha
      add("chatgpt-canvas", 2, "Canvas can iterate React/HTML, but it is less focused
      break;

    case "product-prototype":
      add("claude-artifacts", 6, "Claude Artifacts is fast for an interactive proof c
      add("v0", 5, "v0 is ideal if the prototype should evolve into the production ap
      add("chatgpt-canvas", 3, "Canvas can support focused React/HTML iteration.");
      break;

    case "marketing-page":
      add("claude-artifacts", 7, "A self-contained marketing or explainer page is an
      add("v0", 5, "v0 is appropriate when the page belongs in a Next.js production w
      add("chatgpt-canvas", 2, "Canvas is useful for copy and component iteration.")
      break;

    case "diagram":
      add("claude-artifacts", 8, "Claude is a strong choice for Mermaid, SVG, structu
      add("chatgpt-canvas", 3, "Canvas is useful for diagram source plus supporting c
      add("v0", 1, "v0 is best only when the diagram is part of a larger application
      break;

    case "document":
      add("chatgpt-canvas", 9, "Canvas is optimized for drafting, editing, revising,
      add("claude-artifacts", 4, "Claude can create polished standalone document arti
      add("v0", 1, "v0 is unnecessary unless the document belongs inside an applicati
      break;

    case "presentation":
      add("chatgpt-canvas", 6, "Canvas supports editorial and export-oriented presen
      add("claude-artifacts", 5, "Claude is useful for visual, shareable deck-like ar
      add("v0", 2, "v0 is useful only when the presentation itself is an interactive
      break;

    case "full-stack-feature":
```

```
      add("v0", 10, "A full-stack feature requires multi-file application structure,
      add("claude-artifacts", 1, "Claude can still provide an early prototype or visu
      add("chatgpt-canvas", 1, "Canvas can help create the PRD or review individual r
      break;

    case "analysis":
      add("chatgpt-canvas", 8, "Canvas is well suited to iterative analysis, Python,
      add("claude-artifacts", 3, "Claude can package an analysis result into a sharea
      add("v0", 3, "v0 is useful if ongoing analysis should become a shared product."
      break;
  }

  switch (answers.interaction) {
    case "none":
      add("chatgpt-canvas", 3, "A read-first deliverable favors an editorial workspac
      add("claude-artifacts", 2, "A static artifact is still practical.");
      break;

    case "simple":
      add("claude-artifacts", 5, "Simple browser-side interaction is a strong Artifac
      add("chatgpt-canvas", 3, "Canvas can preview simple React or HTML interaction."
      add("v0", 2, "v0 works, but may be more project overhead than necessary.");
      break;

    case "complex":
      add("claude-artifacts", 5, "A local-state tracker or interactive prototype can
      add("v0", 6, "Complex interaction benefits from durable multi-file React/Next.j
      add("chatgpt-canvas", 3, "Canvas can preview component behavior but is less pro
      break;

    case "multi-user":
      add("v0", 10, "Shared multi-user workflows require auth, authorization, persis
      break;
  }

  switch (answers.backend) {
    case "none":
      add("claude-artifacts", 4, "No backend requirement favors a portable, self-con
      add("chatgpt-canvas", 3, "A document or code-preview workflow can be sufficien
      break;

    case "file-export":
      add("claude-artifacts", 5, "Artifacts work well for client-side downloads such
      add("chatgpt-canvas", 5, "Canvas works well for document and code-file export."
      add("v0", 3, "v0 can add export endpoints inside a production application.");
      break;

    case "database":
      add("v0", 8, "Persistent records need a production application with database in
      add("claude-artifacts", 1, "Use sample data in the artifact, then hand off to a
      break;

    case "server-api":
      add("v0", 9, "Server routes, external services, and webhook workflows favor a f
      break;
```

```
    case "auth-secrets":
      add("v0", 10, "Authentication, permissions, billing, private API access, and se
      break;
  }

  switch (answers.techStack) {
    case "html":
      add("claude-artifacts", 5, "A contained HTML/CSS/JS deliverable is ideal for ar
      add("chatgpt-canvas", 2, "Canvas can edit and preview the page.");
      break;

    case "react":
      add("claude-artifacts", 5, "React is useful for self-contained interactive arti
      add("chatgpt-canvas", 4, "Canvas supports React/HTML preview and code iteratior
      add("v0", 5, "v0 is appropriate when React needs a multi-file production struct
      break;

    case "nextjs":
      add("v0", 9, "A multi-file Next.js application is a direct v0 workflow.");
      break;

    case "nextjs-postgres":
      add("v0", 10, "Next.js plus Postgres requires environment variables, server da
      break;

    case "nextjs-supabase":
      add("v0", 10, "Next.js with Supabase requires real client/server separation, au
      break;
  }

  const needsSupabase =
    answers.techStack === "nextjs-supabase" ||
    answers.supabaseCapability !== "none";

  const needsSupabaseAuth =
    answers.supabaseCapability === "auth" ||
    answers.supabaseCapability === "multi-tenant";

  const needsTenantIsolation =
    answers.supabaseCapability === "multi-tenant";

  if (needsSupabase) {
    add("v0", 10, "Supabase integration requires a multi-file application with enviro
    add("claude-artifacts", 1, "Claude remains useful for prototyping the UI with loc
    add("chatgpt-canvas", 2, "Canvas remains useful for drafting SQL, RLS policies, a
  }

  if (answers.supabaseCapability === "database-only") {
    add("v0", 4, "Persistent Postgres records require server-aware implementation and
  }

  if (answers.supabaseCapability === "auth") {
    add("v0", 7, "Session handling and authenticated data access require server/clien
  }

  if (answers.supabaseCapability === "storage") {
```

```
    add("v0", 7, "File uploads and protected storage need authenticated policies and
  }

  if (answers.supabaseCapability === "realtime") {
    add("v0", 8, "Realtime subscriptions need a persistent app architecture and caref
  }

  if (answers.supabaseCapability === "multi-tenant") {
    add("v0", 10, "Multi-tenant workspaces require organizations, memberships, row-le
  }

  switch (answers.handoff) {
    case "share-now":
      add("claude-artifacts", 6, "Immediate sharing favors a finished visual artifac
      add("chatgpt-canvas", 3, "Canvas can also share editable content for review.")
      break;

    case "doc-export":
      add("chatgpt-canvas", 7, "Canvas directly supports document and code-file expor
      add("claude-artifacts", 4, "Claude supports source-oriented handoff: HTML, JSX
      break;

    case "github":
      add("v0", 7, "A GitHub-backed workflow favors a project-oriented environment."
      add("claude-artifacts", 3, "Artifact source can become versioned repository inp
      add("chatgpt-canvas", 3, "Canvas exports can be committed into a repository.")
      break;

    case "deploy":
      add("v0", 10, "Deployment strongly favors v0 and Vercel's application workflow
      add("claude-artifacts", 1, "Claude is best used for prototype/source handoff ra
      break;
  }

  const ranked = (Object.keys(scores) as PlatformId[])
    .map((id) => ({
      ...getPlatform(id),
      score: scores[id],
      reasons: reasons[id],
    }))
    .sort((a, b) => b.score - a.score);

  const primary = ranked[^8_0];
  const secondary = ranked[^8_1];

  const requiresProductionEscalation =
    answers.backend === "database" ||
    answers.backend === "server-api" ||
    answers.backend === "auth-secrets" ||
    answers.interaction === "multi-user" ||
    answers.techStack === "nextjs" ||
    answers.techStack === "nextjs-postgres" ||
    answers.techStack === "nextjs-supabase" ||
    answers.handoff === "deploy" ||
    needsSupabase;
```

```
  return {
    primary,
    secondary,
    ranked,
    needsSupabase,
    needsSupabaseAuth,
    needsTenantIsolation,
    requiresProductionEscalation,
    confidence: Math.min(
      97,
      Math.max(
        62,
        68 +
          (primary.score - secondary.score) * 4 +
          (requiresProductionEscalation && primary.id === "v0" ? 10 : 0)
      )
    ),
  };
}

function createSupabasePlan(
  answers: Answers,
  recommendation: Recommendation
) {
  if (!recommendation.needsSupabase) {
    return `Supabase is not required by the current choices.

Use local sample data while validating the workflow. If the app later needs persister
  }

  const capabilities: string[] = [];

  if (
    answers.supabaseCapability === "database-only" ||
    answers.supabaseCapability === "auth" ||
    answers.supabaseCapability === "storage" ||
    answers.supabaseCapability === "realtime" ||
    answers.supabaseCapability === "multi-tenant"
  ) {
    capabilities.push("Postgres database with migration-managed schema");
  }

  if (recommendation.needsSupabaseAuth) {
    capabilities.push("Supabase Auth with server/client session handling");
  }

  if (answers.supabaseCapability === "storage") {
    capabilities.push("Supabase Storage with authenticated upload/download policies"
  }

  if (answers.supabaseCapability === "realtime") {
    capabilities.push("Realtime subscriptions scoped to authorized records");
  }

  if (recommendation.needsTenantIsolation) {
    capabilities.push("Organizations, memberships, roles, and Row Level Security poli
```

```
  }

  return `Supabase implementation plan for Semester:

Required capabilities:
${capabilities.map((item) => `- ${item}`).join("\n")}

Recommended project structure:
- app/ for Next.js App Router pages and route handlers
- components/ for UI
- lib/supabase/client.ts for browser client creation
- lib/supabase/server.ts for server client creation
- lib/supabase/middleware.ts or proxy integration for auth session refresh
- supabase/migrations/ for database migrations and RLS policies
- types/database.ts for generated or maintained database types

Environment variables:
- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
- Keep any service-role key server-only; never expose it in client components or pre

Required controls:
- Validate user input with a schema validator.
- Enforce data access with Row Level Security, not only UI conditions.
- Scope every tenant query by organization membership.
- Put privileged operations in server actions or route handlers.
- Add database indexes for organization_id, status, format, category, and updated_at
- Add audit records for sensitive changes if shared workspaces or compliance matter.
- Store migrations and RLS policies in GitHub.
- Use preview environments with separate Supabase projects or safely isolated test da
}

function createGitHubPlan(
  answers: Answers,
  recommendation: Recommendation
) {
  const needsRepository =
    answers.handoff === "github" ||
    answers.handoff === "deploy" ||
    recommendation.requiresProductionEscalation;

  if (!needsRepository) {
    return `GitHub is optional for this scope. Keep the initial artifact self-contai
  }

  if (recommendation.primary.id === "v0") {
    return `GitHub push plan for the v0 implementation:

1. Build the first working multi-file app in v0.
2. Open the v0 project menu and connect GitHub under Settings → GitHub.
3. Create a private Semester repository or connect the existing repository.
4. Use main only for reviewed code; create a feature branch for new work.
5. Confirm .gitignore excludes .env, .env.local, and generated local secrets.
6. Add a .env.example that names required variables without containing values.
7. Review changed files and the v0 preview deployment.
8. Push or sync the branch to GitHub.
```

```
9. Open a pull request and run typecheck, lint, test, and build checks.
10. Merge only after reviewing database migrations, Row Level Security policies, and
11. Deploy with Vercel and configure all production environment variables there.
12. Never commit Supabase service-role credentials or database connection strings to
  }

  if (recommendation.primary.id === "claude-artifacts") {
    return `GitHub push plan after Claude Artifacts:

1. Export or copy source as HTML, JSX/TSX, SVG, Mermaid, Markdown, JSON, or CSV.
2. Create a local Semester repository and organize source deliberately:
   - app/ or src/ for React or Next.js source
   - docs/ for Markdown, Mermaid, policies, PRDs, and decision records
   - public/ for SVG, images, and static files
3. Add README.md with purpose, assumptions, source format, and regeneration instruct
4. Convert the artifact into a normal framework project before adding auth, database
5. Create a feature branch, commit, and push:
   git checkout -b feat/semester-artifact-router
   git add .
   git commit -m "feat: add artifact workflow router"
   git push -u origin feat/semester-artifact-router
6. Open a pull request and use a v0/Next.js implementation when the prototype needs p
  }

  return `GitHub push plan after ChatGPT Canvas:

1. Export source as Markdown, code files, PDF, or DOCX.
2. Prefer committing Markdown and source code as the durable editable record.
3. Keep PDF/DOCX as review or distribution outputs, not the sole source of truth.
4. Organize code under app/, components/, lib/, and tests/ when moving to a product r
5. Create a README that records the intent, assumptions, decisions, and regeneration
6. Create a feature branch, commit, push, open a pull request, and merge after reviev
7. If the deliverable requires Supabase, auth, persistent records, or Vercel deployme
}

function createBuildPrompt(
  answers: Answers,
  recommendation: Recommendation,
  supabasePlan: string
) {
  const platformName = recommendation.primary.platform;

  const platformInstructions: Record<string, string> = {
    "Claude Artifacts": `Build this as a polished, self-contained Claude Artifact.
- Keep it client-side and dependency-light.
- Use realistic local sample data.
- Do not assume production authentication, persistent data, private API access, or se
- Include empty, loading, success, and error states where relevant.
- Make the output portable as HTML, JSX, SVG, Mermaid, Markdown, JSON, or CSV.`,
    "ChatGPT Canvas": `Build this as an editable ChatGPT Canvas deliverable.
- Optimize for iterative revision and a clean document or code-file structure.
- State assumptions and open questions clearly.
- Keep React/HTML preview requirements contained where possible.
- Prepare the work for export as PDF, DOCX, Markdown, or a language-specific source f
    v0: `Build this as a production-oriented Next.js application in v0.
- Use TypeScript, Next.js App Router, and a clean multi-file structure.
```

```
- Use accessible and responsive components.
- Separate browser/client logic from server logic correctly.
- Use environment-variable placeholders rather than hardcoded credentials.
- Include loading, empty, error, and success states.
- Add a README with setup, environment variables, database migrations, test steps, G
  };

  const productionChecklist = recommendation.requiresProductionEscalation
    ? `
Production requirements:
- Add authentication and authorization.
- Define data ownership, organization boundaries, and role permissions.
- Validate all input at server boundaries.
- Use server-side secrets only in server actions, route handlers, or server-only modu
- Add migrations, seed data, indexes, and rollback planning.
- Add error monitoring, logs, analytics, audit records, type checks, linting, and tes
- Confirm accessibility, mobile behavior, performance, privacy, and security before p
    : "";

  const supabaseRequirements = recommendation.needsSupabase
    ? `
${supabasePlan}

Use Supabase integration rules:
- Use @supabase/ssr.
- Create browser and server clients in separate files.
- Use a publishable key in browser-safe configuration only.
- Keep privileged service-role access server-only.
- Add Row Level Security policies before treating the system as production-ready.
- Do not rely on client-side filtering as authorization.`
    : "";

  return `Build this for Semester.

Recommended platform: ${platformName}
Secondary option: ${recommendation.secondary.platform}
Confidence: ${recommendation.confidence}%

Project requirements:
- Deliverable: ${answers.deliverable}
- Interaction: ${answers.interaction}
- Backend requirement: ${answers.backend}
- Tech target: ${answers.techStack}
- Supabase requirement: ${answers.supabaseCapability}
- Handoff: ${answers.handoff}

${platformInstructions[platformName]}

Required deliverables:
- Explain the architecture and important assumptions.
- Use realistic Semester sample data.
- Build the primary workflow so it can be reviewed immediately.
- Include an explicit list of non-goals.
- Include instructions to run, edit, export, or deploy the project.
${productionChecklist}
${supabaseRequirements}
```

```
Design direction:
Use a clean premium B2B software aesthetic, dense-but-readable information design, ac
}

function downloadFile(filename: string, content: string, type: string) {
  const blob = new Blob([content], { type });
  const url = URL.createObjectURL(blob);
  const anchor = document.createElement("a");

  anchor.href = url;
  anchor.download = filename;
  anchor.click();

  URL.revokeObjectURL(url);
}

async function copyToClipboard(value: string) {
  await navigator.clipboard?.writeText(value);
}

function Badge({
  children,
  className = "",
}: {
  children: React.ReactNode;
  className?: string;
}) {
  return (
    <span
      className={`inline-flex items-center rounded-full border px-2 py-0.5 text-xs fo
    >
      {children}
    </span>
  );
}

function PlatformBadge({ platform }: { platform: string }) {
  const styles: Record<string, string> = {
    "Claude Artifacts": "border-amber-200 bg-amber-50 text-amber-800",
    "ChatGPT Canvas": "border-emerald-200 bg-emerald-50 text-emerald-800",
    v0: "border-slate-800 bg-slate-950 text-white",
  };

  return <Badge className={styles[platform]}>{platform}</Badge>;
}

function CopyButton({
  value,
  label = "Copy",
}: {
  value: string;
  label?: string;
}) {
  const [copied, setCopied] = useState(false);
```

```
  const handleCopy = async () => {
    try {
      await copyToClipboard(value);
      setCopied(true);
      window.setTimeout(() => setCopied(false), 1500);
    } catch {
      setCopied(false);
    }
  };

  return (
    <button
      onClick={handleCopy}
      className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 text-xs fon
    >
      {copied ? "Copied" : label}
    </button>
  );
}

function SelectSection<K extends keyof Answers>({
  label,
  field,
  choices,
  value,
  onChange,
}: {
  label: string;
  field: K;
  choices: readonly (readonly [string, string])[];
  value: Answers[K];
  onChange: (field: K, value: Answers[K]) => void;
}) {
  return (
    <fieldset className="rounded-xl border border-slate-200 bg-slate-50 p-4">
      <legend className="px-1 text-sm font-bold text-slate-900">{label}</legend>

      <div className="mt-2 space-y-2">
        {choices.map(([optionValue, optionLabel]) => (
          <label
            key={optionValue}
            className="flex cursor-pointer items-start gap-2 text-sm text-slate-700"
          >
            <input
              type="radio"
              name={field}
              value={optionValue}
              checked={value === optionValue}
              onChange={() => onChange(field, optionValue as Answers[K])}
              className="mt-1 accent-indigo-600"
            />
            <span>{optionLabel}</span>
          </label>
        ))}
      </div>
    </fieldset>
```

```
  );
}

function SupabasePanel({
  recommendation,
  supabasePlan,
}: {
  recommendation: Recommendation;
  supabasePlan: string;
}) {
  return (
    <section className="rounded-2xl border border-emerald-200 bg-emerald-50 p-5 shad
      <div className="flex flex-col justify-between gap-3 md:flex-row md:items-end">
        <div>
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-emerald-70
            Supabase integration
          </p>
          <h2 className="mt-1 text-xl font-bold text-emerald-950">
            {recommendation.needsSupabase
              ? "Production data and auth plan"
              : "Supabase is currently optional"}
          </h2>
        </div>

        <CopyButton value={supabasePlan} label="Copy Supabase plan" />
      </div>

      <pre className="mt-4 whitespace-pre-wrap rounded-xl bg-slate-950 p-4 text-xs le
        {supabasePlan}
      </pre>
    </section>
  );
}

export default function SemesterWorkflowRouter() {
  const [answers, setAnswers] = useState<Answers>(DEFAULT_ANSWERS);
  const [sort, setSort] = useState<{
    key:
      | "platform"
      | "primaryUse"
      | "execution"
      | "reactSupport"
      | "projectShape"
      | "exportFormats"
      | "productionFit";
    direction: "asc" | "desc";
  }>({
    key: "platform",
    direction: "asc",
  });

  const [selectedPlatform, setSelectedPlatform] = useState<Platform | null>(
    null
  );

  const recommendation = useMemo(
```

```
    () => recommendPlatform(answers),
    [answers]
  );

  const supabasePlan = useMemo(
    () => createSupabasePlan(answers, recommendation),
    [answers, recommendation]
  );

  const buildPrompt = useMemo(
    () => createBuildPrompt(answers, recommendation, supabasePlan),
    [answers, recommendation, supabasePlan]
  );

  const githubPlan = useMemo(
    () => createGitHubPlan(answers, recommendation),
    [answers, recommendation]
  );

  const markdownExport = useMemo(() => {
    return `# Semester AI Workflow Recommendation

## Recommendation

- Primary platform: ${recommendation.primary.platform}
- Secondary platform: ${recommendation.secondary.platform}
- Confidence: ${recommendation.confidence}%
- Production escalation: ${
      recommendation.requiresProductionEscalation ? "Required" : "Not currently requ
    }
- Supabase required: ${recommendation.needsSupabase ? "Yes" : "No"}

## Inputs

| Requirement | Selected value |
|---|---|
| Deliverable | ${answers.deliverable} |
| Interaction | ${answers.interaction} |
| Backend | ${answers.backend} |
| Technology target | ${answers.techStack} |
| Supabase capability | ${answers.supabaseCapability} |
| Handoff | ${answers.handoff} |

## Why this recommendation

${recommendation.primary.reasons.map((reason) => `- ${reason}`).join("\n")}

## Build prompt

\`\`\`
${buildPrompt}
\`\`\`

## GitHub handoff

${githubPlan}
```

```
## Supabase implementation plan

${supabasePlan}
`;
  }, [answers, recommendation, buildPrompt, githubPlan, supabasePlan]);

  const sortedPlatforms = useMemo(() => {
    return [...PLATFORM_DATA].sort((a, b) => {
      const aValue = a[sort.key];
      const bValue = b[sort.key];

      return sort.direction === "asc"
        ? String(aValue).localeCompare(String(bValue))
        : String(bValue).localeCompare(String(aValue));
    });
  }, [sort]);

  const updateAnswer = <K extends keyof Answers>(
    field: K,
    value: Answers[K]
  ) => {
    setAnswers((current) => ({ ...current, [field]: value }));
  };

  const toggleSort = (key: typeof sort.key) => {
    setSort((current) => ({
      key,
      direction:
        current.key === key && current.direction === "asc" ? "desc" : "asc",
    }));
  };

  const matrixHeader = (label: string, key: typeof sort.key) => (
    <button
      onClick={() => toggleSort(key)}
      className="flex items-center gap-1 text-left text-xs font-bold uppercase track
    >
      {label}
      {sort.key === key ? (sort.direction === "asc" ? "↑" : "↓") : "↕"}
    </button>
  );

  return (
    <main className="min-h-screen bg-slate-50 p-4 text-slate-900 md:p-8">
      <div className="mx-auto max-w-7xl space-y-6">
        <header className="rounded-2xl bg-gradient-to-br from-slate-950 via-slate-900
          <p className="text-xs font-bold uppercase tracking-[0.16em] text-indigo-300
            Semester build workflow
          </p>

          <h1 className="mt-2 text-3xl font-bold tracking-tight">
            Claude Artifacts vs Canvas vs v0 Router
          </h1>

          <p className="mt-2 max-w-3xl text-sm leading-6 text-slate-300">
```

```
            Choose the right workspace based on project scope, coding workflow,
            persistence, Supabase requirements, GitHub handoff, and deployment.
          </p>
        </header>

        <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
              Project decision tree
            </p>

            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Select what the project needs
            </h2>

            <p className="mt-1 text-sm text-slate-600">
              The recommendation shifts to v0 when the project requires a real
              Next.js architecture, Postgres persistence, Supabase Auth, RLS,
              multi-tenancy, server routes, or deployment.
            </p>
          </div>

          <div className="mt-5 grid gap-4 md:grid-cols-2 xl:grid-cols-3">
            <SelectSection
              label="Deliverable"
              field="deliverable"
              choices={OPTIONS.deliverable}
              value={answers.deliverable}
              onChange={updateAnswer}
            />

            <SelectSection
              label="Interaction"
              field="interaction"
              choices={OPTIONS.interaction}
              value={answers.interaction}
              onChange={updateAnswer}
            />

            <SelectSection
              label="Data and backend"
              field="backend"
              choices={OPTIONS.backend}
              value={answers.backend}
              onChange={updateAnswer}
            />

            <SelectSection
              label="Technology target"
              field="techStack"
              choices={OPTIONS.techStack}
              value={answers.techStack}
              onChange={updateAnswer}
            />

            <SelectSection
```

```
            label="Supabase capability"
            field="supabaseCapability"
            choices={OPTIONS.supabaseCapability}
            value={answers.supabaseCapability}
            onChange={updateAnswer}
          />

          <SelectSection
            label="Handoff"
            field="handoff"
            choices={OPTIONS.handoff}
            value={answers.handoff}
            onChange={updateAnswer}
          />
        </div>

        <div className="mt-5 rounded-2xl border border-indigo-200 bg-indigo-50 p-5"
          <div className="flex flex-col justify-between gap-4 lg:flex-row lg:items-
            <div>
              <p className="text-xs font-bold uppercase tracking-[0.14em] text-indi
                Primary recommendation
              </p>

              <div className="mt-2 flex flex-wrap items-center gap-2">
                <h3 className="text-2xl font-bold text-slate-950">
                  {recommendation.primary.platform}
                </h3>

                <PlatformBadge platform={recommendation.primary.platform} />

                <Badge className="border-indigo-200 bg-white text-indigo-700">
                  {recommendation.confidence}% confidence
                </Badge>
              </div>

              <ul className="mt-4 max-w-3xl space-y-2 text-sm leading-6 text-slate-
                {recommendation.primary.reasons.map((reason) => (
                  <li key={reason}>• {reason}</li>
                ))}
              </ul>
            </div>

            <div className="rounded-xl border border-indigo-100 bg-white p-4 lg:w-8
              <p className="text-xs font-bold uppercase tracking-wide text-slate-50
                Secondary option
              </p>

              <p className="mt-1 text-base font-bold text-slate-900">
                {recommendation.secondary.platform}
              </p>

              <p className="mt-2 text-sm leading-6 text-slate-600">
                {recommendation.secondary.primaryUse}
              </p>
            </div>
          </div>
```

```
            {recommendation.requiresProductionEscalation && (
              <div className="mt-5 rounded-xl border border-amber-200 bg-amber-50 p-4
                <p className="text-sm font-bold text-amber-900">
                  Production handoff required
                </p>

                <p className="mt-1 text-sm leading-6 text-amber-800">
                  This project needs infrastructure beyond a self-contained
                  artifact: persistent records, server logic, Supabase, auth,
                  tenant boundaries, private integrations, or deployment. Use
                  Claude or Canvas for prototypes and specifications, then
                  implement the durable version in a repository-backed v0 and
                  Next.js project.
                </p>
              </div>
            )}
          </div>
        </section>

        <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
          <div className="flex flex-col justify-between gap-3 md:flex-row md:items-en
            <div>
              <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo
                Ready-to-copy build prompt
              </p>

              <h2 className="mt-1 text-xl font-bold text-slate-950">
                Platform-specific implementation brief
              </h2>
            </div>

            <div className="flex flex-wrap gap-2">
              <CopyButton value={buildPrompt} label="Copy prompt" />

              <button
                onClick={() =>
                  downloadFile(
                    "semester-ai-workflow-recommendation.md",
                    markdownExport,
                    "text/markdown;charset=utf-8"
                  )
                }
                className="rounded-lg bg-slate-900 px-3 py-1.5 text-xs font-bold tex
              >
                Export Markdown
              </button>

              <button
                onClick={() =>
                  downloadFile(
                    "semester-ai-workflow-recommendation.json",
                    JSON.stringify(
                      {
                        answers,
                        recommendation: {
```

```
                      primary: recommendation.primary.platform,
                      secondary: recommendation.secondary.platform,
                      confidence: recommendation.confidence,
                      needsSupabase: recommendation.needsSupabase,
                      needsSupabaseAuth: recommendation.needsSupabaseAuth,
                      needsTenantIsolation:
                        recommendation.needsTenantIsolation,
                      requiresProductionEscalation:
                        recommendation.requiresProductionEscalation,
                      reasons: recommendation.primary.reasons,
                    },
                    buildPrompt,
                    githubPlan,
                    supabasePlan,
                  },
                  null,
                  2
                ),
                "application/json;charset=utf-8"
              )
            }
            className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 te
          >
            Export JSON
          </button>
        </div>
      </div>

      <pre className="mt-4 whitespace-pre-wrap rounded-xl bg-slate-950 p-4 text-x
        {buildPrompt}
      </pre>
    </section>

    <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
      <div className="flex flex-col justify-between gap-3 md:flex-row md:items-en
        <div>
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo
            GitHub handoff
          </p>

          <h2 className="mt-1 text-xl font-bold text-slate-950">
            Version-control and deployment plan
          </h2>
        </div>

        <CopyButton value={githubPlan} label="Copy GitHub plan" />
      </div>

      <pre className="mt-4 whitespace-pre-wrap rounded-xl bg-slate-950 p-4 text-x
        {githubPlan}
      </pre>
    </section>

    <SupabasePanel
      recommendation={recommendation}
      supabasePlan={supabasePlan}
```

```
        />

        <section className="overflow-hidden rounded-2xl border border-slate-200 bg-wh
          <div className="border-b border-slate-200 p-5">
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
              Sortable platform matrix
            </p>

            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Execution limits, React support, and export formats
            </h2>

            <p className="mt-1 text-sm text-slate-600">
              Sort the table by any column, then open a platform for full
              implementation and sandbox details.
            </p>
          </div>

          <div className="overflow-x-auto">
            <table className="w-full min-w-[1500px] border-collapse">
              <thead className="bg-slate-50">
                <tr>
                  <th className="px-4 py-3">
                    {matrixHeader("Platform", "platform")}
                  </th>

                  <th className="px-4 py-3">
                    {matrixHeader("Primary use", "primaryUse")}
                  </th>

                  <th className="px-4 py-3">
                    {matrixHeader("Execution model", "execution")}
                  </th>

                  <th className="px-4 py-3">
                    {matrixHeader("React support", "reactSupport")}
                  </th>

                  <th className="px-4 py-3">
                    {matrixHeader("Project shape", "projectShape")}
                  </th>

                  <th className="px-4 py-3">
                    {matrixHeader("Export formats", "exportFormats")}
                  </th>

                  <th className="px-4 py-3">
                    {matrixHeader("Production fit", "productionFit")}
                  </th>

                  <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                    Details
                  </th>
                </tr>
              </thead>
```

```
                <tbody>
                  {sortedPlatforms.map((platform) => (
                    <tr
                      key={platform.id}
                      className="border-t border-slate-100 align-top transition hover:b
                    >
                      <td className="px-4 py-4">
                        <p className="font-bold text-slate-950">
                          {platform.platform}
                        </p>

                        <div className="mt-2 flex flex-wrap gap-1">
                          <Badge className="border-slate-200 bg-slate-50 text-slate-600
                            Prototype {platform.scores.prototype}/5
                          </Badge>

                          <Badge className="border-slate-200 bg-slate-50 text-slate-600
                            Full stack {platform.scores.fullStack}/5
                          </Badge>

                          <Badge className="border-slate-200 bg-slate-50 text-slate-600
                            Deploy {platform.scores.deployment}/5
                          </Badge>
                        </div>
                      </td>

                      <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                        {platform.primaryUse}
                      </td>

                      <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                        {platform.execution}
                      </td>

                      <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                        {platform.reactSupport}
                      </td>

                      <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                        {platform.projectShape}
                      </td>

                      <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                        {platform.exportFormats}
                      </td>

                      <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                        {platform.productionFit}
                      </td>

                      <td className="px-4 py-4">
                        <button
                          onClick={() => setSelectedPlatform(platform)}
                          className="rounded-lg border border-slate-200 bg-white px-3 p
                        >
                          Open
```

```
                    </button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </section>

      {selectedPlatform && (
        <div className="fixed inset-0 z-50 flex justify-end bg-slate-950/30">
          <button
            aria-label="Close platform detail panel"
            className="absolute inset-0 cursor-default"
            onClick={() => setSelectedPlatform(null)}
          />

          <aside className="relative h-full w-full max-w-2xl overflow-y-auto bg-wh
            <div className="flex items-start justify-between gap-4">
              <div>
                <PlatformBadge platform={selectedPlatform.platform} />

                <h2 className="mt-3 text-2xl font-bold text-slate-950">
                  {selectedPlatform.platform}
                </h2>
              </div>

              <button
                onClick={() => setSelectedPlatform(null)}
                className="rounded-lg border border-slate-200 px-3 py-1.5 text-sm
              >
                Close
              </button>
            </div>

            <div className="mt-6 space-y-5">
              {[
                ["Primary use", selectedPlatform.primaryUse],
                ["Execution model", selectedPlatform.execution],
                ["Sandbox limits", selectedPlatform.sandboxLimits],
                ["React support", selectedPlatform.reactSupport],
                ["Project shape", selectedPlatform.projectShape],
                ["Code and export options", selectedPlatform.exportFormats],
                ["GitHub workflow", selectedPlatform.githubWorkflow],
                ["Production handoff", selectedPlatform.productionFit],
                ["Best Semester use cases", selectedPlatform.bestFor],
              ].map(([label, value]) => (
                <div key={label}>
                  <h3 className="text-sm font-bold text-slate-900">
                    {label}
                  </h3>

                  <p className="mt-1 text-sm leading-6 text-slate-600">
                    {value}
                  </p>
                </div>
```

```
              ))}
            </div>
          </aside>
        </div>
      )}
    </div>
  </main>
);
}
```

Supabase client files

When the router recommends **v0 + Next.js + Supabase**, add the following files rather than putting database access directly into the client component.

Environment variables

```
# .env.local
NEXT_PUBLIC_SUPABASE_URL=https://YOUR_PROJECT.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=YOUR_PUBLISHABLE_KEY
```

The Supabase docs recommend separate browser and server clients for Next.js SSR. The browser client is for Client Components; the server client is for Server Components, Server Actions, and Route Handlers. [107] [104]

Browser client

```
// lib/supabase/client.ts
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!
  );
}
```

Server client

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
```

```
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) => {
              cookieStore.set(name, value, options);
            });
          } catch {
            // A Server Component cannot always write cookies.
            // Refresh sessions in middleware/proxy as needed.
          }
        },
      },
    }
  );
}
```

Server Action example

```
// app/artifacts/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";

type CreateArtifactInput = {
  title: string;
  category: string;
  format: string;
  lifecycle: string;
  priority: string;
  suggestedPrompt: string;
  organizationId: string;
};

export async function createArtifact(input: CreateArtifactInput) {
  const supabase = await createClient();

  const { data: authData, error: authError } =
    await supabase.auth.getUser();

  if (authError || !authData.user) {
    throw new Error("You must be signed in to create an artifact.");
  }

  const { error } = await supabase.from("artifacts").insert({
    organization_id: input.organizationId,
    title: input.title,
    category: input.category,
    format: input.format,
    lifecycle: input.lifecycle,
    priority: input.priority,
    suggested_prompt: input.suggestedPrompt,
```

```
    created_by: authData.user.id,
  });

  if (error) {
    throw new Error(error.message);
  }

  revalidatePath("/artifacts");
}
```

Schema and RLS

For Semester's artifact catalog, persist records in Postgres but enforce organization membership with **Row Level Security**. UI filters are not authorization.

```
create table public.organizations (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  created_at timestamptz not null default now()
);

create table public.organization_members (
  organization_id uuid not null references public.organizations(id) on delete cascade
  user_id uuid not null references auth.users(id) on delete cascade,
  role text not null check (role in ('owner', 'admin', 'member', 'viewer')),
  created_at timestamptz not null default now(),
  primary key (organization_id, user_id)
);

create table public.artifacts (
  id uuid primary key default gen_random_uuid(),
  organization_id uuid not null references public.organizations(id) on delete cascade
  created_by uuid not null references auth.users(id),
  title text not null,
  category text not null,
  format text not null,
  use_case text,
  lifecycle text not null,
  audience text not null default 'Internal',
  priority text not null default 'P2',
  status text not null default 'Backlog',
  owner_name text,
  tags text[] not null default '{}',
  suggested_prompt text not null,
  metadata jsonb not null default '{}',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index artifacts_organization_id_idx
  on public.artifacts (organization_id);

create index artifacts_category_idx
  on public.artifacts (category);
```

```
create index artifacts_format_idx
  on public.artifacts (format);

create index artifacts_status_idx
  on public.artifacts (status);

create index artifacts_updated_at_idx
  on public.artifacts (updated_at desc);

alter table public.organizations enable row level security;
alter table public.organization_members enable row level security;
alter table public.artifacts enable row level security;

create policy "Members can view organization artifacts"
on public.artifacts
for select
to authenticated
using (
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = artifacts.organization_id
      and membership.user_id = auth.uid()
  )
);

create policy "Members can create organization artifacts"
on public.artifacts
for insert
to authenticated
with check (
  created_by = auth.uid()
  and exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = artifacts.organization_id
      and membership.user_id = auth.uid()
      and membership.role in ('owner', 'admin', 'member')
  )
);

create policy "Editors can update organization artifacts"
on public.artifacts
for update
to authenticated
using (
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = artifacts.organization_id
      and membership.user_id = auth.uid()
      and membership.role in ('owner', 'admin', 'member')
  )
)
with check (
```

```
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = artifacts.organization_id
      and membership.user_id = auth.uid()
      and membership.role in ('owner', 'admin', 'member')
  )
);

create policy "Admins can delete organization artifacts"
on public.artifacts
for delete
to authenticated
using (
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = artifacts.organization_id
      and membership.user_id = auth.uid()
      and membership.role in ('owner', 'admin')
  )
);
```

How the routing changes

The decision tree now follows these practical rules:

Requirement chosen	Recommendation
Single-page prototype, diagram, calculator, or local-data tracker	Claude Artifacts
PRD, policy, strategy memo, research, code review, or exportable document	ChatGPT Canvas
React app with local state but no backend	Claude Artifacts, Canvas, or v0 depending on whether you need quick sharing versus project structure
Next.js application	v0
Postgres, Supabase database, server actions, APIs, or webhooks	v0
Supabase Auth, Storage, Realtime, or multi-tenant teams	v0 + Next.js + Supabase
GitHub workflow and production deployment	v0 + GitHub + Vercel

For Semester, the likely production path is:

```
Claude Artifact
  → Validate the decision-tree UX and artifact-catalog interaction
  → Export initial JSX / HTML / Mermaid / Markdown

ChatGPT Canvas
  → Create the product requirements, RLS policy explanation,
```

```
    operating docs, legal first drafts, and implementation notes
  → Export Markdown or code files

v0 + GitHub + Vercel + Supabase
  → Build the multi-file Next.js app
  → Add persistent artifact records and organization membership
  → Add auth, Row Level Security, exports, audit trail, and deployment
```

Supabase's official Next.js guidance says that `NEXT_PUBLIC_*` variables are intended for browser exposure while other environment variables remain server-only by default in Next.js. Its starter/docs also use `@supabase/ssr` to create separate browser and server clients for client components, server components, Server Actions, and Route Handlers. [105] [106] [104]

⁂

Claude Artifacts vs ChatGPT Canvas vs v0 sandbox: sortable feature matrix for execution limits, React support, and export formats

Computer
Build a prompt decision tree app for choosing between Claude Artifacts, ChatGPT Canvas, and v0 based on your specific project scope
Computer
Show me the rest of the React component code
Add a feature to copy the generated Supabase prompt
Export this whole app layout as a markdown document

Here is the final missing section: it adds a **Copy Supabase Prompt** control and exports the **entire app layout/specification** as a Markdown document—not merely the current recommendation.

The implementation keeps the app self-contained for prototype use, while the Supabase prompt intentionally routes persistent, authenticated, multi-tenant work to a v0/Next.js implementation. That is the correct boundary: Claude Artifacts and Canvas are strong for preview and iteration; v0 is the better target for a deployable app with a real backend. [120] [121]

Add this helper

Add this function near the other generators, after `createBuildPrompt`:

```
function createSupabasePrompt(
  answers: Answers,
  recommendation: Recommendation
) {
  if (!recommendation.needsSupabase) {
    return `Supabase is not required for the current project choices.

Build the first version with local sample data only. If this project later needs pers
  }
```

```
  const needsAuth = recommendation.needsSupabaseAuth;
  const needsTenancy = recommendation.needsTenantIsolation;
  const needsStorage = answers.supabaseCapability === "storage";
  const needsRealtime = answers.supabaseCapability === "realtime";

  return `Build the Semester project as a production-oriented Next.js App Router appl

Project requirements:
- Deliverable: ${answers.deliverable}
- Interaction model: ${answers.interaction}
- Backend requirement: ${answers.backend}
- Technology target: ${answers.techStack}
- Supabase capability: ${answers.supabaseCapability}
- Handoff requirement: ${answers.handoff}

Stack:
- Next.js with App Router and TypeScript
- Tailwind CSS
- Supabase Postgres
- @supabase/ssr for browser and server clients
- Vercel deployment
- GitHub repository with branch-based preview deployments

Required file structure:
- app/ for pages, layouts, route handlers, and server actions
- components/ for reusable client and server UI components
- lib/supabase/client.ts for createBrowserClient
- lib/supabase/server.ts for createServerClient
- lib/supabase/middleware.ts or proxy integration for auth-session refresh
- lib/validation/ for Zod validation schemas
- lib/db/ for query and domain helpers
- supabase/migrations/ for SQL migrations and RLS policies
- types/database.ts for database types
- .env.example with variable names only
- README.md with setup, migration, test, GitHub, and Vercel deployment instructions

Environment variables:
- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
- Never expose service-role credentials to the browser.
- Do not use NEXT_PUBLIC_ for secrets.
- Store production secrets in Vercel environment variables.

Database requirements:
- Create an artifacts table with:
  id, organization_id, created_by, title, category, format, use_case,
  lifecycle, audience, priority, status, owner_name, tags,
  suggested_prompt, metadata, created_at, updated_at.
${needsTenancy ? `- Create organizations and organization_members tables.
- Support owner, admin, member, and viewer roles.
- Scope every artifact record to organization_id.
- Add Row Level Security policies that enforce organization membership.
- Never rely on client-side UI logic alone for authorization.` : ""}
${needsAuth ? `- Use Supabase Auth for sign-up, sign-in, sign-out, and session-aware
- Protect authenticated routes on the server.
```

```
- Fetch the current user on the server before privileged mutations.` : ""}
${needsStorage ? `- Add Supabase Storage buckets for artifact exports and uploaded s
- Add object-storage policies so users can access only authorized organization assets
${needsRealtime ? `- Add Realtime only for records that benefit from live collaborat
- Scope channels to authorized organization data and handle reconnect/error states.`

Security and quality requirements:
- Enable Row Level Security on all exposed tables.
- Validate every write with Zod before database access.
- Enforce authorization in server actions and route handlers.
- Use database migrations committed to GitHub.
- Add indexes for organization_id, category, format, status, priority, and updated_a
- Use server-side operations for sensitive or privileged actions.
- Add loading, empty, error, and success states.
- Add typecheck, lint, and production build scripts.
- Add a test strategy for authorization and organization-scoped access.
- Add a basic audit-log strategy for sensitive changes.

Build the user interface as a polished, responsive B2B dashboard for Semester:
- Sortable and filterable artifact catalog
- Detail drawer for artifact metadata and generation prompts
- Prompt decision tree
- Copy-to-clipboard controls
- Markdown and JSON export
- Organization-aware records
- Clear permissions-aware UI states
- Accessible keyboard navigation and readable status/priority badges.`;
}
```

Supabase's Next.js SSR guidance recommends `@supabase/ssr` for separate browser/server client creation, and its Next.js setup requires browser-safe public environment variables while sensitive values remain server-only. [122] [123] [124]

Add Supabase copy control

Inside `SemesterWorkflowRouter`, directly after the existing `supabasePlan` declaration, add:

```
const supabasePrompt = useMemo(
  () => createSupabasePrompt(answers, recommendation),
  [answers, recommendation]
);
```

Then replace your current `SupabasePanel` component with this version:

```
function SupabasePanel({
  recommendation,
  supabasePlan,
  supabasePrompt,
}: {
  recommendation: Recommendation;
  supabasePlan: string;
  supabasePrompt: string;
}) {
```

```
  return (
    <section className="rounded-2xl border border-emerald-200 bg-emerald-50 p-5 shado
      <div className="flex flex-col justify-between gap-3 md:flex-row md:items-end">
        <div>
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-emerald-70
            Supabase integration
          </p>

          <h2 className="mt-1 text-xl font-bold text-emerald-950">
            {recommendation.needsSupabase
              ? "Production data, auth, and authorization plan"
              : "Supabase is currently optional"}
          </h2>

          <p className="mt-1 max-w-3xl text-sm leading-6 text-emerald-900">
            The generated prompt is designed for v0 or a normal Next.js project.
            It includes server/client Supabase boundaries, database structure,
            RLS requirements, GitHub handoff, and Vercel deployment guardrails.
          </p>
        </div>

        <div className="flex flex-wrap gap-2">
          <CopyButton value={supabasePlan} label="Copy Supabase plan" />
          <CopyButton value={supabasePrompt} label="Copy Supabase prompt" />
        </div>
      </div>

      <div className="mt-5 grid gap-4 xl:grid-cols-2">
        <div>
          <p className="text-sm font-bold text-emerald-950">
            Implementation plan
          </p>

          <pre className="mt-2 max-h-[440px] overflow-auto whitespace-pre-wrap rounde
            {supabasePlan}
          </pre>
        </div>

        <div>
          <p className="text-sm font-bold text-emerald-950">
            Ready-to-paste v0 / Next.js prompt
          </p>

          <pre className="mt-2 max-h-[440px] overflow-auto whitespace-pre-wrap rounde
            {supabasePrompt}
          </pre>
        </div>
      </div>
    </section>
  );
}
```

Finally, replace the existing `SupabasePanel` call in the main render with:

```
<SupabasePanel
  recommendation={recommendation}
  supabasePlan={supabasePlan}
  supabasePrompt={supabasePrompt}
/>
```

Export whole layout

Add the following helper above the main component. It exports a complete Markdown specification of the **router app itself**: product purpose, UI layout, matrix data, decision logic, generated recommendation, Supabase plan, GitHub plan, and source-style component map.

```
function createWholeAppMarkdown(
  answers: Answers,
  recommendation: Recommendation,
  buildPrompt: string,
  githubPlan: string,
  supabasePlan: string,
  supabasePrompt: string
) {
  const matrixRows = PLATFORM_DATA.map(
    (platform) => `| ${platform.platform} | ${platform.primaryUse} | ${platform.execu
  ).join("\n");

  const decisionGroups = Object.entries(OPTIONS)
    .map(([group, choices]) => {
      const rows = choices
        .map(([value, label]) => `| \`${value}\` | ${label} |`)
        .join("\n");

      return `### ${group}

| Value | Meaning |
|---|---|
${rows}`;
    })
    .join("\n\n");

  const recommendationReasons = recommendation.primary.reasons
    .map((reason) => `- ${reason}`)
    .join("\n");

  return `# Semester AI Workflow Router

## Purpose

A sortable feature matrix and interactive decision-tree application that helps a sof

1. Claude Artifacts
2. ChatGPT Canvas
3. v0

The app evaluates project scope, interaction complexity, persistence, backend needs,
```

Main UI Layout

\`\`\`text
SemesterWorkflowRouter
├── Header
│   ├── Product label
│   ├── App title
│   └── Purpose statement
├── Project Decision Tree
│   ├── Deliverable selector
│   ├── Interaction selector
│   ├── Backend selector
│   ├── Technology target selector
│   ├── Supabase capability selector
│   └── Handoff selector
├── Recommendation Card
│   ├── Primary recommendation
│   ├── Secondary option
│   ├── Confidence score
│   ├── Decision reasons
│   └── Production escalation warning
├── Build Prompt Panel
│   ├── Generated platform-specific prompt
│   ├── Copy prompt button
│   ├── Export current recommendation as Markdown
│   └── Export current recommendation as JSON
├── GitHub Handoff Panel
│   ├── Branch/repository/deployment plan
│   └── Copy GitHub plan button
├── Supabase Integration Panel
│   ├── Supabase implementation plan
│   ├── Copy Supabase plan button
│   ├── Ready-to-paste Supabase prompt
│   └── Copy Supabase prompt button
├── Sortable Platform Matrix
│   ├── Platform
│   ├── Primary use
│   ├── Execution model
│   ├── React support
│   ├── Project shape
│   ├── Export formats
│   ├── Production fit
│   └── Detail drawer trigger
└── Platform Detail Drawer
    ├── Sandbox limits
    ├── React support
    ├── Export options
    ├── GitHub workflow
    └── Best Semester use cases
\`\`\`

Feature Matrix

Platform	Primary use	Execution model	React support	Export formats	Produc

```
${matrixRows}

## Decision Inputs

${decisionGroups}

## Current Selection

| Requirement | Current selection |
|---|---|
| Deliverable | ${answers.deliverable} |
| Interaction | ${answers.interaction} |
| Backend | ${answers.backend} |
| Technology target | ${answers.techStack} |
| Supabase capability | ${answers.supabaseCapability} |
| Handoff | ${answers.handoff} |

## Current Recommendation

- **Primary platform:** ${recommendation.primary.platform}
- **Secondary platform:** ${recommendation.secondary.platform}
- **Confidence:** ${recommendation.confidence}%
- **Needs production escalation:** ${recommendation.requiresProductionEscalation ? "
- **Needs Supabase:** ${recommendation.needsSupabase ? "Yes" : "No"}
- **Needs Supabase Auth:** ${recommendation.needsSupabaseAuth ? "Yes" : "No"}
- **Needs tenant isolation:** ${recommendation.needsTenantIsolation ? "Yes" : "No"}

### Why

${recommendationReasons}

## Generated Build Prompt

\`\`\`text
${buildPrompt}
\`\`\`

## GitHub Handoff Plan

\`\`\`text
${githubPlan}
\`\`\`

## Supabase Implementation Plan

\`\`\`text
${supabasePlan}
\`\`\`

## Generated Supabase Prompt

\`\`\`text
${supabasePrompt}
\`\`\`

## Component Architecture
```

```
\`\`\`text
app/workflow-router/page.tsx
├── SemesterWorkflowRouter
├── SelectSection
├── SupabasePanel
├── PlatformBadge
├── Badge
├── CopyButton
├── createSupabasePrompt
├── createSupabasePlan
├── createBuildPrompt
├── createGitHubPlan
├── recommendPlatform
├── createWholeAppMarkdown
└── downloadFile
\`\`\`

## Production Handoff

For a durable Semester application:

\`\`\`text
Claude Artifact or Canvas
  → prototype UX, strategy, diagrams, and document drafts
  → export JSX / HTML / Mermaid / SVG / Markdown

GitHub
  → store source, migrations, RLS policies, documentation, and tests
  → use branches and pull requests

v0 + Next.js
  → production UI, routes, server actions, forms, and dashboard logic

Supabase
  → Postgres, authentication, organization membership, RLS, storage, realtime

Vercel
  → preview deployments, environment variables, and production deployment
\`\`\`
`;
}
```

Then add this memo inside `SemesterWorkflowRouter`, after `markdownExport`:

```
const wholeAppMarkdown = useMemo(
  () =>
    createWholeAppMarkdown(
      answers,
      recommendation,
      buildPrompt,
      githubPlan,
      supabasePlan,
      supabasePrompt
    ),
```

```
  [
    answers,
    recommendation,
    buildPrompt,
    githubPlan,
    supabasePlan,
    supabasePrompt,
  ]
);
```

Add a new button beside your existing Markdown and JSON export buttons:

```
<button
  onClick={() =>
    downloadFile(
      "semester-ai-workflow-router-layout.md",
      wholeAppMarkdown,
      "text/markdown;charset=utf-8"
    )
  }
  className="rounded-lg border border-indigo-200 bg-indigo-50 px-3 py-1.5 text-xs for
>
  Export whole app layout
</button>
```

This exports a Markdown artifact containing the current recommendation **and** the application's complete product/layout specification, decision-tree inputs, platform matrix, component hierarchy, GitHub plan, Supabase plan, and generated Supabase prompt.

Final button group

Your completed build-prompt action area should look like this:

```
<div className="flex flex-wrap gap-2">
  <CopyButton value={buildPrompt} label="Copy prompt" />

  <button
    onClick={() =>
      downloadFile(
        "semester-ai-workflow-recommendation.md",
        markdownExport,
        "text/markdown;charset=utf-8"
      )
    }
    className="rounded-lg bg-slate-900 px-3 py-1.5 text-xs font-bold text-white hover
  >
    Export recommendation
  </button>

  <button
    onClick={() =>
      downloadFile(
        "semester-ai-workflow-router-layout.md",
```

```
        wholeAppMarkdown,
        "text/markdown;charset=utf-8"
      )
    }
    className="rounded-lg border border-indigo-200 bg-indigo-50 px-3 py-1.5 text-xs
  >
    Export whole app layout
  </button>

  <button
    onClick={() =>
      downloadFile(
        "semester-ai-workflow-recommendation.json",
        JSON.stringify(
          {
            answers,
            recommendation: {
              primary: recommendation.primary.platform,
              secondary: recommendation.secondary.platform,
              confidence: recommendation.confidence,
              needsSupabase: recommendation.needsSupabase,
              needsSupabaseAuth: recommendation.needsSupabaseAuth,
              needsTenantIsolation: recommendation.needsTenantIsolation,
              requiresProductionEscalation:
                recommendation.requiresProductionEscalation,
              reasons: recommendation.primary.reasons,
            },
            buildPrompt,
            githubPlan,
            supabasePlan,
            supabasePrompt,
          },
          null,
          2
        ),
        "application/json;charset=utf-8"
      )
    }
    className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 text-xs font-
  >
    Export JSON
  </button>
</div>
```

Platform boundary

Platform	Best initial output	Export/handoff	Do not use as final implementation when
Claude Artifacts	Single-file React tool, HTML page, SVG, Mermaid diagram, interactive prototype	Export/copy source to GitHub	You need real auth, durable records, server secrets, database access, multi-tenancy, or deployment
ChatGPT Canvas	Editable document, code file, analysis, policy, PRD, strategy memo	PDF, DOCX, Markdown, language-specific source	You need an ongoing application runtime or server-integrated product

Platform	Best initial output	Export/handoff	Do not use as final implementation when
v0	Multi-file React/Next.js application	GitHub sync, Vercel preview, production deployment	You only need a short-lived document or lightweight artifact
Supabase	Postgres, Auth, Storage, Realtime, organization data, Row Level Security	Migrations and policies committed to GitHub	You merely need local mock data for a prototype

Claude Design's current export options include ZIP, PDF, PPTX, Google Slides, standalone HTML, handoff to Claude Code/local coding agents, and sends to platforms including v0 and Vercel. The practical workflow remains: create the fast interactive artifact or document first, then use GitHub + v0 + Supabase when the work needs production persistence, permissions, and deployment. [120]

⁂

Sortable comparison matrix: Claude Artifacts vs ChatGPT Canvas vs v0 sandbox — execution limits, React support, and export formats compared

Computer
Build a prompt decision tree web app that maps your exact workflow to Claude Artifacts, ChatGPT Canvas, or v0 sandbox based on your tech stack
Computer
Show me the full decision tree app code
Add a feature matrix table to the app
Export the code as a single HTML file

Below is a complete **single-file HTML application**—not a React component—that you can save as `semester-ai-workflow-router.html` and open directly in a browser. It includes:

- A sortable comparison matrix for Claude Artifacts, ChatGPT Canvas, and v0
- Execution limits, React support, project shape, and export-format comparisons
- A requirements-based decision tree that chooses the best platform
- Tech-stack and backend rules, including Next.js, Postgres, and Supabase
- A generated platform-specific implementation prompt
- A feature matrix table
- Copy buttons
- Browser downloads for the current recommendation as Markdown/JSON
- A button that exports the entire app source as a single HTML file

Claude Artifacts can create interactive pages, dashboards, designs, decks, and documents, and Claude documents export to common formats such as PowerPoint, PDF, and HTML; published

Claude Code artifacts use HTML, HTM, or Markdown source and can generate viewer downloads such as CSV or PNG. [133] [134]

Full standalone HTML

Save the complete block below as `semester-ai-workflow-router.html`:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1"
  />
  <meta
    name="description"
    content="A sortable platform comparison matrix and workflow decision tree for Cla
  />
  <title>Semester AI Workflow Router</title>

  <style>
    :root {
      --bg: #f8fafc;
      --surface: #ffffff;
      --surface-muted: #f1f5f9;
      --border: #e2e8f0;
      --border-strong: #cbd5e1;
      --text: #0f172a;
      --text-muted: #64748b;
      --dark: #0f172a;
      --dark-2: #172554;
      --indigo: #4f46e5;
      --indigo-light: #eef2ff;
      --indigo-border: #c7d2fe;
      --green-light: #ecfdf5;
      --green-border: #a7f3d0;
      --green-text: #047857;
      --amber-light: #fffbeb;
      --amber-border: #fde68a;
      --amber-text: #92400e;
      --rose-light: #fff1f2;
      --rose-border: #fecdd3;
      --rose-text: #be123c;
      --shadow: 0 10px 30px rgba(15, 23, 42, 0.08);
      --radius: 18px;
      --radius-sm: 10px;
    }

    * {
      box-sizing: border-box;
    }

    html {
      scroll-behavior: smooth;
```

```
}

body {
  margin: 0;
  min-width: 320px;
  background: var(--bg);
  color: var(--text);
  font-family:
    Inter,
    ui-sans-serif,
    system-ui,
    -apple-system,
    BlinkMacSystemFont,
    "Segoe UI",
    sans-serif;
  line-height: 1.5;
}

button,
input,
select {
  font: inherit;
}

button {
  cursor: pointer;
}

.page {
  width: min(1440px, calc(100% - 32px));
  margin: 0 auto;
  padding: 32px 0 56px;
}

.hero {
  background:
    radial-gradient(circle at top right, rgba(99, 102, 241, 0.28), transparent 38
    linear-gradient(135deg, #020617, #0f172a 48%, #1e1b4b);
  border-radius: var(--radius);
  padding: 32px;
  color: #fff;
  box-shadow: var(--shadow);
}

.eyebrow {
  margin: 0;
  color: #a5b4fc;
  font-size: 11px;
  font-weight: 800;
  letter-spacing: 0.16em;
  text-transform: uppercase;
}

h1,
h2,
h3,
```

```
p {
  margin-top: 0;
}

h1 {
  max-width: 920px;
  margin: 8px 0 0;
  font-size: clamp(28px, 4vw, 44px);
  line-height: 1.08;
  letter-spacing: -0.04em;
}

.hero-copy {
  max-width: 820px;
  margin: 14px 0 0;
  color: #cbd5e1;
  font-size: 15px;
  line-height: 1.7;
}

.hero-stats {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 12px;
  margin-top: 28px;
}

.hero-stat {
  border: 1px solid rgba(255, 255, 255, 0.12);
  border-radius: 14px;
  padding: 14px;
  background: rgba(255, 255, 255, 0.08);
}

.hero-stat strong {
  display: block;
  font-size: 22px;
}

.hero-stat span {
  display: block;
  margin-top: 3px;
  color: #cbd5e1;
  font-size: 12px;
}

.section {
  margin-top: 24px;
  border: 1px solid var(--border);
  border-radius: var(--radius);
  background: var(--surface);
  box-shadow: 0 3px 16px rgba(15, 23, 42, 0.04);
}

.section-header {
  display: flex;
```

```
    align-items: flex-end;
    justify-content: space-between;
    gap: 20px;
    padding: 22px 24px 0;
  }

  .section-title {
    margin: 4px 0 0;
    font-size: 22px;
    letter-spacing: -0.02em;
  }

  .section-copy {
    max-width: 850px;
    margin: 8px 0 0;
    color: var(--text-muted);
    font-size: 14px;
    line-height: 1.65;
  }

  .section-body {
    padding: 24px;
  }

  .decision-grid {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 16px;
  }

  .field-card {
    border: 1px solid var(--border);
    border-radius: 14px;
    background: #f8fafc;
    padding: 16px;
  }

  .field-card h3 {
    margin: 0 0 12px;
    color: #1e293b;
    font-size: 14px;
  }

  .choice-list {
    display: grid;
    gap: 9px;
  }

  .choice {
    display: flex;
    align-items: flex-start;
    gap: 9px;
    color: #334155;
    font-size: 13px;
    line-height: 1.4;
    cursor: pointer;
```

```
}

.choice input {
  margin: 2px 0 0;
  accent-color: var(--indigo);
}

.recommendation {
  display: grid;
  grid-template-columns: minmax(0, 1.7fr) minmax(260px, 0.7fr);
  gap: 20px;
  margin-top: 20px;
  border: 1px solid var(--indigo-border);
  border-radius: 16px;
  padding: 20px;
  background: linear-gradient(135deg, var(--indigo-light), #fff);
}

.recommendation-label {
  margin: 0;
  color: var(--indigo);
  font-size: 11px;
  font-weight: 800;
  letter-spacing: 0.14em;
  text-transform: uppercase;
}

.recommendation-title-row {
  display: flex;
    align-items: center;
  flex-wrap: wrap;
  gap: 9px;
  margin-top: 6px;
}

.recommendation-title {
  margin: 0;
  font-size: 28px;
  letter-spacing: -0.03em;
}

.badge {
  display: inline-flex;
  align-items: center;
  border: 1px solid;
  border-radius: 999px;
  padding: 3px 9px;
  font-size: 11px;
  font-weight: 800;
  white-space: nowrap;
}

.badge-indigo {
  border-color: var(--indigo-border);
  background: #fff;
  color: var(--indigo);
```

```
}

.badge-claude {
  border-color: #fde68a;
  background: #fffbeb;
  color: #92400e;
}

.badge-canvas {
  border-color: #a7f3d0;
  background: #ecfdf5;
  color: #047857;
}

.badge-v0 {
  border-color: #0f172a;
  background: #0f172a;
  color: #fff;
}

.reason-list {
  display: grid;
  gap: 7px;
  margin: 16px 0 0;
  padding: 0;
  list-style: none;
  color: #334155;
  font-size: 14px;
  line-height: 1.5;
}

.reason-list li::before {
  content: "•";
  margin-right: 8px;
  color: var(--indigo);
  font-weight: 900;
}

.secondary-card {
  border: 1px solid var(--indigo-border);
  border-radius: 12px;
  padding: 16px;
  background: rgba(255, 255, 255, 0.86);
}

.secondary-card p:last-child {
  margin-bottom: 0;
}

.secondary-label {
  margin-bottom: 4px;
  color: var(--text-muted);
  font-size: 11px;
  font-weight: 800;
  letter-spacing: 0.12em;
  text-transform: uppercase;
```

```
}

.warning {
  margin-top: 16px;
  border: 1px solid var(--amber-border);
  border-radius: 12px;
  padding: 14px;
  background: var(--amber-light);
  color: var(--amber-text);
  font-size: 14px;
  line-height: 1.6;
}

.warning strong {
  display: block;
  margin-bottom: 4px;
}

.button-row {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

.button {
  border: 1px solid var(--border-strong);
  border-radius: 9px;
  padding: 9px 12px;
  background: #fff;
  color: #334155;
  font-size: 12px;
  font-weight: 800;
  transition: 160ms ease;
}

.button:hover {
  border-color: #a5b4fc;
  color: var(--indigo);
  transform: translateY(-1px);
}

.button-primary {
  border-color: var(--dark);
  background: var(--dark);
  color: #fff;
}

.button-primary:hover {
  border-color: #334155;
  background: #334155;
  color: #fff;
}

.button-indigo {
  border-color: var(--indigo);
  background: var(--indigo);
```

```
  color: #fff;
}

.button-indigo:hover {
  border-color: #4338ca;
  background: #4338ca;
  color: #fff;
}

.button-soft {
  border-color: var(--indigo-border);
  background: var(--indigo-light);
  color: #4338ca;
}

.button-soft:hover {
  background: #e0e7ff;
  color: #3730a3;
}

.prompt-grid {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
  gap: 18px;
  margin-top: 16px;
}

.code-panel {
  min-width: 0;
}

.code-panel-title {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 8px;
}

.code-panel-title h3 {
  margin: 0;
  font-size: 14px;
}

pre {
  max-height: 430px;
  overflow: auto;
  margin: 0;
  border-radius: 12px;
  padding: 16px;
  background: #020617;
  color: #dbeafe;
  font-family:
    "SFMono-Regular",
    Consolas,
    "Liberation Mono",
```

```
      Menlo,
      monospace;
    font-size: 12px;
    line-height: 1.65;
    white-space: pre-wrap;
    word-break: break-word;
  }

  .matrix-wrap {
    overflow-x: auto;
    padding: 0 24px 24px;
  }

  table {
    width: 100%;
    min-width: 1500px;
    border-collapse: collapse;
  }

  thead {
    background: #f8fafc;
  }

  th,
  td {
    border-top: 1px solid var(--border);
    padding: 15px;
    vertical-align: top;
    text-align: left;
  }

  th {
    border-top: 0;
    color: var(--text-muted);
    font-size: 11px;
    font-weight: 800;
    letter-spacing: 0.05em;
    text-transform: uppercase;
  }

  th button {
    border: 0;
    padding: 0;
    background: transparent;
    color: inherit;
    font-size: inherit;
    font-weight: inherit;
    letter-spacing: inherit;
    text-transform: inherit;
  }

  th button:hover {
    color: var(--indigo);
  }

  td {
```

```
      max-width: 290px;
      color: #475569;
      font-size: 13px;
      line-height: 1.6;
    }

    .platform-name {
      color: var(--text);
      font-size: 14px;
      font-weight: 800;
    }

    .score-row {
      display: flex;
      flex-wrap: wrap;
      gap: 4px;
      margin-top: 8px;
    }

    .mini-badge {
      border: 1px solid var(--border);
      border-radius: 999px;
      padding: 2px 6px;
      background: #f8fafc;
      color: #64748b;
      font-size: 10px;
      font-weight: 700;
    }

    .feature-table {
      min-width: 1000px;
    }

    .feature-status {
      font-size: 12px;
      font-weight: 800;
    }

    .feature-high {
      color: #047857;
    }

    .feature-medium {
      color: #b45309;
    }

    .feature-low {
      color: #b91c1c;
    }

    .modal {
      position: fixed;
      z-index: 20;
      inset: 0;
      display: none;
      justify-content: flex-end;
```

```
    background: rgba(2, 6, 23, 0.4);
  }

  .modal.open {
    display: flex;
  }

  .drawer {
    width: min(640px, 100%);
    height: 100%;
    overflow-y: auto;
    padding: 28px;
    background: #fff;
    box-shadow: -20px 0 50px rgba(2, 6, 23, 0.18);
  }

  .drawer-header {
    display: flex;
    align-items: flex-start;
    justify-content: space-between;
    gap: 16px;
  }

  .drawer-section {
    margin-top: 24px;
  }

  .drawer-section h3 {
    margin-bottom: 6px;
    font-size: 14px;
  }

  .drawer-section p {
    margin-bottom: 0;
    color: #475569;
    font-size: 14px;
    line-height: 1.65;
  }

  .footer-note {
    margin: 20px 0 0;
    color: #64748b;
    font-size: 12px;
    line-height: 1.6;
  }

  @media (max-width: 1080px) {
    .decision-grid {
      grid-template-columns: repeat(2, minmax(0, 1fr));
    }

    .hero-stats {
      grid-template-columns: repeat(2, minmax(0, 1fr));
    }
  }
```

```
    @media (max-width: 760px) {
      .page {
        width: min(100% - 20px, 1440px);
        padding-top: 10px;
      }

      .hero,
      .section-body {
        padding: 18px;
      }

      .section-header {
        align-items: flex-start;
        flex-direction: column;
        padding: 18px 18px 0;
      }

      .decision-grid,
      .recommendation,
      .prompt-grid {
        grid-template-columns: 1fr;
      }

      .matrix-wrap {
        padding: 0 18px 18px;
      }

      .hero-stats {
        grid-template-columns: 1fr 1fr;
      }
    }
  </style>
</head>

<body>
  <main class="page">
    <header class="hero">
      <p class="eyebrow">Semester build workflow</p>

      <h1>Claude Artifacts vs ChatGPT Canvas vs v0</h1>

      <p class="hero-copy">
        Compare execution boundaries, React support, export paths, and project
        shape. Then use the workflow decision tree to choose the right starting
        environment for a specific Semester deliverable.
      </p>

      <div class="hero-stats">
        <div class="hero-stat">
          <strong>3</strong>
          <span>Platforms compared</span>
        </div>
        <div class="hero-stat">
          <strong>6</strong>
          <span>Decision dimensions</span>
        </div>
```

```
        <div class="hero-stat">
          <strong>7</strong>
          <span>Project workflow types</span>
        </div>
        <div class="hero-stat">
          <strong>HTML</strong>
          <span>Standalone export</span>
        </div>
      </div>
    </header>

    <section class="section">
      <div class="section-header">
        <div>
          <p class="eyebrow" style="color:#4f46e5;">Decision tree</p>
          <h2 class="section-title">Map project scope to the right platform</h2>
          <p class="section-copy">
            Select the project characteristics that matter. The scoring logic
            recommends Claude Artifacts for portable, self-contained work;
            Canvas for document and code iteration; and v0 for real Next.js
            applications, Postgres, Supabase, authentication, GitHub, and Vercel depl
          </p>
        </div>
      </div>

      <div class="section-body">
        <div id="decision-grid" class="decision-grid"></div>

        <div id="recommendation" class="recommendation"></div>
      </div>
    </section>

    <section class="section">
      <div class="section-header">
        <div>
          <p class="eyebrow" style="color:#4f46e5;">Generated handoff</p>
          <h2 class="section-title">Platform prompt, GitHub plan, and export</h2>
          <p class="section-copy">
            The recommendation creates a build prompt and a repository handoff
            plan. Export the current recommendation as Markdown or JSON, or
            download this complete application as a standalone HTML file.
          </p>
        </div>

        <div class="button-row">
          <button id="export-markdown" class="button button-primary">
            Export recommendation
          </button>

          <button id="export-json" class="button">
            Export JSON
          </button>

          <button id="export-html" class="button button-soft">
            Export full HTML app
          </button>
```

```
        </div>
      </div>

      <div class="section-body">
        <div class="prompt-grid">
          <div class="code-panel">
            <div class="code-panel-title">
              <h3>Ready-to-copy build prompt</h3>
              <button id="copy-build-prompt" class="button">Copy prompt</button>
            </div>

            <pre id="build-prompt-output"></pre>
          </div>

          <div class="code-panel">
            <div class="code-panel-title">
              <h3>GitHub and production handoff</h3>
              <button id="copy-github-plan" class="button">Copy plan</button>
            </div>

            <pre id="github-plan-output"></pre>
          </div>
        </div>

        <p class="footer-note">
          This browser-only prototype stores selections in memory. For a shared
          Semester workspace, move the decision data into a Next.js project,
          store records in Supabase/Postgres, enforce organization-scoped access
          with Row Level Security, and deploy with GitHub plus Vercel.
        </p>
      </div>
    </section>

    <section class="section">
      <div class="section-header">
        <div>
          <p class="eyebrow" style="color:#4f46e5;">Feature matrix</p>
          <h2 class="section-title">Capability comparison</h2>
          <p class="section-copy">
            This is a fast operating view of what each environment is best
            positioned to do. “High” means a natural primary fit, not merely
            that the task is technically possible.
          </p>
        </div>
      </div>

      <div class="matrix-wrap">
        <table class="feature-table">
          <thead>
            <tr>
              <th>Feature</th>
              <th>Claude Artifacts</th>
              <th>ChatGPT Canvas</th>
              <th>v0</th>
            </tr>
          </thead>
```

```
          <tbody id="feature-matrix-body"></tbody>
        </table>
      </div>
    </section>

    <section class="section">
      <div class="section-header">
        <div>
          <p class="eyebrow" style="color:#4f46e5;">Sortable platform matrix</p>
          <h2 class="section-title">
            Execution limits, React support, and export formats
          </h2>
          <p class="section-copy">
            Click a header to sort. Use the detail button to review sandbox
            constraints, recommended handoff, and ideal Semester use cases.
          </p>
        </div>
      </div>

      <div class="matrix-wrap">
        <table>
          <thead>
            <tr>
              <th><button data-sort="platform">Platform ↕</button></th>
              <th><button data-sort="primaryUse">Primary use ↕</button></th>
              <th><button data-sort="execution">Execution model ↕</button></th>
              <th><button data-sort="reactSupport">React support ↕</button></th>
              <th><button data-sort="projectShape">Project shape ↕</button></th>
              <th><button data-sort="exportFormats">Export formats ↕</button></th>
              <th><button data-sort="productionFit">Production fit ↕</button></th>
              <th>Details</th>
            </tr>
          </thead>

          <tbody id="platform-matrix-body"></tbody>
        </table>
      </div>
    </section>
  </main>

  <div id="detail-modal" class="modal" aria-hidden="true">
    <aside
      class="drawer"
      role="dialog"
      aria-modal="true"
      aria-labelledby="drawer-title"
    >
      <div class="drawer-header">
        <div>
          <div id="drawer-badge"></div>
          <h2 id="drawer-title" class="section-title"></h2>
        </div>

        <button id="close-drawer" class="button">Close</button>
      </div>
```

```
    <div id="drawer-content"></div>
  </aside>
</div>

<script>
  const PLATFORM_DATA = [
    {
      id: "claude-artifacts",
      platform: "Claude Artifacts",
      primaryUse:
        "Shareable self-contained artifacts, interactive tools, diagrams, visual ex
      execution:
        "Contained rendered artifact experience. Best treated as a client-side arti
      sandboxLimits:
        "Do not rely on it as the final home for durable records, server secrets, p
      reactSupport:
        "Good for contained React-style interactive tools, trackers, calculators, l
      projectShape:
        "Usually a single contained artifact, page, component, HTML file, SVG, Merm
      exportFormats:
        "HTML, React/JSX-style source, Markdown, SVG, Mermaid, plus browser-generat
      githubWorkflow:
        "Export or copy source into GitHub. Store HTML/JSX in app or src, Mermaid/M
      productionFit:
        "Prototype or visual specification → export source → GitHub → Next.js/Supab
      bestFor:
        "Semester artifact catalog prototype, format decision tree, ROI calculator
      scores: {
        prototype: 5,
        documents: 4,
        diagrams: 5,
        fullStack: 1,
        deployment: 2
      }
    },
    {
      id: "chatgpt-canvas",
      platform: "ChatGPT Canvas",
      primaryUse:
        "Iterative document and code workspace for writing, editing, revision, tran
      execution:
        "React and HTML preview in an isolated workspace environment. Python can su
      sandboxLimits:
        "Treat Canvas as a drafting and preview environment rather than a durable b
      reactSupport:
        "Useful for iterating React/HTML and individual code files. Strongest for e
      projectShape:
        "Typically an editable document, code file, or contained work item optimize
      exportFormats:
        "Documents: PDF, DOCX, Markdown. Code: language-specific files such as Java
      githubWorkflow:
        "Export Markdown/source, commit editable source to GitHub, and treat PDF/DO
      productionFit:
        "Draft/revise/export → GitHub or documentation system → production codebase
      bestFor:
```

```
        "Semester PRDs, strategy, policies, legal first drafts, research synthesis
      scores: {
        prototype: 3,
        documents: 5,
        diagrams: 3,
        fullStack: 1,
        deployment: 1
      }
    },
    {
      id: "v0",
      platform: "v0",
      primaryUse:
        "Production-oriented React and Next.js apps, multi-file projects, backend
      execution:
        "VM-backed isolated Node.js development environment for multi-file projects
      sandboxLimits:
        "A preview is still not production. Verify package versions, environment va
      reactSupport:
        "Native fit for React and Next.js, including multi-file components, App Rou
      projectShape:
        "Multi-file repository-style project with components, routes, dependencies
      exportFormats:
        "Application project code, GitHub sync/export, existing-codebase integratio
      githubWorkflow:
        "Connect or create a GitHub repository, work in branches, review preview de
      productionFit:
        "Best for database-backed, authenticated, multi-user Next.js applications w
      bestFor:
        "Semester production artifact catalog, shared team dashboard, authenticated
      scores: {
        prototype: 4,
        documents: 2,
        diagrams: 2,
        fullStack: 5,
        deployment: 5
      }
    }
  ];

  const FEATURE_MATRIX = [
    {
      feature: "Self-contained interactive prototype",
      claude: ["High", "feature-high"],
      canvas: ["Medium", "feature-medium"],
      v0: ["High", "feature-high"]
    },
    {
      feature: "PRDs, policies, strategy, research, and editing",
      claude: ["High", "feature-high"],
      canvas: ["High", "feature-high"],
      v0: ["Low", "feature-low"]
    },
    {
      feature: "Mermaid, SVG, and visual explainers",
      claude: ["High", "feature-high"],
```

```
      canvas: ["Medium", "feature-medium"],
      v0: ["Medium", "feature-medium"]
    },
    {
      feature: "React component experimentation",
      claude: ["High", "feature-high"],
      canvas: ["High", "feature-high"],
      v0: ["High", "feature-high"]
    },
    {
      feature: "Multi-file Next.js application",
      claude: ["Low", "feature-low"],
      canvas: ["Low", "feature-low"],
      v0: ["High", "feature-high"]
    },
    {
      feature: "Server routes and server-side secrets",
      claude: ["Low", "feature-low"],
      canvas: ["Low", "feature-low"],
      v0: ["High", "feature-high"]
    },
    {
      feature: "Postgres or Supabase persistence",
      claude: ["Low", "feature-low"],
      canvas: ["Low", "feature-low"],
      v0: ["High", "feature-high"]
    },
    {
      feature: "Authentication and multi-tenant authorization",
      claude: ["Low", "feature-low"],
      canvas: ["Low", "feature-low"],
      v0: ["High", "feature-high"]
    },
    {
      feature: "Document export",
      claude: ["Medium", "feature-medium"],
      canvas: ["High", "feature-high"],
      v0: ["Low", "feature-low"]
    },
    {
      feature: "GitHub workflow and Vercel deployment",
      claude: ["Medium", "feature-medium"],
      canvas: ["Medium", "feature-medium"],
      v0: ["High", "feature-high"]
    }
  ];

  const DECISION_OPTIONS = {
    deliverable: {
      label: "Deliverable",
      choices: [
        ["interactive-tool", "Interactive tool, tracker, dashboard, calculator, or
        ["product-prototype", "Clickable product prototype or interface explorati
        ["marketing-page", "Marketing site, landing page, one-page demo, or visual
        ["diagram", "Architecture, process, ERD, sequence, roadmap, or system diag
        ["document", "PRD, policy, research, strategy, report, or operating docume
```

```
        ["presentation", "Pitch, board update, sales narrative, launch, or training
        ["full-stack-feature", "Authenticated, data-backed, multi-user product fea
        ["analysis", "Python analysis, research, data cleanup, or generated-file wo
      ]
    },
    interaction: {
      label: "Interaction",
      choices: [
        ["none", "No interaction; people mainly read or review"],
        ["simple", "Simple client-side interaction: tabs, toggles, calculators, sar
        ["complex", "Stateful interface: filters, tables, forms, drawers, editable
        ["multi-user", "Shared records, multiple users, roles, permissions, or coll
      ]
    },
    backend: {
      label: "Data and backend",
      choices: [
        ["none", "No backend; static result or browser-only sample data"],
        ["file-export", "Needs browser-generated CSV, Markdown, SVG, JSON, PDF, or
        ["database", "Needs persistent database records"],
        ["server-api", "Needs API routes, server logic, webhooks, or third-party se
        ["auth-secrets", "Needs sign-in, permissions, billing, private APIs, or ser
      ]
    },
    techStack: {
      label: "Technology target",
      choices: [
        ["none", "No prescribed code stack"],
        ["html", "Plain HTML, CSS, and JavaScript"],
        ["react", "React component or client-side application"],
        ["nextjs", "Multi-file Next.js application"],
        ["nextjs-postgres", "Next.js with Postgres"],
        ["nextjs-supabase", "Next.js with Supabase, Postgres, and Auth"]
      ]
    },
    handoff: {
      label: "Handoff",
      choices: [
        ["share-now", "Share immediately as a completed artifact"],
        ["doc-export", "Export PDF, DOCX, Markdown, or source files"],
        ["github", "Push versioned source into GitHub"],
        ["deploy", "Deploy a real web application"]
      ]
    },
    persistence: {
      label: "Persistence and access",
      choices: [
        ["none", "No persistence; local sample data is enough"],
        ["database", "Persistent artifact/catalog records"],
        ["auth", "User accounts and session-aware access"],
        ["multi-tenant", "Organizations, memberships, roles, and tenant isolation"
        ["storage", "Uploaded files, exports, screenshots, or generated assets"]
      ]
    }
  };
```

```
const DEFAULT_ANSWERS = {
  deliverable: "interactive-tool",
  interaction: "complex",
  backend: "none",
  techStack: "react",
  handoff: "share-now",
  persistence: "none"
};

let answers = { ...DEFAULT_ANSWERS };
let sortState = {
  key: "platform",
  direction: "asc"
};

function getPlatform(id) {
  return PLATFORM_DATA.find((platform) => platform.id === id);
}

function escapeHtml(value) {
  return String(value)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll('"', "&quot;")
    .replaceAll("'", "&#039;");
}

function platformBadge(platform) {
  const classMap = {
    "Claude Artifacts": "badge-claude",
    "ChatGPT Canvas": "badge-canvas",
    "v0": "badge-v0"
  };

  return `<span class="badge ${classMap[platform]}">${escapeHtml(platform)}</spa
}

function addScore(scores, reasons, platform, points, reason) {
  scores[platform] += points;
  reasons[platform].push(reason);
}

function recommendPlatform() {
  const scores = {
    "claude-artifacts": 0,
    "chatgpt-canvas": 0,
    "v0": 0
  };

  const reasons = {
    "claude-artifacts": [],
    "chatgpt-canvas": [],
    "v0": []
  };
```

```
  const add = (platform, points, reason) =>
    addScore(scores, reasons, platform, points, reason);

  switch (answers.deliverable) {
    case "interactive-tool":
      add("claude-artifacts", 7, "A self-contained interactive tool is a direct C
      add("v0", 4, "v0 is a strong follow-on when the tool becomes a durable appl
      add("chatgpt-canvas", 2, "Canvas can iterate code, but it is less oriented
      break;

    case "product-prototype":
      add("claude-artifacts", 6, "Claude Artifacts can rapidly validate an intera
      add("v0", 5, "v0 is strong when the prototype should become the real applic
      add("chatgpt-canvas", 3, "Canvas can support focused React/HTML prototype i
      break;

    case "marketing-page":
      add("claude-artifacts", 7, "A polished, self-contained visual page is well
      add("v0", 5, "v0 becomes the better fit when the page ships as part of a Ne
      add("chatgpt-canvas", 2, "Canvas can support copy and smaller code iteratio
      break;

    case "diagram":
      add("claude-artifacts", 8, "Claude is well suited to Mermaid, SVG, diagrams
      add("chatgpt-canvas", 3, "Canvas can hold diagram source and the associated
      add("v0", 1, "v0 is appropriate only if the diagram is part of a larger pro
      break;

    case "document":
      add("chatgpt-canvas", 9, "Canvas is optimized for drafting, revising, and e
      add("claude-artifacts", 4, "Claude can create polished document artifacts."
      add("v0", 1, "v0 is unnecessary unless the document needs to live inside an
      break;

    case "presentation":
      add("chatgpt-canvas", 6, "Canvas supports an editorial, revision-first pres
      add("claude-artifacts", 5, "Claude can make visual deck-like deliverables."
      add("v0", 2, "v0 makes sense only if the presentation is an interactive web
      break;

    case "full-stack-feature":
      add("v0", 10, "A full-stack feature requires multi-file structure, server l
      add("claude-artifacts", 1, "Claude can still prototype the interaction befo
      add("chatgpt-canvas", 1, "Canvas can help create the specification or revie
      break;

    case "analysis":
      add("chatgpt-canvas", 8, "Canvas is strong for iterative analysis, Python,
      add("claude-artifacts", 3, "Claude can package analysis results into a shar
      add("v0", 3, "v0 is useful if recurring analysis should become a shared app
      break;
  }

  switch (answers.interaction) {
    case "none":
      add("chatgpt-canvas", 3, "A read-first deliverable favors an editorial worl
```

```
      add("claude-artifacts", 2, "A static artifact is still viable.");
      break;

    case "simple":
      add("claude-artifacts", 5, "Simple browser-side interaction fits an artifac
      add("chatgpt-canvas", 3, "Canvas can preview lightweight React or HTML inte
      add("v0", 2, "v0 can handle it, though it can be more project overhead.");
      break;

    case "complex":
      add("claude-artifacts", 5, "A stateful local prototype or tracker can work
      add("v0", 6, "Complex workflows benefit from durable multi-file React/Next
      add("chatgpt-canvas", 3, "Canvas can preview components but is less focused
      break;

    case "multi-user":
      add("v0", 10, "Shared workflows need identity, authorization, persistence,
      break;
  }

  switch (answers.backend) {
    case "none":
      add("claude-artifacts", 4, "No backend favors a portable and self-contained
      add("chatgpt-canvas", 3, "A document or preview workflow can be sufficient
      break;

    case "file-export":
      add("claude-artifacts", 5, "Artifacts can produce client-side downloads suc
      add("chatgpt-canvas", 5, "Canvas is strong for document and source-code exp
      add("v0", 3, "v0 can add export endpoints to a production application.");
      break;

    case "database":
      add("v0", 8, "Persistent data requires a project with a real database integ
      add("claude-artifacts", 1, "Use mock data in the artifact, then hand it off
      break;

    case "server-api":
      add("v0", 9, "Server routes, API integrations, and webhooks favor a real No
      break;

    case "auth-secrets":
      add("v0", 10, "Authentication, authorization, billing, private APIs, and se
      break;
  }

  switch (answers.techStack) {
    case "html":
      add("claude-artifacts", 5, "A contained HTML/CSS/JS deliverable is a natura
      add("chatgpt-canvas", 2, "Canvas can edit and preview the page.");
      break;

    case "react":
      add("claude-artifacts", 5, "React is a good fit for contained interactive a
      add("chatgpt-canvas", 4, "Canvas can preview and iteratively edit React/HTM
      add("v0", 5, "v0 is ideal when React must become a multi-file project.");
```

```
      break;

    case "nextjs":
      add("v0", 9, "A multi-file Next.js app is a direct v0 workflow.");
      break;

    case "nextjs-postgres":
      add("v0", 10, "Next.js plus Postgres requires environment variables, server
      break;

    case "nextjs-supabase":
      add("v0", 10, "Next.js plus Supabase requires client/server boundaries, au
      break;
  }

  switch (answers.persistence) {
    case "none":
      break;

    case "database":
      add("v0", 8, "Persistent data should live in a real database-backed app.")
      break;

    case "auth":
      add("v0", 9, "Authentication requires a durable application and server-awar
      break;

    case "multi-tenant":
      add("v0", 10, "Organization workspaces require roles, membership, tenant is
      break;

    case "storage":
      add("v0", 8, "Uploaded files and protected storage require production polic
      break;
  }

  switch (answers.handoff) {
    case "share-now":
      add("claude-artifacts", 6, "Immediate sharing favors a completed artifact."
      add("chatgpt-canvas", 3, "Canvas can also share or export reviewable work."
      break;

    case "doc-export":
      add("chatgpt-canvas", 7, "Canvas is strong for PDF, DOCX, Markdown, and sou
      add("claude-artifacts", 4, "Claude supports source-oriented handoffs such a
      break;

    case "github":
      add("v0", 7, "A repository-backed workflow is a primary v0 use case.");
      add("claude-artifacts", 3, "Artifact source can be exported and versioned i
      add("chatgpt-canvas", 3, "Canvas source can be exported into a repository."
      break;

    case "deploy":
      add("v0", 10, "Production deployment strongly favors v0 and Vercel.");
      add("claude-artifacts", 1, "Claude should hand off source instead of servir
```

```
          break;
      }

      const ranked = Object.keys(scores)
        .map((id) => ({
          ...getPlatform(id),
          score: scores[id],
          reasons: reasons[id]
        }))
        .sort((a, b) => b.score - a.score);

      const primary = ranked[^10_0];
      const secondary = ranked[^10_1];

      const productionEscalation =
        answers.backend === "database" ||
        answers.backend === "server-api" ||
        answers.backend === "auth-secrets" ||
        answers.interaction === "multi-user" ||
        answers.techStack === "nextjs" ||
        answers.techStack === "nextjs-postgres" ||
        answers.techStack === "nextjs-supabase" ||
        answers.persistence !== "none" ||
        answers.handoff === "deploy";

      const confidence = Math.min(
        97,
        Math.max(
          62,
          68 +
            (primary.score - secondary.score) * 4 +
            (productionEscalation && primary.id === "v0" ? 10 : 0)
        )
      );

      return {
        primary,
        secondary,
        ranked,
        productionEscalation,
        confidence
      };
    }

    function createBuildPrompt(recommendation) {
      const platform = recommendation.primary.platform;

      const platformDirections = {
        "Claude Artifacts": `Build this as a polished, self-contained Claude Artifac
- Keep it client-side and dependency-light.
- Use realistic local sample data.
- Do not assume a production database, full authentication, server secrets, or priva
- Include empty, loading, success, and error states where relevant.
- Make the result portable as HTML, JSX, SVG, Mermaid, Markdown, JSON, or CSV.`,

        "ChatGPT Canvas": `Build this as an editable ChatGPT Canvas deliverable.
```

```
- Optimize for iterative revision and a clean document or code-file structure.
- State assumptions and open questions clearly.
- Keep React/HTML preview requirements contained where possible.
- Prepare the final output for PDF, DOCX, Markdown, or language-specific source expor

        "v0": `Build this as a production-oriented Next.js application in v0.
- Use TypeScript, App Router, accessible responsive components, and a clean multi-fil
- Separate browser/client behavior from server logic.
- Use environment-variable placeholders and never hardcode secrets.
- Include loading, empty, error, and success states.
- Include README setup, testing, GitHub workflow, database, and Vercel deployment ins
      };

      const productionRequirements = recommendation.productionEscalation
        ? `
Production requirements:
- Add authentication and authorization where needed.
- Define data ownership and organization/tenant boundaries.
- Validate input at server boundaries.
- Store secrets in server-only environment variables.
- Add migrations, indexes, seed data, rollback planning, and tests.
- Add monitoring, error reporting, logging, analytics, and audit records where appro
- Confirm accessibility, mobile behavior, privacy, and security before release.`
        : "";

      const supabaseRequirements =
        answers.techStack === "nextjs-supabase" ||
        answers.persistence === "database" ||
        answers.persistence === "auth" ||
        answers.persistence === "multi-tenant" ||
        answers.persistence === "storage"
          ? `
Supabase implementation requirements:
- Use Supabase Postgres for persistent records.
- Use separate browser and server clients.
- Keep service-role access server-only.
- Add Row Level Security for every exposed table.
- Scope organization data through organization_id.
- Do not use client-side UI state as authorization.
- Store SQL migrations and RLS policies in GitHub.`
          : "";

      return `Build this for Semester.

Recommended platform: ${platform}
Secondary option: ${recommendation.secondary.platform}
Recommendation confidence: ${recommendation.confidence}%

Project requirements:
- Deliverable: ${answers.deliverable}
- Interaction: ${answers.interaction}
- Backend: ${answers.backend}
- Technology target: ${answers.techStack}
- Persistence and access: ${answers.persistence}
- Handoff: ${answers.handoff}
```

```
${platformDirections[platform]}

Required deliverables:
- Explain architecture and important assumptions.
- Include realistic Semester sample data.
- Build the primary workflow so it is immediately reviewable.
- List non-goals and known constraints.
- Include exact run, edit, export, GitHub, and deployment instructions.
${productionRequirements}
${supabaseRequirements}

Design direction:
Create a clean, premium B2B software interface. Use accessible controls, readable in
    }

    function createGithubPlan(recommendation) {
      const repositoryNeeded =
        answers.handoff === "github" ||
        answers.handoff === "deploy" ||
        recommendation.productionEscalation;

      if (!repositoryNeeded) {
        return `GitHub is optional for this scope.

Keep this initial version self-contained while validating the idea. Move the source
      }

      if (recommendation.primary.id === "v0") {
        return `GitHub plan for v0:

1. Build the first working multi-file project in v0.
2. Connect the v0 project to the appropriate GitHub account or organization.
3. Create a repository or connect an existing Semester repository.
4. Use main only for reviewed changes; create feature branches for meaningful work.
5. Add .env.example with variable names only.
6. Confirm .env and .env.local are ignored by Git.
7. Review the preview deployment and changed files.
8. Push or sync the feature branch to GitHub.
9. Open a pull request and run lint, typecheck, tests, and production build.
10. Review migrations, Row Level Security, and environment variable changes before me
11. Deploy through Vercel after approval.
12. Never commit API secrets, database URLs, service-role keys, or credentials.`;
      }

      if (recommendation.primary.id === "claude-artifacts") {
        return `GitHub plan after Claude Artifacts:

1. Export or copy the artifact source as HTML, JSX/TSX, SVG, Mermaid, Markdown, JSON
2. Create a Semester repository and structure it intentionally:
   - app/ or src/ for React and Next.js code
   - docs/ for PRDs, decisions, Markdown, and Mermaid diagrams
   - public/ for SVG and static assets
3. Add README.md with artifact purpose, assumptions, and regeneration instructions.
4. Create a feature branch:
   git checkout -b feat/semester-workflow-router
5. Commit and push:
```

```
   git add .
   git commit -m "feat: add workflow router prototype"
   git push -u origin feat/semester-workflow-router
6. If the tool needs auth, data persistence, API routes, secrets, or multi-user acces
      }

      return `GitHub plan after ChatGPT Canvas:

1. Export Markdown and source code as the primary editable record.
2. Treat PDF and DOCX as review/distribution outputs, not the only source of truth.
3. Create a repository and organize content:
   - docs/ for Markdown and decisions
   - app/ and components/ if code becomes a web app
   - scripts/ for Python or utility scripts
4. Add a README explaining purpose, assumptions, and regeneration steps.
5. Create a feature branch, commit, push, and open a pull request.
6. If the project requires data, auth, organization access, or deployment, move into
    }

    function createMarkdownExport(recommendation, buildPrompt, githubPlan) {
      return `# Semester AI Workflow Recommendation

## Recommendation

- **Primary platform:** ${recommendation.primary.platform}
- **Secondary platform:** ${recommendation.secondary.platform}
- **Confidence:** ${recommendation.confidence}%
- **Production escalation required:** ${recommendation.productionEscalation ? "Yes"

## Selected workflow inputs

| Requirement | Selection |
|---|---|
| Deliverable | ${answers.deliverable} |
| Interaction | ${answers.interaction} |
| Backend | ${answers.backend} |
| Technology target | ${answers.techStack} |
| Persistence and access | ${answers.persistence} |
| Handoff | ${answers.handoff} |

## Why this was recommended

${recommendation.primary.reasons.map((reason) => `- ${reason}`).join("\n")}

## Ready-to-copy implementation prompt

\`\`\`text
${buildPrompt}
\`\`\`

## GitHub and production handoff

\`\`\`text
${githubPlan}
\`\`\`
```

```
## Platform summary

| Platform | Primary use | Export options | Production handoff |
|---|---|---|---|
${PLATFORM_DATA.map(
  (platform) =>
    `| ${platform.platform} | ${platform.primaryUse} | ${platform.exportFormats} | $
).join("\n")}

## Product boundary

- Use **Claude Artifacts** for fast, self-contained interactive prototypes, diagrams
- Use **ChatGPT Canvas** for document and code iteration, editing, analysis, and file
- Use **v0** for multi-file React/Next.js applications, Postgres or Supabase persiste
`;
    }

    function renderDecisionGrid() {
      const grid = document.getElementById("decision-grid");

      grid.innerHTML = Object.entries(DECISION_OPTIONS)
        .map(([field, config]) => {
          const choices = config.choices
            .map(
              ([value, label]) => `
                <label class="choice">
                  <input
                    type="radio"
                    name="${field}"
                    value="${value}"
                    ${answers[field] === value ? "checked" : ""}
                  />
                  <span>${escapeHtml(label)}</span>
                </label>
              `
            )
            .join("");

          return `
            <section class="field-card">
              <h3>${escapeHtml(config.label)}</h3>
              <div class="choice-list">${choices}</div>
            </section>
          `;
        })
        .join("");

      grid.querySelectorAll("input[type=radio]").forEach((input) => {
        input.addEventListener("change", (event) => {
          answers[event.target.name] = event.target.value;
          render();
        });
      });
    }

    function renderRecommendation(recommendation) {
```

```
  const target = document.getElementById("recommendation");
  const reasons = recommendation.primary.reasons.length
    ? recommendation.primary.reasons
        .map((reason) => `<li>${escapeHtml(reason)}</li>`)
        .join("")
    : "<li>Best overall fit for the selected workflow.</li>";

  const warning = recommendation.productionEscalation
    ? `
      <div class="warning">
        <strong>Production handoff required</strong>
        Your choices require infrastructure beyond a self-contained artifact:
        persistent records, authentication, server logic, Supabase/Postgres,
        organizational tenancy, private integrations, or deployment. Use
        Claude or Canvas for early validation, then implement the durable
        version in a GitHub-backed v0/Next.js project.
      </div>
    `
    : "";

  target.innerHTML = `
    <div>
      <p class="recommendation-label">Primary recommendation</p>

      <div class="recommendation-title-row">
        <h3 class="recommendation-title">
          ${escapeHtml(recommendation.primary.platform)}
        </h3>

        ${platformBadge(recommendation.primary.platform)}

        <span class="badge badge-indigo">
          ${recommendation.confidence}% confidence
        </span>
      </div>

      <ul class="reason-list">
        ${reasons}
      </ul>

      ${warning}
    </div>

    <aside class="secondary-card">
      <p class="secondary-label">Secondary option</p>
      <p style="margin:0;font-size:17px;font-weight:800;color:#0f172a;">
        ${escapeHtml(recommendation.secondary.platform)}
      </p>
      <p style="margin-top:8px;color:#64748b;font-size:13px;line-height:1.6;">
        ${escapeHtml(recommendation.secondary.primaryUse)}
      </p>
    </aside>
  `;
}

function renderFeatureMatrix() {
```

```
  const tbody = document.getElementById("feature-matrix-body");

  tbody.innerHTML = FEATURE_MATRIX.map(
    (row) => `
      <tr>
        <td class="platform-name">${escapeHtml(row.feature)}</td>
        <td><span class="feature-status ${row.claude[^10_1]}">${row.claude[^10_0
        <td><span class="feature-status ${row.canvas[^10_1]}">${row.canvas[^10_0
        <td><span class="feature-status ${row.v0[^10_1]}">${row.v0[^10_0]}</span>
      </tr>
    `
  ).join("");
}

function renderPlatformMatrix() {
  const sorted = [...PLATFORM_DATA].sort((a, b) => {
    const aValue = String(a[sortState.key] || "");
    const bValue = String(b[sortState.key] || "");

    return sortState.direction === "asc"
      ? aValue.localeCompare(bValue)
      : bValue.localeCompare(aValue);
  });

  const tbody = document.getElementById("platform-matrix-body");

  tbody.innerHTML = sorted
    .map(
      (platform) => `
        <tr>
          <td>
            <div class="platform-name">${escapeHtml(platform.platform)}</div>

            <div class="score-row">
              <span class="mini-badge">Prototype ${platform.scores.prototype}/5</
              <span class="mini-badge">Documents ${platform.scores.documents}/5</
              <span class="mini-badge">Diagrams ${platform.scores.diagrams}/5</sp
              <span class="mini-badge">Full stack ${platform.scores.fullStack}/5<
              <span class="mini-badge">Deploy ${platform.scores.deployment}/5</sp
            </div>
          </td>

          <td>${escapeHtml(platform.primaryUse)}</td>
          <td>${escapeHtml(platform.execution)}</td>
          <td>${escapeHtml(platform.reactSupport)}</td>
          <td>${escapeHtml(platform.projectShape)}</td>
          <td>${escapeHtml(platform.exportFormats)}</td>
          <td>${escapeHtml(platform.productionFit)}</td>

          <td>
            <button
              class="button"
              data-open-platform="${platform.id}"
            >
              Open
            </button>
```

```
            </td>
          </tr>
        `
      )
      .join("");

    tbody.querySelectorAll("[data-open-platform]").forEach((button) => {
      button.addEventListener("click", () => {
        const platform = getPlatform(button.dataset.openPlatform);
        openDrawer(platform);
      });
    });
  }

  function openDrawer(platform) {
    const modal = document.getElementById("detail-modal");
    const title = document.getElementById("drawer-title");
    const badge = document.getElementById("drawer-badge");
    const content = document.getElementById("drawer-content");

    title.textContent = platform.platform;
    badge.innerHTML = platformBadge(platform.platform);

    content.innerHTML = [
      ["Primary use", platform.primaryUse],
      ["Execution model", platform.execution],
      ["Sandbox limits", platform.sandboxLimits],
      ["React support", platform.reactSupport],
      ["Project shape", platform.projectShape],
      ["Code and export options", platform.exportFormats],
      ["GitHub workflow", platform.githubWorkflow],
      ["Production handoff", platform.productionFit],
      ["Best Semester use cases", platform.bestFor]
    ]
      .map(
        ([label, value]) => `
          <section class="drawer-section">
            <h3>${escapeHtml(label)}</h3>
            <p>${escapeHtml(value)}</p>
          </section>
        `
      )
      .join("");

    modal.classList.add("open");
    modal.setAttribute("aria-hidden", "false");
  }

  function closeDrawer() {
    const modal = document.getElementById("detail-modal");
    modal.classList.remove("open");
    modal.setAttribute("aria-hidden", "true");
  }

  async function copyText(value, button) {
    try {
```

```
      await navigator.clipboard.writeText(value);
      const original = button.textContent;
      button.textContent = "Copied";
      setTimeout(() => {
        button.textContent = original;
      }, 1400);
    } catch {
      window.prompt("Copy the text below:", value);
    }
  }

  function downloadFile(filename, content, mimeType) {
    const blob = new Blob([content], {
      type: mimeType
    });

    const url = URL.createObjectURL(blob);
    const anchor = document.createElement("a");

    anchor.href = url;
    anchor.download = filename;
    anchor.click();

    URL.revokeObjectURL(url);
  }

  function exportCurrentHtml() {
    const source = "<!doctype html>\\n" + document.documentElement.outerHTML;

    downloadFile(
      "semester-ai-workflow-router.html",
      source,
      "text/html;charset=utf-8"
    );
  }

  function render() {
    const recommendation = recommendPlatform();
    const buildPrompt = createBuildPrompt(recommendation);
    const githubPlan = createGithubPlan(recommendation);

    renderDecisionGrid();
    renderRecommendation(recommendation);
    renderFeatureMatrix();
    renderPlatformMatrix();

    document.getElementById("build-prompt-output").textContent = buildPrompt;
    document.getElementById("github-plan-output").textContent = githubPlan;

    document
      .getElementById("copy-build-prompt")
      .replaceWith(
        createCopyButton(
          "copy-build-prompt",
          "Copy prompt",
          buildPrompt
```

```
      )
    );

  document
    .getElementById("copy-github-plan")
    .replaceWith(
      createCopyButton(
        "copy-github-plan",
        "Copy plan",
        githubPlan
      )
    );

  document
    .getElementById("export-markdown")
    .replaceWith(
      createActionButton(
        "export-markdown",
        "Export recommendation",
        "button button-primary",
        () => {
          downloadFile(
            "semester-ai-workflow-recommendation.md",
            createMarkdownExport(recommendation, buildPrompt, githubPlan),
            "text/markdown;charset=utf-8"
          );
        }
      )
    );

  document
    .getElementById("export-json")
    .replaceWith(
      createActionButton(
        "export-json",
        "Export JSON",
        "button",
        () => {
          downloadFile(
            "semester-ai-workflow-recommendation.json",
            JSON.stringify(
              {
                answers,
                recommendation: {
                  primary: recommendation.primary.platform,
                  secondary: recommendation.secondary.platform,
                  confidence: recommendation.confidence,
                  productionEscalation: recommendation.productionEscalation,
                  reasons: recommendation.primary.reasons
                },
                buildPrompt,
                githubPlan
              },
              null,
              2
            ),
```

```
              "application/json;charset=utf-8"
            );
          }
        )
      );

    document
      .getElementById("export-html")
      .replaceWith(
        createActionButton(
          "export-html",
          "Export full HTML app",
          "button button-soft",
          exportCurrentHtml
        )
      );
  }

  function createCopyButton(id, label, value) {
    const button = document.createElement("button");
    button.id = id;
    button.className = "button";
    button.textContent = label;
    button.addEventListener("click", () => copyText(value, button));
    return button;
  }

  function createActionButton(id, label, className, handler) {
    const button = document.createElement("button");
    button.id = id;
    button.className = className;
    button.textContent = label;
    button.addEventListener("click", handler);
    return button;
  }

  document.addEventListener("click", (event) => {
    const sortButton = event.target.closest("[data-sort]");

    if (sortButton) {
      const key = sortButton.dataset.sort;

      sortState = {
        key,
        direction:
          sortState.key === key && sortState.direction === "asc"
            ? "desc"
            : "asc"
      };

      renderPlatformMatrix();
    }
  });

  document
    .getElementById("close-drawer")
```

```
      .addEventListener("click", closeDrawer);

    document
      .getElementById("detail-modal")
      .addEventListener("click", (event) => {
        if (event.target.id === "detail-modal") {
          closeDrawer();
        }
      });

    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") {
        closeDrawer();
      }
    });

    render();
  </script>
</body>
</html>
```

How the decision tree works

The scoring logic considers six dimensions:

Input	Why it matters	Strongest effect
Deliverable type	Determines whether the core work is visual, editorial, interactive, or full stack	Documents favor Canvas; diagrams/tools favor Claude; product features favor v0
Interaction complexity	Distinguishes a static or simple tool from a stateful multi-user product	Complex local UI can use Claude; multi-user behavior goes to v0
Backend need	Identifies whether server-side work is needed	Database, API, auth, or secrets route to v0
Tech stack	Identifies project shape	Plain HTML favors Claude; Next.js/Postgres/Supabase favor v0
Handoff target	Distinguishes immediate sharing, file export, GitHub, and deployment	Document export favors Canvas; deployment favors v0
Persistence/access	Detects whether local state is sufficient	Database, auth, storage, and multi-tenancy require a real backend workflow

Export behavior

The standalone app includes three browser-only export actions:

- **Export recommendation:** downloads the current decision, reasons, build prompt, GitHub plan, and platform summary as Markdown.
- **Export JSON:** downloads the selected decision inputs and machine-readable recommendation.
- **Export full HTML app:** serializes the current page source and downloads a standalone `.html` file.

The exported HTML is a snapshot of the full interface and contains no external dependencies, bundler requirement, framework runtime, or backend requirement. It is suitable for a local browser, a static host, a GitHub repository, or an initial handoff to a v0/Next.js project.

Production boundary

Use this file as a product-spec and prototype asset. When Semester needs shared saved recommendations, organization-level workspaces, persistent artifact records, roles, approvals, audit history, or real API integrations, move the same decision data into a v0-built Next.js app and store it in Supabase/Postgres.

Claude's current Artifact feature supports interactive pages alongside documents, dashboards, designs, and decks; it also supports common exports including HTML, PDF, PowerPoint, and Google Slides-related outputs. Claude Code artifact documentation additionally states that published artifact sources are HTML/HTM/Markdown and can offer viewer downloads such as CSV or PNG. [134] [133]

⁂

Build a prompt decision tree web app that maps your exact coding or writing workflow to Claude Artifacts, ChatGPT Canvas, or v0 sandbox

Computer
Prompt-to-tool decision tree: test Claude Artifacts, ChatGPT Canvas, and v0 sandbox against 15 real-world prompts to see which wins
Computer
Show me how to add Supabase auth to this app
Add a feature checklist view to the app
Export this app as a React component for Nextjs

Below is a production-oriented **Next.js React export** of the decision-tree app, plus Supabase Auth integration and a feature-checklist view. It evaluates 15 real-world prompts, recommends Claude Artifacts, ChatGPT Canvas, or v0, and lets authenticated users save their recommendations to Supabase.

For a real shared Semester application, use Next.js App Router with cookie-based Supabase Auth, separate browser/server clients through `@supabase/ssr`, and server-side authorization checks. Supabase's current Next.js guidance explicitly supports this model and provides a `with-supabase` starter with App Router, TypeScript, Tailwind, and cookie-based Auth. [144] [145] [146]

Project structure

```
semester-workflow-router/
├── app/
│   ├── auth/
│   │   ├── sign-in/page.tsx
```

```
│   │   ├── sign-up/page.tsx
│   │   └── confirm/route.ts
│   ├── workflow-router/
│   │   ├── actions.ts
│   │   └── page.tsx
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── auth-button.tsx
│   ├── feature-checklist.tsx
│   └── workflow-router.tsx
├── lib/
│   └── supabase/
│       ├── client.ts
│       ├── middleware.ts
│       └── server.ts
├── supabase/
│   └── migrations/
│       └── 001_create_workflow_recommendations.sql
├── middleware.ts
├── .env.example
└── package.json
```

Install and configure

```
npx create-next-app@latest semester-workflow-router --ts --tailwind --app
cd semester-workflow-router

npm install @supabase/ssr @supabase/supabase-js
```

Create `.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=https://YOUR_PROJECT.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=YOUR_PUBLISHABLE_KEY
```

Use the browser-safe URL and publishable key in `NEXT_PUBLIC_*` variables. Keep any service-role key server-only and do not commit `.env.local`. Supabase's quickstart uses this App Router configuration, while Next.js keeps non-`NEXT_PUBLIC_` variables server-only by default.[146] [147]

Supabase clients

`lib/supabase/client.ts`

```
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!
```

```
  );
}
```

`lib/supabase/server.ts`

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },

        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) => {
              cookieStore.set(name, value, options);
            });
          } catch {
            // Server Components cannot always set cookies.
            // Session refresh happens through middleware.
          }
        },
      },
    }
  );
}
```

`lib/supabase/middleware.ts`

```
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function updateSession(request: NextRequest) {
  let response = NextResponse.next({
    request,
  });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
```

```
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value)
          );

          response = NextResponse.next({
            request,
          });

          cookiesToSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options)
          );
        },
      },
    }
  );

  await supabase.auth.getUser();

  return response;
}
```

`middleware.ts`

```
import type { NextRequest } from "next/server";
import { updateSession } from "@/lib/supabase/middleware";

export async function middleware(request: NextRequest) {
  return updateSession(request);
}

export const config = {
  matcher: [
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*
  ],
};
```

Supabase's SSR package provides distinct browser and server client creation functions, and its SSR documentation explains that sessions are stored in cookies so they are available to both client and server rendering. [145] [144]

Database migration

`supabase/migrations/001_create_workflow_recommendations.sql`

```
create table public.workflow_recommendations (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  title text not null default 'Untitled workflow decision',
  inputs jsonb not null,
  recommendation jsonb not null,
```

```
  build_prompt text not null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index workflow_recommendations_user_id_idx
  on public.workflow_recommendations(user_id);

create index workflow_recommendations_created_at_idx
  on public.workflow_recommendations(created_at desc);

alter table public.workflow_recommendations enable row level security;

create policy "Users can view their own workflow recommendations"
on public.workflow_recommendations
for select
to authenticated
using (user_id = auth.uid());

create policy "Users can create their own workflow recommendations"
on public.workflow_recommendations
for insert
to authenticated
with check (user_id = auth.uid());

create policy "Users can update their own workflow recommendations"
on public.workflow_recommendations
for update
to authenticated
using (user_id = auth.uid())
with check (user_id = auth.uid());

create policy "Users can delete their own workflow recommendations"
on public.workflow_recommendations
for delete
to authenticated
using (user_id = auth.uid());
```

This model saves each recommendation as structured JSON plus the generated prompt. The Row Level Security policies ensure users can only access their own records. For a future Semester team workspace, add `organization_id`, organization membership, role checks, and organization-scoped RLS policies.

Server action

`app/workflow-router/actions.ts`

```
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";

type SaveRecommendationInput = {
```

```
  title: string;
  inputs: Record<string, string>;
  recommendation: Record<string, unknown>;
  buildPrompt: string;
};

export async function saveRecommendation(input: SaveRecommendationInput) {
  const supabase = await createClient();

  const {
    data: { user },
    error: authError,
  } = await supabase.auth.getUser();

  if (authError || !user) {
    throw new Error("You must be signed in to save a workflow recommendation.");
  }

  const { error } = await supabase
    .from("workflow_recommendations")
    .insert({
      user_id: user.id,
      title: input.title.trim() || "Untitled workflow decision",
      inputs: input.inputs,
      recommendation: input.recommendation,
      build_prompt: input.buildPrompt,
    });

  if (error) {
    throw new Error(error.message);
  }

  revalidatePath("/workflow-router");

  return { success: true };
}
```

Full decision-tree component

`components/workflow-router.tsx`

```
"use client";

import { useMemo, useState } from "react";
import { saveRecommendation } from "@/app/workflow-router/actions";

type PlatformId = "claude-artifacts" | "chatgpt-canvas" | "v0";

type Answers = {
  deliverable:
    | "interactive-tool"
    | "product-prototype"
    | "marketing-page"
    | "diagram"
```

```
    | "document"
    | "presentation"
    | "full-stack-feature"
    | "analysis";
  interaction: "none" | "simple" | "complex" | "multi-user";
  backend: "none" | "file-export" | "database" | "server-api" | "auth-secrets";
  techStack:
    | "none"
    | "html"
    | "react"
    | "nextjs"
    | "nextjs-postgres"
    | "nextjs-supabase";
  persistence:
    | "none"
    | "database"
    | "auth"
    | "multi-tenant"
    | "storage";
  handoff: "share-now" | "doc-export" | "github" | "deploy";
};

type Platform = {
  id: PlatformId;
  platform: string;
  primaryUse: string;
  execution: string;
  sandboxLimits: string;
  reactSupport: string;
  projectShape: string;
  exportFormats: string;
  productionFit: string;
  bestFor: string;
  scores: {
    prototype: number;
    documents: number;
    diagrams: number;
    fullStack: number;
    deployment: number;
  };
};

type RankedPlatform = Platform & {
  score: number;
  reasons: string[];
};

type Recommendation = {
  primary: RankedPlatform;
  secondary: RankedPlatform;
  ranked: RankedPlatform[];
  confidence: number;
  needsProductionHandoff: boolean;
  needsSupabase: boolean;
};
```

```
type RealWorldPrompt = {
  id: number;
  prompt: string;
  category: string;
  answers: Answers;
  expectedWinner: PlatformId;
};

const DEFAULT_ANSWERS: Answers = {
  deliverable: "interactive-tool",
  interaction: "complex",
  backend: "none",
  techStack: "react",
  persistence: "none",
  handoff: "share-now",
};

const PLATFORM_DATA: Platform[] = [
  {
    id: "claude-artifacts",
    platform: "Claude Artifacts",
    primaryUse:
      "Shareable self-contained artifacts, interactive tools, diagrams, visual expla
    execution:
      "Contained client-side artifact experience; strongest for a self-contained resu
    sandboxLimits:
      "Do not treat it as the final home for durable records, secrets, complete authe
    reactSupport:
      "Good for local-state React interfaces, trackers, calculators, decision trees,
    projectShape:
      "Usually one contained artifact, a component, HTML page, SVG, Mermaid diagram,
    exportFormats:
      "HTML, JSX-style source, Markdown, SVG, Mermaid, and browser-generated files su
    productionFit:
      "Prototype → source export → GitHub → production Next.js application when infra
    bestFor:
      "Semester decision-tree prototypes, artifact catalogs with mock data, ROI calcu
    scores: {
      prototype: 5,
      documents: 4,
      diagrams: 5,
      fullStack: 1,
      deployment: 2,
    },
  },
  {
    id: "chatgpt-canvas",
    platform: "ChatGPT Canvas",
    primaryUse:
      "Editable workspace for writing, code iteration, research, Python analysis, rev
    execution:
      "Isolated workspace preview for React/HTML and supported coding workflows; suit
    sandboxLimits:
      "Do not use it as a production database, authentication, secret-management, API
    reactSupport:
      "Useful for editing and previewing React/HTML components and focused files. Str
```

```
    projectShape:
      "Document, code file, research artifact, analysis, or focused component prepare
    exportFormats:
      "Documents: PDF, DOCX, Markdown. Code: language-specific source such as JavaScr
    productionFit:
      "Draft/revise/export → GitHub, docs, or an application codebase.",
    bestFor:
      "Semester PRDs, strategy, policies, research synthesis, legal drafts, code refa
    scores: {
      prototype: 3,
      documents: 5,
      diagrams: 3,
      fullStack: 1,
      deployment: 1,
    },
  },
  {
    id: "v0",
    platform: "v0",
    primaryUse:
      "Production-oriented React and Next.js applications, multi-file project work, c
    execution:
      "VM-backed Node.js development sandbox for multi-file projects, server logic, r
    sandboxLimits:
      "Preview success is not production readiness. Verify migrations, environment va
    reactSupport:
      "Native fit for React and Next.js: App Router, Tailwind, multi-file components
    projectShape:
      "Repository-style project with dependencies, components, pages, API routes, ser
    exportFormats:
      "Project code, GitHub sync/export, preview deployments, existing-codebase integ
    productionFit:
      "Best for Postgres/Supabase applications, authenticated workflows, multi-user p
    bestFor:
      "Semester production tracker, authenticated team dashboard, multi-tenant worksp
    scores: {
      prototype: 4,
      documents: 2,
      diagrams: 2,
      fullStack: 5,
      deployment: 5,
    },
  },
];

const FEATURE_CHECKLIST = [
  {
    feature: "Self-contained interactive prototype",
    claude: "High",
    canvas: "Medium",
    v0: "High",
  },
  {
    feature: "PRDs, policies, strategy, research, and editing",
    claude: "High",
    canvas: "High",
```

```
    v0: "Low",
  },
  {
    feature: "Mermaid, SVG, and visual explainers",
    claude: "High",
    canvas: "Medium",
    v0: "Medium",
  },
  {
    feature: "React component experimentation",
    claude: "High",
    canvas: "High",
    v0: "High",
  },
  {
    feature: "Multi-file Next.js app",
    claude: "Low",
    canvas: "Low",
    v0: "High",
  },
  {
    feature: "Server routes and private server secrets",
    claude: "Low",
    canvas: "Low",
    v0: "High",
  },
  {
    feature: "Postgres or Supabase persistence",
    claude: "Low",
    canvas: "Low",
    v0: "High",
  },
  {
    feature: "Authentication and row-level permissions",
    claude: "Low",
    canvas: "Low",
    v0: "High",
  },
  {
    feature: "Document-oriented export",
    claude: "Medium",
    canvas: "High",
    v0: "Low",
  },
  {
    feature: "GitHub and Vercel production workflow",
    claude: "Medium",
    canvas: "Medium",
    v0: "High",
  },
];

const REAL_WORLD_PROMPTS: RealWorldPrompt[] = [
  {
    id: 1,
    category: "Product",
```

```
    prompt:
      "Build a sortable artifact catalog with 100 sample rows, client-side filters, p
    answers: {
      deliverable: "interactive-tool",
      interaction: "complex",
      backend: "file-export",
      techStack: "react",
      persistence: "none",
      handoff: "share-now",
    },
    expectedWinner: "claude-artifacts",
  },
  {
    id: 2,
    category: "Product",
    prompt:
      "Write a detailed PRD for Semester's primary workflow, including acceptance cri
    answers: {
      deliverable: "document",
      interaction: "none",
      backend: "none",
      techStack: "none",
      persistence: "none",
      handoff: "doc-export",
    },
    expectedWinner: "chatgpt-canvas",
  },
  {
    id: 3,
    category: "Engineering",
    prompt:
      "Generate a Mermaid system architecture diagram for a Next.js app, Supabase Aut
    answers: {
      deliverable: "diagram",
      interaction: "none",
      backend: "none",
      techStack: "none",
      persistence: "none",
      handoff: "share-now",
    },
    expectedWinner: "claude-artifacts",
  },
  {
    id: 4,
    category: "Engineering",
    prompt:
      "Build an authenticated Next.js App Router dashboard backed by Supabase Postgre
    answers: {
      deliverable: "full-stack-feature",
      interaction: "multi-user",
      backend: "auth-secrets",
      techStack: "nextjs-supabase",
      persistence: "multi-tenant",
      handoff: "deploy",
    },
    expectedWinner: "v0",
```

```
  },
  {
    id: 5,
    category: "Brand",
    prompt:
      "Create six editable SVG logo mark concepts for Semester with clear visual rat
    answers: {
      deliverable: "diagram",
      interaction: "none",
      backend: "none",
      techStack: "html",
      persistence: "none",
      handoff: "share-now",
    },
    expectedWinner: "claude-artifacts",
  },
  {
    id: 6,
    category: "GTM",
    prompt:
      "Draft a launch strategy, messaging architecture, email sequence, press releas
    answers: {
      deliverable: "document",
      interaction: "none",
      backend: "file-export",
      techStack: "none",
      persistence: "none",
      handoff: "doc-export",
    },
    expectedWinner: "chatgpt-canvas",
  },
  {
    id: 7,
    category: "Finance",
    prompt:
      "Create a Python workflow that cleans a CSV export, calculates monthly retentio
    answers: {
      deliverable: "analysis",
      interaction: "none",
      backend: "file-export",
      techStack: "none",
      persistence: "none",
      handoff: "doc-export",
    },
    expectedWinner: "chatgpt-canvas",
  },
  {
    id: 8,
    category: "Marketing",
    prompt:
      "Create a polished interactive HTML landing page prototype for Semester with pr
    answers: {
      deliverable: "marketing-page",
      interaction: "simple",
      backend: "none",
      techStack: "html",
```

```
      persistence: "none",
      handoff: "share-now",
    },
    expectedWinner: "claude-artifacts",
  },
  {
    id: 9,
    category: "Product",
    prompt:
      "Build a customer-facing ROI calculator with saved scenarios, signed-in users,
    answers: {
      deliverable: "interactive-tool",
      interaction: "multi-user",
      backend: "auth-secrets",
      techStack: "nextjs-supabase",
      persistence: "multi-tenant",
      handoff: "deploy",
    },
    expectedWinner: "v0",
  },
  {
    id: 10,
    category: "Legal",
    prompt:
      "Draft a first version of Terms of Service, a Privacy Policy, and a Data Proces
    answers: {
      deliverable: "document",
      interaction: "none",
      backend: "none",
      techStack: "none",
      persistence: "none",
      handoff: "doc-export",
    },
    expectedWinner: "chatgpt-canvas",
  },
  {
    id: 11,
    category: "Operations",
    prompt:
      "Build a shared operating dashboard where department leads update quarterly OKR
    answers: {
      deliverable: "full-stack-feature",
      interaction: "multi-user",
      backend: "database",
      techStack: "nextjs-postgres",
      persistence: "multi-tenant",
      handoff: "deploy",
    },
    expectedWinner: "v0",
  },
  {
    id: 12,
    category: "Sales",
    prompt:
      "Create a sales deck with a category narrative, product demo flow, objection ha
    answers: {
```

```
      deliverable: "presentation",
      interaction: "none",
      backend: "none",
      techStack: "none",
      persistence: "none",
      handoff: "doc-export",
    },
    expectedWinner: "chatgpt-canvas",
  },
  {
    id: 13,
    category: "Design",
    prompt:
      "Create a clickable product prototype for a new workflow with sample data, emp
    answers: {
      deliverable: "product-prototype",
      interaction: "complex",
      backend: "none",
      techStack: "react",
      persistence: "none",
      handoff: "share-now",
    },
    expectedWinner: "claude-artifacts",
  },
  {
    id: 14,
    category: "Security",
    prompt:
      "Create an internal security data-flow document and threat model for an upcomi
    answers: {
      deliverable: "document",
      interaction: "none",
      backend: "none",
      techStack: "none",
      persistence: "none",
      handoff: "doc-export",
    },
    expectedWinner: "chatgpt-canvas",
  },
  {
    id: 15,
    category: "Engineering",
    prompt:
      "Build a production admin console with audit logs, role controls, server-side A
    answers: {
      deliverable: "full-stack-feature",
      interaction: "multi-user",
      backend: "auth-secrets",
      techStack: "nextjs-supabase",
      persistence: "multi-tenant",
      handoff: "deploy",
    },
    expectedWinner: "v0",
  },
];
```

```
function getPlatform(id: PlatformId) {
  const platform = PLATFORM_DATA.find((item) => item.id === id);

  if (!platform) {
    throw new Error(`Missing platform ${id}`);
  }

  return platform;
}

function recommendPlatform(answers: Answers): Recommendation {
  const scores: Record<PlatformId, number> = {
    "claude-artifacts": 0,
    "chatgpt-canvas": 0,
    v0: 0,
  };

  const reasons: Record<PlatformId, string[]> = {
    "claude-artifacts": [],
    "chatgpt-canvas": [],
    v0: [],
  };

  const add = (id: PlatformId, points: number, reason: string) => {
    scores[id] += points;
    reasons[id].push(reason);
  };

  switch (answers.deliverable) {
    case "interactive-tool":
      add("claude-artifacts", 7, "A self-contained interactive tool is a direct Arti
      add("v0", 4, "v0 is the right follow-on if the tool becomes shared and persiste
      add("chatgpt-canvas", 2, "Canvas can preview code but is less focused on finish
      break;
    case "product-prototype":
      add("claude-artifacts", 6, "Artifacts are fast for interactive product prototyp
      add("v0", 5, "v0 is ideal if the prototype turns into the production app.");
      add("chatgpt-canvas", 3, "Canvas supports focused React/HTML iteration.");
      break;
    case "marketing-page":
      add("claude-artifacts", 7, "A polished one-page visual output suits an Artifac
      add("v0", 5, "v0 is better when the page ships inside a Next.js website.");
      add("chatgpt-canvas", 2, "Canvas is useful for copy and smaller front-end edits
      break;
    case "diagram":
      add("claude-artifacts", 8, "Artifacts are excellent for Mermaid, SVG, and visua
      add("chatgpt-canvas", 3, "Canvas can support diagram source plus analysis.");
      add("v0", 1, "v0 is better only if the diagram is part of a bigger app.");
      break;
    case "document":
      add("chatgpt-canvas", 9, "Canvas is optimized for drafting, revising, and expor
      add("claude-artifacts", 4, "Artifacts can still create polished standalone docu
      add("v0", 1, "v0 is unnecessary unless the document belongs in an app.");
      break;
    case "presentation":
      add("chatgpt-canvas", 6, "Canvas suits editorial and export-first narrative wor
```

```
      add("claude-artifacts", 5, "Artifacts can create shareable visual presentation
      add("v0", 2, "v0 is useful only for an interactive web presentation.");
      break;
    case "full-stack-feature":
      add("v0", 10, "Full-stack work requires project structure, server logic, persis
      add("claude-artifacts", 1, "Artifacts can provide a useful UI prototype.");
      add("chatgpt-canvas", 1, "Canvas can help write specs and review code.");
      break;
    case "analysis":
      add("chatgpt-canvas", 8, "Canvas suits iterative Python, research, and file-ge
      add("claude-artifacts", 3, "Artifacts can package analysis into a shareable vis
      add("v0", 3, "v0 is useful if analysis evolves into a shared app.");
      break;
  }

  switch (answers.interaction) {
    case "none":
      add("chatgpt-canvas", 3, "Read-first deliverables favor an editing workspace."
      add("claude-artifacts", 2, "Static artifacts remain useful.");
      break;
    case "simple":
      add("claude-artifacts", 5, "Simple client-side interaction is a strong Artifac
      add("chatgpt-canvas", 3, "Canvas can preview basic React/HTML interaction.");
      add("v0", 2, "v0 works but can add project overhead.");
      break;
    case "complex":
      add("claude-artifacts", 5, "A local-state prototype can still work as an Artifa
      add("v0", 6, "Complex UI gains from multi-file React/Next.js architecture.");
      add("chatgpt-canvas", 3, "Canvas supports code preview but is less lifecycle-or
      break;
    case "multi-user":
      add("v0", 10, "Shared workflows require identity, authorization, persistence, a
      break;
  }

  switch (answers.backend) {
    case "none":
      add("claude-artifacts", 4, "No backend favors a self-contained deliverable.");
      add("chatgpt-canvas", 3, "Canvas can handle a document/code-preview workflow."
      break;
    case "file-export":
      add("claude-artifacts", 5, "Artifacts work well for browser-generated files suc
      add("chatgpt-canvas", 5, "Canvas supports document and source-file exports.");
      add("v0", 3, "v0 can add exports to a real product.");
      break;
    case "database":
      add("v0", 8, "Persistent data requires a real database-backed application.");
      break;
    case "server-api":
      add("v0", 9, "Server routes, APIs, and webhooks favor v0/Next.js.");
      break;
    case "auth-secrets":
      add("v0", 10, "Auth, permissions, secrets, and billing require production appli
      break;
  }
```

```
  switch (answers.techStack) {
    case "html":
      add("claude-artifacts", 5, "Standalone HTML is a natural Artifact handoff.");
      add("chatgpt-canvas", 2, "Canvas can edit and preview the HTML.");
      break;
    case "react":
      add("claude-artifacts", 5, "Contained React tools work well as Artifacts.");
      add("chatgpt-canvas", 4, "Canvas supports React editing and preview.");
      add("v0", 5, "v0 is ideal when React becomes a multi-file codebase.");
      break;
    case "nextjs":
      add("v0", 9, "A multi-file Next.js project is a v0 workflow.");
      break;
    case "nextjs-postgres":
      add("v0", 10, "Next.js plus Postgres needs migrations, server access, environme
      break;
    case "nextjs-supabase":
      add("v0", 10, "Next.js plus Supabase needs browser/server clients, Auth, RLS, a
      break;
  }

  switch (answers.persistence) {
    case "database":
      add("v0", 8, "Persistent records require a production data layer.");
      break;
    case "auth":
      add("v0", 9, "Authentication requires a durable application and session-aware s
      break;
    case "multi-tenant":
      add("v0", 10, "Multi-tenancy requires organizations, memberships, roles, and se
      break;
    case "storage":
      add("v0", 8, "Protected uploads and files require storage policies and authoriz
      break;
  }

  switch (answers.handoff) {
    case "share-now":
      add("claude-artifacts", 6, "Immediate sharing favors a completed artifact.");
      add("chatgpt-canvas", 3, "Canvas can share or export reviewable work.");
      break;
    case "doc-export":
      add("chatgpt-canvas", 7, "Canvas is strongest for PDF, DOCX, Markdown, and code
      add("claude-artifacts", 4, "Artifacts offer HTML, JSX, Mermaid, SVG, and browse
      break;
    case "github":
      add("v0", 7, "Repository-backed code workflow is a central v0 use case.");
      add("claude-artifacts", 3, "Artifact source can be versioned after export.");
      add("chatgpt-canvas", 3, "Canvas code/doc exports can be committed.");
      break;
    case "deploy":
      add("v0", 10, "Deployment strongly favors v0 and Vercel.");
      break;
  }

  const ranked = (Object.keys(scores) as PlatformId[])
```

```
    .map((id) => ({
      ...getPlatform(id),
      score: scores[id],
      reasons: reasons[id],
    }))
    .sort((a, b) => b.score - a.score);

  const primary = ranked[^11_0];
  const secondary = ranked[^11_1];

  const needsProductionHandoff =
    answers.backend === "database" ||
    answers.backend === "server-api" ||
    answers.backend === "auth-secrets" ||
    answers.interaction === "multi-user" ||
    answers.techStack === "nextjs" ||
    answers.techStack === "nextjs-postgres" ||
    answers.techStack === "nextjs-supabase" ||
    answers.persistence !== "none" ||
    answers.handoff === "deploy";

  const needsSupabase =
    answers.techStack === "nextjs-supabase" ||
    answers.persistence !== "none";

  return {
    primary,
    secondary,
    ranked,
    needsProductionHandoff,
    needsSupabase,
    confidence: Math.min(
      97,
      Math.max(
        62,
        68 +
          (primary.score - secondary.score) * 4 +
          (needsProductionHandoff && primary.id === "v0" ? 10 : 0)
      )
    ),
  };
}

function createBuildPrompt(
  answers: Answers,
  recommendation: Recommendation
) {
  const platformInstructions: Record<string, string> = {
    "Claude Artifacts": `Build this as a polished, self-contained Claude Artifact.
- Keep the deliverable client-side and dependency-light.
- Use realistic local sample data.
- Do not assume a durable database, full authentication, private APIs, or server-side
- Include empty, loading, success, and error states.
- Keep source portable as HTML, JSX, Mermaid, SVG, Markdown, JSON, or CSV.`,
    "ChatGPT Canvas": `Build this as an editable ChatGPT Canvas deliverable.
- Optimize for a clear document or code-file workflow.
```

```
- State assumptions and open questions.
- Keep preview requirements contained.
- Prepare it for document or language-specific source export.
- Use the canvas for revision and review, then hand off durable source to GitHub.`,
    v0: `Build this as a production-oriented Next.js App Router application in v0.
- Use TypeScript, Tailwind, accessible components, and a clean multi-file structure.
- Separate Client Components from server logic.
- Use environment variable placeholders, never hardcoded credentials.
- Include loading, empty, error, and success states.
- Add README instructions for setup, GitHub, database migrations, testing, and Vercel
  };

  const supabaseBlock = recommendation.needsSupabase
    ? `
Supabase requirements:
- Use @supabase/ssr with separate browser and server client files.
- Use Supabase Postgres for persistent data.
- Enable Row Level Security on exposed tables.
- Keep service-role credentials server-only.
- Use server actions or route handlers for sensitive writes.
- Validate writes before database access.
- Scope data by user_id and organization_id when multi-tenant behavior is required.
- Store migrations and RLS policies in GitHub.`
    : "";

  const productionBlock = recommendation.needsProductionHandoff
    ? `
Production requirements:
- Add server-side authorization checks.
- Add schema validation.
- Add database migrations, indexes, and seed data.
- Add linting, type checking, tests, and production build verification.
- Add monitoring, logging, error handling, and audit controls where needed.
- Review privacy, security, accessibility, and mobile behavior before release.`
    : "";

  return `Build this for Semester.

Recommended platform: ${recommendation.primary.platform}
Secondary option: ${recommendation.secondary.platform}
Confidence: ${recommendation.confidence}%

Workflow:
- Deliverable: ${answers.deliverable}
- Interaction: ${answers.interaction}
- Backend: ${answers.backend}
- Technology: ${answers.techStack}
- Persistence: ${answers.persistence}
- Handoff: ${answers.handoff}

${platformInstructions[recommendation.primary.platform]}

Required output:
- Explain architecture and assumptions.
- Use realistic Semester data and terminology.
- Include the primary workflow and a clear non-goals section.
```

```
- Include instructions to run, edit, export, and hand off the deliverable.
${supabaseBlock}
${productionBlock}

Design direction:
Use a premium B2B software aesthetic with clear hierarchy, accessible controls, reada
}

function downloadFile(filename: string, content: string, type: string) {
  const blob = new Blob([content], { type });
  const url = URL.createObjectURL(blob);
  const link = document.createElement("a");

  link.href = url;
  link.download = filename;
  link.click();

  URL.revokeObjectURL(url);
}

async function copyText(value: string) {
  await navigator.clipboard.writeText(value);
}

function getBadgeClass(platform: string) {
  if (platform === "Claude Artifacts") {
    return "border-amber-200 bg-amber-50 text-amber-800";
  }

  if (platform === "ChatGPT Canvas") {
    return "border-emerald-200 bg-emerald-50 text-emerald-800";
  }

  return "border-slate-800 bg-slate-950 text-white";
}

function getStatusClass(value: string) {
  if (value === "High") {
    return "border-emerald-200 bg-emerald-50 text-emerald-700";
  }

  if (value === "Medium") {
    return "border-amber-200 bg-amber-50 text-amber-700";
  }

  return "border-rose-200 bg-rose-50 text-rose-700";
}

function Badge({
  children,
  className = "",
}: {
  children: React.ReactNode;
  className?: string;
}) {
  return (
```

```
    <span
      className={`inline-flex items-center rounded-full border px-2 py-0.5 text-xs fo
    >
      {children}
    </span>
  );
}

function CopyButton({
  value,
  label,
}: {
  value: string;
  label: string;
}) {
  const [copied, setCopied] = useState(false);

  const handleCopy = async () => {
    try {
      await copyText(value);
      setCopied(true);
      window.setTimeout(() => setCopied(false), 1400);
    } catch {
      setCopied(false);
    }
  };

  return (
    <button
      onClick={handleCopy}
      className="rounded-lg border border-slate-200 bg-white px-3 py-1.5 text-xs font
    >
      {copied ? "Copied" : label}
    </button>
  );
}

function FeatureChecklistView() {
  const [checked, setChecked] = useState<Record<string, boolean>>({});

  const completed = Object.values(checked).filter(Boolean).length;
  const total = FEATURE_CHECKLIST.length;

  return (
    <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
      <div className="flex flex-col justify-between gap-3 md:flex-row md:items-end">
        <div>
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-600
            Feature checklist
          </p>
          <h2 className="mt-1 text-xl font-bold text-slate-950">
            Evaluate your project requirements
          </h2>
          <p className="mt-1 text-sm text-slate-600">
            Mark the capabilities your Semester project actually needs. High
            capability indicates the platform is a natural primary fit.
```

```
          </p>
        </div>

        <Badge className="border-indigo-200 bg-indigo-50 text-indigo-700">
          {completed} / {total} checked
        </Badge>
      </div>

      <div className="mt-5 overflow-x-auto">
        <table className="w-full min-w-[900px] border-collapse">
          <thead className="bg-slate-50">
            <tr>
              <th className="w-12 px-4 py-3 text-left text-xs font-bold uppercase tra
                Use
              </th>
              <th className="px-4 py-3 text-left text-xs font-bold uppercase tracking
                Feature
              </th>
              <th className="px-4 py-3 text-left text-xs font-bold uppercase tracking
                Claude Artifacts
              </th>
              <th className="px-4 py-3 text-left text-xs font-bold uppercase tracking
                ChatGPT Canvas
              </th>
              <th className="px-4 py-3 text-left text-xs font-bold uppercase tracking
                v0
              </th>
            </tr>
          </thead>

          <tbody>
            {FEATURE_CHECKLIST.map((item) => (
              <tr key={item.feature} className="border-t border-slate-100">
                <td className="px-4 py-3">
                  <input
                    type="checkbox"
                    checked={Boolean(checked[item.feature])}
                    onChange={(event) =>
                      setChecked((current) => ({
                        ...current,
                        [item.feature]: event.target.checked,
                      }))
                    }
                    className="h-4 w-4 accent-indigo-600"
                  />
                </td>

                <td className="px-4 py-3 text-sm font-semibold text-slate-800">
                  {item.feature}
                </td>

                <td className="px-4 py-3">
                  <Badge className={getStatusClass(item.claude)}>
                    {item.claude}
                  </Badge>
                </td>
```

```
                <td className="px-4 py-3">
                  <Badge className={getStatusClass(item.canvas)}>
                    {item.canvas}
                  </Badge>
                </td>

                <td className="px-4 py-3">
                  <Badge className={getStatusClass(item.v0)}>
                    {item.v0}
                  </Badge>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </section>
  );
}

export function WorkflowRouter({
  userId,
}: {
  userId?: string;
}) {
  const [answers, setAnswers] = useState<Answers>(DEFAULT_ANSWERS);
  const [title, setTitle] = useState("Semester workflow decision");
  const [sort, setSort] = useState<{
    key:
      | "platform"
      | "primaryUse"
      | "execution"
      | "reactSupport"
      | "projectShape"
      | "exportFormats"
      | "productionFit";
    direction: "asc" | "desc";
  }>({
    key: "platform",
    direction: "asc",
  });

  const [selectedPromptId, setSelectedPromptId] = useState<number | null>(
    null
  );
  const [saveState, setSaveState] = useState<
    "idle" | "saving" | "saved" | "error"
  >("idle");

  const recommendation = useMemo(
    () => recommendPlatform(answers),
    [answers]
  );

  const buildPrompt = useMemo(
```

```
    () => createBuildPrompt(answers, recommendation),
    [answers, recommendation]
  );

  const sortedPlatforms = useMemo(() => {
    return [...PLATFORM_DATA].sort((a, b) => {
      const aValue = String(a[sort.key]);
      const bValue = String(b[sort.key]);

      return sort.direction === "asc"
        ? aValue.localeCompare(bValue)
        : bValue.localeCompare(aValue);
    });
  }, [sort]);

  const promptTestResults = useMemo(
    () =>
      REAL_WORLD_PROMPTS.map((item) => {
        const result = recommendPlatform(item.answers);

        return {
          ...item,
          winner: result.primary.id,
          winnerName: result.primary.platform,
          matchedExpectedWinner: result.primary.id === item.expectedWinner,
          confidence: result.confidence,
        };
      }),
    []
  );

  const updateAnswer = <K extends keyof Answers>(
    key: K,
    value: Answers[K]
  ) => {
    setAnswers((current) => ({
      ...current,
      [key]: value,
    }));
  };

  const toggleSort = (key: typeof sort.key) => {
    setSort((current) => ({
      key,
      direction:
        current.key === key && current.direction === "asc" ? "desc" : "asc",
    }));
  };

  const usePromptScenario = (scenario: RealWorldPrompt) => {
    setAnswers(scenario.answers);
    setSelectedPromptId(scenario.id);
    window.scrollTo({
      top: 0,
      behavior: "smooth",
    });
```

```
  };

  const exportMarkdown = () => {
    const markdown = `# Semester AI Workflow Recommendation

## Current recommendation

- Primary platform: ${recommendation.primary.platform}
- Secondary platform: ${recommendation.secondary.platform}
- Confidence: ${recommendation.confidence}%
- Production handoff required: ${
      recommendation.needsProductionHandoff ? "Yes" : "No"
    }
- Supabase recommended: ${recommendation.needsSupabase ? "Yes" : "No"}

## Project inputs

| Input | Selected value |
|---|---|
| Deliverable | ${answers.deliverable} |
| Interaction | ${answers.interaction} |
| Backend | ${answers.backend} |
| Technology | ${answers.techStack} |
| Persistence | ${answers.persistence} |
| Handoff | ${answers.handoff} |

## Why

${recommendation.primary.reasons.map((reason) => `- ${reason}`).join("\n")}

## Build prompt

\`\`\`text
${buildPrompt}
\`\`\`

## Platform comparison

| Platform | Primary use | React support | Export formats | Production fit |
|---|---|---|---|---|
${PLATFORM_DATA.map(
  (platform) =>
    `| ${platform.platform} | ${platform.primaryUse} | ${platform.reactSupport} | ${p
).join("\n")}

## Feature checklist

| Feature | Claude Artifacts | ChatGPT Canvas | v0 |
|---|---|---|---|
${FEATURE_CHECKLIST.map(
  (item) =>
    `| ${item.feature} | ${item.claude} | ${item.canvas} | ${item.v0} |`
).join("\n")}
`;

    downloadFile(
```

```
      "semester-workflow-recommendation.md",
      markdown,
      "text/markdown;charset=utf-8"
    );
  };

  const saveCurrentRecommendation = async () => {
    if (!userId) {
      setSaveState("error");
      return;
    }

    setSaveState("saving");

    try {
      await saveRecommendation({
        title,
        inputs: answers,
        recommendation: {
          primary: recommendation.primary.platform,
          secondary: recommendation.secondary.platform,
          confidence: recommendation.confidence,
          reasons: recommendation.primary.reasons,
          needsProductionHandoff: recommendation.needsProductionHandoff,
          needsSupabase: recommendation.needsSupabase,
        },
        buildPrompt,
      });

      setSaveState("saved");
      window.setTimeout(() => setSaveState("idle"), 1800);
    } catch {
      setSaveState("error");
    }
  };

  const inputGroups: Array<{
    key: keyof Answers;
    label: string;
    options: Array<[Answers[keyof Answers], string]>;
  }> = [
    {
      key: "deliverable",
      label: "Deliverable",
      options: [
        ["interactive-tool", "Interactive tool, tracker, dashboard, calculator, or co
        ["product-prototype", "Clickable product prototype or interface exploration"
        ["marketing-page", "Marketing site, landing page, one-page demo, or visual ex
        ["diagram", "Architecture, process, ERD, sequence, roadmap, or system diagram
        ["document", "PRD, policy, research, strategy, report, or operating document"
        ["presentation", "Pitch, board update, sales narrative, launch, or training"
        ["full-stack-feature", "Authenticated, data-backed, multi-user product featur
        ["analysis", "Python analysis, research, data cleanup, or generated-file work
      ],
    },
    {
```

```
      key: "interaction",
      label: "Interaction",
      options: [
        ["none", "No interaction; people mainly read or review"],
        ["simple", "Simple client-side interaction: tabs, toggles, calculators, or sa
        ["complex", "Stateful UI: filters, tables, forms, drawers, and editable work
        ["multi-user", "Shared records, multiple users, roles, permissions, or collab
      ],
    },
    {
      key: "backend",
      label: "Data and backend",
      options: [
        ["none", "No backend; static content or browser-only sample data"],
        ["file-export", "Needs CSV, Markdown, SVG, JSON, PDF, or related downloads"]
        ["database", "Needs persistent database records"],
        ["server-api", "Needs API routes, webhooks, server logic, or external service
        ["auth-secrets", "Needs auth, private APIs, permissions, billing, or server s
      ],
    },
    {
      key: "techStack",
      label: "Technology target",
      options: [
        ["none", "No prescribed code stack"],
        ["html", "Plain HTML, CSS, and JavaScript"],
        ["react", "React component or client-side application"],
        ["nextjs", "Multi-file Next.js application"],
        ["nextjs-postgres", "Next.js with Postgres"],
        ["nextjs-supabase", "Next.js with Supabase, Postgres, and Auth"],
      ],
    },
    {
      key: "persistence",
      label: "Persistence and access",
      options: [
        ["none", "No persistence; sample data is sufficient"],
        ["database", "Persistent records"],
        ["auth", "User accounts and session-aware access"],
        ["multi-tenant", "Organizations, memberships, roles, and tenant isolation"],
        ["storage", "Uploaded files, source assets, screenshots, or exports"],
      ],
    },
    {
      key: "handoff",
      label: "Handoff",
      options: [
        ["share-now", "Share immediately as a finished artifact"],
        ["doc-export", "Export PDF, DOCX, Markdown, or source"],
        ["github", "Push versioned source into GitHub"],
        ["deploy", "Deploy a production web application"],
      ],
    },
  ];

  const header = (label: string, key: typeof sort.key) => (
```

```
      <button
        onClick={() => toggleSort(key)}
        className="flex items-center gap-1 text-left text-xs font-bold uppercase tracki
      >
        {label}
        {sort.key === key ? (sort.direction === "asc" ? "↑" : "↓") : "↕"}
      </button>
    );

    return (
      <main className="min-h-screen bg-slate-50 p-4 text-slate-900 md:p-8">
        <div className="mx-auto max-w-7xl space-y-6">
          <header className="rounded-2xl bg-gradient-to-br from-slate-950 via-slate-900
            <p className="text-xs font-bold uppercase tracking-[0.16em] text-indigo-300
              Semester build workflow
            </p>

            <h1 className="mt-2 text-3xl font-bold tracking-tight">
              Claude Artifacts vs Canvas vs v0 Router
            </h1>

            <p className="mt-2 max-w-3xl text-sm leading-6 text-slate-300">
              Map an exact coding or writing workflow to the best AI workspace,
              compare platform limits, test 15 real-world prompts, and save
              durable decisions to Supabase.
            </p>
          </header>

          <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
            <div>
              <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
                Project decision tree
              </p>

              <h2 className="mt-1 text-xl font-bold text-slate-950">
                Choose project requirements
              </h2>

              <p className="mt-1 text-sm text-slate-600">
                The router selects v0 when your project needs a real Next.js app,
                Postgres/Supabase, authentication, multi-tenant access, private
                integrations, GitHub workflow, or deployment.
              </p>
            </div>

            <div className="mt-5 grid gap-4 md:grid-cols-2 xl:grid-cols-3">
              {inputGroups.map((group) => (
                <fieldset
                  key={group.key}
                  className="rounded-xl border border-slate-200 bg-slate-50 p-4"
                >
                  <legend className="px-1 text-sm font-bold text-slate-900">
                    {group.label}
                  </legend>

                  <div className="mt-2 space-y-2">
```

```
                {group.options.map(([value, label]) => (
                  <label
                    key={String(value)}
                    className="flex cursor-pointer items-start gap-2 text-sm text-s
                  >
                    <input
                      type="radio"
                      name={group.key}
                      checked={answers[group.key] === value}
                      onChange={() =>
                        updateAnswer(group.key, value as never)
                      }
                      className="mt-1 accent-indigo-600"
                    />

                    <span>{label}</span>
                  </label>
                ))}
              </div>
            </fieldset>
          ))}
        </div>

        <div className="mt-5 rounded-2xl border border-indigo-200 bg-indigo-50 p-5"
          <div className="flex flex-col justify-between gap-4 lg:flex-row lg:items-
            <div>
              <p className="text-xs font-bold uppercase tracking-[0.14em] text-indi
                Primary recommendation
              </p>

              <div className="mt-2 flex flex-wrap items-center gap-2">
                <h3 className="text-2xl font-bold text-slate-950">
                  {recommendation.primary.platform}
                </h3>

                <Badge
                  className={getBadgeClass(
                    recommendation.primary.platform
                  )}
                >
                  {recommendation.primary.platform}
                </Badge>

                <Badge className="border-indigo-200 bg-white text-indigo-700">
                  {recommendation.confidence}% confidence
                </Badge>
              </div>

              <ul className="mt-4 space-y-2 text-sm leading-6 text-slate-700">
                {recommendation.primary.reasons.map((reason) => (
                  <li key={reason}>• {reason}</li>
                ))}
              </ul>
            </div>

            <aside className="rounded-xl border border-indigo-100 bg-white p-4 lg:w
```

```
              <p className="text-xs font-bold uppercase tracking-wide text-slate-50
                Secondary option
              </p>

              <p className="mt-1 text-base font-bold text-slate-900">
                {recommendation.secondary.platform}
              </p>

              <p className="mt-2 text-sm leading-6 text-slate-600">
                {recommendation.secondary.primaryUse}
              </p>
            </aside>
          </div>

          {recommendation.needsProductionHandoff && (
            <div className="mt-5 rounded-xl border border-amber-200 bg-amber-50 p-4
              <p className="text-sm font-bold text-amber-900">
                Production handoff required
              </p>

              <p className="mt-1 text-sm leading-6 text-amber-800">
                Your choices require a durable application boundary:
                persistent data, database access, authentication, server
                routes, tenant access, secrets, or deployment. Prototype in
                Claude or Canvas if helpful, then move into a v0/Next.js
                codebase with Supabase and GitHub.
              </p>
            </div>
          )}
        </div>
      </section>

      <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
        <div className="flex flex-col justify-between gap-3 md:flex-row md:items-en
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo
              Generated build prompt
            </p>

            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Copy, export, or save the recommendation
            </h2>
          </div>

          <div className="flex flex-wrap gap-2">
            <CopyButton value={buildPrompt} label="Copy prompt" />

            <button
              onClick={exportMarkdown}
              className="rounded-lg bg-slate-900 px-3 py-1.5 text-xs font-bold tex
            >
              Export Markdown
            </button>
          </div>
        </div>
```

```
          <pre className="mt-4 max-h-[520px] overflow-auto whitespace-pre-wrap rounde
            {buildPrompt}
          </pre>

          <div className="mt-5 flex flex-col gap-3 border-t border-slate-100 pt-5 md
            <label className="flex-1 text-sm font-bold text-slate-700">
              Saved recommendation title
              <input
                value={title}
                onChange={(event) => setTitle(event.target.value)}
                className="mt-1 w-full rounded-lg border border-slate-200 px-3 py-2
              />
            </label>

            <button
              onClick={saveCurrentRecommendation}
              disabled={saveState === "saving"}
              className="rounded-lg bg-indigo-600 px-4 py-2 text-sm font-bold text-wh
            >
              {saveState === "saving"
                ? "Saving..."
                : userId
                ? "Save to Supabase"
                : "Sign in to save"}
            </button>
          </div>

          {saveState === "saved" && (
            <p className="mt-3 text-sm font-semibold text-emerald-700">
              Recommendation saved to Supabase.
            </p>
          )}

          {saveState === "error" && (
            <p className="mt-3 text-sm font-semibold text-rose-700">
              {userId
                ? "The recommendation could not be saved. Check Supabase schema, Auth
                : "Sign in before saving recommendations."}
            </p>
          )}
        </section>

        <FeatureChecklistView />

        <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
              Prompt-to-tool test suite
            </p>

            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Test 15 real-world prompts
            </h2>

            <p className="mt-1 text-sm text-slate-600">
              Each scenario applies the same decision engine. Click “Use
```

```
            scenario" to load that workflow into the decision tree.
          </p>
        </div>

        <div className="mt-5 overflow-x-auto">
          <table className="w-full min-w-[1100px] border-collapse">
            <thead className="bg-slate-50">
              <tr>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  #
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Category
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Prompt
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Winner
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Confidence
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Test
                </th>
              </tr>
            </thead>

            <tbody>
              {promptTestResults.map((item) => (
                <tr
                  key={item.id}
                  className={`border-t border-slate-100 ${
                    selectedPromptId === item.id ? "bg-indigo-50" : ""
                  }`}
                >
                  <td className="px-4 py-3 text-sm font-bold text-slate-700">
                    {item.id}
                  </td>

                  <td className="px-4 py-3 text-sm text-slate-600">
                    {item.category}
                  </td>

                  <td className="max-w-xl px-4 py-3 text-sm leading-6 text-slate-70
                    {item.prompt}
                  </td>

                  <td className="px-4 py-3">
                    <Badge className={getBadgeClass(item.winnerName)}>
                      {item.winnerName}
                    </Badge>
                  </td>

                  <td className="px-4 py-3 text-sm font-semibold text-slate-700">
```

```
                    {item.confidence}%
                  </td>

                  <td className="px-4 py-3">
                    <button
                      onClick={() => usePromptScenario(item)}
                      className="rounded-lg border border-slate-200 bg-white px-3 p
                    >
                      Use scenario
                    </button>

                    <p
                      className={`mt-2 text-xs font-bold ${
                        item.matchedExpectedWinner
                          ? "text-emerald-700"
                          : "text-rose-700"
                      }`}
                    >
                      {item.matchedExpectedWinner
                        ? "Expected result"
                        : "Review scoring"}
                    </p>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </section>

      <section className="overflow-hidden rounded-2xl border border-slate-200 bg-wh
        <div className="border-b border-slate-200 p-5">
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
            Sortable comparison matrix
          </p>

          <h2 className="mt-1 text-xl font-bold text-slate-950">
            Execution limits, React support, and export formats
          </h2>
        </div>

        <div className="overflow-x-auto">
          <table className="w-full min-w-[1450px] border-collapse">
            <thead className="bg-slate-50">
              <tr>
                <th className="px-4 py-3">
                  {header("Platform", "platform")}
                </th>
                <th className="px-4 py-3">
                  {header("Primary use", "primaryUse")}
                </th>
                <th className="px-4 py-3">
                  {header("Execution model", "execution")}
                </th>
                <th className="px-4 py-3">
                  {header("React support", "reactSupport")}
```

```
                </th>
                <th className="px-4 py-3">
                  {header("Project shape", "projectShape")}
                </th>
                <th className="px-4 py-3">
                  {header("Export formats", "exportFormats")}
                </th>
                <th className="px-4 py-3">
                  {header("Production fit", "productionFit")}
                </th>
              </tr>
            </thead>

            <tbody>
              {sortedPlatforms.map((platform) => (
                <tr
                  key={platform.id}
                  className="border-t border-slate-100 align-top hover:bg-indigo-50
                >
                  <td className="px-4 py-4">
                    <p className="font-bold text-slate-950">
                      {platform.platform}
                    </p>

                    <div className="mt-2 flex flex-wrap gap-1">
                      <Badge className="border-slate-200 bg-slate-50 text-slate-600
                        Prototype {platform.scores.prototype}/5
                      </Badge>
                      <Badge className="border-slate-200 bg-slate-50 text-slate-600
                        Docs {platform.scores.documents}/5
                      </Badge>
                      <Badge className="border-slate-200 bg-slate-50 text-slate-600
                        Full stack {platform.scores.fullStack}/5
                      </Badge>
                      <Badge className="border-slate-200 bg-slate-50 text-slate-600
                        Deploy {platform.scores.deployment}/5
                      </Badge>
                    </div>
                  </td>

                  <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    {platform.primaryUse}
                  </td>

                  <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    {platform.execution}
                  </td>

                  <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    {platform.reactSupport}
                  </td>

                  <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                    {platform.projectShape}
                  </td>
```

```
                    <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                      {platform.exportFormats}
                    </td>

                    <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                      {platform.productionFit}
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </section>
      </div>
    </main>
  );
}
```

Next.js page

app/workflow-router/page.tsx

```
import { WorkflowRouter } from "@/components/workflow-router";
import { createClient } from "@/lib/supabase/server";

export default async function WorkflowRouterPage() {
  const supabase = await createClient();

  const {
    data: { user },
  } = await supabase.auth.getUser();

  return <WorkflowRouter userId={user?.id} />;
}
```

Authentication UI

components/auth-button.tsx

```
import Link from "next/link";
import { createClient } from "@/lib/supabase/server";

export async function AuthButton() {
  const supabase = await createClient();

  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) {
    return (
```

```
      <Link
        href="/auth/sign-in"
        className="rounded-lg border border-slate-200 bg-white px-3 py-2 text-sm fon
      >
        Sign in
      </Link>
    );
  }

  return (
    <form action="/auth/sign-out" method="post">
      <button className="rounded-lg bg-slate-900 px-3 py-2 text-sm font-bold text-wh
        Sign out
      </button>
    </form>
  );
}
```

app/auth/sign-in/page.tsx

```
"use client";

import { useState } from "react";
import { createClient } from "@/lib/supabase/client";

export default function SignInPage() {
  const [email, setEmail] = useState("");
  const [status, setStatus] = useState<"idle" | "loading" | "sent" | "error">(
    "idle"
  );

  const signIn = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    setStatus("loading");

    const supabase = createClient();

    const { error } = await supabase.auth.signInWithOtp({
      email,
      options: {
        emailRedirectTo: `${window.location.origin}/workflow-router`,
      },
    });

    setStatus(error ? "error" : "sent");
  };

  return (
    <main className="min-h-screen bg-slate-50 p-6">
      <form
        onSubmit={signIn}
        className="mx-auto mt-24 max-w-md rounded-2xl border border-slate-200 bg-whi
      >
        <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-600">
          Semester
```

```
        </p>

        <h1 className="mt-2 text-2xl font-bold text-slate-950">
          Sign in to save workflow decisions
        </h1>

        <p className="mt-2 text-sm leading-6 text-slate-600">
          Enter your email and we will send you a magic sign-in link.
        </p>

        <label className="mt-5 block text-sm font-bold text-slate-700">
          Email address
          <input
            required
            type="email"
            value={email}
            onChange={(event) => setEmail(event.target.value)}
            placeholder="you@example.com"
            className="mt-1 w-full rounded-lg border border-slate-200 px-3 py-2 font-
          />
        </label>

        <button
          disabled={status === "loading"}
          className="mt-5 w-full rounded-lg bg-indigo-600 px-4 py-2.5 text-sm font-bo
        >
          {status === "loading" ? "Sending..." : "Send magic link"}
        </button>

        {status === "sent" && (
          <p className="mt-4 text-sm font-semibold text-emerald-700">
            Check your email for the sign-in link.
          </p>
        )}

        {status === "error" && (
          <p className="mt-4 text-sm font-semibold text-rose-700">
            We could not send the sign-in link. Verify your Supabase Auth setup.
          </p>
        )}
      </form>
    </main>
  );
}
```

Add Supabase Auth safely

1. In Supabase, enable your desired login method—email magic link is the simplest first option.
2. Add the local and production callback URLs in **Authentication → URL Configuration**:
   - `http://localhost:3000/workflow-router`
   - Your Vercel production URL plus `/workflow-router`

3. Add the public Supabase URL and publishable key to `.env.local` and Vercel environment variables.
4. Apply the migration so saved decisions have a table and Row Level Security policies.
5. Verify user identity with `supabase.auth.getUser()` in server actions and Server Components before reads/writes.
6. Never place a Supabase service-role key in a Client Component, browser environment variable, or repository.

Supabase recommends cookie-based SSR integration for Next.js and specifically distinguishes `createBrowserClient` for client code from `createServerClient` for server code. Its official quickstart provides a Next.js template with cookie Auth, TypeScript, and Tailwind; Vercel also offers Supabase-auth templates for Next.js App Router.[148] [144] [145] [146]

How the test suite routes prompts

The 15 scenarios deliberately demonstrate the operating boundary:

Prompt family	Typical winner	Why
Sortable client-side tracker with sample data	Claude Artifacts	High-value interactive artifact with no durable backend requirement
PRD, policy, strategy, legal first draft, research	ChatGPT Canvas	Revision, structured writing, and document export are the primary workflow
Mermaid or SVG architecture/visual system map	Claude Artifacts	Best fit for fast visual and diagram outputs
Python cleanup, CSV analysis, generated reports	ChatGPT Canvas	Analysis and file-oriented coding workflow
Authenticated Next.js dashboard with Supabase	v0	Requires project files, server/client boundaries, database, Auth, RLS, and deployment
Multi-tenant team workspace	v0	Requires roles, organization membership, server checks, and Postgres persistence
Marketing-page prototype	Claude Artifacts	Shareable standalone visual output is sufficient
Production landing site connected to a real codebase	v0	Requires a maintainable Next.js project and deployment pipeline

Recommended deployment path

```
Claude Artifacts
  → Prototype the decision-tree UX, visual explainers, diagrams, and local-data tool
  → Export JSX, HTML, Mermaid, SVG, Markdown, or JSON

ChatGPT Canvas
  → Create PRDs, research, legal/operations first drafts, code refinements, and analy
  → Export Markdown, PDF, DOCX, SQL, Python, or source code
```

```
v0 + GitHub
  → Build the multi-file Next.js production application
  → Add client/server Supabase integration
  → Add migrations, RLS, Auth, tests, environment variables, and preview deployments

Vercel + Supabase
  → Deploy the application
  → Use Vercel environment variables
  → Keep user/session data in Supabase Auth and persistent records in Supabase Postgr
```

Supabase and Vercel describe this pairing as a standard Next.js App Router path: Supabase supplies Postgres, Auth, Storage, Edge Functions, and Realtime, while Vercel provides application deployment; their starter/template flow uses cookie-based Auth with TypeScript and Tailwind. [149] [150] [148]

⁂

Interactive prompt-routing tool: compare Claude Artifacts, ChatGPT Canvas, and v0 sandbox across 12 workflow types

Computer
Benchmark test suite: grade Claude Artifacts, ChatGPT Canvas, and v0 sandbox on 10 identical code-generation prompts
Computer
Show the database migration code
Show the workflow-router page code
Show the Supabase client files

Below are the database migration, the server-rendered `workflow-router` page, and the Supabase client files for a Next.js App Router implementation. The app can support a 12-workflow router and persist benchmark runs for 10 identical prompts.

Use `@supabase/ssr`, not the deprecated Next.js Auth Helpers package. Supabase's current migration guidance recommends `createBrowserClient` and `createServerClient`, with cookie-backed sessions available to both browser and server rendering; a middleware/proxy pattern refreshes those sessions before the app renders. [161] [162]

Database migration

Save this as:

```
supabase/migrations/001_create_workflow_router.sql
```

```
create extension if not exists pgcrypto;

-- Optional team/organization layer. Keep it even if the first version is
```

```
-- individual-only: it makes Semester multi-tenant capable later.
create table public.organizations (
  id uuid primary key default gen_random_uuid(),
  name text not null check (char_length(trim(name)) > 0),
  slug text unique,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.organization_members (
  organization_id uuid not null
    references public.organizations(id)
    on delete cascade,
  user_id uuid not null
    references auth.users(id)
    on delete cascade,
  role text not null default 'member'
    check (role in ('owner', 'admin', 'member', 'viewer')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  primary key (organization_id, user_id)
);

-- A saved output from the prompt-routing decision tree.
create table public.workflow_recommendations (
  id uuid primary key default gen_random_uuid(),

  -- Nullable initially allows a private, personal recommendation history.
  -- Make this NOT NULL once every record belongs to a workspace.
  organization_id uuid
    references public.organizations(id)
    on delete cascade,

  user_id uuid not null
    references auth.users(id)
    on delete cascade,

  title text not null default 'Untitled workflow decision'
    check (char_length(trim(title)) > 0),

  -- The current answers from the six decision-tree dimensions.
  -- Example:
  -- {
  --   "deliverable": "full-stack-feature",
  --   "interaction": "multi-user",
  --   "backend": "auth-secrets",
  --   "techStack": "nextjs-supabase",
  --   "persistence": "multi-tenant",
  --   "handoff": "deploy"
  -- }
  inputs jsonb not null default '{}'::jsonb,

  -- The calculated result.
  -- Example:
  -- {
  --   "primary": "v0",
```

```
  --   "secondary": "Claude Artifacts",
  --   "confidence": 97,
  --   "reasons": ["..."],
  --   "needsProductionHandoff": true,
  --   "needsSupabase": true
  -- }
  recommendation jsonb not null default '{}'::jsonb,

  build_prompt text not null,

  tags text[] not null default '{}',

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

-- A benchmark suite defines one comparable set of prompts.
create table public.benchmark_suites (
  id uuid primary key default gen_random_uuid(),

  organization_id uuid
    references public.organizations(id)
    on delete cascade,

  created_by uuid not null
    references auth.users(id)
    on delete cascade,

  title text not null
    check (char_length(trim(title)) > 0),

  description text,

  benchmark_scope text not null default 'code-generation'
    check (
      benchmark_scope in (
        'code-generation',
        'writing',
        'analysis',
        'design',
        'mixed'
      )
    ),

  status text not null default 'draft'
    check (status in ('draft', 'active', 'archived')),

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

-- One of the 10 identical prompts in a suite.
create table public.benchmark_prompts (
  id uuid primary key default gen_random_uuid(),

  suite_id uuid not null
```

```
    references public.benchmark_suites(id)
    on delete cascade,

  prompt_number integer not null
    check (prompt_number > 0),

  title text not null
    check (char_length(trim(title)) > 0),

  category text not null default 'Engineering',

  prompt_text text not null
    check (char_length(trim(prompt_text)) > 0),

  requirements jsonb not null default '{}'::jsonb,

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  unique (suite_id, prompt_number)
);

-- A benchmark run represents executing the same prompt against a tool.
create table public.benchmark_runs (
  id uuid primary key default gen_random_uuid(),

  suite_id uuid not null
    references public.benchmark_suites(id)
    on delete cascade,

  prompt_id uuid not null
    references public.benchmark_prompts(id)
    on delete cascade,

  organization_id uuid
    references public.organizations(id)
    on delete cascade,

  created_by uuid not null
    references auth.users(id)
    on delete cascade,

  platform text not null
    check (platform in ('claude-artifacts', 'chatgpt-canvas', 'v0')),

  run_status text not null default 'planned'
    check (
      run_status in (
        'planned',
        'running',
        'completed',
        'blocked',
        'discarded'
      )
    ),
```

```
  -- Optional metadata for reproducibility.
  model_name text,
  model_version text,
  sandbox_notes text,

  -- Preserve the exact text that was executed.
  executed_prompt text not null,

  -- Link to an artifact, Canvas share link, GitHub PR, v0 deployment,
  -- or exported project/document when available.
  output_url text,

  output_summary text,

  execution_seconds numeric(10, 2)
    check (execution_seconds is null or execution_seconds >= 0),

  -- Integer scoring standardizes comparison.
  correctness_score smallint
    check (correctness_score between 0 and 5),

  ui_quality_score smallint
    check (ui_quality_score between 0 and 5),

  code_quality_score smallint
    check (code_quality_score between 0 and 5),

  maintainability_score smallint
    check (maintainability_score between 0 and 5),

  export_handoff_score smallint
    check (export_handoff_score between 0 and 5),

  sandbox_fit_score smallint
    check (sandbox_fit_score between 0 and 5),

  overall_score numeric(4, 2)
    generated always as (
      case
        when correctness_score is null
          or ui_quality_score is null
          or code_quality_score is null
          or maintainability_score is null
          or export_handoff_score is null
          or sandbox_fit_score is null
        then null
        else round(
          (
            correctness_score +
            ui_quality_score +
            code_quality_score +
            maintainability_score +
            export_handoff_score +
            sandbox_fit_score
          )::numeric / 6,
          2
```

```
        )
      end
    ) stored,

  rubric_notes text,

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  unique (suite_id, prompt_id, platform)
);

-- Helpful indexes for catalog, recommendation history, and benchmark dashboards.
create index workflow_recommendations_user_id_created_at_idx
  on public.workflow_recommendations (user_id, created_at desc);

create index workflow_recommendations_organization_id_created_at_idx
  on public.workflow_recommendations (organization_id, created_at desc);

create index benchmark_suites_organization_id_created_at_idx
  on public.benchmark_suites (organization_id, created_at desc);

create index benchmark_prompts_suite_id_prompt_number_idx
  on public.benchmark_prompts (suite_id, prompt_number);

create index benchmark_runs_suite_id_platform_idx
  on public.benchmark_runs (suite_id, platform);

create index benchmark_runs_prompt_id_platform_idx
  on public.benchmark_runs (prompt_id, platform);

create index benchmark_runs_organization_id_created_at_idx
  on public.benchmark_runs (organization_id, created_at desc);

-- Generic updated_at trigger.
create or replace function public.set_updated_at()
returns trigger
language plpgsql
security invoker
set search_path = public
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

create trigger organizations_set_updated_at
before update on public.organizations
for each row
execute function public.set_updated_at();

create trigger organization_members_set_updated_at
before update on public.organization_members
for each row
execute function public.set_updated_at();
```

```
create trigger workflow_recommendations_set_updated_at
before update on public.workflow_recommendations
for each row
execute function public.set_updated_at();

create trigger benchmark_suites_set_updated_at
before update on public.benchmark_suites
for each row
execute function public.set_updated_at();

create trigger benchmark_prompts_set_updated_at
before update on public.benchmark_prompts
for each row
execute function public.set_updated_at();

create trigger benchmark_runs_set_updated_at
before update on public.benchmark_runs
for each row
execute function public.set_updated_at();

-- Enable Row Level Security.
alter table public.organizations enable row level security;
alter table public.organization_members enable row level security;
alter table public.workflow_recommendations enable row level security;
alter table public.benchmark_suites enable row level security;
alter table public.benchmark_prompts enable row level security;
alter table public.benchmark_runs enable row level security;

-- Organizations: a member can see its workspace.
create policy "Organization members can view organizations"
on public.organizations
for select
to authenticated
using (
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = organizations.id
      and membership.user_id = auth.uid()
  )
);

create policy "Authenticated users can create organizations"
on public.organizations
for insert
to authenticated
with check (true);

create policy "Organization owners and admins can update organizations"
on public.organizations
for update
to authenticated
using (
  exists (
    select 1
```

```
    from public.organization_members membership
    where membership.organization_id = organizations.id
      and membership.user_id = auth.uid()
      and membership.role in ('owner', 'admin')
  )
)
with check (
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = organizations.id
      and membership.user_id = auth.uid()
      and membership.role in ('owner', 'admin')
  )
);

-- Membership visibility.
create policy "Organization members can view memberships"
on public.organization_members
for select
to authenticated
using (
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = organization_members.organization_id
      and membership.user_id = auth.uid()
  )
);

create policy "Owners and admins can manage memberships"
on public.organization_members
for all
to authenticated
using (
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = organization_members.organization_id
      and membership.user_id = auth.uid()
      and membership.role in ('owner', 'admin')
  )
)
with check (
  exists (
    select 1
    from public.organization_members membership
    where membership.organization_id = organization_members.organization_id
      and membership.user_id = auth.uid()
      and membership.role in ('owner', 'admin')
  )
);

-- Personal records OR organization-scoped records:
-- - User-owned private record: organization_id is null and user_id matches.
-- - Shared record: caller must be a member of that organization.
```

```
create policy "Users can view allowed workflow recommendations"
on public.workflow_recommendations
for select
to authenticated
using (
  user_id = auth.uid()
  or (
    organization_id is not null
    and exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = workflow_recommendations.organization_id
        and membership.user_id = auth.uid()
    )
  )
);

create policy "Users can create their own workflow recommendations"
on public.workflow_recommendations
for insert
to authenticated
with check (
  user_id = auth.uid()
  and (
    organization_id is null
    or exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = workflow_recommendations.organization_id
        and membership.user_id = auth.uid()
        and membership.role in ('owner', 'admin', 'member')
    )
  )
);

create policy "Owners can update their private recommendations and editors can update
on public.workflow_recommendations
for update
to authenticated
using (
  user_id = auth.uid()
  or (
    organization_id is not null
    and exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = workflow_recommendations.organization_id
        and membership.user_id = auth.uid()
        and membership.role in ('owner', 'admin', 'member')
    )
  )
)
with check (
  user_id = auth.uid()
  or (
    organization_id is not null
```

```
    and exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = workflow_recommendations.organization_id
        and membership.user_id = auth.uid()
        and membership.role in ('owner', 'admin', 'member')
    )
  )
);

create policy "Owners can delete their private recommendations and admins can delete
on public.workflow_recommendations
for delete
to authenticated
using (
  user_id = auth.uid()
  or (
    organization_id is not null
    and exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = workflow_recommendations.organization_id
        and membership.user_id = auth.uid()
        and membership.role in ('owner', 'admin')
    )
  )
);

-- Benchmark suites: creator can always access own suites;
-- workspace members can access organization-scoped suites.
create policy "Users can view allowed benchmark suites"
on public.benchmark_suites
for select
to authenticated
using (
  created_by = auth.uid()
  or (
    organization_id is not null
    and exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = benchmark_suites.organization_id
        and membership.user_id = auth.uid()
    )
  )
);

create policy "Authenticated users can create benchmark suites"
on public.benchmark_suites
for insert
to authenticated
with check (
  created_by = auth.uid()
  and (
    organization_id is null
    or exists (
```

```
      select 1
      from public.organization_members membership
      where membership.organization_id = benchmark_suites.organization_id
        and membership.user_id = auth.uid()
        and membership.role in ('owner', 'admin', 'member')
    )
  )
);

create policy "Creators and workspace editors can update benchmark suites"
on public.benchmark_suites
for update
to authenticated
using (
  created_by = auth.uid()
  or (
    organization_id is not null
    and exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = benchmark_suites.organization_id
        and membership.user_id = auth.uid()
        and membership.role in ('owner', 'admin', 'member')
    )
  )
)
with check (
  created_by = auth.uid()
  or (
    organization_id is not null
    and exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = benchmark_suites.organization_id
        and membership.user_id = auth.uid()
        and membership.role in ('owner', 'admin', 'member')
    )
  )
);

create policy "Creators and workspace admins can delete benchmark suites"
on public.benchmark_suites
for delete
to authenticated
using (
  created_by = auth.uid()
  or (
    organization_id is not null
    and exists (
      select 1
      from public.organization_members membership
      where membership.organization_id = benchmark_suites.organization_id
        and membership.user_id = auth.uid()
        and membership.role in ('owner', 'admin')
    )
  )
```

```
);

-- Benchmark prompt visibility is inherited from the suite.
create policy "Users can view benchmark prompts from accessible suites"
on public.benchmark_prompts
for select
to authenticated
using (
  exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_prompts.suite_id
      and (
        suite.created_by = auth.uid()
        or (
          suite.organization_id is not null
          and exists (
            select 1
            from public.organization_members membership
            where membership.organization_id = suite.organization_id
              and membership.user_id = auth.uid()
          )
        )
      )
  )
);

create policy "Editors can manage prompts for accessible benchmark suites"
on public.benchmark_prompts
for all
to authenticated
using (
  exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_prompts.suite_id
      and (
        suite.created_by = auth.uid()
        or (
          suite.organization_id is not null
          and exists (
            select 1
            from public.organization_members membership
            where membership.organization_id = suite.organization_id
              and membership.user_id = auth.uid()
              and membership.role in ('owner', 'admin', 'member')
          )
        )
      )
  )
)
with check (
  exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_prompts.suite_id
```

```
      and (
        suite.created_by = auth.uid()
        or (
          suite.organization_id is not null
          and exists (
            select 1
            from public.organization_members membership
            where membership.organization_id = suite.organization_id
              and membership.user_id = auth.uid()
              and membership.role in ('owner', 'admin', 'member')
          )
        )
      )
  )
);

-- Benchmark run visibility and write permissions are inherited from the suite.
create policy "Users can view benchmark runs from accessible suites"
on public.benchmark_runs
for select
to authenticated
using (
  exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_runs.suite_id
      and (
        suite.created_by = auth.uid()
        or (
          suite.organization_id is not null
          and exists (
            select 1
            from public.organization_members membership
            where membership.organization_id = suite.organization_id
              and membership.user_id = auth.uid()
          )
        )
      )
  )
);

create policy "Editors can manage benchmark runs from accessible suites"
on public.benchmark_runs
for all
to authenticated
using (
  exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_runs.suite_id
      and (
        suite.created_by = auth.uid()
        or (
          suite.organization_id is not null
          and exists (
            select 1
```

```
            from public.organization_members membership
            where membership.organization_id = suite.organization_id
              and membership.user_id = auth.uid()
              and membership.role in ('owner', 'admin', 'member')
          )
        )
      )
  )
)
with check (
  created_by = auth.uid()
  and exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_runs.suite_id
      and (
        suite.created_by = auth.uid()
        or (
          suite.organization_id is not null
          and exists (
            select 1
            from public.organization_members membership
            where membership.organization_id = suite.organization_id
              and membership.user_id = auth.uid()
              and membership.role in ('owner', 'admin', 'member')
          )
        )
      )
  )
);
```

Notes on the migration

- The benchmark schema keeps the **identical prompt** in `benchmark_prompts` and creates one run per platform in `benchmark_runs`, enforced by `unique (suite_id, prompt_id, platform)`.
- `overall_score` is generated from six 0–5 rubric scores: correctness, UI quality, code quality, maintainability, export/handoff, and sandbox fit.
- Start with individual records by leaving `organization_id` null; add organization membership when the Semester team needs shared benchmark suites.
- RLS checks membership in the database. Never rely only on hiding buttons in the UI.

Benchmark seed: 10 identical prompts

Add this after creating a suite, or keep it in a separate seed migration after replacing the suite UUID:

```
-- Replace this with a real benchmark_suites.id.
-- Example workflow:
-- 1. Insert a suite through the app.
-- 2. Copy the returned id.
```

```
-- 3. Insert these ten prompts.
--
-- insert into public.benchmark_prompts (
--   suite_id,
--   prompt_number,
--   title,
--   category,
--   prompt_text,
--   requirements
-- )
-- values
--   (
--     'YOUR_SUITE_UUID',
--     1,
--     'Sortable catalog',
--     'React UI',
--     'Build a responsive React dashboard that displays 100 artifact records. Includ
--     '{"requiresReact":true,"requiresExport":true,"requiresBackend":false}'::jsonb
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     2,
--     'Next.js authentication',
--     'Full stack',
--     'Build a Next.js App Router application with email magic-link authentication,
--     '{"requiresNextjs":true,"requiresAuth":true,"requiresServer":true}'::jsonb
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     3,
--     'Supabase artifact tracker',
--     'Full stack',
--     'Build a multi-file Next.js artifact tracker backed by Supabase Postgres. Incl
--     '{"requiresNextjs":true,"requiresDatabase":true,"requiresRLS":true,"requiresMu
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     4,
--     'Landing page',
--     'Marketing',
--     'Build a polished responsive B2B SaaS landing page with hero, product workflow
--     '{"requiresHTML":true,"requiresResponsiveUI":true,"requiresBackend":false}'::j
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     5,
--     'Data visualization dashboard',
--     'React UI',
--     'Build a React dashboard with realistic sample SaaS metrics, KPI cards, filter
--     '{"requiresReact":true,"requiresState":true,"requiresCharts":true}'::jsonb
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     6,
--     'CSV analysis script',
--     'Python',
```

```
--     'Write a Python script that reads a CSV of subscription events, computes montl
--     '{"requiresPython":true,"requiresFileIO":true,"requiresAnalysis":true}'::jsonl
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     7,
--     'API route',
--     'Backend',
--     'Build a Next.js route handler for creating artifact records. Validate input,
--     '{"requiresNextjs":true,"requiresAPI":true,"requiresAuth":true,"requiresValida
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     8,
--     'File upload workflow',
--     'Full stack',
--     'Build a secure Next.js file upload workflow with Supabase Storage. Require au
--     '{"requiresNextjs":true,"requiresStorage":true,"requiresAuth":true,"requiresRl
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     9,
--     'Mermaid architecture map',
--     'Diagram',
--     'Create a Mermaid architecture diagram for a multi-tenant Next.js app using Su
--     '{"requiresMermaid":true,"requiresDocumentation":true,"requiresBackend":false}
--   ),
--   (
--     'YOUR_SUITE_UUID',
--     10,
--     'Technical design document',
--     'Engineering documentation',
--     'Write a technical design document for a multi-tenant artifact catalog. Includ
--     '{"requiresDocumentation":true,"requiresArchitecture":true,"requiresPlanning"
--   );
```

Workflow-router page

Save this as:

```
app/workflow-router/page.tsx
```

This is a **Server Component**: it retrieves the signed-in user and the user's latest saved decisions on the server, then passes serializable data to the client-side interactive router.

```
import Link from "next/link";
import { createClient } from "@/lib/supabase/server";
import { WorkflowRouter } from "@/components/workflow-router";

type SavedRecommendation = {
  id: string;
  title: string;
  inputs: Record<string, string>;
```

```
  recommendation: {
    primary?: string;
    secondary?: string;
    confidence?: number;
    reasons?: string[];
    needsProductionHandoff?: boolean;
    needsSupabase?: boolean;
  };
  build_prompt: string;
  created_at: string;
};

export default async function WorkflowRouterPage() {
  const supabase = await createClient();

  const {
    data: { user },
  } = await supabase.auth.getUser();

  const { data: savedRecommendations, error } = user
    ? await supabase
        .from("workflow_recommendations")
        .select(
          "id, title, inputs, recommendation, build_prompt, created_at"
        )
        .eq("user_id", user.id)
        .order("created_at", { ascending: false })
        .limit(12)
    : {
        data: [],
        error: null,
      };

  if (error) {
    console.error("Unable to load saved workflow recommendations:", error);
  }

  const normalizedRecommendations: SavedRecommendation[] =
    (savedRecommendations ?? []).map((item) => ({
      id: item.id,
      title: item.title,
      inputs: (item.inputs ?? {}) as Record<string, string>,
      recommendation: (item.recommendation ?? {}) as SavedRecommendation["recommenda
      build_prompt: item.build_prompt,
      created_at: item.created_at,
    }));

  return (
    <main className="min-h-screen bg-slate-50 p-4 text-slate-900 md:p-8">
      <div className="mx-auto max-w-7xl space-y-6">
        <header className="rounded-2xl bg-gradient-to-br from-slate-950 via-slate-900
          <p className="text-xs font-bold uppercase tracking-[0.16em] text-indigo-300
            Semester build workflow
          </p>

          <div className="mt-2 flex flex-col justify-between gap-4 md:flex-row md:ite
```

```
            <div>
              <h1 className="text-3xl font-bold tracking-tight">
                AI Workflow Router
              </h1>

              <p className="mt-2 max-w-3xl text-sm leading-6 text-slate-300">
                Route coding, writing, product, design, analysis, and production
                work to Claude Artifacts, ChatGPT Canvas, or v0—then save the
                decision and benchmark identical prompts across all three.
              </p>
            </div>

            <div className="rounded-xl border border-white/10 bg-white/10 px-4 py-3 t
              {user ? (
                <>
                  <p className="font-semibold text-white">
                    Signed in
                  </p>
                  <p className="mt-1 truncate text-xs text-slate-300">
                    {user.email}
                  </p>
                </>
              ) : (
                <>
                  <p className="font-semibold text-white">
                    Prototype mode
                  </p>
                  <p className="mt-1 text-xs text-slate-300">
                    Sign in to save recommendations and benchmarks.
                  </p>
                </>
              )}
            </div>
          </div>
        </header>

        {!user && (
          <section className="rounded-2xl border border-amber-200 bg-amber-50 p-5">
            <h2 className="text-base font-bold text-amber-950">
              Save and benchmark features require sign-in
            </h2>

            <p className="mt-2 text-sm leading-6 text-amber-900">
              You can use the decision tree without an account. Sign in with
              Supabase Auth to save workflow decisions, create benchmark suites,
              record runs, and share them with a Semester organization later.
            </p>

            <Link
              href="/auth/sign-in"
              className="mt-4 inline-flex rounded-lg bg-amber-900 px-3 py-2 text-sm t
            >
              Sign in
            </Link>
          </section>
        )}
```

```
        <WorkflowRouter
          userId={user?.id}
          userEmail={user?.email ?? null}
          initialRecommendations={normalizedRecommendations}
        />
      </div>
    </main>
  );
}
```

Supabase client files

1. Browser client

Save as:

```
lib/supabase/client.ts
```

```
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!
  );
}
```

Use this client only in Client Components, browser event handlers, and client-side UI code.

2. Server client

Save as:

```
lib/supabase/server.ts
```

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
```

```
        },

        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) => {
              cookieStore.set(name, value, options);
            });
          } catch {
            /*
             * In a Server Component, cookie writes can be unavailable.
             * The middleware/proxy session refresh handles session updates
             * before render and writes the returned cookies to the response.
             */
          }
        },
      },
    }
  );
}
```

Use this in Server Components, Server Actions, and Route Handlers. It can read the same cookie-backed auth session created in the browser.

3. Session-refresh middleware helper

Save as:

```
lib/supabase/middleware.ts
```

```
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function updateSession(request: NextRequest) {
  let response = NextResponse.next({
    request,
  });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },

        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) => {
            request.cookies.set(name, value);
          });

          response = NextResponse.next({
            request,
```

```
          });

          cookiesToSet.forEach(({ name, value, options }) => {
            response.cookies.set(name, value, options);
          });
        },
      },
    }
  );

  /*
   * Do not remove this call.
   * It refreshes and validates the auth session before protected pages render.
   */
  await supabase.auth.getUser();

  return response;
}
```

4. Root middleware

Save as:

```
middleware.ts
```

```
import type { NextRequest } from "next/server";
import { updateSession } from "@/lib/supabase/middleware";

export async function middleware(request: NextRequest) {
  return updateSession(request);
}

export const config = {
  matcher: [
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*
  ],
};
```

5. Environment file

Save this locally as:

```
.env.local
```

```
NEXT_PUBLIC_SUPABASE_URL=https://YOUR_PROJECT.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=YOUR_PUBLISHABLE_KEY
```

Commit only the template:

```
.env.example
```

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
```

Do not put a service-role secret in a `NEXT_PUBLIC_` variable. Next.js embeds `NEXT_PUBLIC_*` values into browser-delivered code, while non-prefixed environment variables remain server-only. [163]

Auth route example

For magic-link confirmation, add:

```
app/auth/confirm/route.ts
```

```
import { type NextRequest, NextResponse } from "next/server";
import { createServerClient } from "@supabase/ssr";

export async function GET(request: NextRequest) {
  const requestUrl = new URL(request.url);
  const tokenHash = requestUrl.searchParams.get("token_hash");
  const type = requestUrl.searchParams.get("type");
  const next = requestUrl.searchParams.get("next") ?? "/workflow-router";

  const response = NextResponse.redirect(new URL(next, requestUrl.origin));

  if (!tokenHash || !type) {
    return NextResponse.redirect(
      new URL("/auth/sign-in?error=missing_token", requestUrl.origin)
    );
  }

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },

        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) => {
            response.cookies.set(name, value, options);
          });
        },
      },
    }
  );

  const { error } = await supabase.auth.verifyOtp({
```

```
    type: type as "magiclink",
    token_hash: tokenHash,
  });

  if (error) {
    return NextResponse.redirect(
      new URL("/auth/sign-in?error=verification_failed", requestUrl.origin)
    );
  }

  return response;
}
```

Benchmark rubric

For the 10 identical prompts, score each platform on the same six dimensions:

Score	Meaning
0	Missing, unusable, or fails the stated requirement
1	Barely addresses the requirement
2	Partially works but has major omissions
3	Usable baseline with some meaningful gaps
4	Strong and close to production or publication quality
5	Excellent: complete, coherent, robust, and well matched to the platform
Dimension	What to grade
---	---
Correctness	Does it satisfy the stated requirements?
UI quality	Is the interaction/design clear, accessible, and polished where relevant?
Code quality	Is the code coherent, typed, structured, and free of obvious defects?
Maintainability	Is the result easy to revise, test, and evolve?
Export/handoff	Can you use the resulting file/code/document in the next workflow stage?
Sandbox fit	Does the task actually fit the platform rather than fighting its constraints?

The database's generated `overall_score` averages these six scores automatically. This makes the benchmark a tool-choice study, not merely a subjective output gallery.

Security boundary

Claude Artifacts can rapidly generate self-contained browser artifacts and lightweight interactive work, but sandbox boundaries mean it should not be the final system for private production data, durable authentication, or backend persistence. ChatGPT Canvas is strongest for revision-centric writing, code, and export; v0 is the appropriate runtime-oriented destination for the Next.js/Supabase portions of the workflow. Supabase's current SSR package

consolidates earlier framework-specific Auth Helpers and uses cookie-backed browser/server clients, with middleware refreshing the session before a server-rendered request is handled. [162] [164] [161]

⁂

Sortable benchmark matrix: Claude Artifacts vs ChatGPT Canvas vs v0 Sandbox across 12 workflows — scoring code quality, rendering, and state management

Computer
Build a benchmark test suite harness that grades Claude Artifacts, ChatGPT Canvas, and v0 sandbox across 12 distinct code-generation tasks
Computer
Show me the middleware code for session refreshing
Show me the UI component for the 12-workflow router
Add benchmark grading logic to the Supabase schema

Below is a production-oriented benchmark layer for the Semester workflow router: a **12-workflow sortable matrix**, a reusable **Supabase session-refresh middleware**, a client-side router UI, and database grading logic that computes weighted benchmark scores.

The core distinction remains: Claude Artifacts is strongest for self-contained interactive artifacts and diagrams, ChatGPT Canvas for document/code iteration and file exports, and v0 for multi-file Next.js apps with real state, server logic, data, and deployment. Vercel Sandbox runs generated or untrusted code in isolated Linux microVMs, which is why it is a better fit for full project execution than a browser-only prototype environment. [177]

Session refresh middleware

Supabase's current Next.js SSR guidance uses a middleware/proxy layer to refresh expired session cookies **before** Server Components render. Call `supabase.auth.getUser()` inside the request flow; do not rely on client-side session state for authorization. [178] [179]

`lib/supabase/middleware.ts`

```
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function updateSession(request: NextRequest) {
  let response = NextResponse.next({
    request,
  });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
```

```
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },

        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) => {
            request.cookies.set(name, value);
          });

          response = NextResponse.next({
            request,
          });

          cookiesToSet.forEach(({ name, value, options }) => {
            response.cookies.set(name, value, options);
          });
        },
      },
    }
  );

  /*
   * Do not remove this call.
   * It validates the current user and refreshes an expiring Supabase Auth
   * access token, then propagates updated cookies onto the response.
   */
  await supabase.auth.getUser();

  return response;
}
```

Root proxy/middleware

For current Next.js conventions, name the file `proxy.ts` if your version uses the Proxy convention; many existing projects still use `middleware.ts`. Keep one approach, not both.

```
import type { NextRequest } from "next/server";
import { updateSession } from "@/lib/supabase/middleware";

export async function proxy(request: NextRequest) {
  return updateSession(request);
}

export const config = {
  matcher: [
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*
  ],
};
```

If your project uses the earlier middleware convention:

```
import type { NextRequest } from "next/server";
import { updateSession } from "@/lib/supabase/middleware";

export async function middleware(request: NextRequest) {
  return updateSession(request);
}

export const config = {
  matcher: [
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).
  ],
};
```

Benchmark grading schema

This migration adds a reusable grading model for **12 workflow types** and platform-specific benchmark runs. It supports raw manual scores, weighted computed scores, evaluator notes, and a sortable leader-board query.

Save as:

```
supabase/migrations/002_add_benchmark_grading.sql
```

```
create extension if not exists pgcrypto;

-- Keep the platform choice constrained and normalized.
do $$
begin
  create type public.ai_platform as enum (
    'claude-artifacts',
    'chatgpt-canvas',
    'v0'
  );
exception
  when duplicate_object then null;
end $$;

-- The 12 workflow types tested in this benchmark.
do $$
begin
  create type public.workflow_type as enum (
    'interactive-tool',
    'product-prototype',
    'marketing-page',
    'diagram',
    'document',
    'presentation',
    'data-analysis',
    'react-component',
    'nextjs-feature',
    'database-workflow',
    'authentication-workflow',
```

```
    'multi-tenant-saas'
  );
exception
  when duplicate_object then null;
end $$;

-- A benchmark suite may contain the complete 12-workflow set or a subset.
create table if not exists public.benchmark_suites (
  id uuid primary key default gen_random_uuid(),
  organization_id uuid,
  created_by uuid not null references auth.users(id) on delete cascade,
  title text not null,
  description text,
  status text not null default 'draft'
    check (status in ('draft', 'active', 'completed', 'archived')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

-- Each benchmark task holds an identical task specification that gets used
-- against all three platforms.
create table if not exists public.benchmark_tasks (
  id uuid primary key default gen_random_uuid(),
  suite_id uuid not null references public.benchmark_suites(id) on delete cascade,

  workflow workflow_type not null,
  task_number integer not null check (task_number between 1 and 12),

  title text not null,
  category text not null,
  task_prompt text not null,

  -- Machine-readable requirements help keep grading reproducible.
  requirements jsonb not null default '{}'::jsonb,

  -- The expected grading emphasis for this task:
  -- e.g. {"code_quality": 0.30, "rendering": 0.20, ...}
  rubric_weights jsonb not null default '{
    "correctness": 0.25,
    "code_quality": 0.20,
    "rendering": 0.15,
    "state_management": 0.15,
    "maintainability": 0.10,
    "handoff": 0.10,
    "sandbox_fit": 0.05
  }'::jsonb,

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  unique (suite_id, task_number),
  unique (suite_id, workflow)
);

-- One run for a platform against an identical benchmark task.
create table if not exists public.benchmark_runs (
```

```
  id uuid primary key default gen_random_uuid(),

  suite_id uuid not null references public.benchmark_suites(id) on delete cascade,
  task_id uuid not null references public.benchmark_tasks(id) on delete cascade,

  organization_id uuid,
  created_by uuid not null references auth.users(id) on delete cascade,

  platform public.ai_platform not null,

  status text not null default 'planned'
    check (status in ('planned', 'running', 'completed', 'blocked', 'discarded')),

  -- Exact prompt and context used for repeatability.
  executed_prompt text not null,
  model_name text,
  model_version text,
  sandbox_notes text,

  -- Link to Claude artifact, Canvas share, GitHub PR, v0 project,
  -- deployed preview, or stored output artifact.
  output_url text,
  output_summary text,

  execution_seconds numeric(10, 2)
    check (execution_seconds is null or execution_seconds >= 0),

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  unique (suite_id, task_id, platform)
);

-- Individual grading events allow more than one evaluator without losing
-- the underlying observations.
create table if not exists public.benchmark_grades (
  id uuid primary key default gen_random_uuid(),

  run_id uuid not null references public.benchmark_runs(id) on delete cascade,
  graded_by uuid not null references auth.users(id) on delete cascade,

  -- Every value is 0 through 5:
  -- 0 = missing or fails
  -- 3 = usable baseline
  -- 5 = excellent / robust / well-matched
  correctness_score smallint not null check (correctness_score between 0 and 5),
  code_quality_score smallint not null check (code_quality_score between 0 and 5),
  rendering_score smallint not null check (rendering_score between 0 and 5),
  state_management_score smallint not null check (state_management_score between 0 ar
  maintainability_score smallint not null check (maintainability_score between 0 and
  handoff_score smallint not null check (handoff_score between 0 and 5),
  sandbox_fit_score smallint not null check (sandbox_fit_score between 0 and 5),

  grader_notes text,
  evidence jsonb not null default '{}'::jsonb,
```

```
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  unique (run_id, graded_by)
);

-- Helpful indexes for matrix and leaderboard queries.
create index if not exists benchmark_tasks_suite_workflow_idx
  on public.benchmark_tasks(suite_id, workflow);

create index if not exists benchmark_runs_suite_platform_idx
  on public.benchmark_runs(suite_id, platform);

create index if not exists benchmark_runs_task_platform_idx
  on public.benchmark_runs(task_id, platform);

create index if not exists benchmark_grades_run_id_idx
  on public.benchmark_grades(run_id);

-- Generic timestamp handler.
create or replace function public.set_updated_at()
returns trigger
language plpgsql
security invoker
set search_path = public
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

drop trigger if exists benchmark_suites_set_updated_at on public.benchmark_suites;
create trigger benchmark_suites_set_updated_at
before update on public.benchmark_suites
for each row
execute function public.set_updated_at();

drop trigger if exists benchmark_tasks_set_updated_at on public.benchmark_tasks;
create trigger benchmark_tasks_set_updated_at
before update on public.benchmark_tasks
for each row
execute function public.set_updated_at();

drop trigger if exists benchmark_runs_set_updated_at on public.benchmark_runs;
create trigger benchmark_runs_set_updated_at
before update on public.benchmark_runs
for each row
execute function public.set_updated_at();

drop trigger if exists benchmark_grades_set_updated_at on public.benchmark_grades;
create trigger benchmark_grades_set_updated_at
before update on public.benchmark_grades
for each row
execute function public.set_updated_at();
```

```
-- Weighted score by evaluator and run.
-- We use the task's rubric_weights JSON so individual tasks can emphasize
-- different criteria, e.g. diagrams emphasize rendering and handoff, while
-- multi-tenant work emphasizes correctness/state/auth architecture.
create or replace view public.benchmark_grade_scores as
select
  grade.id as grade_id,
  grade.run_id,
  grade.graded_by,
  run.platform,
  run.task_id,
  task.suite_id,
  task.workflow,
  task.task_number,
  task.title as task_title,

  grade.correctness_score,
  grade.code_quality_score,
  grade.rendering_score,
  grade.state_management_score,
  grade.maintainability_score,
  grade.handoff_score,
  grade.sandbox_fit_score,

  round(
    (
      grade.correctness_score *
        coalesce((task.rubric_weights ->> 'correctness')::numeric, 0.25) +
      grade.code_quality_score *
        coalesce((task.rubric_weights ->> 'code_quality')::numeric, 0.20) +
      grade.rendering_score *
        coalesce((task.rubric_weights ->> 'rendering')::numeric, 0.15) +
      grade.state_management_score *
        coalesce((task.rubric_weights ->> 'state_management')::numeric, 0.15) +
      grade.maintainability_score *
        coalesce((task.rubric_weights ->> 'maintainability')::numeric, 0.10) +
      grade.handoff_score *
        coalesce((task.rubric_weights ->> 'handoff')::numeric, 0.10) +
      grade.sandbox_fit_score *
        coalesce((task.rubric_weights ->> 'sandbox_fit')::numeric, 0.05)
    ),
    2
  ) as weighted_score_out_of_5,

  round(
    (
      grade.correctness_score *
        coalesce((task.rubric_weights ->> 'correctness')::numeric, 0.25) +
      grade.code_quality_score *
        coalesce((task.rubric_weights ->> 'code_quality')::numeric, 0.20) +
      grade.rendering_score *
        coalesce((task.rubric_weights ->> 'rendering')::numeric, 0.15) +
      grade.state_management_score *
        coalesce((task.rubric_weights ->> 'state_management')::numeric, 0.15) +
      grade.maintainability_score *
        coalesce((task.rubric_weights ->> 'maintainability')::numeric, 0.10) +
```

```
      grade.handoff_score *
        coalesce((task.rubric_weights ->> 'handoff')::numeric, 0.10) +
      grade.sandbox_fit_score *
        coalesce((task.rubric_weights ->> 'sandbox_fit')::numeric, 0.05)
    ) * 20,
    1
  ) as weighted_score_out_of_100,

  grade.grader_notes,
  grade.evidence,
  grade.created_at
from public.benchmark_grades grade
join public.benchmark_runs run
  on run.id = grade.run_id
join public.benchmark_tasks task
  on task.id = run.task_id;

-- Average all evaluators for each platform/task pair.
create or replace view public.benchmark_task_leaderboard as
select
  suite_id,
  workflow,
  task_number,
  task_title,
  platform,

  round(avg(weighted_score_out_of_5), 2) as average_score_out_of_5,
  round(avg(weighted_score_out_of_100), 1) as average_score_out_of_100,

  round(avg(correctness_score), 2) as average_correctness,
  round(avg(code_quality_score), 2) as average_code_quality,
  round(avg(rendering_score), 2) as average_rendering,
  round(avg(state_management_score), 2) as average_state_management,
  round(avg(maintainability_score), 2) as average_maintainability,
  round(avg(handoff_score), 2) as average_handoff,
  round(avg(sandbox_fit_score), 2) as average_sandbox_fit,

  count(*) as grader_count
from public.benchmark_grade_scores
group by
  suite_id,
  workflow,
  task_number,
  task_title,
  platform;

-- Overall platform result across all tasks in a suite.
create or replace view public.benchmark_platform_leaderboard as
select
  suite_id,
  platform,

  round(avg(average_score_out_of_5), 2) as suite_average_out_of_5,
  round(avg(average_score_out_of_100), 1) as suite_average_out_of_100,

  round(avg(average_correctness), 2) as average_correctness,
```

```
  round(avg(average_code_quality), 2) as average_code_quality,
  round(avg(average_rendering), 2) as average_rendering,
  round(avg(average_state_management), 2) as average_state_management,
  round(avg(average_maintainability), 2) as average_maintainability,
  round(avg(average_handoff), 2) as average_handoff,
  round(avg(average_sandbox_fit), 2) as average_sandbox_fit,

  count(*) as task_platform_result_count
from public.benchmark_task_leaderboard
group by suite_id, platform;

-- Row Level Security.
alter table public.benchmark_suites enable row level security;
alter table public.benchmark_tasks enable row level security;
alter table public.benchmark_runs enable row level security;
alter table public.benchmark_grades enable row level security;

-- Initial simple policy model:
-- creators own their suites and all related tasks/runs/grades.
-- Replace/extend with organization membership checks for team workspaces.
create policy "Creators can manage benchmark suites"
on public.benchmark_suites
for all
to authenticated
using (created_by = auth.uid())
with check (created_by = auth.uid());

create policy "Suite creators can manage benchmark tasks"
on public.benchmark_tasks
for all
to authenticated
using (
  exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_tasks.suite_id
      and suite.created_by = auth.uid()
  )
)
with check (
  exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_tasks.suite_id
      and suite.created_by = auth.uid()
  )
);

create policy "Suite creators can manage benchmark runs"
on public.benchmark_runs
for all
to authenticated
using (
  exists (
    select 1
    from public.benchmark_suites suite
```

```
    where suite.id = benchmark_runs.suite_id
      and suite.created_by = auth.uid()
  )
)
with check (
  created_by = auth.uid()
  and exists (
    select 1
    from public.benchmark_suites suite
    where suite.id = benchmark_runs.suite_id
      and suite.created_by = auth.uid()
  )
);

create policy "Users can create and read their own benchmark grades"
on public.benchmark_grades
for all
to authenticated
using (
  graded_by = auth.uid()
  or exists (
    select 1
    from public.benchmark_runs run
    join public.benchmark_suites suite on suite.id = run.suite_id
    where run.id = benchmark_grades.run_id
      and suite.created_by = auth.uid()
  )
)
with check (
  graded_by = auth.uid()
);
```

Scoring model

The weighted score is out of 100:

$$\text{Score} = 20 \times \sum_{i} (\text{criterion score}_i \times \text{criterion weight}_i)$$

Default weights:

Criterion	Default weight	What it measures
Correctness	25%	Did the output satisfy the task requirements?
Code quality	20%	Structure, types, errors, clarity, and defects
Rendering	15%	Visual output, responsiveness, UX, and polish
State management	15%	Filters, forms, async state, persistence boundaries, and predictable behavior
Maintainability	10%	Modularity, readability, extensibility, and testability
Handoff	10%	Quality of export, source, repository, or deployment handoff
Sandbox fit	5%	Whether the task suits the platform's execution model

For database, authentication, and multi-tenant tasks, increase `correctness`, `state_management`, and `sandbox_fit`; for interactive design work, increase `rendering`; for docs, raise `handoff` and lower `state_management`.

Seed the 12 workflows

Insert this after creating a benchmark suite. Replace `YOUR_SUITE_UUID` with the suite ID.

```
insert into public.benchmark_tasks (
  suite_id,
  task_number,
  workflow,
  title,
  category,
  task_prompt,
  requirements,
  rubric_weights
)
values
  (
    'YOUR_SUITE_UUID',
    1,
    'interactive-tool',
    'Sortable artifact tracker',
    'React UI',
    'Build a responsive client-side artifact tracker with 100 sample records, search
    '{"react": true, "clientState": true, "csvExport": true, "backend": false}'::jso
    '{"correctness":0.25,"code_quality":0.20,"rendering":0.20,"state_management":0.2
  ),
  (
    'YOUR_SUITE_UUID',
    2,
    'product-prototype',
    'Interactive onboarding prototype',
    'Product design',
    'Build a clickable product onboarding prototype with an empty state, a multi-ste
    '{"react": true, "prototype": true, "statefulWorkflow": true}'::jsonb,
    '{"correctness":0.20,"code_quality":0.15,"rendering":0.30,"state_management":0.2
  ),
  (
    'YOUR_SUITE_UUID',
    3,
    'marketing-page',
    'B2B SaaS landing page',
    'Marketing',
    'Build a polished responsive B2B SaaS landing page with navigation, hero, produc
    '{"responsive": true, "marketing": true, "backend": false}'::jsonb,
    '{"correctness":0.20,"code_quality":0.15,"rendering":0.35,"state_management":0.0
  ),
  (
    'YOUR_SUITE_UUID',
    4,
    'diagram',
    'Multi-tenant system architecture',
    'Engineering',
```

```
    'Create a Mermaid architecture diagram for a Next.js application using Supabase A
    '{"mermaid": true, "architecture": true, "documentation": true}'::jsonb,
    '{"correctness":0.30,"code_quality":0.05,"rendering":0.30,"state_management":0.05
  ),
  (
    'YOUR_SUITE_UUID',
    5,
    'document',
    'Technical design document',
    'Engineering documentation',
    'Write a technical design document for a multi-tenant artifact catalog. Include c
    '{"documentation": true, "technicalDesign": true}'::jsonb,
    '{"correctness":0.35,"code_quality":0.05,"rendering":0.10,"state_management":0.05
  ),
  (
    'YOUR_SUITE_UUID',
    6,
    'presentation',
    'Product launch narrative',
    'GTM',
    'Create a slide-by-slide product launch narrative for a B2B SaaS feature. Cover a
    '{"slides": true, "gtm": true, "narrative": true}'::jsonb,
    '{"correctness":0.25,"code_quality":0.05,"rendering":0.25,"state_management":0.05
  ),
  (
    'YOUR_SUITE_UUID',
    7,
    'data-analysis',
    'Subscription CSV analysis',
    'Analytics',
    'Write a Python script that reads subscription events from CSV, computes monthly
    '{"python": true, "csv": true, "analysis": true, "files": true}'::jsonb,
    '{"correctness":0.30,"code_quality":0.25,"rendering":0.05,"state_management":0.10
  ),
  (
    'YOUR_SUITE_UUID',
    8,
    'react-component',
    'Metric dashboard component',
    'React UI',
    'Build a reusable TypeScript React dashboard component with KPI cards, loading ar
    '{"react": true, "typescript": true, "component": true}'::jsonb,
    '{"correctness":0.25,"code_quality":0.25,"rendering":0.20,"state_management":0.20
  ),
  (
    'YOUR_SUITE_UUID',
    9,
    'nextjs-feature',
    'Authenticated route handler',
    'Next.js',
    'Build a Next.js App Router route handler for creating artifact records. Require
    '{"nextjs": true, "api": true, "validation": true, "auth": true}'::jsonb,
    '{"correctness":0.30,"code_quality":0.25,"rendering":0.05,"state_management":0.15
  ),
  (
    'YOUR_SUITE_UUID',
```

```
    10,
    'database-workflow',
    'Supabase artifact CRUD',
    'Full stack',
    'Build a Next.js App Router artifact catalog backed by Supabase Postgres. Include
    '{"nextjs": true, "supabase": true, "postgres": true, "rls": true}'::jsonb,
    '{"correctness":0.30,"code_quality":0.20,"rendering":0.10,"state_management":0.20
  ),
  (
    'YOUR_SUITE_UUID',
    11,
    'authentication-workflow',
    'Cookie-based Supabase Auth',
    'Authentication',
    'Build Supabase Auth for Next.js App Router using separate browser/server clients
    '{"nextjs": true, "supabaseAuth": true, "ssr": true, "cookies": true}'::jsonb,
    '{"correctness":0.35,"code_quality":0.20,"rendering":0.05,"state_management":0.20
  ),
  (
    'YOUR_SUITE_UUID',
    12,
    'multi-tenant-saas',
    'Organization-scoped workspace',
    'SaaS architecture',
    'Build a multi-tenant Next.js workspace with organizations, memberships, owner/ad
    '{"nextjs": true, "multiTenant": true, "supabase": true, "rls": true, "auth": tru
    '{"correctness":0.35,"code_quality":0.20,"rendering":0.10,"state_management":0.20
```

12-workflow router UI

Save as:

```
components/workflow-router-12.tsx
```

This is a client component. It includes:

- A 12-workflow selector
- Project-requirement scoring
- A sortable platform benchmark matrix
- An editable grading form
- Benchmark scorecards across code quality, rendering, and state management
- A recommendation panel that responds to the selected workflow

```
"use client";

import { useMemo, useState } from "react";

type PlatformId = "claude-artifacts" | "chatgpt-canvas" | "v0";

type WorkflowId =
```

```
  | "interactive-tool"
  | "product-prototype"
  | "marketing-page"
  | "diagram"
  | "document"
  | "presentation"
  | "data-analysis"
  | "react-component"
  | "nextjs-feature"
  | "database-workflow"
  | "authentication-workflow"
  | "multi-tenant-saas";

type Platform = {
  id: PlatformId;
  name: string;
  execution: string;
  reactSupport: string;
  exportFormats: string;
  limits: string;
  baseScores: {
    codeQuality: number;
    rendering: number;
    stateManagement: number;
    sandboxFit: number;
  };
};

type Workflow = {
  id: WorkflowId;
  number: number;
  title: string;
  category: string;
  description: string;
  requirements: {
    requiresRendering: boolean;
    requiresState: boolean;
    requiresServer: boolean;
    requiresDatabase: boolean;
    requiresAuth: boolean;
    requiresExport: boolean;
    requiresMultiFile: boolean;
  };
};

type Grade = {
  codeQuality: number;
  rendering: number;
  stateManagement: number;
  correctness: number;
  maintainability: number;
  handoff: number;
  sandboxFit: number;
};

const PLATFORMS: Platform[] = [
```

```
  {
    id: "claude-artifacts",
    name: "Claude Artifacts",
    execution:
      "Contained rendered artifact experience, strongest for self-contained client-s
    reactSupport:
      "Strong for lightweight React-style local-state prototypes, interactive pages,
    exportFormats:
      "HTML, JSX-style source, Markdown, SVG, Mermaid, CSV/PNG-style browser exports
    limits:
      "Do not rely on it for durable persistence, full auth, private server secrets,
    baseScores: {
      codeQuality: 4,
      rendering: 5,
      stateManagement: 3,
      sandboxFit: 5,
    },
  },
  {
    id: "chatgpt-canvas",
    name: "ChatGPT Canvas",
    execution:
      "Isolated document/code workspace suited to editing, revision, Python analysis
    reactSupport:
      "Useful for React/HTML code iteration and focused files, but less oriented towa
    exportFormats:
      "PDF, DOCX, Markdown, and language-specific code files such as JS, TS, Python,
    limits:
      "Not a durable backend, authentication system, database host, or deployment run
    baseScores: {
      codeQuality: 4,
      rendering: 3,
      stateManagement: 2,
      sandboxFit: 4,
    },
  },
  {
    id: "v0",
    name: "v0",
    execution:
      "VM-backed Node.js sandbox suited to multi-file projects, server logic, databas
    reactSupport:
      "Native fit for React and Next.js App Router, including server/client boundarie
    exportFormats:
      "Project code, GitHub sync, preview deployments, Vercel deployment, and existin
    limits:
      "Still validate migrations, environment variables, authorization, tests, secur
    baseScores: {
      codeQuality: 5,
      rendering: 4,
      stateManagement: 5,
      sandboxFit: 5,
    },
  },
];
```

```
const WORKFLOWS: Workflow[] = [
  {
    id: "interactive-tool",
    number: 1,
    title: "Sortable artifact tracker",
    category: "React UI",
    description:
      "A client-side tracker with filters, sorting, a detail drawer, sample data, and
    requirements: {
      requiresRendering: true,
      requiresState: true,
      requiresServer: false,
      requiresDatabase: false,
      requiresAuth: false,
      requiresExport: true,
      requiresMultiFile: false,
    },
  },
  {
    id: "product-prototype",
    number: 2,
    title: "Interactive onboarding prototype",
    category: "Product Design",
    description:
      "A clickable onboarding flow with empty, error, success, and completion states
    requirements: {
      requiresRendering: true,
      requiresState: true,
      requiresServer: false,
      requiresDatabase: false,
      requiresAuth: false,
      requiresExport: false,
      requiresMultiFile: false,
    },
  },
  {
    id: "marketing-page",
    number: 3,
    title: "B2B SaaS landing page",
    category: "Marketing",
    description:
      "A polished responsive site with a hero, product sections, proof, FAQ, and CTA
    requirements: {
      requiresRendering: true,
      requiresState: false,
      requiresServer: false,
      requiresDatabase: false,
      requiresAuth: false,
      requiresExport: false,
      requiresMultiFile: false,
    },
  },
  {
    id: "diagram",
    number: 4,
    title: "System architecture map",
```

```
    category: "Engineering",
    description:
      "A Mermaid or SVG map of services, databases, auth, integrations, and trust bou
    requirements: {
      requiresRendering: true,
      requiresState: false,
      requiresServer: false,
      requiresDatabase: false,
      requiresAuth: false,
      requiresExport: true,
      requiresMultiFile: false,
    },
  },
  {
    id: "document",
    number: 5,
    title: "Technical design document",
    category: "Documentation",
    description:
      "A structured technical specification with data model, API design, risks, and r
    requirements: {
      requiresRendering: false,
      requiresState: false,
      requiresServer: false,
      requiresDatabase: false,
      requiresAuth: false,
      requiresExport: true,
      requiresMultiFile: false,
    },
  },
  {
    id: "presentation",
    number: 6,
    title: "Product launch narrative",
    category: "GTM",
    description:
      "A presentation covering positioning, launch plan, sales enablement, metrics, a
    requirements: {
      requiresRendering: true,
      requiresState: false,
      requiresServer: false,
      requiresDatabase: false,
      requiresAuth: false,
      requiresExport: true,
      requiresMultiFile: false,
    },
  },
  {
    id: "data-analysis",
    number: 7,
    title: "Subscription CSV analysis",
    category: "Analytics",
    description:
      "A Python workflow that computes retention and churn metrics, then writes repor
    requirements: {
      requiresRendering: false,
```

```
      requiresState: false,
      requiresServer: false,
      requiresDatabase: false,
      requiresAuth: false,
      requiresExport: true,
      requiresMultiFile: false,
    },
  },
  {
    id: "react-component",
    number: 8,
    title: "Reusable React metric dashboard",
    category: "React UI",
    description:
      "A typed dashboard component with KPI cards, filters, tables, dialogs, and mocl
    requirements: {
      requiresRendering: true,
      requiresState: true,
      requiresServer: false,
      requiresDatabase: false,
      requiresAuth: false,
      requiresExport: false,
      requiresMultiFile: false,
    },
  },
  {
    id: "nextjs-feature",
    number: 9,
    title: "Authenticated route handler",
    category: "Next.js",
    description:
      "A typed route handler with validation, auth, organization checks, and structu
    requirements: {
      requiresRendering: false,
      requiresState: true,
      requiresServer: true,
      requiresDatabase: false,
      requiresAuth: true,
      requiresExport: false,
      requiresMultiFile: true,
    },
  },
  {
    id: "database-workflow",
    number: 10,
    title: "Supabase artifact CRUD",
    category: "Full Stack",
    description:
      "A Next.js catalog backed by Postgres with migrations, RLS, server actions, and
    requirements: {
      requiresRendering: true,
      requiresState: true,
      requiresServer: true,
      requiresDatabase: true,
      requiresAuth: true,
      requiresExport: false,
```

```
      requiresMultiFile: true,
    },
  },
  {
    id: "authentication-workflow",
    number: 11,
    title: "Cookie-based Supabase Auth",
    category: "Authentication",
    description:
      "Browser/server clients, session refresh, protected routes, magic links, and s
    requirements: {
      requiresRendering: true,
      requiresState: true,
      requiresServer: true,
      requiresDatabase: true,
      requiresAuth: true,
      requiresExport: false,
      requiresMultiFile: true,
    },
  },
  {
    id: "multi-tenant-saas",
    number: 12,
    title: "Organization-scoped SaaS workspace",
    category: "SaaS Architecture",
    description:
      "Organizations, memberships, roles, RLS, audit logs, protected APIs, and admin
    requirements: {
      requiresRendering: true,
      requiresState: true,
      requiresServer: true,
      requiresDatabase: true,
      requiresAuth: true,
      requiresExport: false,
      requiresMultiFile: true,
    },
  },
];

const DEFAULT_GRADES: Record<PlatformId, Grade> = {
  "claude-artifacts": {
    codeQuality: 4,
    rendering: 5,
    stateManagement: 3,
    correctness: 4,
    maintainability: 3,
    handoff: 4,
    sandboxFit: 5,
  },
  "chatgpt-canvas": {
    codeQuality: 4,
    rendering: 3,
    stateManagement: 2,
    correctness: 4,
    maintainability: 4,
    handoff: 5,
```

```
    sandboxFit: 4,
  },
  v0: {
    codeQuality: 5,
    rendering: 4,
    stateManagement: 5,
    correctness: 5,
    maintainability: 5,
    handoff: 5,
    sandboxFit: 5,
  },
};

function getPlatformStyle(name: string) {
  if (name === "Claude Artifacts") {
    return "border-amber-200 bg-amber-50 text-amber-800";
  }

  if (name === "ChatGPT Canvas") {
    return "border-emerald-200 bg-emerald-50 text-emerald-800";
  }

  return "border-slate-900 bg-slate-950 text-white";
}

function calculateWorkflowScore(platform: Platform, workflow: Workflow) {
  let score = 0;
  const reasons: string[] = [];

  if (workflow.requirements.requiresRendering) {
    score += platform.baseScores.rendering * 4;
    reasons.push("Rendering quality matters for this workflow.");
  }

  if (workflow.requirements.requiresState) {
    score += platform.baseScores.stateManagement * 4;
    reasons.push("State management is required.");
  }

  if (workflow.requirements.requiresServer) {
    if (platform.id === "v0") {
      score += 20;
      reasons.push("Server-side execution is required.");
    } else {
      score -= 12;
    }
  }

  if (workflow.requirements.requiresDatabase) {
    if (platform.id === "v0") {
      score += 20;
      reasons.push("Database integration is required.");
    } else {
      score -= 15;
    }
  }
```

```
  if (workflow.requirements.requiresAuth) {
    if (platform.id === "v0") {
      score += 20;
      reasons.push("Authentication and authorization are required.");
    } else {
      score -= 15;
    }
  }

  if (workflow.requirements.requiresMultiFile) {
    if (platform.id === "v0") {
      score += 15;
      reasons.push("A multi-file project structure is required.");
    } else {
      score -= 10;
    }
  }

  if (workflow.requirements.requiresExport) {
    if (platform.id === "chatgpt-canvas") {
      score += 15;
      reasons.push("Document/code export is a major requirement.");
    }

    if (platform.id === "claude-artifacts") {
      score += 12;
      reasons.push("Artifact source and browser export are useful.");
    }
  }

  score += platform.baseScores.codeQuality * 2;
  score += platform.baseScores.sandboxFit * 2;

  return {
    score,
    reasons,
  };
}

function getRecommendation(workflow: Workflow) {
  const ranked = PLATFORMS.map((platform) => {
    const result = calculateWorkflowScore(platform, workflow);

    return {
      platform,
      score: result.score,
      reasons: result.reasons,
    };
  }).sort((a, b) => b.score - a.score);

  return {
    primary: ranked[^13_0],
    secondary: ranked[^13_1],
    all: ranked,
  };
```

```
}

function weightedGradeScore(grade: Grade) {
  const weighted =
    grade.correctness * 0.25 +
    grade.codeQuality * 0.2 +
    grade.rendering * 0.15 +
    grade.stateManagement * 0.15 +
    grade.maintainability * 0.1 +
    grade.handoff * 0.1 +
    grade.sandboxFit * 0.05;

  return Number((weighted * 20).toFixed(1));
}

function Badge({
  children,
  className = "",
}: {
  children: React.ReactNode;
  className?: string;
}) {
  return (
    <span
      className={`inline-flex items-center rounded-full border px-2 py-0.5 text-xs fo
    >
      {children}
    </span>
  );
}

function ScoreCell({
  value,
  onChange,
}: {
  value: number;
  onChange: (value: number) => void;
}) {
  return (
    <select
      value={value}
      onChange={(event) => onChange(Number(event.target.value))}
      className="rounded-lg border border-slate-200 bg-white px-2 py-1 text-xs font-s
    >
      {[0, 1, 2, 3, 4, 5].map((score) => (
        <option key={score} value={score}>
          {score}/5
        </option>
      ))}
    </select>
  );
}

export function WorkflowRouter12() {
  const [selectedWorkflowId, setSelectedWorkflowId] =
    useState<WorkflowId>("interactive-tool");
```

```
  const [sort, setSort] = useState<{
    key: "name" | "codeQuality" | "rendering" | "stateManagement" | "sandboxFit";
    direction: "asc" | "desc";
  }>({
    key: "name",
    direction: "asc",
  });

  const [grades, setGrades] =
    useState<Record<PlatformId, Grade>>(DEFAULT_GRADES);

  const selectedWorkflow = useMemo(
    () => WORKFLOWS.find((workflow) => workflow.id === selectedWorkflowId)!,
    [selectedWorkflowId]
  );

  const recommendation = useMemo(
    () => getRecommendation(selectedWorkflow),
    [selectedWorkflow]
  );

  const sortedPlatforms = useMemo(() => {
    return [...PLATFORMS].sort((a, b) => {
      const getValue = (platform: Platform) => {
        if (sort.key === "name") {
          return platform.name;
        }

        const field =
          sort.key === "codeQuality"
            ? "codeQuality"
            : sort.key === "rendering"
            ? "rendering"
            : sort.key === "stateManagement"
            ? "stateManagement"
            : "sandboxFit";

        return platform.baseScores[field];
      };

      const aValue = getValue(a);
      const bValue = getValue(b);

      if (typeof aValue === "number" && typeof bValue === "number") {
        return sort.direction === "asc" ? aValue - bValue : bValue - aValue;
      }

      return sort.direction === "asc"
        ? String(aValue).localeCompare(String(bValue))
        : String(bValue).localeCompare(String(aValue));
    });
  }, [sort]);

  const updateGrade = (
    platformId: PlatformId,
```

```
    field: keyof Grade,
    value: number
  ) => {
    setGrades((current) => ({
      ...current,
      [platformId]: {
        ...current[platformId],
        [field]: value,
      },
    }));
  };

  const toggleSort = (key: typeof sort.key) => {
    setSort((current) => ({
      key,
      direction:
        current.key === key && current.direction === "asc" ? "desc" : "asc",
    }));
  };

  const exportBenchmarkMarkdown = () => {
    const markdown = `# Semester AI Platform Benchmark

## Selected workflow

- Workflow: ${selectedWorkflow.title}
- Category: ${selectedWorkflow.category}
- Description: ${selectedWorkflow.description}

## Router result

- Primary recommendation: ${recommendation.primary.platform.name}
- Secondary recommendation: ${recommendation.secondary.platform.name}

## Grading matrix

| Platform | Correctness | Code quality | Rendering | State management | Maintainabi
|---|---:|---:|---:|---:|---:|---:|---:|---:|
${PLATFORMS.map((platform) => {
  const grade = grades[platform.id];

  return `| ${platform.name} | ${grade.correctness} | ${grade.codeQuality} | ${grade
}).join("\n")}

## Evaluation rubric

- Correctness: requirement satisfaction
- Code quality: structure, type safety, errors, readability
- Rendering: visual quality, responsiveness, interaction clarity
- State management: predictable UI/data state and appropriate persistence boundary
- Maintainability: modularity, testability, and ease of iteration
- Handoff: exports, repository compatibility, and deployment path
- Sandbox fit: whether the workflow suits the platform
`;

    const blob = new Blob([markdown], {
```

```
      type: "text/markdown;charset=utf-8",
    });

    const url = URL.createObjectURL(blob);
    const anchor = document.createElement("a");

    anchor.href = url;
    anchor.download = "semester-platform-benchmark.md";
    anchor.click();

    URL.revokeObjectURL(url);
  };

  const header = (
    label: string,
    key: typeof sort.key
  ) => (
    <button
      onClick={() => toggleSort(key)}
      className="flex items-center gap-1 text-left text-xs font-bold uppercase track
    >
      {label}
      {sort.key === key ? (sort.direction === "asc" ? "↑" : "↓") : "↕"}
    </button>
  );

  return (
    <main className="min-h-screen bg-slate-50 p-4 text-slate-900 md:p-8">
      <div className="mx-auto max-w-7xl space-y-6">
        <header className="rounded-2xl bg-gradient-to-br from-slate-950 via-slate-900
          <p className="text-xs font-bold uppercase tracking-[0.16em] text-indigo-300
            Semester benchmark lab
          </p>

          <h1 className="mt-2 text-3xl font-bold tracking-tight">
            Claude Artifacts vs Canvas vs v0
          </h1>

          <p className="mt-2 max-w-3xl text-sm leading-6 text-slate-300">
            A 12-workflow router and benchmark harness for scoring code quality,
            rendering, state management, export handoff, and sandbox fit.
          </p>
        </header>

        <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
          <div>
            <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
              Workflow router
            </p>

            <h2 className="mt-1 text-xl font-bold text-slate-950">
              Choose one of 12 coding and content workflows
            </h2>
          </div>

          <div className="mt-5 grid gap-3 md:grid-cols-2 xl:grid-cols-3">
```

```
            {WORKFLOWS.map((workflow) => {
              const isSelected = workflow.id === selectedWorkflow.id;

              return (
                <button
                  key={workflow.id}
                  onClick={() => setSelectedWorkflowId(workflow.id)}
                  className={`rounded-xl border p-4 text-left transition ${
                    isSelected
                      ? "border-indigo-500 bg-indigo-50 shadow-sm"
                      : "border-slate-200 bg-white hover:border-indigo-300 hover:bg-s
                  }`}
                >
                  <div className="flex items-start justify-between gap-3">
                    <Badge
                      className={
                        isSelected
                          ? "border-indigo-200 bg-white text-indigo-700"
                          : "border-slate-200 bg-slate-50 text-slate-600"
                      }
                    >
                      {workflow.number}
                    </Badge>

                    <span className="text-xs font-bold text-slate-400">
                      {workflow.category}
                    </span>
                  </div>

                  <h3 className="mt-3 text-sm font-bold text-slate-950">
                    {workflow.title}
                  </h3>

                  <p className="mt-2 text-xs leading-5 text-slate-600">
                    {workflow.description}
                  </p>
                </button>
              );
            })}
          </div>

          <div className="mt-5 rounded-2xl border border-indigo-200 bg-indigo-50 p-5"
            <div className="flex flex-col justify-between gap-4 lg:flex-row lg:items-
              <div>
                <p className="text-xs font-bold uppercase tracking-[0.14em] text-indi
                  Router recommendation
                </p>

                <div className="mt-2 flex flex-wrap items-center gap-2">
                  <h3 className="text-2xl font-bold text-slate-950">
                    {recommendation.primary.platform.name}
                  </h3>

                  <Badge
                    className={getPlatformStyle(
                      recommendation.primary.platform.name
```

```
                )}
              >
                Primary
              </Badge>

              <Badge className="border-indigo-200 bg-white text-indigo-700">
                Workflow score {recommendation.primary.score}
              </Badge>
            </div>

            <ul className="mt-4 space-y-2 text-sm leading-6 text-slate-700">
              {recommendation.primary.reasons.length ? (
                recommendation.primary.reasons.map((reason) => (
                  <li key={reason}>• {reason}</li>
                ))
              ) : (
                <li>• Best overall baseline for the selected workflow.</li>
              )}
            </ul>
          </div>

          <aside className="rounded-xl border border-indigo-100 bg-white p-4 lg:w
            <p className="text-xs font-bold uppercase tracking-wide text-slate-50
              Secondary option
            </p>

            <p className="mt-1 text-base font-bold text-slate-900">
              {recommendation.secondary.platform.name}
            </p>

            <p className="mt-2 text-sm leading-6 text-slate-600">
              {recommendation.secondary.platform.primaryUse}
            </p>
          </aside>
        </div>
      </div>
    </section>

    <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
      <div className="flex flex-col justify-between gap-3 md:flex-row md:items-en
        <div>
          <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo
            Sortable platform benchmark matrix
          </p>

          <h2 className="mt-1 text-xl font-bold text-slate-950">
            Execution limits, React support, and baseline platform fit
          </h2>
        </div>

        <button
          onClick={exportBenchmarkMarkdown}
          className="rounded-lg bg-slate-900 px-3 py-2 text-xs font-bold text-whi
        >
          Export benchmark Markdown
        </button>
```

```
          </div>

          <div className="mt-5 overflow-x-auto">
            <table className="w-full min-w-[1220px] border-collapse">
              <thead className="bg-slate-50">
                <tr>
                  <th className="px-4 py-3">{header("Platform", "name")}</th>
                  <th className="px-4 py-3">
                    {header("Code quality", "codeQuality")}
                  </th>
                  <th className="px-4 py-3">
                    {header("Rendering", "rendering")}
                  </th>
                  <th className="px-4 py-3">
                    {header("State management", "stateManagement")}
                  </th>
                  <th className="px-4 py-3">
                    {header("Sandbox fit", "sandboxFit")}
                  </th>
                  <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                    Execution
                  </th>
                  <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                    React support
                  </th>
                  <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                    Exports
                  </th>
                </tr>
              </thead>

              <tbody>
                {sortedPlatforms.map((platform) => (
                  <tr
                    key={platform.id}
                    className="border-t border-slate-100 align-top hover:bg-indigo-50
                  >
                    <td className="px-4 py-4">
                      <p className="font-bold text-slate-950">
                        {platform.name}
                      </p>

                      <p className="mt-2 text-xs leading-5 text-slate-500">
                        {platform.limits}
                      </p>
                    </td>

                    <td className="px-4 py-4">
                      <Badge className="border-slate-200 bg-slate-50 text-slate-700">
                        {platform.baseScores.codeQuality}/5
                      </Badge>
                    </td>

                    <td className="px-4 py-4">
                      <Badge className="border-slate-200 bg-slate-50 text-slate-700">
                        {platform.baseScores.rendering}/5
```

```
                  </Badge>
                </td>

                <td className="px-4 py-4">
                  <Badge className="border-slate-200 bg-slate-50 text-slate-700">
                    {platform.baseScores.stateManagement}/5
                  </Badge>
                </td>

                <td className="px-4 py-4">
                  <Badge className="border-slate-200 bg-slate-50 text-slate-700">
                    {platform.baseScores.sandboxFit}/5
                  </Badge>
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.execution}
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.reactSupport}
                </td>

                <td className="max-w-xs px-4 py-4 text-sm leading-6 text-slate-60
                  {platform.exportFormats}
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </section>

    <section className="rounded-2xl border border-slate-200 bg-white p-5 shadow-s
      <div>
        <p className="text-xs font-bold uppercase tracking-[0.14em] text-indigo-6
          Manual grading harness
        </p>

        <h2 className="mt-1 text-xl font-bold text-slate-950">
          Score the same workflow across all three platforms
        </h2>

        <p className="mt-1 text-sm text-slate-600">
          Enter evaluator scores from 0 to 5. The component calculates a
          weighted result out of 100 using the same dimensions stored in the
          Supabase grading schema.
        </p>
      </div>

      <div className="mt-5 overflow-x-auto">
        <table className="w-full min-w-[1200px] border-collapse">
          <thead className="bg-slate-50">
            <tr>
              <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                Platform
```

```
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Correctness
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Code quality
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Rendering
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  State
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Maintainability
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Handoff
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Sandbox fit
                </th>
                <th className="px-4 py-3 text-left text-xs font-bold uppercase trac
                  Weighted
                </th>
              </tr>
            </thead>

            <tbody>
              {PLATFORMS.map((platform) => {
                const grade = grades[platform.id];

                return (
                  <tr
                    key={platform.id}
                    className="border-t border-slate-100"
                  >
                    <td className="px-4 py-3">
                      <Badge className={getPlatformStyle(platform.name)}>
                        {platform.name}
                      </Badge>
                    </td>

                    <td className="px-4 py-3">
                      <ScoreCell
                        value={grade.correctness}
                        onChange={(value) =>
                          updateGrade(platform.id, "correctness", value)
                        }
                      />
                    </td>

                    <td className="px-4 py-3">
                      <ScoreCell
                        value={grade.codeQuality}
                        onChange={(value) =>
```

```
                updateGrade(platform.id, "codeQuality", value)
              }
            />
          </td>

          <td className="px-4 py-3">
            <ScoreCell
              value={grade.rendering}
              onChange={(value) =>
                updateGrade(platform.id, "rendering", value)
              }
            />
          </td>

          <td className="px-4 py-3">
            <ScoreCell
              value={grade.stateManagement}
              onChange={(value) =>
                updateGrade(platform.id, "stateManagement", value)
              }
            />
          </td>

          <td className="px-4 py-3">
            <ScoreCell
              value={grade.maintainability}
              onChange={(value) =>
                updateGrade(platform.id, "maintainability", value)
              }
            />
          </td>

          <td className="px-4 py-3">
            <ScoreCell
              value={grade.handoff}
              onChange={(value) =>
                updateGrade(platform.id, "handoff", value)
              }
            />
          </td>

          <td className="px-4 py-3">
            <ScoreCell
              value={grade.sandboxFit}
              onChange={(value) =>
                updateGrade(platform.id, "sandboxFit", value)
              }
            />
          </td>

          <td className="px-4 py-3">
            <Badge className="border-indigo-200 bg-indigo-50 text-indigo-
              {weightedGradeScore(grade)} / 100
            </Badge>
          </td>
        </tr>
```

```
                  );
                })}
              </tbody>
            </table>
          </div>
        </section>
      </div>
    </main>
  );
}
```

Persist a grade

Use a Server Action for grades rather than inserting directly from the client. This ensures the authenticated evaluator is derived on the server.

```
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";

type GradeInput = {
  runId: string;
  correctnessScore: number;
  codeQualityScore: number;
  renderingScore: number;
  stateManagementScore: number;
  maintainabilityScore: number;
  handoffScore: number;
  sandboxFitScore: number;
  graderNotes?: string;
  evidence?: Record<string, unknown>;
};

export async function saveBenchmarkGrade(input: GradeInput) {
  const supabase = await createClient();

  const {
    data: { user },
    error: authError,
  } = await supabase.auth.getUser();

  if (authError || !user) {
    throw new Error("You must be signed in to grade a benchmark run.");
  }

  const scoreValues = [
    input.correctnessScore,
    input.codeQualityScore,
    input.renderingScore,
    input.stateManagementScore,
    input.maintainabilityScore,
    input.handoffScore,
    input.sandboxFitScore,
```

```
  ];

  if (
    scoreValues.some(
      (score) => !Number.isInteger(score) || score < 0 || score > 5
    )
  ) {
    throw new Error("Every benchmark score must be an integer from 0 to 5.");
  }

  const { error } = await supabase.from("benchmark_grades").upsert(
    {
      run_id: input.runId,
      graded_by: user.id,
      correctness_score: input.correctnessScore,
      code_quality_score: input.codeQualityScore,
      rendering_score: input.renderingScore,
      state_management_score: input.stateManagementScore,
      maintainability_score: input.maintainabilityScore,
      handoff_score: input.handoffScore,
      sandbox_fit_score: input.sandboxFitScore,
      grader_notes: input.graderNotes ?? null,
      evidence: input.evidence ?? {},
    },
    {
      onConflict: "run_id,graded_by",
    }
  );

  if (error) {
    throw new Error(error.message);
  }

  revalidatePath("/benchmark");
  return { success: true };
}
```

Recommended evaluation workflow

1. Create one benchmark suite for the 12 standardized tasks.
2. Run the **exact same prompt** in Claude Artifacts, Canvas, and v0.
3. Record the actual output location, model/version, time, and sandbox notes.
4. Grade each run independently using the 0–5 rubric.
5. Use `benchmark_task_leaderboard` for per-workflow results and `benchmark_platform_leaderboard` for aggregated platform results.
6. Keep the raw artifact, export, project, or preview URL attached to every run so ratings can be audited.

This approach keeps subjective evaluation explicit while preserving the evidence necessary to revise the scoring later.

⁂

Claude Artifacts vs ChatGPT Canvas vs v0 Sandbox: a 12-workflow benchmark matrix scoring code quality, rendering, and state management

Computer
Build a test harness app that runs automated prompts against Claude, ChatGPT Canvas, and v0 Sandbox to score code output and state persistence
Computer
Show the rest of the benchmark matrix code
Add the 12 workflows to the sortable matrix
Show the test suite harness configuration
Sources
195

A robust benchmark should separate **routing guidance** from **actual execution results**: the matrix can recommend a platform for each of 12 workflows, while the harness stores reproducible runs, raw outputs, evaluator scores, and state-persistence checks. Fully automating all three products may require each platform's supported API, export, or connected workflow; do not assume their interactive sandboxes expose identical programmatic execution interfaces.

For Semester, the most reliable design is: normalize each task into a test specification, run it through approved platform-specific adapters, store all evidence in Supabase, then grade code quality, rendering, state management, and persistence from the resulting output. v0's sandbox is built around executing untrusted or agent-generated code in an isolated environment, making it a natural target for multi-file project tests; Claude Artifacts and Canvas are better represented by artifact/document/code outputs plus explicit export and preview evidence. [196] [197]

Benchmark matrix design

Use these 12 workflows as the canonical benchmark set:

#	Workflow	Primary capability under test	Likely best fit
1	Sortable artifact tracker	Client-side filtering, local state, CSV export	Claude Artifacts
2	Interactive onboarding prototype	UI rendering, local multi-step state	Claude Artifacts
3	B2B SaaS landing page	Responsive rendering and visual polish	Claude Artifacts
4	System architecture diagram	Mermaid/SVG rendering and export	Claude Artifacts
5	Technical design document	Structured writing, editing, export	ChatGPT Canvas
6	Product-launch presentation	Linear narrative, slide/document handoff	ChatGPT Canvas
7	Subscription CSV analysis	Python/data output and report generation	ChatGPT Canvas
8	Reusable React dashboard component	Component quality, TypeScript, local state	v0 or Claude Artifacts

#	Workflow	Primary capability under test	Likely best fit
9	Authenticated Next.js route handler	Server code, validation, auth boundaries	v0
10	Supabase artifact CRUD	Database schema, RLS, server actions, UI state	v0
11	Cookie-based Supabase Auth	SSR sessions, middleware, protected routes	v0
12	Multi-tenant SaaS workspace	Organizations, roles, RLS, audit trail, deployment	v0

The key benchmark rule: **all three platforms receive the same canonical task prompt and acceptance criteria**. The platform adapter may wrap that prompt with necessary platform-specific execution instructions, but it must preserve the required outputs and test criteria.

Harness configuration

Save this as:

```
benchmark/benchmark.config.ts
```

```
export type PlatformId =
  | "claude-artifacts"
  | "chatgpt-canvas"
  | "v0";

export type WorkflowId =
  | "interactive-tool"
  | "product-prototype"
  | "marketing-page"
  | "diagram"
  | "technical-document"
  | "presentation"
  | "data-analysis"
  | "react-component"
  | "nextjs-route-handler"
  | "supabase-crud"
  | "supabase-auth"
  | "multi-tenant-saas";

export type RubricWeights = {
  correctness: number;
  codeQuality: number;
  rendering: number;
  stateManagement: number;
  maintainability: number;
  handoff: number;
  sandboxFit: number;
  persistence: number;
};

export type BenchmarkTask = {
  id: WorkflowId;
  number: number;
  title: string;
```

```
  category: string;
  prompt: string;
  requirements: {
    react?: boolean;
    typescript?: boolean;
    html?: boolean;
    mermaid?: boolean;
    python?: boolean;
    nextjs?: boolean;
    supabase?: boolean;
    database?: boolean;
    auth?: boolean;
    rls?: boolean;
    storage?: boolean;
    serverCode?: boolean;
    localState?: boolean;
    persistentState?: boolean;
    export?: boolean;
    multiFile?: boolean;
  };
  acceptanceCriteria: string[];
  rubricWeights: RubricWeights;
  recommendedPlatform: PlatformId;
};

export type PlatformCapabilities = {
  id: PlatformId;
  name: string;
  supportsClientRendering: boolean;
  supportsLocalState: boolean;
  supportsMultiFileProjects: boolean;
  supportsServerCode: boolean;
  supportsDatabaseWorkflow: boolean;
  supportsAuthWorkflow: boolean;
  supportsNativeDocumentExport: boolean;
  supportsProjectExport: boolean;
  supportsDeployWorkflow: boolean;
  statePersistence: "none" | "local-preview" | "project-backed";
  executionModel: string;
};

export const RUBRIC_DEFAULTS: RubricWeights = {
  correctness: 0.22,
  codeQuality: 0.18,
  rendering: 0.15,
  stateManagement: 0.15,
  maintainability: 0.1,
  handoff: 0.08,
  sandboxFit: 0.07,
  persistence: 0.05,
};

export const PLATFORM_CAPABILITIES: PlatformCapabilities[] = [
  {
    id: "claude-artifacts",
    name: "Claude Artifacts",
```

```
    supportsClientRendering: true,
    supportsLocalState: true,
    supportsMultiFileProjects: false,
    supportsServerCode: false,
    supportsDatabaseWorkflow: false,
    supportsAuthWorkflow: false,
    supportsNativeDocumentExport: true,
    supportsProjectExport: true,
    supportsDeployWorkflow: false,
    statePersistence: "local-preview",
    executionModel:
      "Self-contained artifact/client-side preview and export workflow.",
  },
  {
    id: "chatgpt-canvas",
    name: "ChatGPT Canvas",
    supportsClientRendering: true,
    supportsLocalState: true,
    supportsMultiFileProjects: false,
    supportsServerCode: false,
    supportsDatabaseWorkflow: false,
    supportsAuthWorkflow: false,
    supportsNativeDocumentExport: true,
    supportsProjectExport: true,
    supportsDeployWorkflow: false,
    statePersistence: "local-preview",
    executionModel:
      "Document/code editing workspace with isolated rendering and export workflow."
  },
  {
    id: "v0",
    name: "v0 Sandbox",
    supportsClientRendering: true,
    supportsLocalState: true,
    supportsMultiFileProjects: true,
    supportsServerCode: true,
    supportsDatabaseWorkflow: true,
    supportsAuthWorkflow: true,
    supportsNativeDocumentExport: false,
    supportsProjectExport: true,
    supportsDeployWorkflow: true,
    statePersistence: "project-backed",
    executionModel:
      "Multi-file Node.js/Next.js project sandbox with server, database, and deployme
  },
];

export const BENCHMARK_TASKS: BenchmarkTask[] = [
  {
    id: "interactive-tool",
    number: 1,
    title: "Sortable artifact tracker",
    category: "React UI",
    recommendedPlatform: "claude-artifacts",
    prompt: `Build a responsive client-side artifact tracker with 100 sample records
- Full-text search
```

```
- Multi-select category and format filters
- Sortable columns
- Status and priority badges
- Editable status
- A detail drawer
- Local state only
- CSV export
- Keyboard-accessible controls
- Empty, loading, and error states
Do not use a backend or external UI library.`,
    requirements: {
      react: true,
      typescript: true,
      localState: true,
      export: true,
    },
    acceptanceCriteria: [
      "Search filters across artifact title, category, owner, and tags",
      "At least two multi-select filters work together",
      "Table sort changes row ordering",
      "CSV export contains filtered rows",
      "UI has accessible labels and keyboard-focusable controls",
      "No server-side dependency is required",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      rendering: 0.2,
      stateManagement: 0.22,
      persistence: 0.01,
    },
  },
  {
    id: "product-prototype",
    number: 2,
    title: "Interactive onboarding prototype",
    category: "Product Design",
    recommendedPlatform: "claude-artifacts",
    prompt: `Build a clickable onboarding prototype for a B2B SaaS product. Include:
- Welcome screen
- Four-step setup flow
- Progress indicator
- Validation errors
- Empty state
- Success state
- Back/next navigation
- Responsive layout
- Realistic sample data
Use local component state and do not require a backend.`,
    requirements: {
      react: true,
      localState: true,
    },
    acceptanceCriteria: [
      "All setup steps are reachable",
      "Back/next buttons preserve entered state",
      "Validation blocks invalid continuation",
```

```
      "Completion state is visible after final step",
      "Layout is responsive",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      rendering: 0.28,
      stateManagement: 0.23,
      persistence: 0.01,
    },
  },
  {
    id: "marketing-page",
    number: 3,
    title: "B2B SaaS landing page",
    category: "Marketing",
    recommendedPlatform: "claude-artifacts",
    prompt: `Build a polished responsive B2B SaaS landing page. Include:
- Navigation
- Hero
- Product workflow section
- Benefits
- Product screenshots/placeholders
- Customer proof
- FAQ
- Primary conversion CTA
- Footer
Use semantic accessible HTML and responsive CSS.`,
    requirements: {
      html: true,
    },
    acceptanceCriteria: [
      "Semantic page landmarks exist",
      "Navigation and CTA links are present",
      "Mobile layout remains usable",
      "Visual hierarchy is clear",
      "No console errors are emitted",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      rendering: 0.35,
      stateManagement: 0.03,
      persistence: 0.01,
    },
  },
  {
    id: "diagram",
    number: 4,
    title: "Multi-tenant system architecture",
    category: "Engineering",
    recommendedPlatform: "claude-artifacts",
    prompt: `Create a Mermaid architecture diagram for a multi-tenant Next.js applica
- Supabase Auth
- Postgres
- Supabase Storage
- Row Level Security
- Stripe billing
```

```
- Webhooks
- Vercel deployment
- Observability
- Audit logs
Show trust boundaries, main data flows, and external dependencies.`,
    requirements: {
      mermaid: true,
      export: true,
    },
    acceptanceCriteria: [
      "Valid Mermaid source is provided",
      "All required systems appear",
      "Data flow and trust boundaries are labelled",
      "Diagram remains readable at normal zoom",
      "Source can be copied into a repository",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      correctness: 0.3,
      rendering: 0.3,
      handoff: 0.15,
      stateManagement: 0.02,
      persistence: 0.01,
    },
  },
  {
    id: "technical-document",
    number: 5,
    title: "Technical design document",
    category: "Engineering Documentation",
    recommendedPlatform: "chatgpt-canvas",
    prompt: `Write a technical design document for a multi-tenant artifact catalog. I
- Context
- Goals and non-goals
- User workflows
- Architecture
- Data model
- Authorization and Row Level Security
- API design
- Risks and failure modes
- Observability
- Rollout
- Test plan
- Open questions`,
    requirements: {
      export: true,
    },
    acceptanceCriteria: [
      "Every requested section is included",
      "Tradeoffs are explicit",
      "Security and authorization assumptions are called out",
      "Document is readable and review-ready",
      "Source can be exported cleanly",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
```

```
      correctness: 0.34,
      handoff: 0.25,
      maintainability: 0.16,
      rendering: 0.07,
      stateManagement: 0.02,
      persistence: 0.01,
    },
  },
  {
    id: "presentation",
    number: 6,
    title: "Product launch narrative",
    category: "GTM",
    recommendedPlatform: "chatgpt-canvas",
    prompt: `Create a slide-by-slide product launch narrative for a B2B SaaS feature
- Target audience
- Problem
- Positioning
- Product walkthrough
- Evidence/proof
- Launch channels
- Sales enablement
- Timeline
- Metrics
- Risks
- Speaker notes`,
    requirements: {
      export: true,
    },
    acceptanceCriteria: [
      "Narrative has a clear beginning, middle, and end",
      "Every slide has a single key message",
      "Speaker notes support delivery",
      "Metrics and risks are explicit",
      "Presentation can be exported or handed off",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      rendering: 0.24,
      handoff: 0.24,
      correctness: 0.25,
      stateManagement: 0.02,
      persistence: 0.01,
    },
  },
  {
    id: "data-analysis",
    number: 7,
    title: "Subscription CSV analysis",
    category: "Analytics",
    recommendedPlatform: "chatgpt-canvas",
    prompt: `Write a Python script that:
- Reads subscription events from CSV
- Computes monthly active customers
- Computes gross logo churn
- Computes gross revenue churn
```

```
- Computes net revenue retention
- Writes a cleaned summary CSV
- Produces a short Markdown report
Handle missing values and malformed rows safely.`,
    requirements: {
      python: true,
      export: true,
    },
    acceptanceCriteria: [
      "Script handles a valid CSV input",
      "Malformed rows are handled or reported safely",
      "All requested metrics are computed",
      "CSV and Markdown outputs are produced",
      "Code is readable and modular",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      correctness: 0.32,
      codeQuality: 0.25,
      handoff: 0.12,
      rendering: 0.03,
      stateManagement: 0.08,
      persistence: 0.01,
    },
  },
  {
    id: "react-component",
    number: 8,
    title: "Reusable React metric dashboard",
    category: "React UI",
    recommendedPlatform: "v0",
    prompt: `Build a reusable TypeScript React metric dashboard component with:
- KPI cards
- Local filters
- Data table
- Detail modal
- Realistic mock data
- Loading state
- Empty state
- Error state
- Accessible controls
Do not use an external UI library.`,
    requirements: {
      react: true,
      typescript: true,
      localState: true,
    },
    acceptanceCriteria: [
      "Component has typed props or data model",
      "Filters change visible data",
      "Modal opens and closes accessibly",
      "All state variants render",
      "Code can be moved into an existing React app",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
```

```
      codeQuality: 0.25,
      rendering: 0.2,
      stateManagement: 0.22,
      persistence: 0.01,
    },
  },
  {
    id: "nextjs-route-handler",
    number: 9,
    title: "Authenticated Next.js route handler",
    category: "Next.js",
    recommendedPlatform: "v0",
    prompt: `Build a Next.js App Router route handler for creating artifact records.
- Require authenticated user access
- Validate request data
- Enforce organization membership
- Return typed JSON errors
- Include request/response examples
- Avoid leaking database errors to clients`,
    requirements: {
      nextjs: true,
      serverCode: true,
      auth: true,
      multiFile: true,
    },
    acceptanceCriteria: [
      "Authentication is checked server-side",
      "Input validation exists",
      "Organization authorization is enforced",
      "Errors use appropriate status codes",
      "Examples are documented",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      correctness: 0.32,
      codeQuality: 0.25,
      stateManagement: 0.16,
      sandboxFit: 0.12,
      rendering: 0.02,
      persistence: 0.01,
    },
  },
  {
    id: "supabase-crud",
    number: 10,
    title: "Supabase artifact CRUD",
    category: "Full Stack",
    recommendedPlatform: "v0",
    prompt: `Build a Next.js App Router artifact catalog backed by Supabase Postgres
- Migration SQL
- Row Level Security
- Server Actions
- Typed queries
- Create/edit/delete workflows
- Search and filters
- Schema validation
```

```
- Loading/error states
- Organization-scoped records`,
    requirements: {
      nextjs: true,
      supabase: true,
      database: true,
      auth: true,
      rls: true,
      serverCode: true,
      persistentState: true,
      multiFile: true,
    },
    acceptanceCriteria: [
      "Schema contains tenant/organization scoping",
      "RLS policies exist",
      "Mutations are validated",
      "CRUD UI handles async states",
      "Server-side authorization is not delegated to UI",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      correctness: 0.32,
      codeQuality: 0.2,
      stateManagement: 0.2,
      sandboxFit: 0.1,
      rendering: 0.08,
      persistence: 0.05,
    },
  },
  {
    id: "supabase-auth",
    number: 11,
    title: "Cookie-based Supabase Auth",
    category: "Authentication",
    recommendedPlatform: "v0",
    prompt: `Build Supabase Auth for Next.js App Router using:
- Separate browser and server clients
- Session-refresh middleware/proxy
- Magic-link sign-in
- Protected routes
- Sign-out
- Secure environment variable guidance
- No service-role key in client code`,
    requirements: {
      nextjs: true,
      supabase: true,
      database: true,
      auth: true,
      serverCode: true,
      persistentState: true,
      multiFile: true,
    },
    acceptanceCriteria: [
      "Browser and server clients are separated",
      "Sessions refresh before protected server rendering",
      "Server validates authenticated identity",
```

```
      "Secret-handling guidance is correct",
      "Sign-in and sign-out flows are complete",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      correctness: 0.35,
      codeQuality: 0.2,
      stateManagement: 0.2,
      sandboxFit: 0.08,
      rendering: 0.04,
      persistence: 0.05,
    },
  },
  {
    id: "multi-tenant-saas",
    number: 12,
    title: "Organization-scoped SaaS workspace",
    category: "SaaS Architecture",
    recommendedPlatform: "v0",
    prompt: `Build a multi-tenant Next.js workspace with:
- Organizations
- Memberships
- Owner/admin/member/viewer roles
- Supabase Auth
- Postgres tables
- Row Level Security
- Server-side authorization
- Audit logs
- Protected APIs
- Responsive admin dashboard`,
    requirements: {
      nextjs: true,
      supabase: true,
      database: true,
      auth: true,
      rls: true,
      serverCode: true,
      persistentState: true,
      multiFile: true,
    },
    acceptanceCriteria: [
      "Organizations and memberships are modeled",
      "RLS enforces tenant isolation",
      "Role rules are explicit",
      "Server-side authorization exists",
      "Audit events are considered",
      "Dashboard reflects access constraints",
    ],
    rubricWeights: {
      ...RUBRIC_DEFAULTS,
      correctness: 0.35,
      codeQuality: 0.2,
      stateManagement: 0.2,
      sandboxFit: 0.08,
      rendering: 0.06,
      persistence: 0.05,
```

```
    },
  },
];
```

Adapter interface

The harness must not fake execution. Each adapter should either call an approved API/workflow or return an explicit `unsupported`/`manual` result.

Save as:

```
benchmark/adapters/types.ts
```

```ts
import type { BenchmarkTask, PlatformId } from "../benchmark.config";

export type RunStatus =
  | "queued"
  | "running"
  | "completed"
  | "manual-review-required"
  | "unsupported"
  | "failed";

export type BenchmarkRunResult = {
  platform: PlatformId;
  taskId: string;
  status: RunStatus;

  executedPrompt: string;

  outputText?: string;
  outputUrl?: string;

  sourceFiles?: Array<{
    path: string;
    content: string;
    language?: string;
  }>;

  consoleOutput?: string;
  executionSeconds?: number;

  persistenceEvidence?: {
    tested: boolean;
    persistedAcrossRefresh?: boolean;
    storageMode?: "none" | "local" | "database" | "session" | "project";
    notes?: string;
  };

  limitations?: string[];
  error?: string;
};
```

```
export interface BenchmarkAdapter {
  readonly platform: PlatformId;

  canRun(task: BenchmarkTask): {
    supported: boolean;
    reason?: string;
    manualReviewRequired?: boolean;
  };

  run(task: BenchmarkTask): Promise<BenchmarkRunResult>;
}
```

Capability guard

```
import type { BenchmarkTask, PlatformId } from "../benchmark.config";
import type { BenchmarkAdapter } from "./types";

export function requireCapability(
  platform: PlatformId,
  task: BenchmarkTask
) {
  const requiresBackend =
    task.requirements.serverCode ||
    task.requirements.database ||
    task.requirements.auth ||
    task.requirements.rls ||
    task.requirements.persistentState;

  if (
    platform !== "v0" &&
    requiresBackend
  ) {
    return {
      supported: false,
      manualReviewRequired: false,
      reason:
        "This task requires production-style server, database, authentication, or pe
    };
  }

  return {
    supported: true,
  };
}
```

v0 adapter skeleton

This is deliberately a harness configuration skeleton, not a claim that an undocumented endpoint should be called. Connect it to your approved v0 project/API/CLI workflow.

```
import type { BenchmarkAdapter, BenchmarkRunResult } from "./types";
import type { BenchmarkTask } from "../benchmark.config";
```

```
import { requireCapability } from "./capability-guard";

export class V0Adapter implements BenchmarkAdapter {
  readonly platform = "v0" as const;

  canRun(task: BenchmarkTask) {
    return requireCapability(this.platform, task);
  }

  async run(task: BenchmarkTask): Promise<BenchmarkRunResult> {
    const capability = this.canRun(task);

    if (!capability.supported) {
      return {
        platform: this.platform,
        taskId: task.id,
        status: "unsupported",
        executedPrompt: task.prompt,
        limitations: [capability.reason ?? "Unsupported task."],
      };
    }

    /*
     * Replace this block with your approved v0 integration:
     * - create/select a benchmark project or branch
     * - submit task.prompt
     * - wait for generated project or preview
     * - capture output URL, source file manifest, logs, and elapsed time
     * - run test/persistence probe if the generated app is executable
     */
    return {
      platform: this.platform,
      taskId: task.id,
      status: "manual-review-required",
      executedPrompt: task.prompt,
      limitations: [
        "Connect this adapter to an approved v0 API, project automation, or manual be
      ],
    };
  }
}
```

Claude Artifact adapter skeleton

```
import type { BenchmarkAdapter, BenchmarkRunResult } from "./types";
import type { BenchmarkTask } from "../benchmark.config";
import { requireCapability } from "./capability-guard";

export class ClaudeArtifactsAdapter implements BenchmarkAdapter {
  readonly platform = "claude-artifacts" as const;

  canRun(task: BenchmarkTask) {
    return requireCapability(this.platform, task);
  }
```

```
  async run(task: BenchmarkTask): Promise<BenchmarkRunResult> {
    const capability = this.canRun(task);

    if (!capability.supported) {
      return {
        platform: this.platform,
        taskId: task.id,
        status: "unsupported",
        executedPrompt: task.prompt,
        limitations: [capability.reason ?? "Unsupported task."],
      };
    }

    /*
     * Use an approved Claude workflow/API or mark as manual:
     * - submit the canonical prompt
     * - collect HTML/JSX/SVG/Mermaid/document source
     * - collect share/export URL if available
     * - record whether the artifact rendered
     * - run client-side static checks on exported source
     */
    return {
      platform: this.platform,
      taskId: task.id,
      status: "manual-review-required",
      executedPrompt: task.prompt,
      limitations: [
        "Connect to an approved Claude Artifact creation/export workflow. Do not trea
      ],
    };
  }
}
```

Canvas adapter skeleton

```
import type { BenchmarkAdapter, BenchmarkRunResult } from "./types";
import type { BenchmarkTask } from "../benchmark.config";
import { requireCapability } from "./capability-guard";

export class ChatGPTCanvasAdapter implements BenchmarkAdapter {
  readonly platform = "chatgpt-canvas" as const;

  canRun(task: BenchmarkTask) {
    return requireCapability(this.platform, task);
  }

  async run(task: BenchmarkTask): Promise<BenchmarkRunResult> {
    const capability = this.canRun(task);

    if (!capability.supported) {
      return {
        platform: this.platform,
        taskId: task.id,
        status: "unsupported",
        executedPrompt: task.prompt,
```

```
        limitations: [capability.reason ?? "Unsupported task."],
      };
    }

    /*
     * Use a supported OpenAI workflow/API or mark as manual:
     * - create the canvas/code/document prompt
     * - export code/document output
     * - capture rendered preview or exported artifact
     * - attach source file and review notes
     */
    return {
      platform: this.platform,
      taskId: task.id,
      status: "manual-review-required",
      executedPrompt: task.prompt,
      limitations: [
        "Connect to an approved Canvas generation/export workflow. Record exported so
      ],
    };
  }
}
```

Automated scoring probes

For outputs that become a project or exported source files, use deterministic checks **in addition to human review**. Never infer architecture quality merely from the generated text.

```
export type StaticProbeResult = {
  passed: boolean;
  score: number;
  evidence: string[];
};

export function scoreSourceStructure(
  files: Array<{ path: string; content: string }>
): StaticProbeResult {
  const evidence: string[] = [];

  const hasReadme = files.some((file) =>
    file.path.toLowerCase().includes("readme")
  );

  const hasTypeScript = files.some((file) =>
    /\.(ts|tsx)$/.test(file.path)
  );

  const hasErrorState = files.some((file) =>
    /error|ErrorBoundary|setError|throw new Error/i.test(file.content)
  );

  const hasLoadingState = files.some((file) =>
    /loading|isLoading|pending|Suspense/i.test(file.content)
  );
```

```
  const hasValidation = files.some((file) =>
    /zod|safeParse|parse\(|validate/i.test(file.content)
  );

  if (hasReadme) evidence.push("README detected");
  if (hasTypeScript) evidence.push("TypeScript source detected");
  if (hasErrorState) evidence.push("Error-state handling detected");
  if (hasLoadingState) evidence.push("Loading-state handling detected");
  if (hasValidation) evidence.push("Validation-related code detected");

  const score = [
    hasReadme,
    hasTypeScript,
    hasErrorState,
    hasLoadingState,
    hasValidation,
  ].filter(Boolean).length;

  return {
    passed: score >= 3,
    score,
    evidence,
  };
}

export function scorePersistenceProbe(input: {
  requiresPersistentState: boolean;
  output: {
    persistenceEvidence?: {
      tested: boolean;
      persistedAcrossRefresh?: boolean;
      storageMode?: string;
      notes?: string;
    };
  };
}): StaticProbeResult {
  if (!input.requiresPersistentState) {
    return {
      passed: true,
      score: 5,
      evidence: ["Persistent state is not required for this workflow."],
    };
  }

  const evidence = input.output.persistenceEvidence;

  if (!evidence?.tested) {
    return {
      passed: false,
      score: 0,
      evidence: ["Persistence was required but no persistence test was recorded."],
    };
  }

  if (!evidence.persistedAcrossRefresh) {
```

```
    return {
      passed: false,
      score: 1,
      evidence: [
        `Persistence test failed or was incomplete. Storage mode: ${evidence.storageM
      ],
    };
  }

  return {
    passed: true,
    score: evidence.storageMode === "database" ? 5 : 3,
    evidence: [
      `State persisted across refresh via ${evidence.storageMode ?? "recorded storage
      evidence.notes ?? "No additional notes.",
    ],
  };
}
```

Benchmark execution configuration

```
import {
  BENCHMARK_TASKS,
  PLATFORM_CAPABILITIES,
  type PlatformId,
} from "./benchmark.config";

export type BenchmarkExecutionConfig = {
  suiteName: string;
  runPlatforms: PlatformId[];
  workflows: string[];
  enforceSamePrompt: boolean;
  enforceFreshSession: boolean;
  captureSourceFiles: boolean;
  capturePreviewUrl: boolean;
  captureConsoleOutput: boolean;
  runStaticChecks: boolean;
  runPersistenceProbe: boolean;
  requireHumanReview: boolean;
  timeoutSeconds: number;
};

export const SEMESTER_BENCHMARK_CONFIG: BenchmarkExecutionConfig = {
  suiteName: "Semester AI Workspace Benchmark — 12 Workflows",
  runPlatforms: [
    "claude-artifacts",
    "chatgpt-canvas",
    "v0",
  ],
  workflows: BENCHMARK_TASKS.map((task) => task.id),
  enforceSamePrompt: true,
  enforceFreshSession: true,
  captureSourceFiles: true,
  capturePreviewUrl: true,
  captureConsoleOutput: true,
```

```
  runStaticChecks: true,
  runPersistenceProbe: true,
  requireHumanReview: true,
  timeoutSeconds: 600,
};

export function validateConfig(config: BenchmarkExecutionConfig) {
  const errors: string[] = [];

  if (!config.enforceSamePrompt) {
    errors.push(
      "Benchmark invalid: all platforms must receive the same canonical prompt."
    );
  }

  if (!config.requireHumanReview) {
    errors.push(
      "Benchmark incomplete: static checks do not replace human review."
    );
  }

  if (config.runPersistenceProbe && !config.enforceFreshSession) {
    errors.push(
      "Persistence testing requires a fresh-session or refresh boundary."
    );
  }

  for (const platformId of config.runPlatforms) {
    const platform = PLATFORM_CAPABILITIES.find(
      (item) => item.id === platformId
    );

    if (!platform) {
      errors.push(`Unknown platform: ${platformId}`);
    }
  }

  return errors;
}
```

What can be automated

Benchmark area	Automate?	Recommended evidence
Canonical prompt consistency	Yes	Persist exact `executed_prompt` per run
Output/source capture	Usually	HTML, JSX/TSX, Markdown, Mermaid, project-file manifest
Static code checks	Yes	TypeScript, lint, build, source-pattern probes
Rendering verification	Partly	Screenshot, preview URL, console logs, human visual review

Benchmark area	Automate?	Recommended evidence
State-management behavior	Partly	Scripted interaction test plus manual inspection
Persistence across reload/session	Yes for deployable projects; limited for sandbox previews	Refresh/session test, DB row check, storage evidence
Auth/RLS correctness	Partly	Automated tests plus security review
Code quality	Partly	Lint/type/build plus evaluator rubric
Design quality	No, not reliably	Human review with documented rubric
Platform suitability	No, not entirely	Rubric score plus architecture evidence

The database schema is designed around that distinction: **automated probes create evidence, while human evaluators grade what cannot be reliably inferred.**

Practical scoring interpretation

- **Claude Artifacts** should score strongly on workflows 1–4: client-side tools, prototypes, visual pages, diagrams, and shareable artifacts.
- **ChatGPT Canvas** should score strongly on workflows 5–7: technical documents, presentations, and Python/data-file outputs.
- **v0** should score strongly on workflows 8–12, especially when code requires a multi-file Next.js structure, server logic, Supabase/Postgres, Auth, RLS, persistent records, or multi-tenant permissions.

Do not treat that expected pattern as results. The purpose of the harness is to record actual results, attach the generated evidence, and reveal where a particular tool/model/version performs differently than the expected platform fit.

⁂

1. https://academy.claude.com/courses/claude-101/creating-with-artifacts
2. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them
3. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf
4. https://www.codecademy.com/article/how-to-use-claude-artifacts-create-share-and-remix-ai-content
5. https://beginnersinai.org/claude-artifacts-guide/
6. https://learning.northeastern.edu/ai-student-guides-visual-learning-with-claude/
7. https://www.reddit.com/r/ClaudeAI/comments/1w88gld/what_are_good_examples_for_artifacts/
8. https://www.getmasset.com/resources/claude-artifacts-for-marketers
9. https://go9x.com/blog/claude-artifacts-examples
10. https://www.totalum.app/blog/claude-artifacts-alternative-2026
11. https://code.claude.com/docs/en/artifacts?stixel_click=eyJzIjoiODNjNjhjNGQtYzk1YS00MmM0LTgxYjctNzU2NWU1Nzg3YzEzIn0=

12. https://support.claude.com/en/articles/9487310-what-are-artifacts-and-how-do-i-use-them%23h_8fd83d1dd6
13. https://p0stman.com/guides/claude-artifacts-limitations
14. https://fast.io/resources/claude-artifacts-guide/
15. https://academy.claude.com/courses/claude-101/creating-with-artifacts
16. https://albato.com/blog/publications/how-to-use-claude-artifacts-guide
17. https://simonwillison.net/2024/Oct/21/claude-artifacts/
18. https://www.reddit.com/r/ClaudeAI/comments/1tb8al4/claude_artifacts_basically_killed_google_slides/
19. https://deckary.com/blog/claude-artifacts-presentations
20. https://www.linkedin.com/posts/christopher-duffy-caio_how-to-create-and-customise-an-artifact-in-activity-7382488637461929984-oANg
21. https://code.claude.com/docs/en/artifacts
22. https://www.netlify.com/knowledge-base/deploy-your-claude-artifacts-to-netlify/
23. https://dev.to/hira_jabeen_ccaa191c13070/ultimate-claude-artifacts-guide-45k3
24. https://www.codecademy.com/article/how-to-use-claude-artifacts-create-share-and-remix-ai-content
25. https://aiwiki.ai/wiki/claude_artifacts
26. https://github.com/mostafaebrahimi/artifact-exporter-for-claude
27. https://albato.com/blog/publications/how-to-use-claude-artifacts-guide
28. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them
29. https://p0stman.com/guides/claude-artifacts-limitations
30. https://www.getmasset.com/resources/claude-artifacts-for-marketers
31. https://simonwillison.net/tags/claude-artifacts/
32. https://www.shareduo.com/blog/claude-artifacts
33. https://openai.com/index/introducing-canvas/
34. https://v0.app/docs/sandbox
35. https://v0.app/docs/faqs
36. https://code.claude.com/docs/en/artifacts
37. https://x.com/OpenAI/status/1933305323672252475
38. https://www.rapidevelopers.com/v0-issues/resolving-build-errors-in-exported-v0-projects
39. https://github.com/claudio-silva/claude-artifact-runner
40. https://www.rapidevelopers.com/v0-issues/exporting-and-sharing-v0-projects-across-environments
41. https://www.reddit.com/r/ClaudeAI/comments/1hv82fg/question_on_exporting_using_react_artifacts/
42. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them
43. https://www.netlify.com/knowledge-base/deploy-your-claude-artifacts-to-netlify/
44. https://www.getmasset.com/resources/claude-artifacts-for-marketers
45. https://p0stman.com/guides/claude-artifacts-limitations
46. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf

47. https://www.appstuck.com/blog/v0-not-working-fix-the-most-common-errors-2026
48. https://docs.anthropic.com/en/docs/claude-code/corporate-proxy
49. https://v0.dev/docs
50. https://help.openai.com/es-419/articles/9930697-qué-es-la-función-de-canvas-en-chatgpt-y-cómo-la-uso
51. https://docs.anthropic.com/en/docs/claude-code/desktop
52. https://help.openai.com/es-es/articles/6825453-chatgpt-release-notes
53. https://v0.app/docs/sandbox
54. https://api2.v0.dev/docs/agentic-features
55. https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/code-execution-tool
56. https://docs.anthropic.com/en/docs/claude-code/sdk/sdk-typescript
57. https://v0.dev/docs/v0-model-api
58. https://docs.anthropic.com/en/release-notes/overview
59. https://docs.anthropic.com/en/docs/claude-code/sdk
60. https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/web-fetch-tool
61. https://docs.anthropic.com/en/docs/claude-code/memory
62. https://docs.anthropic.com/en/release-notes/claude-code
63. https://docs.anthropic.com/en/docs/claude-code/hooks
64. https://docs.anthropic.com/en/docs/claude-code/corporate-proxy
65. https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/code-execution-tool
66. https://help.openai.com/en/articles/9930697-using-canvas-in-chatgpt
67. https://v0.app/docs/sandbox
68. https://api2.v0.dev/docs/agentic-features
69. https://help.openai.com/en/articles/6825453-chatgpt-release-not
70. https://help.openai.com/en/articles/10128477-chatgpt-enterprise-edu-release-notesOpenAI
71. https://v0.dev/docs/github
72. https://chat.v0.dev/docs/faqs.mdx
73. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf
74. https://v0.dev/docs/design-systems
75. https://help.openai.com/en/articles/10128477-chatgpt-enterprise-edu-release-notesGoogle
76. https://help.openai.com/en/articles/9930697-what-is-the-canvas-feature-in-chatgpt-and-how-do-i-use-it&rut=abc3b0f17817b5bba0db1aa06b04d192fadeea43d8cefd274497bd76329e8130
77. https://help.openai.com/resources/translation-view-redirect?articleId=10128477&routeLanguage=cs-cz&redirectTo=/en/articles/10128477-chatgpt-enterprise-a-edu-poznámky-k-vydání
78. https://help.openai.com/resources/translation-view-redirect?articleId=10128477&routeLanguage=pl-pl&redirectTo=/en/articles/10128477-chatgpt-enterprise-i-edu-informacje-o-wersji
79. https://help.openai.com/es-es/articles/6825453-chatgpt-release-notes
80. https://v0.dev/docs

81. https://v0.dev/docs/v0-model-api
82. https://help.openai.com/es-419/articles/9930697-qué-es-la-función-de-canvas-en-chatgpt-y-cómo-la-uso
83. https://nextjs.org/learn/dashboard-app/setting-up-your-database
84. https://vercel.com/kb/guide/nextjs-prisma-postgres
85. https://v0.app/docs/sandbox
86. https://api2.v0.dev/docs/agentic-features
87. https://v0.dev/docs/github
88. https://chat.v0.dev/docs/faqs.mdx
89. https://help.openai.com/en/articles/9930697-using-canvas-in-chatgpt
90. https://unmarkdown.com/blog/claude-artifacts-vs-chatgpt-canvas
91. https://instapods.com/blog/claude-artifacts-vs-chatgpt-canvas/
92. https://github.com/vercel/next.js/discussions/71534
93. https://www.qwe.edu.pl/tutorial/claude-artifacts-interactive-content-tutorial/
94. https://github.com/vercel/postgres-next-starter
95. https://github.com/orgs/community/discussions/141404
96. https://www.reddit.com/r/ChatGPTCoding/comments/1dsx7cb/chatgpt_artifacts/
97. https://github.com/orgs/community/discussions/172355
98. https://www.reddit.com/r/nextjs/comments/1qb9bsw/nextjs_prisma_migrations_in_github_actions_but/
99. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them
100. https://www.netlify.com/knowledge-base/deploy-your-claude-artifacts-to-netlify/
101. https://simonwillison.net/tags/claude-artifacts/
102. https://github.com/claudio-silva/claude-artifact-runner
103. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf
104. https://supabase.com/docs/guides/auth/server-side/creating-a-client
105. https://supabase.com/docs/guides/getting-started/tutorials/with-nextjs
106. https://nextjs.org/docs/pages/guides/environment-variables
107. https://supabase.com/docs/guides/getting-started/quickstarts/nextjs
108. https://github.com/supabase/server/blob/main/docs/ssr-frameworks.md
109. https://stackoverflow.com/questions/78615007/how-to-create-a-shareable-supbase-server-client
110. https://www.jigz.dev/blogs/supabase-server-vs-browser-client-best-use-cases-in-next-js
111. https://www.reddit.com/r/Supabase/comments/17wjtie/nextjs_supabasessr_oauth_am_i_doing_this_right/
112. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them
113. https://simonwillison.net/tags/claude-artifacts/
114. https://www.netlify.com/knowledge-base/deploy-your-claude-artifacts-to-netlify/
115. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf

116. https://supabase.com/partners/catalog/vercel
117. https://developer.paddle.com/sdks/starter-kits/nextjs
118. https://vercel.com/templates/next.js/supabase
119. https://www.npmjs.com/package/@supabase/ssr
120. https://support.claude.com/en/articles/14604416-get-started-with-claude-design
121. https://livemy.app/blog/deploy-claude-artifacts
122. https://supabase.com/docs/guides/auth/server-side/creating-a-client
123. https://supabase.com/docs/guides/getting-started/tutorials/with-nextjs
124. https://nextjs.org/docs/pages/guides/environment-variables
125. https://simonwillison.net/tags/claude-artifacts/
126. https://www.netlify.com/knowledge-base/deploy-your-claude-artifacts-to-netlify/
127. https://github.com/claudio-silva/claude-artifact-runner
128. https://instapods.com/blog/claude-artifacts-vs-chatgpt-canvas/
129. https://www.shareduo.com/blog/claude-artifacts-vs-chatgpt-canvas
130. https://www.ai-toolbox.co/claude-management-and-productivity/how-to-use-claude-artifacts-guide-2026
131. https://www.reddit.com/r/ClaudeAI/comments/1qqbc5s/psa_why_your_external_images_break_in_claude/
132. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them
133. https://code.claude.com/docs/zh-CN/artifacts
134. https://claude.com/features/artifacts
135. https://dev.to/hira_jabeen_ccaa191c13070/ultimate-claude-artifacts-guide-45k3
136. https://instapods.com/blog/claude-artifacts-vs-chatgpt-canvas/
137. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf
138. https://platform.claude.com/docs/en/home
139. https://bestremotetools.com/how-to-use-claude-artifacts-for-rapid-prototyping-react-components/
140. https://code.claude.com/docs/en/artifacts
141. https://support.claude.com/en/articles/14604416-get-started-with-claude-design
142. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them
143. https://www.shareduo.com/blog/claude-artifacts-vs-chatgpt-canvas
144. https://supabase.com/docs/guides/auth/server-side
145. https://supabase.com/docs/guides/auth/server-side/migrating-to-ssr-from-auth-helpers
146. https://supabase.com/docs/guides/auth/quickstarts/nextjs
147. https://nextjs.org/docs/pages/guides/environment-variables
148. https://vercel.com/templates/supabase-auth
149. https://supabase.com/blog/supabase-vercel-partnership
150. https://vercel.com/marketplace/supabase
151. https://dev.to/jais_mukesh/managing-supabase-auth-state-across-server-client-components-in-nextjs-2h2b

152. https://github.com/wpcodevo/nextjs14-supabase-ssr-authentication
153. https://supabase.com/docs/guides/auth/server-side/creating-a-client
154. https://vercel.com/templates/next.js/supabase
155. https://www.npmjs.com/package/@supabase/ssr
156. https://www.shareduo.com/blog/claude-artifacts-vs-chatgpt-canvas
157. https://www.reddit.com/r/Supabase/comments/17wjtie/nextjs_supabasessr_oauth_am_i_doing_this_right/
158. https://instapods.com/blog/claude-artifacts-vs-chatgpt-canvas/
159. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf
160. https://v0.app/community/Q8ZxIpTnhbZ
161. https://supabase.com/docs/guides/troubleshooting/how-to-migrate-from-supabase-auth-helpers-to-ssr-package-5NRunM
162. https://github.com/supabase/ssr
163. https://nextjs.org/docs/pages/guides/environment-variables
164. https://valueaddvc.com/blog/claude-artifacts-the-feature-thats-turning-claude-into-a-mini-app-builder
165. https://supabase.com/docs/guides/getting-started/quickstarts/nextjs
166. https://supabase.com/docs/guides/auth/server-side/migrating-to-ssr-from-auth-helpers
167. https://academy.claude.com/courses/claude-101/creating-with-artifacts
168. https://simonwillison.net/tags/claude-artifacts/
169. https://www.netlify.com/knowledge-base/deploy-your-claude-artifacts-to-netlify/
170. https://www.jigz.dev/blogs/supabase-server-vs-browser-client-best-use-cases-in-next-js
171. https://www.reddit.com/r/Supabase/comments/17wjtie/nextjs_supabasessr_oauth_am_i_doing_this_right/
172. https://nextjs.org/
173. https://nextjs.org/learn
174. https://v0.app/docs/sandbox
175. https://supabase.com/docs/guides/auth/server-side
176. https://supabase.com/docs/guides/auth/server-side/creating-a-client
177. https://vercel.com/docs/sandbox
178. https://supabase.com/docs/guides/troubleshooting/how-do-you-troubleshoot-nextjs---supabase-auth-issues-riMCZV
179. https://github.com/orgs/supabase/discussions/28069
180. https://www.majidpro.com/blog/nextjs-supabase-auth-ssr
181. https://www.iloveblogs.blog/guides/supabase-auth-complete-session-middleware-guide
182. https://www.faisalkarkoh.com/blog/v0-vercel-review-ui-generation-production
183. https://v0.app/docs/sandbox
184. https://supabase.com/docs/guides/auth/server-side/creating-a-client
185. https://nextjs.org/
186. https://nextjs.org/learn

187. https://github.com/claudio-silva/claude-artifact-runner

188. https://www.shareduo.com/blog/claude-artifacts-vs-chatgpt-canvas

189. https://www.reddit.com/r/Supabase/comments/17wjtie/nextjs_supabasessr_oauth_am_i_doing_this_right/

190. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf

191. https://academy.claude.com/courses/claude-101/creating-with-artifacts

192. https://learning.northeastern.edu/ai-student-guides-visual-learning-with-claude/

193. https://support.claude.com/en/articles/17153992-what-are-artifacts-and-how-do-i-use-them

194. https://www.reddit.com/r/ClaudeAI/comments/1w88gld/what_are_good_examples_for_artifacts/

195. https://egghead.io/lessons/supabase-dynamically-render-ui-based-on-user-session-with-ssr-in-next-js-client-components

196. https://vercel.com/docs/sandbox

197. https://valueaddvc.com/blog/claude-artifacts-the-feature-thats-turning-claude-into-a-mini-app-builder